

The Denotational Semantics of SSA

JAD GHALAYINI and NEEL KRISHNASWAMI

Static single assignment form, or SSA, has been the dominant compiler intermediate representation for decades. In this paper, we give a type theory for a variant of SSA, including its equational theory, which are strong enough to validate a variety of control and data flow transformations. We also give a categorical semantics for SSA, and show that the type theory is sound and complete with respect to the categorical axiomatization. We demonstrate the utility of our model by exhibiting a variety of concrete models satisfying our axioms, including in particular a model of TSO weak memory. The correctness of the syntactic metatheory, as well as the completeness proof has been mechanized in the Lean proof assistant.

CCS Concepts: • **Theory of computation** → **Denotational semantics; Categorical semantics; Type theory.**

Additional Key Words and Phrases: SSA, Categorical Semantics, Elgot Structure, Effectful Category

ACM Reference Format:

Jad Ghalayini and Neel Krishnaswami. 2024. The Denotational Semantics of SSA. 1, 1 (June 2024), 100 pages. <https://doi.org/XXXXXXX.XXXXXXX>

This paper is dedicated to the memory of Alan Jeffrey, who taught us about both premonoidal categories and the semantics of weak memory, and who never shied away from either theory or implementation.

1 Introduction

Static single assignment form, or SSA form, has been the dominant compiler intermediate representation since its introduction by Alpern et al. [4] and Rosen et al. [58] in the late 1980s. Most major compilers – GCC, Clang, MLIR, Cranelift – use this representation, because it makes many optimizations much easier to do than traditional 3-address code IRs.

The key idea behind SSA is to adapt an idea from functional programming: namely, every variable is defined only once. This means that substitution is unconditionally valid, without first requiring a dataflow analysis to compute where definitions reach. Furthermore, because variables are immutable, they can have at most one value, which means that a program analysis only needs to store an abstract value once per variable, rather than once per variable per program point, reducing memory overheads from quadratic to linear. Unlike in functional programming, though, scoping of definitions in SSA is traditionally not lexical. Instead, scoping is defined by *dominance*: every variable occurrence must be dominated by a single assignment in the control flow graph.

Traditionally, the semantics of SSA has been handled informally. Since it was conceived of as a simple first-order imperative programming language, whether a rewrite is sound or not was usually obvious, without needing any complex correctness arguments. Unfortunately, all of computers, languages and compilers have become more complex since the late 1980s.

Essentially all modern computers are multicore and feature many levels of caching. As a result, the semantics of memory can no longer be correctly modelled as a big array of bytes, and the

Authors' Contact Information: [Jad Ghalayini](mailto:Jad.Ghalayini@cl.cam.ac.uk), jeg74@cl.cam.ac.uk; [Neel Krishnaswami](mailto:Neel.Krishnaswami@cl.cam.ac.uk), nk480@cl.cam.ac.uk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 ACM.

ACM XXXX-XXXX/2024/6-ART

<https://doi.org/XXXXXXX.XXXXXXX>

execution of a multithreaded program can no longer be viewed as an interleaving of its threads' sequential execution traces. Finding good semantics for modern weak memory systems remains an ongoing challenge. Furthermore, each programming language's semantics must also have its own model of weak memory which both abstracts over the differing weak memory models of all the architectures the language gets compiled to, and also validates the optimizations which compiler writers wish to perform. Finally, modern compilers exploit semantic properties like undefined behaviour to transform programs much more aggressively than they did in the previous millenium.

As a result, it is no longer correct to justify compiler optimizations in terms of the simple imperative model, and it is an open question which equations should hold of an SSA program. Note that the pressure on SSA as an intermediate representation comes from all directions: the machine semantics, the language semantics, and the compiler optimizations have all become more complicated. Once all of these concerns are tangled together, it is very hard to decide if a particular transformation should hold or not.

To resolve this issue, we propose studying the equational theory of SSA. Having a well-defined equational theory for SSA will let us disentangle these concerns, because the equational theory can serve as an interface which both compiler writers and hardware designers can use. The compiler writers can rely upon the equational theory of SSA when justifying optimizations, without needing to know all the details of the memory model at all times. Conversely, memory models could be validated by seeing if they satisfy the equations of SSA, without needing to study every possible compiler optimization.

Defining an equational theory for SSA is a nontrivial problem, for both minor and major reasons. Traditionally, SSA is presented as a collection of basic blocks augmented with ϕ -functions to handle control-flow merges. This is a good representation for compiler engineering, but creates a large number of papercuts when defining an operational semantics or equational theory. Operational semantics for SSA have to keep track of the execution history to correctly interpret ϕ -nodes, and the lack of compound expressions makes it awkward to define and use primitives like parallel substitution. In addition, when all one has is an operational semantics, the only natural notion of program equivalence is contextual equivalence. Unfortunately, contextual equivalence is extremely sensitive to the effects available in the language (and SSA often has to deal with very complex effects like weak memory), and this makes it difficult to formulate a criterion for when the equational theory is complete.

If we had a class of denotational models for SSA, then it would be possible to give an equational theory, and show that it is complete (or not) relative to this class of models. But this is precisely the classical methodology of type theory and categorical logic! To fully understand a programming language, in general one wants a type theory equipped with an equational theory, a proof that it is sound and complete with respect to a categorical axiomatization, and a variety of interesting concrete models satisfying those axioms. Moreover, it would be best if the categorical axiomatization was constructed from "standard parts": the less novel the categorical axioms, the better, because standard constructions are better-studied and have more theorems already proved about them. This is exactly what we provide in this paper. Concretely, our contributions are as follows:

- First, we give a type-theoretic presentation of SSA, with both typing rules (in Section 3) and an equational theory (in Section 4) for well-typed terms. We also prove the correctness of suitable substitution properties for this calculus.
- Next, in Section 5, we give a categorical semantics for this type theory, in terms of distributive Elgot categories, which are Freyd categories equipped with distributive coproducts and a strong Conway iteration operator. This demonstrates that a categorical semantics for SSA can be constructed from well-known structures. We use Freyd categories to model

sequencing of imperative computations, we use coproducts to model conditionals, and we use Conway iteration to model looping.

We show that any category with this structure is a model of SSA. This establishes that all of the equations we give are sound with respect to the categorical structure.

- We also show, in Section 5.5, that syntax quotiented by the equational theory yields the initial distributive Elgot category. This establishes that our set of syntactic equations is complete, and that there are no equations which the denotational semantics validates, but which cannot be proved syntactically.

Theoretically, this leads us to the surprising discovery that SSA is actually a syntactic presentation of distributive Elgot categories, thereby establishing a surprising connection between compiler IRs and category theory. (In fact, our calculus is also the first syntactic presentation of distributive Elgot categories.)

Practically, it means that we can freely mix equational and semantic reasoning depending on what is convenient for the problem at hand: a semantic proof which holds in all λ_{SSA} -models guarantees the existence of a corresponding syntactic sequence of rewrites.

- We proceed in Section 6 to show that this denotational axiomatization is useful in practice. We give a model of TSO weak memory based on Kavanagh and Brookes [38] in Section 6.3, We also give a family of concrete models based on Brookes-style traces [12], which can be instantiated to support the release/acquire model of Dvir et al. [18]. This demonstrates that it is possible to give realistic weak memory models, in a variety of semantic styles, which do not disturb the structure of SSA in fundamental ways. Furthermore, our results constitute the first proof that SSA-based loop transformations are compatible with weak memory.
- Finally, we have substantially mechanized our proofs using the Lean 4 proof assistant. We have mechanized proofs of substitution for our type theory, as well as proofs that the syntax forms the initial model, and that the SPARC TSO semantics forms a valid model of SSA. The denotational semantics and its proof of the soundness of substitution are done on paper.

2 Static Single Assignment Form

2.1 From Three Address Code to SSA

Directly optimizing a source language can be difficult, because surface languages are often very large and have features (such as type inference and overloading) which make it difficult to express sound program equivalences. Elaborating a surface language to a simpler intermediate representation makes it easier to write program analyses and optimizations. One of the earliest compiler IRs introduced is *three-address code* [3] (also known as *register transfer language (RTL)*).

3-address programs consists of a *control-flow graph* (CFG) G with a distinguished, nameless entry block. Each node of the CFG corresponds to a *basic block* β , which is a straight-line sequence of *instructions* $x = f(y, z)$ (hence the name *3-address code*, referring to the typical three variables x, y, z) followed by a *terminator* τ , which can be a (conditional) branch to another basic block. We give a grammar for 3-address code in Figure 1, with some slight adjustments to the usual presentation:

- *Constants* c are interpreted as nullary instructions $c()$.
- We only support nullary tuples $()$ and binary tuples (v, v') as composite values; n -ary tuples (x, y, z) can be encoded as nested binary tuples $((x, y), z)$.
- We allow conditional branches if $o \{ \tau \}$ else $\{ \tau' \}$ to be *nested*. This allows us to emulate switch-statements $\text{switch}(o) \{ c_1 : \tau_1, \dots, c_n : \tau_n \}$ as nested if-statements without introducing spurious basic blocks. Likewise, for uniformity, a return-statement may appear in the branch of a conditional branch.

148	let $n = 10$;	$n = 10$;
149	let mut $i = 1$;	$i = 1$;
150	let mut $a = 1$;	$a = 1$;
151	while $i < n$ {	br loop;
152	$a = a * (i + 1)$	loop : if $i < n$ { br body } else { ret a };
153	$i = i + 1$;	body : $t = i + 1$;
154	}	$a = a * t$;
155	ret a	$i = i + 1$;
156		br loop
157		
158		
159		
160	(a) As an imperative program	(b) As 3-address code

Fig. 2. A simple, slightly suboptimal program to compute $10!$ via multiplication in a loop, represented as typical imperative code and in 3-address code.

$v ::= x \mid (v, v') \mid ()$
 $o ::= v \mid f \ v \mid \iota_l \ v \mid \iota_r \ v \mid \text{abort } v$
 $\beta ::= x = o; \beta \mid (x, y) = o; \beta \mid \tau$
 $\tau ::= \text{br } \ell \mid \text{ret } v \mid \text{if } o \{ \tau \} \text{ else } \{ \tau' \}$
 $G ::= \beta \mid G; \ell : \beta$

Fig. 1. Grammar for 3-address code

As a concrete example, consider the simple imperative program to compute $10!$ given in Figure 2. We can normalize our code into 3-address code, as in Figure 2b, by:

- Converting structured control flow (e.g., while) into unstructured jumps between basic blocks.
- Converting composite expressions like $a * (i + 1)$ into a sequence of definitions naming each subexpression. Here, expressions like $a + b$ are syntactic sugar for primitive operations $+(a, b)$.

While functional languages typically rely on *lexical scoping*, where the scope of a variable is determined by its position within the code's nested structure, 3-address code uses a different scoping mechanism based on *dominance*. In particular, a variable x is considered to be in scope at a specific point P if and only if all execution paths from the program's entry point to P pass through a definition D for x . In this case, we say that the definition D *dominates* P . The relation on basic blocks " A dominates B " can in fact be viewed as a tree rooted at the entry block: every pair of basic blocks A, B have a least common ancestor C which dominates them both; we call this tree the *dominator tree* [16].

Even though three address code was designed to simplify flow analysis, many optimizations remain difficult to express in this format. Because a variable's value may be set by multiple definitions throughout the program's execution, variables do not have stable values, and so it is not in general safe to substitute a definition for a variable. To improve our ability to reason about programs, we

introduce the *static single assignment* restriction, originally proposed by Alpern et al. [4], which states that every variable must be defined at exactly one point in the program. Because there is a unique definition for each variable, substitution is valid.

We can intuitively think of each variable as being defined by an immutable let-binding, and a variable x is in scope at a program point P , if and only if its unique definition site D_x strictly dominates P .

A given basic block can be converted to SSA form by numbering each definition of a variable, effectively changing references to x to references to x_t , i.e. “ x at time t .” For example, we could rewrite

$$\begin{array}{lll} x = 3y + 5; & & \text{let } x_0 = 3y + 5; \\ x = 3x + 2; & \approx & \text{let } x_1 = 3x_0 + 2; \\ \text{ret } (3x + 1) & & \text{ret } (3x_1 + 1) \end{array}$$

This transformation enables algebraic reasoning about expressions involving each x_t . However, since we can only define a variable once in SSA form, expressing programs with loops and branches becomes challenging. For example, naïvely trying to lower the program in Figure 2b into SSA form would not work, since the reference to i in the right-hand-side of the statement $i = i + 1$ can refer to *either* the previous value of i from the last iteration of the loop *or* the original value $i = 1$. The classical solution is to introduce ϕ -nodes, which select a value based on the predecessor block from which control arrived. We give the lowering of our program into SSA with ϕ -nodes in Figure 3b.

Cytron et al. [16] introduced the first efficient algorithm to lower a program in 3-address code to valid SSA while introducing a minimum number of ϕ -nodes, making SSA practical for widespread use as an intermediate representation. Unfortunately, ϕ -nodes do not have an obvious operational semantics.

Additionally, they require us to adopt more complex scoping rules than simple dominance-based scoping. For example, in the basic block loop in Figure 3b, i_0 evaluates to 1 if we came from the entry block and to i_1 if we came from body. Similarly, a_0 evaluates to either 1 or a_1 based on the predecessor block. This does not obey dominance-based scoping, since i_0 and i_1 are defined *after* the predecessor block. This seems counterintuitive – after all, variables are typically used after they are defined. In fact, since the value of a ϕ -node is determined by which basic block is our immediate predecessor, we instead need to use the rule that expressions in ϕ -node branches with source S can use any variable y defined at the *end* of S . Note that this is a strict superset of the variables visible for a normal instruction x , which can only use variables y which *dominate* x – i.e., such that *every* path from the entry block to the definition of x goes through y , rather than only those paths which also go through S .

While this rule can be quite confusing, and in particular makes it non-obvious how to assign an operational semantics to ϕ -nodes, the fact that the scoping for ϕ -node branches is based on the source block, rather than the block in which the ϕ -node itself appears, hints at a possible solution. By *moving* the expression in each branch to the *call-site*, we can transition to an isomorphic syntax called basic blocks with arguments (BBA), as illustrated in Figure 5b. In this approach, each ϕ -node – since it lacks side effects and has scoping rules independent of its position in the basic block, depending only on the source of each branch – can be moved to the top of the block. This reorganization allows us to treat each ϕ -node as equivalent to an argument for the basic block, with the corresponding values passed at the jump site. Converting a program from BBA format back to standard SSA form with ϕ -nodes is straightforward: introduce a ϕ -node for each argument of a basic block, and for each branch corresponding to the ϕ -node, add an argument to the jump instruction from the appropriate source block.

246	$n = 10;$	$\text{let } n = 10;$
247	$i = 1;$	br loop
248	$a = 1;$	$\text{loop : let } i_0 = \phi(\text{entry} : 1, \text{body} : i_1);$
249	$\text{br loop};$	$\text{let } a_0 = \phi(\text{entry} : 1, \text{body} : a_1);$
250	$\text{loop : if } i < n \{ \text{br body} \} \text{ else } \{ \text{ret } a \};$	$\text{if } i_0 < n \{ \text{br body} \} \text{ else } \{ \text{ret } a_0 \};$
251	$\text{body : let } t = i + 1;$	$\text{body : let } t = i_0 + 1$
252	$a = a * t;$	$\text{let } a_1 = a_0 * t$
253	$i = i + 1;$	$\text{let } i_1 = i_0 + 1$
254	br loop	br loop
255	(a) 3-address code	(b) Converted to SSA form

Fig. 3. Conversion of three address code for the program in Figure 2 to SSA form, requiring the insertion of ϕ -nodes for i and a due to control-flow dependent updates. Note how SSA-form can be viewed as “three address code in which all let-bindings are immutable.”

We give a formal grammar for basic blocks-with-arguments SSA in Figure 4¹. One of the other changes we make is replacing conditional branches $\text{if } o \{ \tau \} \text{ else } \{ \tau' \}$ with *case-statements* $\text{case } o \{ \iota_l y : \tau, \iota_r z : \tau' \}$. In particular, a case-statement $\text{if } o \{ \tau \} \text{ else } \{ \tau' \}$ may be desugared into a case-statement on a Boolean value $o : 1 + 1$.

Note that this grammar no longer needs a separate terminator for returns: we can treat the return point as a distinguished label (with argument) that a program can jump to.

$v ::= x \mid (v, v') \mid ()$
 $o ::= v \mid f v \mid \iota_l v \mid \iota_r v \mid \text{abort } v$
 $\beta ::= \text{let } x = o; \beta \mid \text{let } (x, y) = o; \beta \mid \tau$
 $\tau ::= \text{br } \ell o \mid \text{case } o \{ \iota_l y : \tau, \iota_r z : \tau' \}$
 $G ::= G \ell(x) : \beta \mid \beta$

Fig. 4. Grammar for basic blocks-with-arguments SSA

This allows us to use dominance-based scoping without any special cases for ϕ -nodes. When considering basic blocks, this means that a variable is visible within the block D where it is defined, starting from the point of its definition. It continues to be visible in all subsequent blocks P that are strictly dominated by D in the control-flow graph (CFG). For example, in Figure 5b:

- The entry block strictly dominates all other blocks by definition; thus, the variable n is visible in loop and body.
- loop strictly dominates body; therefore, the parameters i_0, a_0 to loop are visible in body without the need to pass them as parameters.

¹Many variants of SSA do not allow variables to appear alone on the right-hand side of assignments, such as $x = y; \beta$. We do not incorporate this restriction, though we could by normalizing even further and substituting $[y/x]\beta$ instead.

295	let $n = 10$;	let $n = 10$;
296	br loop;	br loop(1 , 1);
297		
298	loop : let $i_0 = \phi(\text{entry} : 1, \text{body} : i_1)$;	loop(i_0, a_0) : if $i_0 < n$ { br body }
299	let $a_0 = \phi(\text{entry} : 1, \text{body} : a_1)$;	else { ret a_0 };
300		
301	if $i_0 < n$ { br body }	body : let $t = i_0 + 1$
302	else { ret a_0 };	let $a_1 = a_0 * t$
303	body : let $t = i_0 + 1$	let $i_1 = i_0 + 1$
304	let $a_1 = a_0 * t$	br loop(i_1, a_1)
305	let $i_1 = i_0 + 1$	
306	br loop	
307		
308	(a) With ϕ -nodes	(b) Basic-blocks with arguments

Fig. 5. The program in Figure 2 written in standard SSA (using ϕ nodes), like in LLVM [41], and in basic-blocks with arguments SSA, like in MLIR [42] and Cranelift [59]. The arguments i_0, a_0 corresponding to the ϕ -nodes i_0, a_0 are colored in red and blue, respectively.

- body does *not* strictly dominate loop, since there is a path from the entry block to loop that does not pass through body.

2.2 Type-theoretic SSA

An important insight provided by the BBA format, as discussed by Appel [5] and Kelsey [39], is that a program in SSA form can be interpreted as a collection of tail-recursive functions, where each basic block and branch correspond to a function and tail call, respectively. This yields a natural framework for defining the semantics of SSA and reasoning about optimizations.

A program in BBA is not quite a functional program, because scoping is dominance-based rather than lexically scoped. However, it turns out to be very easy to convert dominance-based scoping into lexical scoping. Observe that the function corresponding to a given basic block B can only be called by other blocks B' having that basic block's parent $P = \text{parent}(B)$ as an ancestor in the dominance tree (as, otherwise, the parent would not actually dominate the block, since we could get to B through B' without passing through P). Moreover, the variables *visible* in B are exactly the variables visible at the end of P ; i.e., the variables visible in P and those defined in P .

So if we make the dominance tree explicit in the syntax and tie the binding of variables to this tree structure, then lexical and dominance-based scoping become one and the same. We use this observation to introduce *lexical SSA* in Figure 6. The key idea of this syntax is to, rather than treating the control-flow graph G as a flat collection of basic blocks (with a distinguished block), to instead consider (subtrees of) the dominance tree r , with the root of the tree implicitly being the entry block. We call such subtrees *regions*: we note that they have a single entry (the root) and multiple exits (the leaves), and so generalize the more standard concept of a single-entry-single-exit region in a CFG.

In particular, a *region* r generalizes a basic block β by annotating the terminator τ with a list L of *labeled branches* " $\ell_i(x_i) : \{t_i\}$," yielding a *where-block* " τ where L ". Each ℓ_i can only be branched to by τ and the regions t_i , thus syntactically enforcing that the basic block at the root of r (made up of its instructions and terminators) *dominates* all the basic blocks in the subregions t_i (which can only be reached through r). The data of a region r is thus exactly the data contained in a basic block


```

344 1  struct BasicBlock {
345 2      // unary/binary let-bindings, collected into a list
346 3      vector<Instruction> instructions;
347 4      Terminator terminator;
348 5      // Dominated basic blocks, forming a subtree of the dominance tree
349 6      // Note that only `terminator` is allowed to jump to blocks in `children`
350 7      map<Label, (Argument, BasicBlock)> children;
351 8  }
352 9

```

Fig. 7. Data encoded by the grammar in Figure 6

β (its instructions and terminator) together with a set of subregions dominated by r ; in C++-like pseudocode, we might represent a region as in Figure 7.

Regions allow us to enforce dominance-based scoping simply by making the variables defined in r visible only in the t_i , which, as previously stated, *must* be dominated by r ; i.e., dominance based scoping becomes lexical scoping of where-blocks. It is easy to see (we demonstrate this more rigorously in Section 4.4) that, given a CFG G , there exists some way to annotate its topological sort w.r.t. the dominance relation with where-blocks to obtain a region r which is lexically well-scoped if and only if C is a valid SSA program; we illustrate this process on our running example in Figure 8. Conversely, erasing the where-blocks from a region r and giving the root a name trivially yields a (topologically sorted!) SSA program, establishing an isomorphism between lexical SSA and standard SSA.

```

367  v ::= x | (v, v') | ()
368
369  o ::= v | f v |  $\iota_l v$  |  $\iota_r v$  | abort v
370
371  r, s, t ::= let x = o; t | let (x, y) = o; t |  $\tau$  where L
372
373   $\tau$  ::= br  $\ell$  o | case o {  $\iota_l y : \tau, \iota_r z : \tau'$  }
374
375  L ::=  $\cdot$  | L,  $\ell(x) : \{t\}$ 

```

Fig. 6. Grammar for lexically-scoped SSA

Lexical scoping allows us to apply many of techniques developed in type theory and functional programming for reasoning about program transformations. Indeed, the result of our conversion to lexical scoping looks a lot like the correspondence between SSA and CPS described in Kelsey [39]. We can use this correspondence to guide us in developing an *equational theory* for SSA programs, with the goal of enabling compositional reasoning about program transformations such as:

- *Control-flow rewrites*, such as jump-threading or fusing two identical branches of an if-statement
- *Algebraic rewrites*, such as simplifying arithmetic expressions
- Combinations of the two, such as rewriting “if $x > 0$ then $0 - x$ else x ” to “ $\text{abs}(x)$ ”.

To help achieve this, we will slightly generalize our syntax by:

- (1) Fusing the syntactic categories o, v of operations and values into the syntactic category a of *expressions*
- (2) Fusing the syntactic category τ of terminators into the syntactic category of regions r .

393	let $n = 10$;	let $n = 10$;
394	br loop(1, 1)	br loop(1, 1)
395		where loop(i_0, a_0) : {
396	loop(i_0, a_0) : if $i_0 < n$ { br body }	if $i_0 < n$ { br body }
397	else { ret a_0 }	else { ret a_0 }
398	body : let $t = i_0 + 1$	where body : {
399	let $a_1 = a_0 * t$	let $t = i_0 + 1$
400	let $i_1 = i_0 + 1$	let $a_1 = a_0 * t$
401	br loop(i_1, a_1)	let $i_1 = i_0 + 1$
402		br loop(i_1, a_1)
403		}
404		}
405		}
406		}
407		}
408		}
409	(a) Dominance-based scoping	(b) Lexical scoping

Fig. 8. Conversion of an SSA program from dominance-based scoping to explicit lexical scoping

- (3) Extending expressions a to allow *let-expressions* “let $x = a$; b ” and *case-expressions* “case a { $l_l x : b, l_r y : c$ }”

This leaves us with our final language, λ_{SSA} , the resulting grammar for which is given in Figure 9. It is easy to see that these changes add no expressive power to lexical SSA: we can desugar 1 by introducing names for anonymous sub-expressions, 2 by introducing names for anonymous sub-regions, and 3 by floating out let-bindings and case-statements in the obvious manner, introducing labels as necessary; we discuss this in more detail in Section 4.4.

Change 1 allows us to effectively reason about *substitution*: replacing the value of a variable (which is a value v) with its definition (which is an instruction o). This can be used as a building block for optimizations such as common subexpression elimination and global value numbering; combined with change 3, we can also reason algebraically about “branching” operations like conditional move and absolute value.

On the other hand, 2 lets us replace an unconditional branch $\text{br } \ell \ a$ (which is a terminator τ) with the code *pointed to* by the label ℓ (which is a region r), allowing us to perform the jump-threading optimization

$$\text{let } x = a; \text{br } \ell \ b \text{ where } \ell(y) : \{r\} \approx \text{let } x = a; \text{let } y = b; r \text{ where } \ell(y) : \{r\}$$

While both sides of this equation are valid lexical SSA programs, by loosening our syntax slightly, we can *unconditionally* replace jumps with regions, without worrying about jumps nested in case statements or fusing where-blocks. This, especially combined with change 3, makes it much easier to verify optimizations such as

$$\begin{aligned} & \text{case } a \{ l_l x : \text{br } \ell \ (l_r x), l_r x : \text{br } \ell \ (l_l x) \} \text{ where } \ell(y) : \{ \text{case } y \{ l_l z : \text{ret } (l_r z), l_r z : \text{ret } (l_l z) \} \} \\ & \approx \text{case } a \{ l_l x : \text{case } l_r x \{ l_l z : \text{ret } (l_r z), l_r z : \text{ret } (l_l z) \} \\ & \quad , l_r x : \text{case } l_l x \{ l_l z : \text{ret } (l_r z), l_r z : \text{ret } (l_l z) \} \} \\ & \approx \text{case } a \{ l_l x : \text{ret}(l_l x), l_r x : \text{ret}(l_r x) \} \approx \text{ret } (\text{case } a \{ l_l x : l_l x, l_r x : l_r x \}) \approx \text{ret } a \end{aligned}$$

by repeatedly applying a set of known-good rules, and, moreover, dramatically simplifies the form of the rules themselves.

$$\begin{aligned}
 a, b, c, e &::= x \mid f \ a \mid \text{let } x = a; e \\
 &\mid () \mid (a, b) \mid \text{let } (x, y) = a; e \\
 &\mid \iota_l \ a \mid \iota_r \ a \mid \text{abort } a \mid \text{case } e \{ \iota_l \ x : a, \iota_r \ y : b \} \\
 r, s, t &::= \text{let } x = a; t \mid \text{let } (x, y) = a; t \mid t \text{ where } L \mid \text{br } \ell \ a \mid \text{case } e \{ \iota_l \ x : s, \iota_r \ y : t \} \\
 L &::= \cdot \mid L, \ell(x) : \{t\}
 \end{aligned}$$

Fig. 9. Grammar for λ_{SSA}

3 Type Theory

We now give a formal account of λ_{SSA} , starting with the types. Our types are first order, and consists of binary sums $A + B$, products $A \otimes B$, the unit type $\mathbf{1}$, and the empty type $\mathbf{0}$, all parameterised over a set of base types $X \in \mathcal{T}$. We write our set of types as $\text{Ty}(X)$.

A (variable) *context* Γ is a list of *typing hypotheses* $x : A$, where x is a variable name and A is the type of that variable. Similarly, we define a *label-context* to be a list of *labels* $\ell(A)$, where A is the parameter type that must be passed on a jump to the label ℓ . The grammar for types, contexts, and label-contexts is given in Figure 10.

$$\begin{aligned}
 A, B, C &::= X \mid A \otimes B \mid \mathbf{1} \mid A + B \mid \mathbf{0} \\
 \Gamma &::= \cdot \mid \Gamma, x : A \\
 L &::= \cdot \mid L, \ell(A)
 \end{aligned}$$

Fig. 10. Grammar for λ_{SSA} types, contexts, and label-contexts

Our grammar in Figure 9 was implicitly parameterised over a set of *primitive instructions* $f \in \mathcal{I}$. In particular, for each pair $A, B \in \text{Ty}(X)$ we specify a set of primitive instructions $f \in \mathcal{I}(A, B)$, with a subset of *pure instructions* $\mathcal{I}_{\perp}(A, B)$. To allow us to write $\mathcal{I}_{\epsilon}$ for an *effect* $\epsilon \in \{\top, \perp\}$, we denote $\mathcal{I}_{\top}(A, B) := \mathcal{I}(A, B)$. In general, we define $\mathcal{I}_{\epsilon} = \bigcup_{A, B} \mathcal{I}_{\epsilon}(A, B)$, and $\mathcal{I} = \bigcup_{\epsilon} \mathcal{I}_{\epsilon}$.

We'll call a tuple $Sg = (\mathcal{T}, \mathcal{I})$ of types and instructions over these types an λ_{SSA} -signature, and, for the rest of this section, work over a fixed signature.

As shown in Figure 9, λ_{SSA} terms are divided into two syntactic categories, each associated with a judgement:

- *Expressions* a, b, c, e , which are typed with the judgement $\Gamma \vdash_e a : A$, which says that under the typing context Γ , the expression a has type A and effect ϵ . We say a term is *pure* if it has effect $\perp$; note that whether an expression is pure or not depends both on the expression itself and on the purity of the variables used in the expression; this is to allow reasoning about impure substitutions.
- *Regions* r, s, t , which recursively define a lexically-scoped SSA program with a single entry and (potentially) multiple exits. This is typed with the judgement $\Gamma \vdash r \triangleright L$, which states that given that Γ is live at the unique entry point, r will either loop forever or branch to one of the exit labels in $\ell(A) \in L$ with an argument of type A .

The typing rules for expressions are given in Figure 11. In particular, expressions may be built up from the following fairly standard primitives:

$$\begin{array}{c}
\boxed{\Gamma \vdash_{\epsilon} a : A} \\
\frac{\Gamma(x) = A}{\Gamma \vdash_{\epsilon} x : A} \text{var} \quad \frac{f \in \mathcal{I}_{\epsilon}(A, B) \quad \Gamma \vdash_{\epsilon} a : A}{\Gamma \vdash_{\epsilon} f a : B} \text{op} \quad \frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma, x : A \vdash_{\epsilon} b : B}{\Gamma \vdash_{\epsilon} \text{let } x = a; b : B} \text{let}_1 \\
\frac{}{\Gamma \vdash_{\epsilon} () : \mathbf{1}} \text{unit} \quad \frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma \vdash_{\epsilon} b : B}{\Gamma \vdash_{\epsilon} (a, b) : A \otimes B} \text{pair} \\
\frac{\Gamma \vdash_{\epsilon} e : A \otimes B \quad \Gamma, x : A, y : B \vdash_{\epsilon} c : C}{\Gamma \vdash_{\epsilon} \text{let } (x, y) = e; c : C} \text{let}_2 \\
\frac{\Gamma \vdash_{\epsilon} a : A}{\Gamma \vdash_{\epsilon} \iota_l a : A + B} \text{inl} \quad \frac{\Gamma \vdash_{\epsilon} b : B}{\Gamma \vdash_{\epsilon} \iota_r b : A + B} \text{inr} \quad \frac{\Gamma \vdash_{\epsilon} a : \mathbf{0}}{\Gamma \vdash_{\epsilon} \text{abort } a : A} \text{abort} \\
\frac{\Gamma \vdash_{\epsilon} e : A + B \quad \Gamma, x : A \vdash_{\epsilon} a : C \quad \Gamma, y : A \vdash_{\epsilon} b : C}{\Gamma \vdash_{\epsilon} \text{case } e \{ \iota_l x : a, \iota_r y : b \} : C} \text{case}
\end{array}$$

Fig. 11. Rules for typing λ_{SSA} expressions

- A variable x in the context Γ , as typed by `var`.
- A *primitive instruction* $f \in \mathcal{I}_{\epsilon}(A, B)$ applied to an expression $\Gamma \vdash_{\epsilon} a : A$, typed by `op`
- Unary and binary *let-bindings*, typed by `let1` and `let2` respectively
- A *pair* of expressions $\Gamma \vdash_{\epsilon} a : A, \Gamma \vdash_{\epsilon} b : B$, typed by `pair`. Operationally, we interpret this as executing a , and then b , and returning the pair of their values.
- An empty tuple $()$, which types in any context by `unit`
- Injections, typed by `inl` and `inr`
- Pattern matching on sum types, typed by `case`. Operationally, we interpret this as executing e , and then, if e is a left injection $\iota_l x$, executing a with its value (x) , otherwise executing b .
- An operator `abort` e allowing us to abort execution if given a value of the empty type. Since the empty type is a 0-ary sum type, `abort` can be seen as a case with no branches. Since the empty type is uninhabited, execution can never reach an abort. This can be viewed as a typesafe version of the unreachable instruction in LLVM IR.

Traditional presentations of SSA use a boolean type instead of sum types. Naturally, booleans can be encoded with sum types as $\mathbf{1} + \mathbf{1}$. If-then-else is then a case which ignores the unit payloads, so that if $e_1 \{e_2\} \text{ else } \{e_3\} := \text{case } e_1 \{ \iota_l () : e_2, \iota_r () : e_3 \}$.

We now move on to *regions*, which can be typed as follows:

- A branch to a label ℓ with pure argument a , typed with `br`.
- Unary and binary *let-bindings*, typed by `let1` and `let2` respectively
- Pattern matching on sum types, typed by `case`. Operationally, we interpret this as executing the expression e , and then, if e is a left injection $\iota_l x$, executing r with its value (x) , otherwise executing s .
- *where-blocks* of the form “ r where $(\ell_i(x_i) : \{t_i\})_i$ ”, which consist of a collection of mutually recursive regions $\ell_i(x_i) : \{t_i\}$ and a *terminator region* r which may branch to one of ℓ_i or an exit label.

3.1 Metatheory

We can now begin to state the syntactic metatheory of λ_{SSA} . One of the most important metatheorems, and a basic sanity check of our type theory, is *weakening*; essentially, if something typechecks in a context Δ , and Γ contains all the variables of Δ (written $\Gamma \leq \Delta$, pronounced “ Γ *weakens* Δ ”),

$$\begin{array}{c}
\boxed{\Gamma \vdash r \triangleright L} \\
\frac{\Gamma \vdash_{\perp} a : A \quad L \ell = A}{\Gamma \vdash \text{br } \ell \ a \triangleright L} \text{br} \quad \frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma, x : A \vdash r \triangleright L}{\Gamma \vdash \text{let } x = a; r \triangleright L} \text{let}_1\text{-r} \\
\frac{\Gamma \vdash_{\epsilon} e : A \otimes B \quad \Gamma, x : A, y : B \vdash r \triangleright L}{\Gamma \vdash \text{let } (x, y) = e; r \triangleright L} \text{let}_2\text{-r} \\
\frac{\Gamma \vdash_{\epsilon} e : A + B \quad \Gamma, x : A \vdash r \triangleright L \quad \Gamma, y : B \vdash s \triangleright L}{\Gamma \vdash \text{case } e \{ \iota_l x : r, \iota_r y : s \} \triangleright L} \text{case-r} \\
\frac{\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_{i \in I} \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I}}{\Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I} \triangleright L} \text{cfg}
\end{array}$$

Fig. 12. Rules for typing λ_{SSA} regions

then it should typecheck in the context Γ as well. Here, the context with fewer variables appears on the *right*, allowing us to compose typing judgements likeso

$$\Gamma \leq \Delta \implies \Delta \vdash r \triangleright L \implies \Gamma \vdash r \triangleright L$$

As our theory has two types of context; we'd also like to define *label-weakening* $L \leq K$, which we should be able to apply in the same manner:

$$\Gamma \vdash r \triangleright L \implies L \leq K \implies \Gamma \vdash r \triangleright K$$

If a region r typechecks with exit labels L , and K contains every label in L , then r should obviously also typecheck in K . It follows that in the judgement $L \leq K$ the context with fewer labels appears on the *left*-hand side of the judgement: this corresponds precisely to the fact that label-weakening (injection into a coproduct) is semantically dual to variable-weakening (projection from a product), and hence the order is flipped.

We give the (standard) formal rules for weakening $\Gamma \leq \Delta$, and their duals, in the first part of Figure 13.

- `wk-nil` and `lwk-nil` say that the empty (label) context weakens itself,
- `wk-skip` says that if Γ weakens Δ , then $\Gamma, x : A$ also weakens Δ for arbitrary (fresh) x . Dually, `lwk-skip` says that if L weakens K , then L also weakens $K, \ell(A)$ for arbitrary (fresh) ℓ .
- `wk-cons` says that if Γ weakens Δ , then Γ with $x : A$ added weakens $\Delta, x : A$. Likewise, `lwk-cons` says that if L weakens K , then L with $\ell(A)$ added weakens $K, \ell(A)$.

It is easy to see that (label) weakening defined in this manner induces a partial order on (label) contexts. Our weakening lemma is then as follows:

LEMMA 3.1 (WEAKENING). *Given $\Gamma \leq \Delta$, $\epsilon \leq \epsilon'$, we have that:*

- If $\Delta \vdash_{\epsilon} a : A$, then $\Gamma \vdash_{\epsilon'} a : A$*
- If $L \leq K$ and $\Delta \vdash r \triangleright L$, then $\Gamma \vdash r \triangleright K$*
- If $\gamma : \Delta \mapsto \Xi$, then $\gamma : \Gamma \mapsto \Xi$*
- If $\Delta \vdash \sigma : L \rightsquigarrow K$, then $\Gamma \vdash \sigma : L \rightsquigarrow K$*

PROOF. These are formalized as:

- `Term.Wf.wk` in `Typing/Term/Basic.lean`
- `Region.Wf.wk` in `Typing/Region/Basic.lean`
- Follows from `Term.Subst.Wf.comp` in `Typing/Term/Subst.lean`
- Follows from `Region.Subst.Wf.vsubst` in `Typing/Region/LSubst.lean`

$$\begin{array}{c}
\boxed{\Gamma \leq \Delta} \\
\frac{}{\cdot \leq \cdot} \text{wk-nil} \quad \frac{\Gamma \leq \Delta}{\Gamma, x : A \leq \Delta} \text{wk-skip} \quad \frac{\Gamma \leq \Delta}{\Gamma, x : A \leq \Delta, x : A} \text{wk-cons} \\
\boxed{L \leq K} \\
\frac{}{\cdot \leq \cdot} \text{lwk-nil} \quad \frac{L \leq K}{L \leq K, \ell(A)} \text{lwk-skip} \quad \frac{L \leq K}{L, \ell(A) \leq K, \ell(A)} \text{lwk-cons} \\
\boxed{\gamma : \Gamma \mapsto \Delta} \\
\frac{}{\cdot : \Gamma \mapsto \cdot} \text{sb-nil} \quad \frac{\gamma : \Gamma \mapsto \Delta \quad \Gamma \vdash_{\perp} e : A}{\gamma, x \mapsto e : \Gamma \mapsto \Delta, x : A} \text{sb-cons} \\
\boxed{\Gamma \vdash \sigma : L \rightsquigarrow K} \\
\frac{}{\Gamma \vdash \cdot : \cdot \rightsquigarrow K} \text{ls-nil} \\
\frac{\Gamma \vdash \sigma : L \rightsquigarrow K \quad \Gamma, x : A \vdash r \triangleright K \quad \Gamma \vdash \sigma : L \rightsquigarrow K}{\Gamma \vdash \sigma, \ell(x) \mapsto r : L, \ell(A) \rightsquigarrow K} \text{ls-cons}
\end{array}$$

Fig. 13. Rules for typing λ_{SSA} weakening and substitution

□

The validity of variable weakening hinges on the fact that all the variables in Δ are also available with the same type in Γ , i.e., if $\Delta \vdash_{\epsilon} x : A \implies \Gamma \vdash_{\epsilon} x : A$, then anything which can be typed in Δ can be typed in Γ . So while weakening on *terms* is just the identity, weakening on *derivations* is essentially replacing “variables from Δ ” with “variables from Γ .” Since none of our typing rules, other than *var*, make use of variable names, we might ask whether we can repeat essentially the same reasoning to reason about the well-typedness of replacing variables in Γ with arbitrary pure expressions of the same type (i.e., perform a substitution). An assignment of such variables $\gamma : x \mapsto \gamma_x$ is called a *substitution*, which we can type with the judgement $\gamma : \Gamma \mapsto \Delta$ as per the rules given in Figure 13. In particular,

- *sb-nil* says that the empty substitution takes every context to the empty context.
- *sb-cons* says that if γ takes Γ to Δ and $\Gamma \vdash_{\perp} e : A$, then γ with the additional substitution $x \mapsto e$ adjoined takes Γ to $\Delta, x : A$

To *use* a substitution, we simply need to perform standard capture-avoiding substitution (see Figure 37 in the appendix). Substitution satisfies the *substitution lemma* as follows:

LEMMA 3.2 (SUBSTITUTION). *Given $\gamma : \Gamma \mapsto \Delta$, we have that:*

- $\Delta \vdash_{\epsilon} a : A \implies \Gamma \vdash_{\epsilon} [\gamma]a : A$
- $\Delta \vdash r \triangleright L \implies \Gamma \vdash [\gamma]r \triangleright L$
- $\gamma_2 : \Delta \mapsto \Xi \implies [\gamma]\gamma_2 : \Gamma \mapsto \Xi$
- $\sigma \vdash \Gamma : L \rightsquigarrow K \implies [\gamma]\sigma \vdash \Delta : L \rightsquigarrow K$

PROOF. These are formalized as:

- Term.Wf.subst in Typing/Term/Subst.lean
- Region.Wf.vsubst in Typing/Region/VSubst.lean
- Term.Subst.Wf.comp in Typing/Term/Subst.lean

(d) `Region.Subst.Wf.vsubst` in `Typing/Region/LSubst.lean`

□

Note in particular that this allows us to take the *composition* $[y']\gamma : \Gamma' \mapsto \Delta$ of substitutions $\gamma' : \Gamma' \mapsto \Gamma$ and $\gamma : \Gamma \mapsto \Delta$; the composition associates as expected: $[[\gamma_1]\gamma_2]\gamma_3 = [\gamma_1]([\gamma_2]\gamma_3)$, and has identity $[\text{id}]\gamma = \gamma$, yielding a category of substitutions with variable contexts Γ as objects.

Given a substitution $\gamma : \Gamma \mapsto \Delta$ and context Ξ disjoint from Γ and Δ , we may define a “left extension” operation $\cdot_{\Xi}^{\uparrow}$ yielding $\gamma_{\Xi}^{\uparrow} : \Xi, \Gamma \mapsto \Xi, \Delta$ which appends the identity substitution for each variable in Ξ in the obvious manner:

$$\gamma^{\uparrow} = \gamma \quad \gamma_{\Xi, x:A}^{\uparrow} = x \mapsto x, \gamma_{\Xi}^{\uparrow} \quad (1)$$

We may similarly define a “right extension” operation $\cdot_{\Xi}^{\downarrow}$ yielding $\gamma_{\Xi}^{\downarrow} : \Gamma, \Xi \mapsto \Delta, \Xi$ as follows:

$$\gamma^{\downarrow} = \gamma \quad \gamma_{\Xi, x:A}^{\downarrow} = \gamma_{\Xi}^{\downarrow}, x \mapsto x \quad (2)$$

In particular, we note that the identity substitution on Γ can be written as $\cdot_{\Gamma}^{\uparrow}$; in general, we have $[\gamma]a = [\gamma_{\Xi}^{\uparrow}]a = [\gamma_{\Xi}^{\downarrow}]a$. We will usually infer Ξ from context.

One other particularly important form of substitution is that of substituting an expression a for an individual variable x , which we will write $[a/x] := (x \mapsto a)^{\uparrow}$.

Finally, just as we can generalize weakening by substituting expressions for variables via substitution, we can generalize label weakening by substituting *labels* for (*parametrized*) *regions* via *label substitution*. In particular, a label-substitution $\sigma \vdash \Gamma : L \rightsquigarrow K$ maps every label $\ell(A) \in L$ to a region $\Gamma, x : A \vdash r \triangleright K$ parametrized by $x : A$. As shown in Figure 14, we may then define label-substitution recursively in the obvious manner, mapping $\text{br } \ell \ a$ to $[a/x]r$ as a base case. Composition of label-substitutions is pointwise. This allows us to state *label substitution* as follows:

LEMMA 3.3 (LABEL SUBSTITUTION). *Given $\Gamma \vdash \sigma : L \rightsquigarrow K$, we have that*

- (a) $\Gamma \vdash r \triangleright L \implies \Gamma \vdash [\sigma]r \triangleright K$
- (b) $\Gamma \vdash \kappa : L \rightsquigarrow J \implies \Gamma \vdash [\sigma]\kappa : K \rightsquigarrow J$

PROOF. These are formalized as:

- (a) `Region.Wf.lsubst` in `Typing/Region/LSubst.lean`
- (b) `Region.Subst.Wf.comp` in `Typing/Region/LSubst.lean`

□

We may similarly define left and right extensions $\Gamma \vdash \sigma_K^{\uparrow} : L, J \rightsquigarrow K, J$ and $\Gamma \vdash \sigma_K^{\downarrow} : L, J \rightsquigarrow K, J$ and for label substitutions $\Gamma \vdash \sigma : L \rightsquigarrow K$ in the obvious manner:

$$\sigma^{\uparrow} = \sigma \quad \sigma_{K, \ell(A)}^{\uparrow} = \sigma_K^{\uparrow}, \ell(x) \mapsto \text{br } \ell \ x \quad (3)$$

$$\sigma^{\downarrow} = \sigma \quad \sigma_{K, \ell(A)}^{\downarrow} = \ell(x) \mapsto \text{br } \ell \ x, \sigma_K^{\downarrow} \quad (4)$$

As for variable substitutions, we will often omit L when it is clear from the context. We also define the shorthand $[\ell/\kappa] = [(\kappa(x) \mapsto \text{br } \ell \ x)^{\uparrow}]$ for single-label substitutions.

4 Equational Theory

4.1 Expressions

We can now give an equational theory for λ_{SSA} expressions. In particular, we will inductively define an equivalence relation $\Gamma \vdash_{\epsilon} a \approx a' : A$ on terms a, a' for each context Γ , effect ϵ , and type A . For each of the rules we will present, we assume the rule is valid if and only if *both sides* of the rule are

$$\begin{aligned}
(\sigma, \ell(x) \mapsto r)(\ell, a) &= [a/x]r & (\sigma, \kappa(x) \mapsto r)(\ell, a) &= \sigma(\ell, a) & (\cdot)(\ell, a) &= \text{br } \ell \ a \\
[\sigma](\text{br } \ell \ a) &= \sigma(\ell, a) & [\sigma](\text{let } x = a; r) &= \text{let } x = a; [\sigma]r \\
[\sigma](\text{let } (x, y) = e; r) &= \text{let } (x, y) = e; [\sigma]r \\
[\sigma](\text{case } e \ \{ \iota_l x : r, \iota_r y : s \}) &= \text{case } e \ \{ \iota_l x : [\sigma]r, \iota_r y : [\sigma]s \} \\
[\sigma](r \text{ where } (\ell_i(x_i) : \{t_i\},)_i) &= ([\sigma]r \text{ where } (\ell_i(x_i) : \{[\sigma]t_i\},)_i) \\
[\sigma](\cdot) &= \cdot & [\sigma](\sigma', \ell(x) \mapsto r) &= ([\sigma]\sigma', \ell(x) \mapsto [\sigma]r)
\end{aligned}$$

Fig. 14. Capture-avoiding label substitution for λ_{SSA} regions and label substitutions; in particular, we assume bound variables and labels are α -converted so as not to appear in σ .

well-typed. We also assume that variables are α -converted as appropriate to avoid shadowing; our formalization uses de Bruijn indices, but we stick with names in this exposition for simplicity.

The rules for this relation can be roughly split into *rewriting rules*, which denote when two particular expressions have equivalent semantics, and *congruence rules*, which govern how rewrites can be composed to enable equational reasoning. In particular, our congruence rules, given in Figure 15, consist of:

- refl, symm, trans, which state that $\Gamma \vdash_{\epsilon} \cdot \approx \cdot : A$ is reflexive, transitive, and symmetric respectively for each choice of Γ, ϵ, A , and therefore an equivalence relation.
- let₁, let₂, pair, inl, inr, case, and abort, which state that $\Gamma \vdash_{\epsilon} \cdot \approx \cdot : A$ is a *congruence* with respect to the corresponding expression constructor, and, in particular, that the expression constructors are well-defined functions on the quotient of expressions up to $\approx$.

We also include the following *type-directed* rules as part of our congruence relation:

- initial, which equates *all* terms in a context containing the empty type 0, since we will deem any such context to be *unreachable* by control flow. In particular, any instruction or function call returning 0 is assumed to diverge.
- terminal, which equates all *pure* terms of unit type 1. Note that *impure* terms may be disequal, since while their result values are the same, their side effects may differ!

We may group the rest of our rules according to the relevant constructor, i.e. let (unary and binary) and case. In particular, for unary let, we have the following rules, summarized in Figure 16:

- let₁- β , which allows us to substitute the bound variable in x the let-statement $\text{let } x = a; b$ with its definition a , yielding $[a/x]b$. Note that we require $\Gamma \vdash_{\perp} a : A$; i.e., a must be *pure*.
- let₁- η , which is the standard η -rule for let. This is included as a separate rule since, while it follows trivially from β for pure a , we also want to consider *impure* expressions.
- Rules let₁-op, let₁-let₁, let₁-let₂, let₁-abort, and let₁-case which allow us to “pull” a let-statement out of any of the other expression constructors; operationally, this is saying that the bound expression we pull out is evaluated before the rest of the let-binding.

For example, let₁-case says that, if both $\text{let } z = \text{case } e \ \{ \iota_l x : a, \iota_r y : b \}; d$ and $\text{case } e \ \{ \iota_l x : \text{let } z = a; d, \iota_r y : \text{let } z = b; d \} y$, are well typed, then both must have the same behaviour:

- (1) Compute e
- (2) If $e = \iota_l e_l$, compute $[e_l/x]a$, else, if $e = \iota_r e_r$, compute $[e_r/y]b$; store this value as z
- (3) Compute d given our value for z

Note in particular that, since both sides are well-typed, d cannot depend on either x or y .

$$\begin{array}{c}
\frac{\Gamma \vdash_{\epsilon} a : A}{\Gamma \vdash_{\epsilon} a \approx a : A} \text{refl} \quad \frac{\Gamma \vdash_{\epsilon} a \approx b : A \quad \Gamma \vdash_{\epsilon} b \approx c : A}{\Gamma \vdash_{\epsilon} a \approx c : A} \text{trans} \quad \frac{\Gamma \vdash_{\epsilon} a \approx b : A}{\Gamma \vdash_{\epsilon} b \approx a : A} \text{symm} \\
\frac{\Gamma \vdash_{\epsilon} a \approx a' : A \quad \Gamma, x : A \vdash_{\epsilon} b \approx b' : B}{\Gamma \vdash_{\epsilon} \text{let } x = a; b \approx \text{let } x = a'; b' : B} \text{let}_1 \\
\frac{\Gamma \vdash_{\epsilon} a \approx a' : A \quad \Gamma \vdash_{\epsilon} b \approx b' : B}{\Gamma \vdash_{\epsilon} (a, b) \approx (a', b) : A \otimes B} \text{pair} \\
\frac{\Gamma \vdash_{\epsilon} e \approx e' : A \otimes B \quad \Gamma, x : A, y : B \vdash_{\epsilon} c \approx c' : C}{\Gamma \vdash_{\epsilon} \text{let } (x, y) = e; c \approx \text{let } (x, y) = e'; c' : C} \text{let}_2 \\
\frac{\Gamma \vdash_{\epsilon} a \approx a' : A}{\Gamma \vdash_{\epsilon} \iota_l a \approx \iota_l a' : A + B} \text{inl} \quad \frac{\Gamma \vdash_{\epsilon} b \approx b' : B}{\Gamma \vdash_{\epsilon} \iota_r b \approx \iota_r b' : A + B} \text{inr} \\
\frac{\Gamma \vdash_{\epsilon} e \approx e' : A + B \quad \Gamma, x : A \vdash_{\epsilon} a \approx a' : C \quad \Gamma, y : B \vdash_{\epsilon} b \approx b' : C}{\Gamma \vdash_{\epsilon} \text{case } e \{ \iota_l x : a, \iota_r y : b \} \approx \text{case } e' \{ \iota_l x : a', \iota_r y : b' \} : C} \text{case} \\
\frac{\Gamma \vdash_{\epsilon} a \approx a' : 0}{\Gamma \vdash_{\epsilon} \text{abort } a \approx \text{abort } a' : A} \text{abort} \\
\frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma \vdash_{\epsilon} a' : A \quad \Gamma \vdash_{\perp} e : 0}{\Gamma \vdash_{\epsilon} a \approx a' : A} \text{initial} \quad \frac{\Gamma \vdash_{\perp} a : 1 \quad \Gamma \vdash_{\perp} a' : 1}{\Gamma \vdash_{\epsilon} a \approx a' : 1} \text{terminal}
\end{array}$$

Fig. 15. Congruence rules for λ_{SSA} expressions

$$\begin{array}{c}
\frac{\Gamma \vdash_{\perp} a : A \quad \Gamma, x : A \vdash_{\epsilon} b : B}{\Gamma \vdash_{\epsilon} \text{let } x = a; b \approx [b/x]a : B} \text{let}_1\text{-}\beta \quad \frac{\Gamma \vdash_{\epsilon} a : A}{\Gamma \vdash_{\epsilon} \text{let } x = a; x \approx a : A} \text{let}_1\text{-}\eta \\
\frac{f \in \mathcal{I}_{\epsilon}(A, B) \quad \Gamma \vdash_{\epsilon} a : A \quad \Gamma, y : B \vdash_{\epsilon} c : C}{\Gamma \vdash_{\epsilon} \text{let } y = f a; c \approx \text{let } x = a; \text{let } y = f x; c : C} \text{let}_1\text{-op} \\
\frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma, x : A \vdash_{\epsilon} b : B \quad \Gamma, y : B \vdash_{\epsilon} c : C}{\Gamma \vdash_{\epsilon} \text{let } y = (\text{let } x = a; b); c \approx \text{let } x = a; \text{let } y = b; c : C} \text{let}_1\text{-let}_1 \\
\frac{\Gamma \vdash_{\epsilon} e : A \times B \quad \Gamma, x : A, y : C \vdash_{\epsilon} c : C \quad \Gamma, z : C \vdash_{\epsilon} d : D}{\Gamma \vdash_{\epsilon} \text{let } z = (\text{let } (x, y) = e; c); d \approx \text{let } (x, y) = e; \text{let } z = c; d : D} \text{let}_1\text{-let}_2 \\
\frac{\Gamma \vdash_{\epsilon} a : 0 \quad \Gamma, y : A \vdash_{\epsilon} b : B}{\Gamma \vdash_{\epsilon} \text{let } y = \text{abort } b; b \approx \text{let } x = a; \text{let } y = \text{abort } x; b : B} \text{let}_1\text{-abort} \\
\frac{\Gamma \vdash_{\epsilon} e : A + B \quad \Gamma, x : A \vdash_{\epsilon} a : C \quad \Gamma, y : B \vdash_{\epsilon} b : C \quad \Gamma, z : C \vdash_{\epsilon} d : D}{\Gamma \vdash_{\epsilon} \text{let } z = (\text{case } e \{ \iota_l x : a, \iota_r y : b \}); d \approx \text{case } e \{ \iota_l x : \text{let } z = a; d, \iota_r y : \text{let } z = b; d \} : D} \text{let}_1\text{-case}
\end{array}$$

Fig. 16. Rewriting rules for λ_{SSA} unary let expressions

Handling the other type constructors is a little simpler: by providing a “binding” rule, we generally only need to specify how to interact with let_1 , as well as an η and β rule; interactions with the other constructors can then be derived. For example, consider the rules for let_2 given in 17; we have:

- $\text{let}_2\text{-}\eta$, which is the standard η -rule for binary let-bindings

$$\begin{array}{c}
\frac{\Gamma \vdash_e a : A \quad \Gamma \vdash_e b : B \quad \Gamma, x : A, y : B \vdash_e c : C}{\Gamma \vdash_e \text{let } (x, y) = (a, b); c \approx \text{let } x = a; \text{let } y = b; c : C} \text{let}_2\text{-pair} \\
\frac{\Gamma \vdash_e e : A \otimes B}{\Gamma \vdash_e \text{let } (x, y) = e; (x, y) \approx e : A \otimes B} \text{let}_2\text{-}\eta \\
\frac{\Gamma \vdash_e e : A \otimes B \quad \Gamma, x : A, y : B \vdash_e c : C}{\Gamma \vdash_e \text{let } (x, y) = e; c \approx \text{let } z = e; \text{let } (x, y) = z; c : C} \text{let}_2\text{-bind} \\
\frac{\Gamma \vdash_e a : A \quad \Gamma, x : A \vdash_e c : C \quad \Gamma, y : B \vdash_e d : C}{\Gamma \vdash_e \text{case } \iota_l a \{ \iota_l x : c, \iota_r y : d \} \approx \text{let } x = a; c : C} \text{case-inl} \\
\frac{\Gamma \vdash_e b : B \quad \Gamma, x : A \vdash_e c : C \quad \Gamma, y : B \vdash_e d : C}{\Gamma \vdash_e \text{case } \iota_r b \{ \iota_l x : c, \iota_r y : d \} \approx \text{let } y = b; d : C} \text{case-inr} \\
\frac{\Gamma \vdash_e e : A + B}{\Gamma \vdash_e \text{case } e \{ \iota_l x : \iota_l x, \iota_r y : \iota_r y \} \approx e : A + B} \text{case-}\eta \\
\frac{\Gamma \vdash_e e : A + B \quad \Gamma, x : A \vdash_e c : C \quad \Gamma, y : B \vdash_e d : C}{\Gamma \vdash_e \text{case } e \{ \iota_l x : c, \iota_r y : d \} \approx \text{let } z = e; \text{case } z \{ \iota_l x : c, \iota_r y : d \} : C} \text{case-bind}
\end{array}$$

Fig. 17. Rewriting rules for λ_{SSA} binary let and case expressions

- $\text{let}_2\text{-pair}$, which acts like a slightly generalized β -rule, since we can derive β reduction as follows: given pure $\Gamma \vdash_{\perp} a : A$ and $\Gamma \vdash_{\perp} b : B$, we have

$$(\text{let } (x, y) = (a, b); c) \approx (\text{let } x = a; \text{let } y = b; c) \approx ([a/x](\text{let } y = b; c)) \approx ([a/x][b/y]c)$$

We state the rule in a more general form to allow for impure a and b , as well as to simplify certain proofs.

- $\text{let}_2\text{-bind}$, which allows us to “pull” out the bound value of a binary let-expression into its own unary let-expression; operationally, this just says that we execute the bound value before executing the binding itself.

This is enough to allow us to define our interactions with the other expression constructors: for example, to show that we can lift an operation f out of a binary let-binding, rather than adding a separate rule, we can instead derive (types omitted for simplicity) it from $\text{let}_2\text{-bind}$ and $\text{let}_1\text{-op}$ as follows:

$$\begin{aligned}
(\text{let } (x, y) = f a; b) &\approx (\text{let } z_f = f a; \text{let } (x, y) = z; b) \\
&\approx (\text{let } z_a = a; \text{let } z_f = f z_a; \text{let } (x, y) = z; b) \\
&\approx (\text{let } z_a = a; \text{let } (x, y) = f z_a; b)
\end{aligned}$$

Similarly, it is enough to give η , β , and binding rules for case expressions. In particular, we have that

- case-inl and case-inr serve as β -reduction rules, telling us that case-expressions given an injection as an argument have the expected operational behaviour. Note that we reduce to a let-expression rather than perform a substitution to allow for impure discriminants.
- $\text{case-}\eta$ is the standard η -rule for case-expressions.
- case-bind allows us to “pull” out the bound value of the discriminant into its own let-expression; again, operationally, this just says that we need to evaluate the discriminant before executing the case-expression.

It's interesting that this is enough, along with the let-case rule and friends, to derive the distributivity properties we would expect well-behaved case-expressions to have. For example, we have that

$$\begin{aligned} f(\text{case } e \{ \iota_l x : a, \iota_r y : b \}) &\approx (\text{let } z = \text{case } e \{ \iota_l x : a, \iota_r y : b \}; f z) \\ &\approx \text{case } e \{ \iota_l x : \text{let } z = a; f z, \iota_r y : \text{let } z = b; f z \} \\ &\approx \text{case } e \{ \iota_l x : f a, \iota_r y : f b \} \end{aligned}$$

and likewise for more complicated distributivity properties involving, e.g., let-bindings.

The case for the other constructors is even more convenient: no additional rules are required at all to handle operations, pairs, and injections. For example, we can derive the expected bind-rule for operations as follows:

$$f a \approx (\text{let } y = f a; y) \approx (\text{let } x = a; \text{let } y = f x; y) \approx (\text{let } x = a; f x)$$

This completes the equational theory for λ_{SSA} terms; in Section 5.5, we will show that this is enough to state a completeness theorem.

4.2 Regions

We now come to the equational theory for regions, which is similar to that for terms, except that we also need to support control-flow graphs. As before, we will split our rules into a set of *congruence rules* and, for each region constructor, *rewriting rules* based on that constructor's semantics. Our congruence rules, given in Figure 18, are quite standard; we have:

- As for terms, refl , trans , and symm state that $\Gamma \vdash \cdot \approx \cdot \triangleright L$ is an equivalence relation for all Γ, L .
- Similarly, let_1 , let_2 , case , and cfg state that $\Gamma \vdash \cdot \approx \cdot \triangleright L$ is a congruence over the respective region constructors; *as well as* the equivalence relation on terms $\Gamma \vdash_e \cdot \approx \cdot : A$.
- initial states that any context containing the empty type $\mathbf{0}$ equates all regions, by a similar reasoning to the rules for terms. Note that we do not require an analogue to the terminal rule (for example, for regions targeting $L = \ell(1)$), since it will follow from the version for terms; this is good, since the concept of a “pure” region has not yet been defined.

Our rewriting rules for unary let-statements, given in Figure 19, are analogous to those for unary let-expressions:

- $\text{let}_1\text{-}\beta$ allows us to perform β -reduction of *pure* expressions into regions; unlike for terms, we do not need an η -rule
- Exactly like for let-expressions, $\text{let}_1\text{-op}$, $\text{let}_1\text{-let}_1$, $\text{let}_1\text{-let}_2$, $\text{let}_1\text{-abort}$, and $\text{let}_1\text{-case}$ allow us to pull out nested subexpressions of the bound value of a let-statement into their own unary let-statement

Similarly to expressions, binary let-statements and case-statements need only the obvious β rule and binding rule, with all the interactions with other constructors derivable; these rules are given in Figure 20. Note in particular that η -rules are not necessary, as these are derivable from binding and the η -rules for expressions.

Dealing with where-blocks, on the other hand, is a little bit more complicated, as shown by the number of rules in Figure 21. One difficulty is that, unlike the other region constructors, we will need an η -rule as well as *two* β -rules. The latter are simple enough to state:

- For ℓ_k defined in a where-block, $\text{cfg-}\beta_1$ says that we can replace a branch to ℓ_k with argument a with a let-statement binding a to the corresponding body t_k 's argument x_k .
- For κ *not* defined in a where-block, $\text{cfg-}\beta_2$ says that a branch to κ within the where-block has the same semantics as if the where-block was not there; hence, it can be removed.

$$\begin{array}{c}
\frac{\Gamma \vdash r \triangleright L}{\Gamma \vdash r \approx r \triangleright L} \text{refl} \quad \frac{\Gamma \vdash r \approx s \triangleright L \quad \Gamma \vdash s \approx t \triangleright L}{\Gamma \vdash r \approx t \triangleright L} \text{trans} \quad \frac{\Gamma \vdash r \approx s \triangleright L}{\Gamma \vdash s \approx r \triangleright L} \text{symm} \\
\frac{\Gamma \vdash_e a \approx a' : A \quad \Gamma, x : A \vdash r \approx r' \triangleright L}{\Gamma \vdash \text{let } x = a; r \approx \text{let } x = a'; r' \triangleright L} \text{let}_1 \quad \frac{\Gamma \vdash_e e \approx e' : A \otimes B \quad \Gamma, x : A, y : B \vdash r \approx r' \triangleright L}{\Gamma \vdash \text{let } (x, y) = e; r \approx \text{let } (x, y) = e'; r' \triangleright L} \text{let}_2 \\
\frac{\Gamma \vdash_e e \approx e' : A + B \quad \Gamma, x : A \vdash r \approx r' \triangleright L \quad \Gamma, y : B \vdash s \approx s' \triangleright L}{\Gamma \vdash \text{case } e \{ \iota_l x : r, \iota_r y : s \} \approx \text{case } e' \{ \iota_l x : r', \iota_r y : s' \} \triangleright L} \text{case} \\
\frac{\Gamma \vdash r \approx r' \triangleright L, (\ell_i(A_i),)_{i \in I} \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \approx t'_i \triangleright L, (\ell_j(A_j),)_{j \in I}}{\Gamma \vdash r \text{ where } (\ell_i(x_i : A_i) : \{t_i\},)_{i \in I} \approx r' \text{ where } (\ell_i(x_i : A_i) : \{t'_i\},)_{i \in I} \triangleright L} \text{cfg} \\
\frac{\Gamma \vdash r \triangleright L \quad \Gamma \vdash s \triangleright L \quad \exists x, \Gamma x = \mathbf{0}}{\Gamma \vdash r \approx s \triangleright L} \text{initial}
\end{array}$$

Fig. 18. Congruence rules for λ_{SSA} regions

$$\begin{array}{c}
\frac{\Gamma \vdash_\perp a : A \quad \Gamma, x : A \vdash r \triangleright L}{\Gamma \vdash \text{let } x = a; r \approx [a/x]r \triangleright L} \text{let}_1\text{-}\beta \\
\frac{f \in \mathcal{I}_e(A, B) \quad \Gamma \vdash_e a : A \quad \Gamma, y : B \vdash r \triangleright L}{\Gamma \vdash \text{let } y = f a; r \approx \text{let } x = a; \text{let } y = f x; r \triangleright L} \text{let}_1\text{-op} \\
\frac{\Gamma \vdash_e a : A \quad \Gamma, x : A \vdash_e b : B \quad \Gamma, y : B \vdash r \triangleright L}{\Gamma \vdash \text{let } y = (\text{let } x = a; b); r \approx \text{let } x = a; \text{let } y = b; r \triangleright L} \text{let}_1\text{-let}_1 \\
\frac{\Gamma \vdash_e e : A \otimes B \quad \Gamma, x : A, y : B \vdash_e c : C \quad \Gamma, z : C \vdash r \triangleright L}{\Gamma \vdash \text{let } z = (\text{let } (x, y) = e; c); r \approx \text{let } (x, y) = e; \text{let } z = c; r \triangleright L} \text{let}_1\text{-let}_2 \\
\frac{\Gamma \vdash_e e : A + B \quad \Gamma, x : A \vdash_e a : C \quad \Gamma, y : B \vdash_e b : C \quad \Gamma, z : C \vdash r \triangleright L}{\Gamma \vdash \text{let } z = (\text{case } e \{ \iota_l x : a, \iota_r y : b \}); r \approx \text{case } e \{ \iota_l x : \text{let } z = a; \iota_r y : \text{let } z = b; r \} \triangleright L} \text{let}_1\text{-case} \\
\frac{\Gamma \vdash_e a : \mathbf{0} \quad \Gamma, y : A \vdash r \triangleright L}{\Gamma \vdash \text{let } y = \text{abort } a; r \approx \text{let } x = a; \text{let } y = \text{abort } x; r \triangleright L} \text{let}_1\text{-abort}
\end{array}$$

Fig. 19. Rewriting rules for λ_{SSA} unary let-statements

To state our η -rule, however, we will need to introduce some more machinery. Given a mapping from a set of labels ℓ_i to associated regions t_i , we may define the *control-flow graph substitution* $\text{cfigs} \{(\ell_i(x_i) : \{t_i\},)_i\}$ pointwise as follows:

$$\text{cfigs} \{(\ell_i(x_i) : \{t_i\},)_i\} \kappa a := (\text{br } \kappa a \text{ where } (\ell_i(x_i) : \{t_i\},)_i) \quad (5)$$

In general, we may derive, for any label-context L (assuming $\text{cfigs} \{ \cdot \}$ acts uniformly on the labels κ in L as described above), the following rule:

$$\frac{\forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I}}{\Gamma \vdash \text{cfigs} \{(\ell_i(x_i) : \{t_i\},)_{i \in I} : L, (\ell_j(A_j),)_{j \in I} \rightsquigarrow L} \text{cfigs} \quad (6)$$

$$\begin{array}{c}
\frac{\Gamma \vdash_e a : A \quad \Gamma \vdash_e b : B \quad \Gamma, x : A, y : B \vdash r \triangleright L}{\Gamma \vdash \text{let } (x, y) = (a, b); r \approx \text{let } x = a; \text{let } y = b; r \triangleright L} \text{let}_2\text{-pair} \\
\frac{\Gamma \vdash_e e : A \otimes B \quad \Gamma, x : A, y : B \vdash r \triangleright L}{\Gamma \vdash \text{let } (x, y) = e; r \approx \text{let } z = e; \text{let } (x, y) = z; r \triangleright L} \text{let}_2\text{-bind} \\
\frac{\Gamma \vdash_e a : A \quad \Gamma, x : A \vdash r \triangleright L \quad \Gamma, y : B \vdash s \triangleright L}{\Gamma \vdash \text{case } \iota_l a \{ \iota_l x : r, \iota_r y : s \} \approx \text{let } x = a; r \triangleright L} \text{case-inl} \\
\frac{\Gamma \vdash_e b : B \quad \Gamma, x : A \vdash r \triangleright L \quad \Gamma, y : B \vdash s \triangleright L}{\Gamma \vdash \text{case } \iota_r b \{ \iota_l x : r, \iota_r y : s \} \approx \text{let } y = b; s \triangleright L} \text{case-inr} \\
\frac{\Gamma \vdash_e e : A + B \quad \Gamma, x : A \vdash r \triangleright L \quad \Gamma, y : B \vdash s \triangleright L}{\Gamma \vdash \text{case } e \{ \iota_l x : r, \iota_r y : s \} \approx \text{let } z = e; \text{case } z \{ \iota_l x : r, \iota_r y : s \} \triangleright L} \text{case-bind}
\end{array}$$

Fig. 20. Rewriting rules for λ_{SSA} binary let-statements and case-statements

Our η -rule, $\text{cfg-}\eta$, says that any where-block of the form r where $(\ell_i(x_i) : \{t_i\},)_i$ has the same semantics as the label-substitution $[\text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\}]r$, which in effect propagates the where-block to the branches of r , if any. While we called this rule $\text{cfg-}\eta$, it also functions similarly to a binding rule in that it allows us to derive many of the expected commutativity properties of where; for example, we have that

$$\begin{aligned}
\text{let } y = a; r \text{ where } (\ell_i(x_i) : \{\text{br } \ell_j a_j\},)_i &\approx [\text{cfgs } \{(\ell_i(x_i) : \{\text{br } \ell_j a_j\},)_i\}](\text{let } y = a; r) \\
&\approx \text{let } y = a; [\text{cfgs } \{(\ell_i(x_i) : \{\text{br } \ell_j a_j\},)_i\}]r \\
&\approx \text{let } y = a; r \text{ where } (\ell_i(x_i) : \{\text{br } \ell_j a_j\},)_i
\end{aligned}$$

One particularly important application of the η -rule for control-flow graphs is in validating the rewrite

$$\frac{\Gamma \vdash_e a : A + B \quad \Gamma, x : A \vdash s \triangleright L \quad \Gamma, y : B \vdash t \triangleright L}{\Gamma \vdash \text{case } a \{ \iota_l x : s, \iota_r y : t \} \approx (\text{case } a \{ \iota_l x : \text{br } \ell x, \iota_r y : \text{br } \ell' y \})} \text{case2cfg}$$

where $\ell(x) : \{s\}, \ell'(y) : \{t\} \triangleright L$ (7)

In addition, we also add as an axiom the ability to get rid of a single, trivially nested where-block; this is given as the rule codiag .

To be able to soundly perform equational rewriting, we will need the *uniformity* property, which is described by the rule uni . In essence, this lets us commute pure expressions with loop bodies, enabling rewrites (in imperative style) like

$$\text{loop } \{x = x + 1; \text{if } p \text{ } 3x \{ \text{ret } 3x \} \} \approx y = 3x; \text{loop } \{y = y + 3; \text{if } p \text{ } y \{ \text{ret } y \} \} \quad (8)$$

Note that substitution alone would not allow us to derive Equation 8 above, since x and y change each iteration, and hence, in SSA, would need to become parameters as follows:

$$\begin{aligned}
&\text{br } \ell x \text{ where } \ell(y) : \{ \text{let } x' = y + 1; \text{if } p \text{ } 3x' \{ \text{ret } 3x' \} \} \text{ else } \{ \text{br } \ell x' \} \\
&\approx \text{let } y = 3x; \text{br } \kappa y \text{ where } \kappa(y) : \{ \text{let } y' = y + 3; \text{if } p \text{ } y' \{ \text{ret } y' \} \} \text{ else } \{ \text{br } \kappa y' \} \quad (9)
\end{aligned}$$

The actual rule is quite complicated, so let's break it down point by point. Assume we are given:

- A region $\Gamma, y : B \vdash s \triangleright L, \kappa(B)$ taking “input” y of type B and, as “output,” jumping to a label κ with an argument of type B . We'll interpret branches to any other label (i.e. any label in L) as a (divergent) “side effect.”

- A region $\Gamma, x : A \vdash t \triangleright L, \ell(A)$ taking “input” x of type A and, as “output,” jumping to a label ℓ with an argument of type A .
- A *pure* expression $\Gamma, x : A \vdash_{\perp} e : B$ parameterised by a value x of type A

Suppose further that the following condition holds:

$$\Gamma, x : A \vdash [e/y]s \approx t \text{ where } \ell(x) : \{\text{br } \kappa e\} \triangleright L, \kappa(B)$$

That is, the following two programs are equivalent:

- (a) Given input x , evaluate e and, taking it’s output to be input y , evaluate s , (implicitly) yielding as output a new value of y . In imperative pseudocode,

$$y = e; y = s$$

- (b) Given input x , evaluate t and, taking it’s output to be the *new* value of x , evaluate e , (implicitly) yielding as output a new value y . In imperative pseudocode,

$$x = t; y = e$$

Then, for any well-typed entry block $\Gamma \vdash r \triangleright L, \ell(A)$ (which can produce an appropriate input $x : A$ at label ℓ), we have that

$$\Gamma \vdash (r \text{ where } \ell(x) : \{\text{br } \kappa e\}) \text{ where } \kappa(y) : \{s\} \approx r \text{ where } t \triangleright L$$

i.e., in imperative pseudocode,

$$x = r; y = e; \text{loop } \{y = s\} \approx x = r; \text{loop } \{x = t\} \quad (10)$$

since

$$y = e; y = s; y = s; \dots \approx x = t; y = e; y = s; \dots \approx x = t; x = t; y = e; \dots \approx \dots \quad (11)$$

where s and t may branch out of the loop. Note that, due to $\text{let}_1\text{-}\beta$, $\text{cfg-}\eta$, and $\text{cfg-}\beta_1$, this is equivalent to the rule

$$\frac{\Gamma, x : A \vdash [e/y]s \approx [\ell(x) \mapsto \text{br } \kappa e]t \triangleright L, \kappa(B)}{\Gamma \vdash (([\ell(x) \mapsto \text{br } \kappa e]r) \text{ where } \kappa(y) : \{s\}) \approx (r \text{ where } \ell(x) : \{t\}) \triangleright L} \text{uni'}$$

$$\text{where } \Gamma \vdash r \triangleright L, \ell(A) \quad \Gamma, x : A \vdash_{\perp} e : B \quad \Gamma, y : B \vdash s \triangleright L, \kappa(B) \quad \Gamma, x : A \vdash t \triangleright L, \ell(A) \quad (12)$$

Going back to our concrete example from Equation 9, if we first substitute the let-binding $y = 3x$ on the RHS, we get

$$\begin{aligned} &\text{br } \ell x \text{ where } \ell(y) : \{\text{let } x' = y + 1; \text{if } p \ 3x' \ \{\text{ret } 3x'\} \text{ else } \{\text{br } \ell x'\}\} \\ &\approx \text{br } \kappa 3x \text{ where } \kappa(y) : \{\text{let } y' = y + 3; \text{if } p \ y' \ \{\text{ret } y'\} \text{ else } \{\text{br } \kappa y'\}\} \end{aligned} \quad (13)$$

Now, instantiate uni' (Equation 12) by taking:

- $s = \text{let } y' = y + 3; \text{if } p \ y' \ \{\text{ret } y'\} \text{ else } \{\text{br } \kappa y'\}$ to be the loop body on the RHS
- $e = 3x$
- $r = \text{br } \ell x$
- $t = \text{let } x' = y + 1; \text{if } p \ 3x' \ \{\text{ret } 3x'\} \text{ else } \{\text{br } \ell x'\}$ to be the loop body on the LHS

$$\begin{array}{c}
\frac{\Gamma \vdash_{\perp} a : A_k \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I}}{\Gamma \vdash \text{br } \ell_k a \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I} \approx (\text{let } x_k = a; t_k) \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I} \triangleright L} \text{cfg-}\beta_1 \\
\frac{\Gamma \vdash_{\perp} b : B \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I} \quad L \kappa = B \quad \kappa \notin \{\ell_i \mid i \in I\}}{\Gamma \vdash \text{br } \kappa b \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I} \approx \text{br } \kappa b \triangleright L} \text{cfg-}\beta_2 \\
\frac{\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_{i \in I} \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I}}{\Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I} \approx [\text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_{i \in I}\}] r \triangleright L} \text{cfg-}\eta \\
\frac{\Gamma \vdash r \triangleright L, \ell(A) \quad \Gamma, y : A \vdash s \triangleright L, \ell(A), \kappa(A)}{\Gamma \vdash r \text{ where } \ell(x) : \{\text{br } \kappa x \text{ where } \kappa(y) : \{s\}\} \approx r \text{ where } \ell(y) : \{[\ell/\kappa]s\} \triangleright L} \text{codiag} \\
\frac{\Gamma \vdash r \triangleright L, \ell(A) \quad \Gamma, x : A \vdash \text{let } y = e; s \approx t \text{ where } \ell(x) : \{\text{br } \kappa e\} \triangleright L, \kappa(B)}{\Gamma \vdash (r \text{ where } \ell(x) : \{\text{br } \kappa e\}) \text{ where } \kappa(y) : \{s\} \approx r \text{ where } t \triangleright L} \text{uni} \\
\text{where } \Gamma, x : A \vdash_{\perp} e : B, \quad \Gamma, y : B \vdash s \triangleright L, \kappa(B), \quad \text{and } \Gamma, x : A \vdash t \triangleright L, \ell(A) \\
\frac{\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_{i \in I} \quad \Gamma \vdash \sigma : (\ell_i(A_i),)_{i \in I} \rightsquigarrow (\kappa_j(B_j),)_{j \in J} \quad \forall j \in J. \Gamma, x_j : B_j \vdash t_j \triangleright L, (\ell_i(A_i),)_{i \in I}}{\Gamma \vdash ([\sigma^1]r) \text{ where } (\kappa_j(x_j) : \{[\sigma^1]t_j\},)_{j \in J} \approx r \text{ where } (\ell_i(x_i) : \{[(\kappa_j(x_j) \mapsto t_j),]_{j \in J}^1(\sigma \ell_i x_i)\},)_{i \in I}} \text{dinat}
\end{array}$$

Fig. 21. Rewriting rules for λ_{SSA} where-blocks

It's easy to see that $(([\ell(x) \mapsto \text{br } \kappa e]r) \text{ where } \kappa(y) : \{s\})$ and $(r \text{ where } t)$ are syntactically equal to the *RHS* and *LHS* of our desired result (Equation 13). So, it suffices to verify that

$$\begin{aligned}
&\Gamma, x : A \vdash [e/y]s \approx \text{let } y' = 3x + 3; \text{ if } p \ y' \ \{\text{ret } y'\} \text{ else } \{\text{br } \kappa \ y'\} \\
&\approx \text{let } y' = 3(x + 1); \text{ if } p \ y' \ \{\text{ret } y'\} \text{ else } \{\text{br } \kappa \ y'\} \\
&\approx \text{let } x' = x + 1; \text{ let } y' = 3x'; \text{ if } p \ y' \ \{\text{ret } y'\} \text{ else } \{\text{br } \kappa \ y'\} \\
&\approx \text{let } x' = x + 1; \text{ if } p \ 3x' \ \{\text{ret } 3x'\} \text{ else } \{\text{br } \kappa \ 3x'\} \\
&\approx [\ell(x) \mapsto \text{br } \kappa e]t
\end{aligned}$$

as desired. The reason why we require e to be *pure* in the uniformity rule is that impure expressions do not necessarily commute with infinite loops, even if they commute with any finite number of iterations of the loop. For example, if hi is some effectful operation (say, printing “hello”), it is quite obvious that,

$$\text{hi}; x = x + 1; \text{ if } x = y \ \{\text{ret } y\} \approx x = x + 1; \text{ if } x = y \ \{\text{hi}; \text{ret } y\}; \text{hi} \quad (14)$$

whereas

$$\text{hi}; \text{loop}\{x = x + 1; \text{ if } x = y \ \{\text{ret } y\}\} \not\approx \text{loop}\{x = x + 1; \text{ if } x = y \ \{\text{hi}; \text{ret } y\}\}; \text{hi} \quad (15)$$

since, in particular, we may have $y \leq x$, in which case the loop will never exit and hence hi will never be executed.

The derivable rule *uni*' (Equation 12) illuminates a very important potential use for uniformity; namely, formalizing rewrites like those in Figure 22. In particular, consider a program of the form

$$\Gamma \vdash ([\ell(x) \mapsto \text{br } \kappa e]r) \text{ where } \kappa(y) : \{[\ell(x) \mapsto \text{br } \kappa e]s\} \triangleright L$$

1079	let $n = 10$;	
1080	br loop(1, 1)	
1081	where loop(i_0, a_0) : {	
1082	if $i_0 < n$ {	let $n = 10$;
1083	br loop($i_0 + 1, a_0 * (i_0 + 1)$)	br loop(
1084	} else {	let $(x, y) = (0, 1)$;
1085	ret a_0	($x + 1, y * (x + 1)$)
1086	}	)
1087	}	where loop(i_0, a_0) : {
1088	}	if $i_0 < n$ {
1089	}	br loop(
1090	}	let $(x, y) = (i_0, a_0)$;
1091	(a) Program from Figure 8 after substituting lets.	($x + 1, y * (x + 1)$)
1092		)
1093	let $n = 10$;	} else {
1094	br loop (0, 1)	ret a_0
1095	where loop(x, y) : {	}
1096	let $(i_0, a_0) = (x + 1, y * (x + 1))$;	}
1097	if $i_0 < n$ {	
1098	br loop(i_0, a_0)	
1099	} else {	
1100	ret a_0	
1101	}	
1102	}	
1103	}	
1104	}	
1105	}	
1106	(b) Equivalent to Figure 22c by <i>dinaturality</i>	(c) Equivalent to Figure 22a by congruence
1107		

Fig. 22. Decomposing multi-block rewrites (from 22c to 22a, and therefore to the more optimal program 22b) into simple algebraic steps. By verifying each step, we can verify complex optimizations through decomposition.

where

- $\Gamma \vdash r \triangleright L, \ell(A)$
- $\Gamma, y : B \vdash s \triangleright L, \ell(A)$
- $\Gamma, x : A \vdash_{\perp} e : B$ is pure

Then we have that

$$\begin{aligned}
 [e/y][\ell(x) \mapsto \text{br } \kappa(e)]s & \approx [\ell(x) \mapsto \text{br } \kappa[e/y](e)][e/y]s \\
 & \approx [\ell(x) \mapsto \text{br } \kappa(e)][e/y]s
 \end{aligned}$$

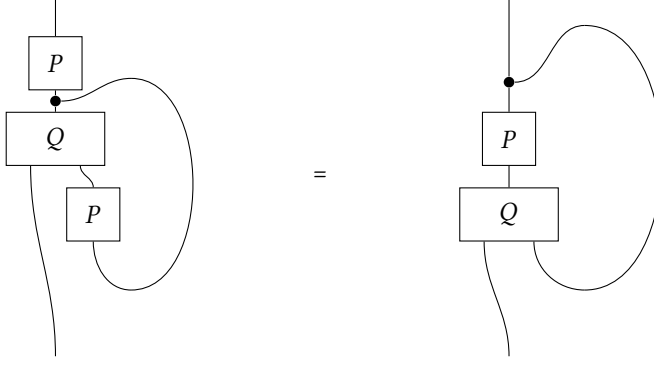


Fig. 23. Dinaturality on a structured loop

and therefore that

$$\begin{aligned} \Gamma \vdash ([\ell(x) \mapsto \text{br } \kappa \ e]r) \text{ where } \kappa(z) : \{[\ell(x) \mapsto \text{br } \kappa \ e]s\} \\ \approx r \text{ where } \ell(x) : \{[e/y]s\} \\ \approx r \text{ where } \ell(x) : \{\text{let } y = e; s\} \end{aligned}$$

In particular, for example, we can then easily derive the rewrite from Figure 22b to Figure 22c by noting the *equalities* (an equivalence would be enough, of course)

$$\begin{aligned} & \text{if } i_0 < n \{ \\ & \quad \text{br loop}(\text{let } (x, y) = (i_0, a_0); (x + 1, y * x + 1)) \\ & \} \text{ else } \{\text{ret}(a_0)\} \\ & = \\ & [\text{loop}(i_0, a_0) \mapsto \text{let } (x, y) = (i_0, a_0); (x + 1, y * x + 1)](\text{if } i_0 < n \{ \\ & \quad \text{br loop}(i_0, a_0) \\ & \} \text{ else } \{\text{ret}(a_0)\}) \end{aligned}$$

and

$$\begin{aligned} & \text{let } n = 10; \text{br loop}(\text{let } (x, y) = (0, 1); (x + 1, y * (x + 1))) \\ & = [\text{loop}(i_0, a_0) \mapsto \text{let } (x, y) = (i_0, a_0); (x + 1, y * x + 1)](\text{let } n = 10; \text{br loop}(0, 1)) \end{aligned}$$

Rewrites like this are an instance of the principle we call *dinaturality*, which, for structured control-flow, can be best expressed as an equivalence between the control-flow graphs in Figure 23. Unlike in the case of uniformity, however, this is true even when the program fragment P is *impure*, since, unlike in the case of general uniformity, we do not commute P over an infinite number of iterations. Our final rewriting rule, *dinat*, generalises the above rewrite from sequential composition on a structured control-flow graph to label substitution on an arbitrary control-flow graph.

We require a separate rule for impure dinaturality as it allows us to relate unary and n -ary where-blocks and, in particular, use this relationship to interconvert between data-flow and control-flow.

This means we now have enough equations to derive the flattening of nested where-blocks:

$$\frac{\begin{array}{c} \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I} \\ \Gamma \vdash r \triangleright L, (\ell_i(A_i),)_{i \in I}, (\kappa_j(B_j),)_{j \in I} \quad \forall i \in I. \Gamma, y_i : B_i \vdash s_i \triangleright L, (\ell_j(A_j),)_{j \in I}, (\kappa_k(B_k),)_{k \in K} \end{array}}{\Gamma \vdash (r \text{ where } (\kappa_k(y_k) : \{s_k\}),)_{k \in K} \text{ where } (\ell_i(x_i) : \{t_i\}),)_{i \in I} \approx r \text{ where } (\kappa_k(y_k) : \{s_k\}),_{k \in K}, (\ell_i(x_i) : \{t_i\}),_{i \in I} \triangleright L} \text{cfg-fuse} \quad (16)$$

Rather than directly giving derivation trees for such auxilliary rules, it is more convenient to give a denotational proof. However, the completeness of our equational theory (proved in Section 5.5) means that the semantic equality implies the existence of the requisite derivation tree. A proof can be found in Lemma C.7 in the appendix. This is one of the benefits of having a completeness result: it lets us switch freely between equational and denotational modes of reasoning.

There are some other basic rules we may want to use which turn out to be derivable from our existing set. For example, while re-ordering labels in a where-block looks like a no-op in our named syntax, to rigorously justify the following rule actually requires dinaturality (with the permutation done via a label-substitution):

$$\frac{\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_{i \in I} \quad \forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I} \quad \sigma \text{ permutation}}{\Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\}),_{i \in I} \approx r \text{ where } (\ell_{\sigma_i}(x_{\sigma_i}) : \{t_{\sigma_i}\}),_{i \in I} \triangleright L} \text{perm-cfg} \quad (17)$$

Note the implicit use of the fact that if some region r typechecks in some label-context L , then it typechecks in any permutation of L , which is again proven by label-substitution.

4.3 Metatheory

We can now begin to investigate the metatheoretic properties of our equational theory. As a first sanity check, we can verify that weakening, label-weakening, and loosening of effects all respect our equivalence relation, as stated in the following lemma:

LEMMA 4.1 (WEAKENING (REWRITING)). *Given $\Gamma \leq \Delta$, $L \leq K$, and $\epsilon \leq \epsilon'$, we have that*

- (a) $\Delta \vdash_\epsilon a \approx a' : A \implies \Gamma \vdash_{\epsilon'} a \approx a' : A$
- (b) $\Delta \vdash r \approx r' \triangleright L \implies \Gamma \vdash r \approx r' \triangleright K$

PROOF. These are formalized as:

- (a) Term.InS.wk_congr and Term.InS.wk_eff_congr in Rewrite/Term/Setoid.lean
- (b) Region.InS.vwk_congr and Region.InS.lwk_congr in Rewrite/Region/Setoid.lean

□

It is straightforward to verify that these are indeed equivalence relations. In fact, it turns out that substitution and label-substitution both respect these equivalences, in the following precise sense:

LEMMA 4.2 (CONGRUENCE (SUBSTITUTION)). *Given $\gamma \approx \gamma' : \Gamma \mapsto \Delta$, we have that*

- (a) $\Delta \vdash_\epsilon a \approx a' : A \implies \Gamma \vdash_\epsilon [\gamma]a \approx [\gamma']a' : A$
- (b) $\Delta \vdash r \approx r' \triangleright L \implies \Gamma \vdash [\gamma]r \approx [\gamma']r' \triangleright L$
- (c) $\rho \approx \rho' : \Delta \mapsto \Xi \implies [\gamma]\rho \approx [\gamma']\rho' : \Gamma \mapsto \Xi$
- (d) $\sigma \approx \sigma' \vdash \Delta : L \rightsquigarrow K \implies [\gamma]\sigma \approx [\gamma']\sigma' \vdash \Gamma : L \rightsquigarrow K$

PROOF. These are formalized as:

- (a) Term.InS.subst_congr in Rewrite/Term/Setoid.lean
- (b) Region.InS.vsubst_congr in Rewrite/Region/Setoid.lean
- (c) Term.Subst.InS.comp_congr in Rewrite/Term/Setoid.lean
- (d) Region.Subst.InS.vsubst_congr in Rewrite/Region/LSubst.lean

$$\begin{array}{c}
\frac{}{\cdot \approx \cdot : \Gamma \mapsto \cdot} \text{sb-nil} \quad \frac{\Gamma \vdash_{\perp} a \approx a' : A \quad \gamma \approx \gamma' : \Gamma \mapsto \Delta}{\gamma, x \mapsto a \approx \gamma', x \mapsto a' : \Gamma \mapsto \Delta, x : A} \text{sb-cons} \\
\frac{\gamma \approx \gamma' : \Gamma \mapsto \Delta}{\gamma, x \mapsto a \approx \gamma' : \Gamma \mapsto \Delta} \text{sb-skip-l} \quad \frac{\gamma \approx \gamma' : \Gamma \mapsto \Delta}{\gamma \approx \gamma', x \mapsto a' : \Gamma \mapsto \Delta} \text{sb-skip-r} \\
\frac{}{\cdot \approx \cdot \vdash \cdot : K \rightsquigarrow} \text{ls-nil} \quad \frac{\Gamma, x : A \vdash r \approx r' \triangleright K \quad \sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K}{\sigma, \ell(x) \mapsto r \approx \sigma', \ell(x) \mapsto r' \vdash \Gamma : L, \ell(A) \rightsquigarrow K} \text{ls-cons} \\
\frac{\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K}{\sigma, \ell(x) \mapsto r \approx \sigma' \vdash \Gamma : L \rightsquigarrow K} \text{ls-skip-l} \quad \frac{\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K}{\sigma \approx \sigma', \ell(x) \mapsto r' \vdash \Gamma : L \rightsquigarrow K} \text{ls-skip-r} \\
\frac{\gamma \approx \gamma' : \Gamma, x : A \mapsto \Delta, x : A}{\gamma \approx \gamma' : \Gamma \mapsto \Delta} \text{sb-id} \quad \frac{\sigma \approx \sigma' \vdash \Gamma : L, \ell(A) \rightsquigarrow K, \ell(A)}{\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K} \text{ls-id}
\end{array}$$

Fig. 24. Rules for the equivalence relation on λ_{SSA} substitutions and label-substitutions

In particular, note that this lemma uses an equivalence relation on substitutions and label-substitutions: this is just the obvious pointwise extension of the equivalence relation on terms and regions respectively. We give the rules for this relation in Figure 24 in the interests of explicitness.

LEMMA 4.3 (CONGRUENCE (LABEL SUBSTITUTION)). *Given $\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K$, we have that*

- (a) $\Gamma \vdash r \approx r' \triangleright L \implies \Gamma \vdash [\sigma]r \approx [\sigma']r' \triangleright K$
- (b) $\kappa \approx \kappa' \vdash \Gamma : L \rightsquigarrow J \implies [\sigma]\kappa \approx [\sigma']\kappa' \vdash \Gamma : K \rightsquigarrow J$

PROOF. These are formalized as:

- (a) Region.InS.lsubst_congr in Rewrite/Region/LSubst.lean
- (b) Region.LSubst.InS.comp_congr in Rewrite/Region/LSubst.lean

This means, in particular, that, substitution and label-substitution are well-defined operators on equivalence classes of terms, which will come in handy later as we set out to prove completeness in Section 5.5.

4.4 Standard SSA

The relaxation of SSA to λ_{SSA} allows us to state our equational theory and handle substitution more conveniently. However, this approach may lead readers to question whether we have truly provided a type-theoretic presentation of SSA as it is used in practice. The argument given in the introduction for why this is the case can be re-stated as the following series of explicit claims:

- (1) Every λ_{SSA} region can be converted to an equivalent lexical SSA region
- (2) Every lexical SSA region can be erased to a well-formed SSA program by removing “where”
- (3) Every well-formed SSA program can be typed as a lexical SSA region purely by adding “where”. Moreover, a way to do this can be found in nearly linear time.
- (4) Two lexical SSA regions which erase to the same SSA program are equivalent

Claim 4 implies that the mappings given in Claim 2 and Claim 3 establish an equivalence (up to our equational theory) between lexical SSA and traditional SSA. This means that the choice of where-bracketing that converts an SSA program into a lexical SSA program is semantically

irrelevant. In other words, where-bracketing only makes dominance syntactically apparent, letting us give syntax-directed rules for typing SSA programs.

We establish this claim in two phases. First, we will show that λ_{SSA} can be transformed (in linear space) into an equivalent lexical SSA program. Next, we will show that ordinary and lexical SSA can be transformed into one another by adding and removing where-blocks.

Converting λ_{SSA} directly to lexical SSA can be unwieldy. Therefore, we break the transformation down into two lowering passes:

- (1) We first convert λ_{SSA} into a subset corresponding to A-normal form (ANF) extended with mutually recursive where-bindings
- (2) We then convert the resulting ANF regions into lexical SSA

4.4.1 From λ_{SSA} to A-Normal Form. Our first step is to extend the work of Chakravarty et al. [14], by providing an algorithm for converting between λ_{SSA} and ANF in an equivalence-preserving way. We define the ANF regions, whose grammar is given in Figure 25, to be λ_{SSA} regions with expressions restricted to operations o ; equivalently, we can view ANF regions as lexical SSA regions with the syntactic category of regions r and terminators τ fused. We may now introduce the following predicates on syntax:

- $\text{IsANF}(r)$ means that the region r is in ANF
- $\text{IsSSA}(r)$ means that the region r is a lexical SSA region. We note that $\text{IsSSA}(r) \implies \text{IsANF}(r)$.
- $\text{IsVal}(a)$ means that the expression a can be parsed as a value v
- $\text{IsOp}(a)$ means that the expression a can be parsed as an instruction o . We note that $\text{IsVal}(a) \implies \text{IsOp}(a)$.

We can define a syntactic function to convert an λ_{SSA} region to ANF inductively as follows:

$$\begin{aligned}
 \text{ANF}(\text{br } \ell \ a) &= \text{ANF}_{\text{let}}(x, a, \text{br } \ell \ x) \\
 \text{ANF}(\text{let } x = a; \ r) &= \text{ANF}_{\text{let}}(x, a, \text{ANF}(r)) \\
 \text{ANF}(\text{let } (x, y) = a; \ r) &= \text{ANF}_{\text{let}}(z, a, \text{let } (x, y) = z; \ \text{ANF}(r)) \\
 \text{ANF}(\text{case } a \ \{ \iota_l \ x : r, \iota_r \ y : s \}) &= \text{ANF}_{\text{let}}(z, a, \text{case } z \ \{ \iota_l \ x : \text{ANF}(r), \iota_r \ y : \text{ANF}(s) \}) \\
 \text{ANF}(r \ \text{where } (\ell_i(x_i) : \{t_i\},)_i) &= \text{ANF}(r) \ \text{where } (\ell_i(x_i) : \{\text{ANF}(t_i)\},)_i
 \end{aligned} \tag{18}$$

where we define $\text{ANF}_{\text{let}}(x, a, r)$ by induction on expressions a as follows

$$\begin{aligned}
 \text{ANF}_{\text{let}}(x, a, r) &= (\text{let } x = a; \ r) && \text{if } \text{IsOp}(a) \\
 \text{ANF}_{\text{let}}(x, f \ e, r) &= \text{ANF}_{\text{let}}(y, e, \text{let } x = f \ y; \ r) \\
 \text{ANF}_{\text{let}}(x, (\text{let } y = e; \ a), r) &= \text{ANF}_{\text{let}}(y, e, \text{ANF}_{\text{let}}(x, a, r)) \\
 \text{ANF}_{\text{let}}(x, (e_1, e_2), r) &= \text{ANF}_{\text{let}}(y_1, e_1, \text{ANF}_{\text{let}}(y_2, e_2, \text{ANF}_{\text{let}}(x, (y_1, y_2), r))) \\
 \text{ANF}_{\text{let}}(x, (\text{let } (y, z) = e; \ a), r) &= \text{ANF}_{\text{let}}(w, e, (\text{let } (y, z) = w; \ \text{ANF}_{\text{let}}(x, a, r))) \\
 \text{ANF}_{\text{let}}(x, \iota_l \ e, r) &= \text{ANF}_{\text{let}}(y, e, (\text{let } x = \iota_l \ y; \ r)) \\
 \text{ANF}_{\text{let}}(x, \iota_r \ e, r) &= \text{ANF}_{\text{let}}(y, e, (\text{let } x = \iota_r \ y; \ r)) \\
 \text{ANF}_{\text{let}}(x, \text{case } e \ \{ \iota_l \ y : a, \iota_r \ z : b \}, r) &= \text{ANF}_{\text{let}}(w, e, \text{case } w \\
 &\quad \{ \iota_l \ y : \text{ANF}_{\text{let}}(x, a, \text{br } \ell \ x), \iota_r \ z : \text{ANF}_{\text{let}}(x, b, \text{br } \ell \ x) \} \\
 &\quad \text{where } \ell(x) : \{ \text{ANF}(r) \}) \\
 \text{ANF}_{\text{let}}(x, \text{abort } e, r) &= \text{ANF}_{\text{let}}(y, e, (\text{let } x = \text{abort } y; \ r))
 \end{aligned} \tag{19}$$

$v ::= x \mid (v, v') \mid ()$
 $o ::= v \mid f \ v \mid \iota_l \ v \mid \iota_r \ v \mid \text{abort } v$
 $r, s, t ::= \text{let } x = o; t \mid \text{let } (x, y) = o; t \mid t \text{ where } L \mid \text{br } \ell \ o \mid \text{case } e \{ \iota_l \ o : s, \iota_r \ y : t \}$
 $L ::= \cdot \mid L, \ell(x) : \{t\}$

Fig. 25. Grammar for ANF regions

We note that, to get to ANF, we don't actually need the new label, and can instead replace each branch with $\text{ANF}_{\text{let}}(x, a, r)$; introducing the label is only necessary to guarantee that the transformation to ANF is *linear-space*. The specification of the functions we just defined is given by the following lemma:

LEMMA 4.4 (A-NORMALIZATION). *Given an arbitrary region r , we have that*

- $\text{IsANF}(\text{ANF}(r))$
- *If $\Gamma \vdash r \triangleright L$, then $\Gamma \vdash r \approx \text{ANF}(r) \triangleright L$*

Similarly, given an arbitrary variable x , expression a and region r If we are also given an arbitrary expression a , then

- $\text{IsANF}(r) \implies \text{IsANF}(\text{ANF}_{\text{let}}(x, a, r))$
- *If $\Gamma \vdash_e a : A$ and $\Gamma, x : A \vdash r \triangleright L$, then $\Gamma \vdash \text{let } x = a; r \approx \text{ANF}_{\text{let}}(x, a, r) \triangleright L$*

PROOF. See Appendix C.2 □

4.4.2 *From ANF to (Lexical) SSA.* We would now like to define a syntactic linear-space transformation from regions r to equivalent lexical SSA regions $\text{SSA}(r)$. We do this by giving a pair of mutually syntactic transformations $\text{SSA}(r)$ converting a region to SSA, and $\text{SSA}_a(r, L)$ from ANF regions r targeting labels in L to lexical SSA.

$$\text{SSA}(r) = \text{SSA}_a(\text{ANF}(r), \cdot)$$

$$\text{SSA}_a(t, (\ell_i(x_i) : \{t_i\},)_i) = t \text{ where } (\ell_i(x_i) : \{t_i\},)_i \quad \text{where } t \text{ is a terminator}$$

$$\text{SSA}_a((\text{let } x = a; r), L) = (\text{let } x = a; \text{SSA}_a(r, L))$$

$$\text{SSA}_a((\text{let } (x, y) = a; r), L) = (\text{let } (x, y) = a; \text{SSA}_a(r, L)) \quad (20)$$

$$\text{SSA}_a((\text{case } a \{ \iota_l \ x : s, \iota_r \ y : t \}), L) = (\text{case } a \{ \iota_l \ x : \text{br } \ell_l \ x, \iota_r \ y : \text{br } \ell_r \ y \})$$

$$\text{where } L, \ell_l(x) : \{\text{SSA}(s)\}, \ell_r(y) : \{\text{SSA}(t)\}$$

$$\text{SSA}_a((r \text{ where } (\ell_i(x_i) : \{t_i\},)_i), L) = \text{SSA}_a(r, (L, (\ell_i(x_i) : \{\text{SSA}(t_i)\},)_i))$$

The specification of these functions is as follows:

LEMMA 4.5 (SSA CONVERSION). *Given an arbitrary region r ,*

- $\text{IsSSA}(\text{SSA}(r))$
- *If $\Gamma \vdash r \triangleright L$, then $\Gamma \vdash r \approx \text{SSA}(r) \triangleright L$*

In particular,

- *If $\text{IsANF}(r)$ and $\forall i, \text{IsSSA}(t_i)$, then $\text{IsSSA}(\text{SSA}_a(r, (\ell_i(x_i) : \{t_i\},)_i))$*
- *If $\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_i$ and $\forall i, \Gamma \vdash t_i \triangleright L, (\ell_i(A_i),)_i$, then $\Gamma \vdash (r \text{ where } (\ell_i(x_i) : \{t_i\},)_i) \approx \text{SSA}_a(r, (\ell_i(x_i) : \{t_i\},)_i) \triangleright L$*

PROOF. See Appendix C.2 □

4.4.3 *From (Lexical) SSA to SSA.* We now come to the final part of our argument: that lexical SSA is equivalent to standard SSA, where we take the basic-blocks-with-arguments dialect described in Figure 4 as standard. We begin by recalling how, in Figure 7, we illustrated how a lexical SSA region can alternatively be interpreted as a tuple of:

- A basic block β (the *entry block*), consisting of,
 - A sequence of instructions $\text{let } x = o$;
 - A terminator τ
- A map from labels ℓ to subtrees r (the region's *children*), L

More formally, given a lexical SSA region r , we can define the following functions to compute r 's entry block and children as follows

$$\begin{aligned} \text{entry}(\text{let } x = a; r) &= (\text{let } x = a; \text{entry}(r)) \\ \text{entry}(\text{let } (x, y) = a; r) &= (\text{let } (x, y) = a; \text{entry}(r)) \end{aligned} \quad (21)$$

$$\begin{aligned} \text{entry}(\tau \text{ where } (\ell_i(x_i) : \{t_i\},)_i) &= \tau \\ \text{children}(\text{let } x = a; r) &= \text{children}(r) \\ \text{children}(\text{let } (x, y) = a; r) &= \text{children}(r) \\ \text{children}(\tau \text{ where } (\ell_i(x_i) : \{t_i\},)_i) &= [\ell_i(x_i) : \{t_i\},]_i \end{aligned} \quad (22)$$

We can similarly define a function to construct a lexical SSA program from a basic block β and a set of children as follows:

$$\begin{aligned} \text{bb}(\text{let } x = a; \beta, L) &= \text{let } x = a; \text{bb}(\beta, L) \\ \text{bb}(\text{let } (x, y) = a; \beta, L) &= \text{let } (x, y) = a; \text{bb}(\beta, L) \\ \text{bb}(\tau, L) &= \tau \text{ where } L \end{aligned} \quad (23)$$

It is easy to see that these functions are mutually inverse: for any lexical SSA region r , we have

$$r = \text{bb}(\text{entry}(r), \text{children}(r)) \quad (24)$$

Some other useful facts about $\text{bb}(\cdot, \cdot)$ include:

- It is a congruence: if $r \approx r'$ and each $t_i \approx t'_i$, $\text{bb}(r, (\ell_i(x_i) : \{t_i\},)_i) \approx \text{bb}(r', (\ell_i(x_i) : \{t'_i\},)_i)$
- It is invariant up to permutations σ : $\text{bb}(r, (\ell_i(x_i) : \{t_i\},)_i) \approx \text{bb}(r, (\ell_{\sigma_i}(x_{\sigma_i}) : \{t_{\sigma_i}\},)_i)$
- Similarly, *if both sides of the equation are well-typed*, i.e.,
 - All t_i do not use y or any of the variables defined in s
 - All branches to ℓ_i come from κ or ℓ_j (and not from either r or G)

$$\text{bb}(r, (G, \kappa(y) : \{s\}, (\ell_i(x_i) : \{t_i\},))) \approx \text{bb}(r, (G, \kappa(y) : \{s \text{ where } (\ell_i(x_i) : \{t_i\},)\},))) \quad (25)$$

and hence

$$\text{bb}(r, (G, \kappa(y) : \{s\}, (\ell_i(x_i) : \{t_i\},))) \approx \text{bb}(r, (G, \kappa(y) : \{\text{bb}(s, (\ell_i(x_i) : \{t_i\},))\},))) \quad (26)$$

In particular, we may apply this rule twice to obtain

$$\text{bb}(r, (G, \kappa(y) : \{\text{bb}(s, G')\}, (\ell_i(x_i) : \{t_i\},))) \approx \text{bb}(r, (G, \kappa(y) : \{\text{bb}(s, G', (\ell_i(x_i) : \{t_i\},))\},))) \quad (27)$$

We may hence define a function $\text{cfg}(\cdot)$ from lexical SSA programs r to control-flow graphs G as follows:

$$\text{cfg}(r) = \text{entry}(r), (\ell_i(x_i) : \{\text{cfg}(t_i)\},)_{(\ell_i(x_i) : \{t_i\}) \in \text{children}(r)} \quad (28)$$

where we recursively flatten

$$G, \ell(x) : \{\beta, G'\} := G, \ell(x) : \{\beta\}, G' \quad (29)$$

At the level of program text, all this function is doing is “removing where-blocks;” so we’ll call the result *reased*. An SSA program is *well-formed* if:

- All variable uses are well-typed
- All variable uses respect dominance-based scoping

It is easy to see that any erased program is well-formed, since our typing rules guarantee every expression is well-typed, while our lexical scoping for values ensures that variables are only visible in the children of the block β in which they are defined, which the lexical scoping of labels guarantees are dominated by β .

On the other hand, we may give an algorithm to convert any well-formed SSA program G into a well-typed lexical SSA program $r = \text{reg}(G)$ as follows:

- (1) Compute the dominance tree of G , rooted at its entry block β .
- (2) For each child $\ell_i(x_i) : \{\beta_i\}$ of β , let G_i denote the CFG composed of the descendants of β_i (with β_i as entry block), given in the order they appear in G . Recursively compute $r_i = \text{reg}(G_i)$.
- (3) Return the program $\text{bb}(\beta, (\ell_i(x_i) : \{r_i\},)_i)$

We will write $G \simeq G'$ to mean “ G is a permutation of G' ” (that is, G and G' have the same entry block, but the other labels may be reordered). It is easy to see that this algorithm yields a lexical SSA program which erases to a permutation of G , i.e., $\text{cfg}(\text{reg}(G)) \simeq G$, as desired. To complete our argument, it hence suffices to show that, given lexical SSA regions $\Gamma \vdash r \triangleright L$, $\Gamma \vdash r' \triangleright L$, such that $\text{cfg}(r) \simeq \text{cfg}(r')$, we have that $\Gamma \vdash r \approx r' \triangleright L$. We break this down into two lemmas:

LEMMA 4.6 (PERMUTATION INVARIANCE). *If $G \simeq G'$ and $\Gamma \vdash \text{reg}(G) \triangleright L$, then $\Gamma \vdash \text{reg}(G') \triangleright L$ and $\Gamma \vdash \text{reg}(G) \approx \text{reg}(G') \triangleright L$*

PROOF. See Appendix C.2 □

LEMMA 4.7 (CFG CONVERSION). *Given a lexical SSA region r (i.e., assuming $\text{lsSSA}(r)$) s.t. $\Gamma \vdash r \triangleright L$, we have that $\Gamma \vdash r \approx \text{reg}(\text{cfg}(r)) \triangleright L$.*

PROOF. See Appendix C.2 □

It follows that, given lexical SSA regions $\Gamma \vdash r \triangleright L$ and $\Gamma \vdash r' \triangleright L$, if $\text{cfg}(r) \simeq \text{cfg}(r')$, we have that $\Gamma \vdash r \approx r' \triangleright L$, as desired, since in particular

$$r \approx \text{reg}(\text{cfg}(r)) \approx \text{reg}(\text{cfg}(r')) \approx r' \quad (30)$$

5 Denotational Semantics

Now that we have given an equational theory for SSA, we would like to build a denotational semantics for SSA: a mapping from well-typed programs $\Gamma \vdash r \triangleright L$ to mathematical objects $\llbracket \Gamma \vdash r \triangleright L \rrbracket$. This naturally gives us another way to reason about whether two programs “ r, r' ” are “the same:” we can ask whether $\llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \vdash r' \triangleright L \rrbracket$. Our goal is to build a denotational semantics which is both *sound* and *complete*; that is, such that

$$\Gamma \vdash r \approx r' \triangleright L \iff \llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \vdash r' \triangleright L \rrbracket$$

More specifically, we’ll begin by defining a notion of a *model* $\mathcal{M}$ of SSA, and then define the denotational semantics w.r.t. a model $\llbracket \Gamma \vdash r \triangleright L \rrbracket_{\mathcal{M}}$. We’ll then prove that, for *any* model of SSA, if $\Gamma \vdash r \approx r' \triangleright L$, then

$$\llbracket \Gamma \vdash r \triangleright L \rrbracket_{\mathcal{M}} = \llbracket \Gamma \vdash r' \triangleright L \rrbracket_{\mathcal{M}}$$

In other words, our equational theory is *sound*, and so never allows us to prove an equation which does not in fact hold. We’ll then give an model $\mathcal{M}$ such that

$$\llbracket \Gamma \vdash r \triangleright L \rrbracket_{\mathcal{M}} = \llbracket \Gamma \vdash r' \triangleright L \rrbracket_{\mathcal{M}} \implies \Gamma \vdash r \approx r' \triangleright L$$

showing that our equational theory is also *complete*: if an equation holds in an *arbitrary* model of SSA, it must in fact be derivable via our theory.

We'd further like our denotational semantics to be *compositional*: the denotation of a program should be a fixed function of the denotations of its parts. To do this, we will give SSA a *categorical* semantics, interpreting (label) contexts as objects and programs as morphisms between them in some target category C . Different program structures, such as branching and sequential composition, will correspond more-or-less directly to structures in our target category.

Before we begin, we fix some notational conventions:

- We denote the *composition* of two morphisms $f : A \rightarrow B$ and $g : B \rightarrow C$ as $f;g : A \rightarrow C$
- We denote the coproduct of a morphism $f : B \rightarrow C$ with the identity id_A as $f + A : B + A \rightarrow C + A$ or $A + f : A + B \rightarrow A + C$.

5.1 Freyd-Elgot Categories

Moggi [51] showed that the Kleisli category of a strong monad over a CCC interprets effectful higher-order functional programs. There are two mismatches in using this as a semantics for SSA. On one hand, SSA has features not necessarily supported by Moggi models such as arbitrary, unstructured cyclic control-flow. On the other hand Moggi models support features SSA (as a first-order language) does not have, such as higher-order functions and first-class computation values.

Given that we want to model SSA with some category C , we hence have to think about what structure we need C to possess so that it has exactly our desired features. Obviously, we need a way to take the product of two objects, to be able to model contexts as well as pairs. The usual way to do this is via *monoidal categories*. However, monoidal categories typically have too many equations: computations operating on independent data must always commute (this is the “sliding rule”), which is obviously not true for effects such as printing, since

$$\text{print}(x); \text{print}(y) \neq \text{print}(y); \text{print}(x)$$

Instead, we will only require that our category be *premonoidal*, as introduced by Power and Robinson [55]: “monoidal, without sliding.” Indeed, the Kleisli category of a strong monad on a CCC is not always monoidal, as demonstrated by the writer monad on Set (which exposes print), but *is* always premonoidal. We define a premonoidal category as follows:

Definition 5.1 (Symmetric Premonoidal Category). We define a *binoidal category* to be a category C equipped with a binary operation $\otimes : |C| \times |C| \rightarrow |C|$ on the objects of C and, for each $A, B \in |C|$, functors $A \otimes -, - \otimes B : C \rightarrow C$. We say a morphism $f : A \rightarrow A'$ in a binoidal category is *central* if, for all $g : B \rightarrow B'$, it satisfies *sliding*:

$$f \otimes B; A' \otimes g = A \otimes g; f \otimes B' \quad B \otimes f; g \otimes A' = g \otimes A; B' \otimes f$$

in which case we may write these morphisms as $f \otimes g : A \otimes B \rightarrow A' \otimes B'$ and $g \otimes f : B \otimes A \rightarrow B' \otimes A'$ respectively. A *premonoidal category* is, then, a binoidal category equipped with:

- An *identity* object $I \in |C|$
- For each triple of objects $A, B, C \in |C|$, a central, natural isomorphism $\alpha_{A,B,C} : (A \otimes B) \otimes C \rightarrow A \otimes (B \otimes C)$, the *associator*
- For each object A , central, natural isomorphisms $\lambda_A : A \otimes I \rightarrow A$ and $\rho_A : I \otimes A \rightarrow A$, the *left* and *right unitors*

satisfying the *triangle* and *pentagon identity*

$$\alpha_{A,I,B} : A \otimes \lambda_B = \rho_A \otimes B \quad \alpha_{A \otimes B, C, D} : \alpha_{A, B, C \otimes D} = \alpha_{A, B, C} \otimes D; \alpha_{A, B \otimes C, D} : A \otimes \alpha_{B, C, D}$$

We say a premonoidal category is *symmetric* if it is also equipped with a central, natural involution $\sigma_{A,B} : A \otimes B \rightarrow B \otimes A$, the *symmetry*, satisfying the *hexagon identity*

$$\alpha_{A,B,C}; \sigma_{A,B \otimes C}; \alpha_{B,C,A} = \sigma_{A,B} \otimes C; \alpha_{B,A,C}; B \otimes \sigma_{A,C}$$

We say a premonoidal category is *monoidal* if every morphism is central.

One important theorem about premonoidal categories is *coherence*:

THEOREM 5.2. *The subcategory $\mathcal{A}$ generated by associators, unitors, and their tensor products is an equivalence relation, i.e., for all $A, B : |\mathcal{A}|$, if $f, g : A \rightarrow B$ can be constructed using only identity, composition, associators, unitors, and their tensor products, then:*

- (a) $f = g$
- (b) f, g are isomorphisms in $\mathcal{A}$

In particular, as a syntactic convenience, we will often simply write “ α ” for the (unique) morphism between objects A and B , when it exists, satisfying the requirements of Theorem 5.2. For example, given $f : A \rightarrow B \otimes ((C \otimes I) \otimes D)$ and $g : ((B \otimes (I \otimes C)) \otimes D) \rightarrow E$, we have

$$f; \alpha; g := f; B \otimes (\lambda_C \otimes D); \alpha_{B,C,D}^{-1}; (B \otimes \rho_C^{-1}) \otimes D; g$$

Just like for higher-order functional languages, we can interpret types A as objects $\llbracket A \rrbracket : |C|$. Similarly, we can interpret variable contexts Γ by taking products of objects, as follows:

$$\llbracket \Gamma \rrbracket : |C| \quad \llbracket \cdot \rrbracket = I \quad \llbracket \Gamma, x : A \rrbracket = \llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket$$

We would like to interpret an expression-in-context $\Gamma \vdash_e a : A$ as a morphism in C from $\llbracket \Gamma \rrbracket$ to $\llbracket A \rrbracket$. However, in standard SSA, it is possible for a variable to be unused, or used multiple times. Our premonoidal structure, however, does not give us any way to *project* out of a product type, making it impossible to interpret expressions-in-context like $x : A, y : B \vdash x : A$. In order to project out individual variables, we need Cartesian structure, but a premonoidal category can only interpret *linear* expressions, that is, those which use every variable exactly once. However, it is too much to require the whole premonoidal category to be Cartesian, because that would validate the sliding rule, which is what we set out to avoid.

In the case of the Kleisli category over a CCC, the Cartesian structure of the CCC becomes the premonoidal structure of the Kleisli category. This lets us use the Cartesian structure to project out the variables, while the product in the Kleisli category still does not satisfy sliding. By analogy with this case, we can suppose that there is a subcategory $C_\perp$ of the premonoidal category C , which has the property that the premonoidal structure in C behaves like a Cartesian product in $C_\perp$.

If we generalize this structure to an arbitrary premonoidal category, we get the notion of a *Freyd category*, as introduced in Levy et al. [47]:

Definition 5.3 (Freyd category). A Freyd category is a premonoidal category C equipped with a wide subcategory $C_\perp \subseteq C$ of *pure* morphisms such that

- $C_\perp$ contains all associators, unitors, and symmetries
- I is a terminal object in $C_\perp$. In particular, this implies the terminal morphisms $!_A : A \rightarrow I$ are pure.
- For each A, B , $A \otimes B$ is a cartesian product of A, B in $C_\perp$
- For pure morphisms f, g , $f \otimes g = \langle \pi_l; f, \pi_r; g \rangle$

Where C is clear from context, we will write $A \rightarrow_\perp B$ to denote a pure morphism $C_\perp(A, B)$.

Alternatively, it is equivalent to require

- $C_\perp$ contains all associators, unitors, and symmetries

- I is a terminal object in $C_\perp$
- For each A , there exists a pure morphism $\Delta_A : A \rightarrow A \otimes A$ forming a comonoid with the (pure) terminal morphism $!_A : A \rightarrow I$, i.e: $\Delta_A; !_A \otimes A; \rho = \Delta_A; A \otimes !_A; \lambda = \text{id}_A$
- For every pure morphism $f : A \rightarrow_\perp B$, $f; \Delta_B = \Delta_A; f \otimes f$ and $f; !_B = !_A$.

In both cases, we have that $\pi_l = A \otimes !_B; \lambda$, $\pi_r = !_A \otimes B; \rho$, $\langle f, g \rangle = \Delta_A; f \otimes g$, and $\Delta_A = \langle \text{id}_A, \text{id}_A \rangle$

For convenience, given $f : A \rightarrow B$ in a Freyd category, we will define the notation

$$\text{let}(f) := \Delta_A; A \otimes f : A \rightarrow A \otimes B \quad (31)$$

Note that this is pure if and only if f is. This has the following useful properties:

- $\text{let}(f); \pi_r = f$ and, for f pure, $\text{let}(f); \pi_l = \text{id}$
- $\text{let}(f; g) = \text{let}(f); A \otimes g$ and $\text{let}(\text{id}_A) = \Delta_A$
- $\text{let}(\text{let}(f)) = \text{let}(f); \Delta_A \otimes B; \alpha_{A,A,B}$, and, therefore, given $g : A \otimes B \rightarrow C$, we have $\text{let}(\text{let}(f); g) = \text{let}(f); \text{let}(g); \pi_l \otimes C$

We now have everything we need to model effectful first-order expressions. For reasoning about substitution, we will also demand that the denotation of “pure” expressions $\Gamma \vdash_\perp a : A$ lies in $C_\perp(\llbracket \Gamma \rrbracket, \llbracket A \rrbracket)$. In general, we will write:

- $C_\top$ to mean just C , allowing us to write C_ϵ for an *effect* $\epsilon \in \{\perp, \top\}$.
- Morphisms in C_ϵ as $A \rightarrow_\epsilon B$, where C is clear from context.

At this point, we still have no way to interpret control-flow, i.e. case-expressions. Furthermore, if we want to model regions as morphisms, we need some way of modelling label-contexts L . At first glance, it seems sufficient for branching control-flow to require the existence of coproducts, and indeed, assuming the existence of all coproducts and an initial object, we may model label-contexts as follows:

$$\boxed{\llbracket L \rrbracket : |C|} \quad \llbracket \cdot \rrbracket = 0 \quad \llbracket L, \ell(A) \rrbracket = \llbracket L \rrbracket + \llbracket A \rrbracket$$

Regions can now be interpreted as morphisms in C from $\llbracket \Gamma \rrbracket$ to $\llbracket L \rrbracket$, as desired. Just like for products, we will write “ α^+ ” for the (unique) morphism between objects A and B , when it exists, satisfying the requirements of Theorem 5.2 where coproducts are taken as the monoidal structure; we will sometimes also write α_B^+ for clarity.

It turns out that our coproducts must be *distributive* to allow us to use variables in scope before a branch. In particular, we define a *distributive* premonoidal category as follows:

Definition 5.4 (Distributive category). A premonoidal category C with all coproducts is *distributive* if, for all A, B, C , the obvious morphism

$$\delta_{A,B,C} = [(A + \iota_l), (A + \iota_r)] : (A \otimes B) + (A \otimes C) \rightarrow A \otimes (B + C)$$

has an inverse δ^{-1} . We will say a Freyd category C is distributive if it has all coproducts and the subcategory of pure morphisms $C_\perp$ is distributive (which implies, in particular, that C is distributive when taken as a premonoidal category).

We note in particular that every cartesian closed category with coproducts is a distributive Freyd category, as is the Kleisli category of a monad over a CCC with coproducts. For any finite coproduct $\Sigma_i B_i$, we will introduce the notation $\delta_\Sigma : \Sigma_i (A \otimes B_i) \rightarrow A \otimes \Sigma_i B_i$ and $\delta_\Sigma^{-1} : A \otimes \Sigma_i B_i \rightarrow \Sigma_i (A \otimes B_i)$ to denote the obvious morphisms.

This gives us enough machinery to express any *acyclic* control-flow graph, however, we still have no way to model loops. What we would like is to equip our category with an *iteration operator* taking a morphism $f : A \rightarrow B + A$, representing a “loop” which, given input A , either produces a result B or continues to another iteration with a new A , to its *fixpoint* $f^\dagger : A \rightarrow B$. Naturally, we

require that this operator satisfies various properties, and in particular is *strong*, that is, compatible with the distributor. Formally, we define a (strong) *Conway iteration operator* as follows:

Definition 5.5 (Conway iteration). A category C with all coproducts is said to have a *iteration operator* if we can define an operator $(-)^{\dagger}$ taking every morphism $f : A \rightarrow B + A$ to a morphism $f^{\dagger} : A \rightarrow B$, the *fixpoint* of f , with the following property: given $f : A \rightarrow B + A$, we have $f^{\dagger} = f; [\text{id}, f^{\dagger}]$. We say this operator is a *Conway iteration operator* if it additionally satisfies the following properties:

- *Naturality*: given $f : A \rightarrow B + A$ and $g : B \rightarrow C$, we have $(f; g + \text{id})^{\dagger} = f^{\dagger}; g : A \rightarrow C$
- *Dinaturality*: given morphisms $g : A \rightarrow B + C$ and $h : C \rightarrow B + A$, we have that $(g; [\iota_l, h])^{\dagger} = g; [\text{id}_B, (h; [\iota_l, g])^{\dagger}]$
- *Codiagonal*: given $f : A \rightarrow (B + A) + A$, we have $(f^{\dagger})^{\dagger} = (f; [\text{id}, \iota_r])^{\dagger} : A \rightarrow B$

If C is distributive, we say this operator is *strong* if

$$\forall f : A \rightarrow B + A, (C \otimes f; \delta^{-1})^{\dagger} = C \otimes f^{\dagger}$$

Given a wide subcategory $\mathcal{K} \subseteq C$, we say C is $\mathcal{K}$ -uniform if, for all $h : A \rightarrow_{\mathcal{K}} B$, $f : B \rightarrow C + B$, and $g : A \rightarrow C + A$, we have that

$$h; f = g; C + h \implies h; f^{\dagger} = g^{\dagger}$$

We will call Freyd categories equipped with an appropriate Elgot structure *strong Elgot categories*. In particular, we define

Definition 5.6 ((Strong) Elgot structure). A Freyd category C with all coproducts is said to have an *Elgot structure* if it has a Conway iteration operator which is $C_{\perp}$ -uniform. If C is distributive, we say C is *strong Elgot* if its iteration operator is strong. In particular, we say a monad is *Elgot* if its Kliesli category has an Elgot structure. Similarly, we say a *strong monad* is *strong Elgot* if its Kliesli category has a strong Elgot structure.

It turns out that, to check something is a strong Elgot category, we do not need to explicitly verify dinaturality. In particular, we may verify the following:

PROPOSITION 5.7. *If $(-)^{\dagger}$ is an iteration operator which satisfies naturality and codiagonal and is $\mathcal{K}$ -uniform for $\mathcal{K}$ co-Cartesian, then it also satisfies dinaturality.*

PROOF. See Lemma 31 of Goncharov and Schröder [28] □

Since in a distributive Freyd category $C_{\perp}$ must be distributive (and hence co-cartesian), dinaturality follows from the other axioms of an Elgot category. Given a distributive Freyd category equipped with a (strong) Elgot structure, we will often want to consider the fixpoint of a morphism $f : R \otimes A \rightarrow B + A$, where our “context” R does not change between iterations. To do this, we first need to build up a morphism

$$\text{rcase}(f) := \text{let}(f); \pi_l \otimes (B + A); \delta^{-1} : R \otimes A \rightarrow R \otimes B + R \otimes A \quad (32)$$

which computes f and then distributes a copy of the read-only state R to each branch of the result. We may then define the fixpoint

$$\text{rfix}(f) := (\text{rcase}(f))^{\dagger}; \pi_r : R \otimes A \rightarrow B \quad (33)$$

We consider some more properties of $\text{rcase}(f)$ and $\text{rfix}(f)$ in Appendix B.

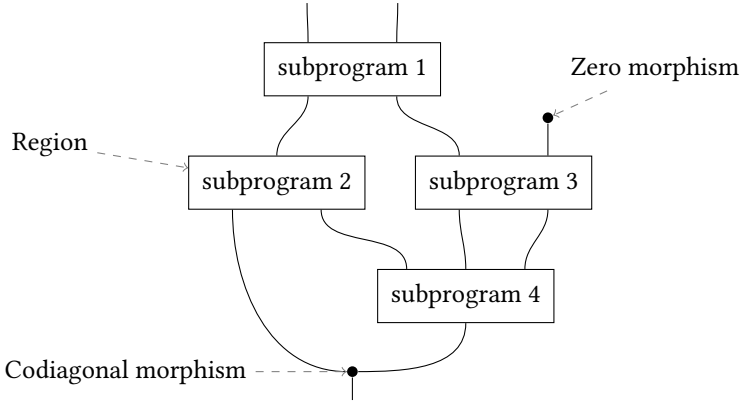


Fig. 26. A string diagram using the coproduct as symmetric monoidal structure, interpreted as a CFG

5.2 String Diagrams

String diagrams provide a graphical calculus for reasoning about (symmetric) monoidal categories, which allows us to succinctly express complex morphisms and rewrites. Since both cartesian and co-cartesian categories are monoidal (with the product and coproduct as tensor, respectively), we can use string diagrams to reason about both. In the co-cartesian case, string diagrams behave much like control-flow diagrams, with boxes representing sub-programs, input wires entry points, and output wires exit points. In particular, a *region* is just a box with a single input wire. Continuing this analogy, we draw the codiagonal morphism $[id_A, id_A]$ as joining two wires, and the zero morphism as a wire coming from nowhere, as in Figure 26.

The power of string diagrams comes from the fact that many syntactically distinct ways to write equal values are obviously graphically equivalent by *isotopy*: essentially, moving boxes and wires around. String diagrams also give us an elegant way to represent and reason about Elgot structures. It turns out that Elgot structures induce a *trace* on the coproduct [30]: given $f : A + C \rightarrow B + C$, we can define

$$\text{Tr}_{A,B}^C(f) = \iota_l; [f; B + \iota_r]^\dagger = \iota_l; f; [id, (\iota_l; f)^\dagger] : A \rightarrow B \quad (34)$$

Since this satisfies the axioms of a trace over a symmetric monoidal category, we can draw it, and therefore the Elgot operator, as in Figure 27. Continuing with the control-flow diagram analogy, such traces can be interpreted as *loops*, with the Elgot axioms, now drawn as diagrams in Figure 28.

Unfortunately, unmodified string diagrams do not work for premonoidal categories, and hence for Freyd categories. The reason is because, since not all morphisms are central, premonoidal categories do not in general validate *sliding*. However, this is easy enough to fix: we can postulate a (dashed red) “state” wire which all impure morphisms require as an input and output, as in Figure 29. Since the state wire linearly threads through all impure boxes, it establishes a unique order in which they must be executed; this construction is shown to be sound in Román [57]. Pure morphisms do not have a state wire, so a diagram representing a pure morphism will simply have a dashed red “stripe” on the side. This gives us a convenient way to distinguish between string diagrams using the monoidal structure induced by the coproduct and those using the premonoidal structure induced by the tensor product in a category having both (such as a distributive premonoidal category): the latter will have a state wire, while the former will not.

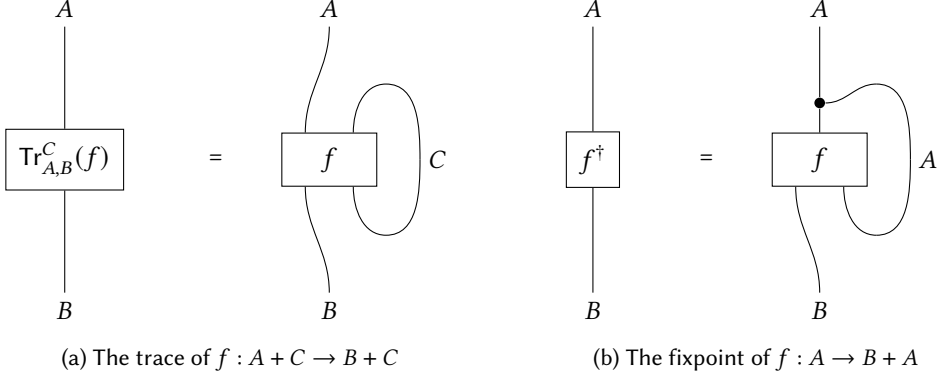


Fig. 27. Representations of the coproduct trace and Elgot structure as string diagrams

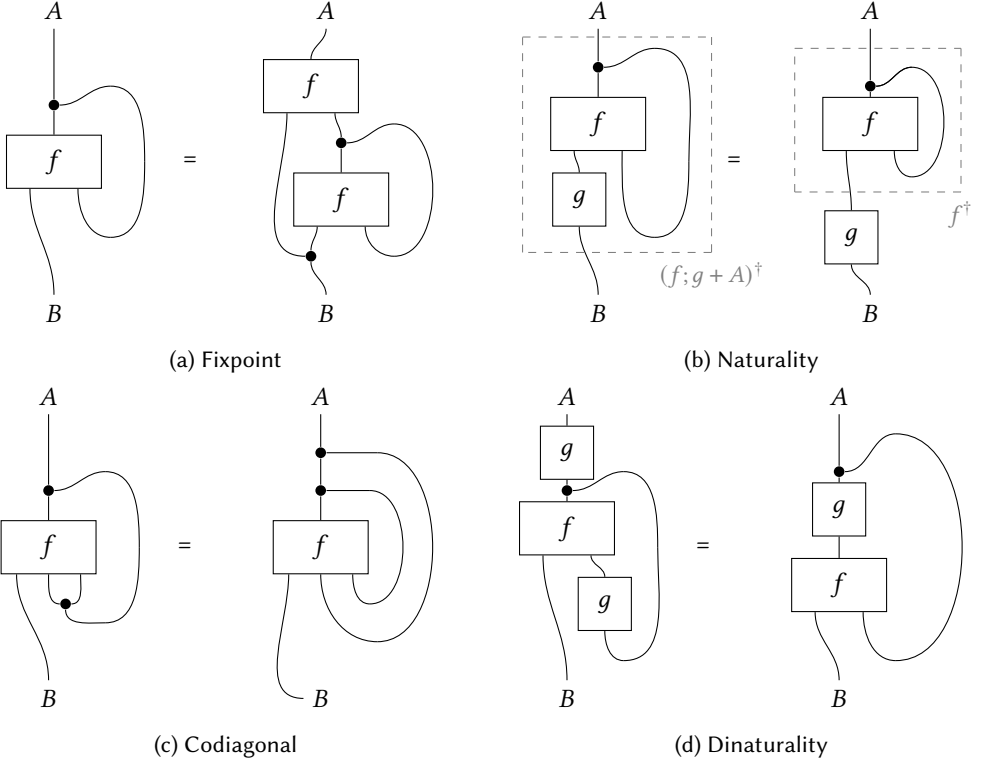


Fig. 28. Representations of the Elgot axioms as string diagrams

5.3 Semantics

We now have all the ingredients we need to give a semantics to λ_{SSA} expressions and regions. In particular, an λ_{SSA} expression model of a signature $Sg = (\mathcal{T}, \mathcal{I})$ consists of:

- An distributive Freyd category C
- A map $\llbracket \cdot \rrbracket$ from base types X to objects $\llbracket X \rrbracket : |C|$

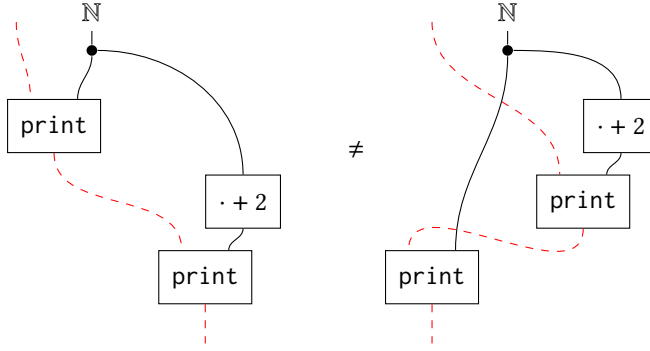


Fig. 29. A string diagram in a premonoidal category, demonstrating the necessity of using a state wire

- A map $\llbracket \cdot \rrbracket$ from primitive instructions $f \in \mathcal{I}_\epsilon(A, B)$ to morphisms $\llbracket f \rrbracket : \llbracket A \rrbracket \rightarrow_\epsilon \llbracket B \rrbracket$, where we model a type A as follows:
 - The unit type 1 is modelled as the monoidal unit I
 - The empty type 0 is modelled as the initial object 0
 - Products $A \otimes B$ are modelled as tensor products $\llbracket A \rrbracket \otimes \llbracket B \rrbracket$
 - Sum types $A + B$ are modelled as coproducts $\llbracket A \rrbracket + \llbracket B \rrbracket$

If an λ_{SSA} expression model additionally has an Elgot structure on $\mathcal{C}$, we will refer to it simply as an λ_{SSA} model. We will model *contexts* and *label contexts* as tensor products and coproducts of the denotations of their parameters, respectively, as in Figure 30.

We can now interpret λ_{SSA} -expressions $\Gamma \vdash_\epsilon a : A$ over a given signature Sg as morphisms $\llbracket \Gamma \rrbracket \rightarrow_\epsilon \llbracket A \rrbracket$ using the rules in Figure 31. Up to this point, both our syntax and semantics are quite standard; in particular:

- Variables x are modelled as projections from the appropriate index in the context's denotation $\pi_{\Gamma, x} : \llbracket \Gamma \rrbracket \rightarrow_\perp \llbracket A \rrbracket$
- Applications of primitive instructions $f \ a$ are modelled as $\llbracket f \rrbracket$ precomposed with the denotation of a .
- Unary let-bindings are modelled by:
 - Duplicating the context using the diagonal morphism $\Delta_{\llbracket \Gamma \rrbracket}$
 - Passing the right copy of the context through $\llbracket \Gamma \vdash_\epsilon a : A \rrbracket$ to get an input of type $\llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket = \llbracket \Gamma, x : A \rrbracket$
 - Passing the result of this through $\llbracket \Gamma, x : A \vdash_\epsilon b : B \rrbracket$ to get the final result of type $\llbracket B \rrbracket$
- Pairs are modelled by passing the result of the diagonal morphism $\Delta_{\llbracket \Gamma \rrbracket}$ to $\llbracket \Gamma \vdash_\epsilon a : A \rrbracket \ltimes \llbracket \Gamma \vdash_\epsilon b : B \rrbracket$ i.e., first passing the left copy through $\llbracket \Gamma \vdash_\epsilon a : A \rrbracket$ and then the right copy through $\llbracket \Gamma \vdash_\epsilon b : B \rrbracket$. By the axioms of a Freyd category, for pure morphisms, this is the same as simply taking the product $\langle \llbracket \Gamma \vdash_\epsilon a : A \rrbracket, \llbracket \Gamma \vdash_\epsilon b : B \rrbracket \rangle$.
- Binary let-bindings are modelled similarly to unary let-bindings, except that after passing the right copy of the context through $\llbracket \Gamma \vdash_\epsilon e : A \otimes B \rrbracket$, we re-associate $\llbracket \Gamma \rrbracket \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket)$ to $(\llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket) \otimes \llbracket B \rrbracket = \llbracket \Gamma, x : A, y : B \rrbracket$ before passing the entire result through $\llbracket \Gamma, x : A, y : B \vdash_\epsilon c : C \rrbracket$.
- The unit value $()$ is modelled as the terminal morphism $1_{\llbracket \Gamma \rrbracket}$, while abort a is modelled as the denotation of a postcomposed with the zero morphism $0_{\llbracket A \rrbracket}$. Injections, similarly, are simply modelled as the appropriate coproduct injections.

$$\begin{array}{c}
\boxed{\llbracket A \rrbracket : |C|} \\
\llbracket 1 \rrbracket = I \quad \llbracket A \otimes B \rrbracket = \llbracket A \rrbracket \otimes \llbracket B \rrbracket \quad \llbracket 0 \rrbracket = 0 \quad \llbracket A + B \rrbracket = \llbracket A \rrbracket + \llbracket B \rrbracket \\
\boxed{\llbracket \Gamma \rrbracket : |C|} \\
\llbracket \cdot \rrbracket = I \quad \llbracket \Gamma, x : A \rrbracket = \llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket \\
\boxed{\llbracket L \rrbracket : |C|} \\
\llbracket \cdot \rrbracket = 0 \quad \llbracket L, \ell(A) \rrbracket = \llbracket L \rrbracket + \llbracket A \rrbracket \\
\boxed{\llbracket \Gamma \leq \Delta \rrbracket : \llbracket \Gamma \rrbracket \rightarrow_{\perp} \llbracket \Delta \rrbracket} \\
\llbracket \cdot \leq \cdot \rrbracket = \text{id}_I \quad \llbracket \Gamma, x : A \leq \Delta \rrbracket = \pi_I; \llbracket \Gamma \leq \Delta \rrbracket \quad \llbracket \Gamma, x : A \leq \Delta, x : A \rrbracket = \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket \\
\boxed{\llbracket L \leq K \rrbracket : \llbracket L \rrbracket \rightarrow_{\perp} \llbracket K \rrbracket} \\
\llbracket \cdot \leq \cdot \rrbracket = \text{id}_0 \quad \llbracket L \leq K, \ell(A) \rrbracket = \llbracket L \leq K \rrbracket; \iota_{\ell} \quad \llbracket L, \ell(A) \leq K, \ell(A) \rrbracket = \llbracket L \leq K \rrbracket + \llbracket A \rrbracket
\end{array}$$

Fig. 30. Denotational semantics for λ_{SSA} types, contexts, and weakenings

- A case-expression is modelled by
 - Duplicating the context using the diagonal morphism $\Delta_{\llbracket \Gamma \rrbracket}$
 - Using the right copy of the context to compute the discriminant using $\llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket$
 - Applying the inverse distributor δ^{-1} to obtain a coproduct $\llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket + \llbracket \Gamma \rrbracket \otimes \llbracket B \rrbracket$
 - Computing $\llbracket \Gamma, x : A \vdash_{\epsilon} a : C \rrbracket$ on the right branch and $\llbracket \Gamma, y : B \vdash_{\epsilon} b : C \rrbracket$ on the left branch

Similarly, if we in fact have an λ_{SSA} model, we can interpret λ_{SSA} regions $\Gamma \vdash r \triangleright L$ as morphisms $\llbracket \Gamma \rrbracket \rightarrow \llbracket L \rrbracket$; note that we don't assume anything about the effect of these morphisms. As L is a coproduct, we can view the result object of a region r as encoding both *data* and *control-flow* information. In particular, we interpret a branch $\text{br } \ell \ a$ as simply the injection of the (pure) expression $\Gamma \vdash_{\perp} a : A$, our *data*, into the element of the coproduct corresponding to ℓ , which encodes the point in control-flow the rest of the program should jump to next. This is in contrast to *expressions*, which purely encode data, with no particular instructions on how to use it afterwards. Our interpretation of let-statements and case-statements, given in Figure 32, is exactly the same as that of the corresponding expressions.

Finally, we come to the interpretation of where-statements, which is where the Elgot structure comes in. The semantics of a where-block $\Gamma \vdash r$ where $(\ell_i(x_i) : \{t_i\},)_i \triangleright L$ can be broken down into two major components:

- The *terminator* $\text{esem}_{\Gamma, L}(r) : \llbracket \Gamma \rrbracket \rightarrow \llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket$, which, given as input the context $\llbracket \Gamma \rrbracket$, executes r and then re-associates the output. The output type $\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket$ expresses that r may either:

$$\begin{aligned}
& \boxed{\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket : \llbracket \Gamma \rrbracket \rightarrow_{\epsilon} \llbracket A \rrbracket} \\
& \llbracket \Gamma \vdash_{\epsilon} x : A \rrbracket = \pi_{\Gamma, x} \\
& \llbracket \Gamma \vdash_{\epsilon} f a : B \rrbracket = \llbracket f \rrbracket \circ \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket \\
& \llbracket \Gamma \vdash_{\epsilon} \text{let } x = a; b : B \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket \Gamma, x : A \vdash_{\epsilon} b : B \rrbracket) \\
& \llbracket \Gamma \vdash_{\epsilon} (a, b) : A \otimes B \rrbracket = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket \ltimes \llbracket \Gamma \vdash_{\epsilon} b : B \rrbracket \\
& \llbracket \Gamma \vdash_{\epsilon} \text{let } (x, y) = e; c : C \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} e : A \otimes B \rrbracket; \alpha; \llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket) \\
& \llbracket \Gamma \vdash_{\epsilon} () : 1 \rrbracket = 1_{\llbracket \Gamma \rrbracket} \\
& \llbracket \Gamma \vdash_{\epsilon} \iota_l a : A + B \rrbracket = \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \iota_l \\
& \llbracket \Gamma \vdash_{\epsilon} \iota_r b : A + B \rrbracket = \llbracket \Gamma \vdash_{\epsilon} b : B \rrbracket; \iota_r \\
& \llbracket \Gamma \vdash_{\epsilon} \text{case } e \{ \iota_l x : a, \iota_r y : b \} : C \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket; \delta_{\llbracket \Gamma \rrbracket}^{-1}; \\
& \quad \llbracket \llbracket \Gamma, x : A \vdash_{\epsilon} a : C \rrbracket, \llbracket \Gamma, y : B \vdash_{\epsilon} b : C \rrbracket \rrbracket) \\
& \llbracket \Gamma \vdash_{\epsilon} \text{abort } a : A \rrbracket = \llbracket \Gamma \vdash_{\epsilon} a : 0 \rrbracket; 0_{\llbracket A \rrbracket} \\
& \text{where } \boxed{\pi_{\Gamma, x} : \llbracket \Gamma \rrbracket \rightarrow_{\perp} \llbracket A \rrbracket} \quad \pi_{(\Gamma, x:A), x} = \pi_r \quad \pi_{(\Gamma, y:B), x} = \pi_l; \pi_{\Gamma, x}
\end{aligned}$$

Fig. 31. Denotational semantics for λ_{SSA} expressions

- Via the left injection, return immediately, jumping to an enclosing label in L
- Via the right injection, jump to the i^{th} basic block t_i in the where-statement by returning a value in $\llbracket A_i \rrbracket$
- The “loop” $\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i) : \llbracket \Gamma \rrbracket \otimes \Sigma_i \llbracket A_i \rrbracket \rightarrow \llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket$ which, given as input the context $\llbracket \Gamma \rrbracket$ and an input $\llbracket A_i \rrbracket$ for the i^{th} basic block t_i , executes t_i and then re-associates the output. The output type $\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket$ again expresses that control-flow may either exit the where-block (via $\llbracket L \rrbracket$) or jump to some other basic block (via $\Sigma_i \llbracket A_i \rrbracket$).

We glue these together in the obvious manner to get the semantics for a where-block:

- Compute $\text{let}(\text{esem}_{\Gamma, L}(r))$, which, given as input the context $\llbracket \Gamma \rrbracket$, copies the context, executes $\text{esem}_{r, \cdot}()$ and then returns the copied context and the output as $\llbracket \Gamma \rrbracket \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket)$
- Distribute the context into each branch of the coproduct, yielding $\llbracket \Gamma \rrbracket \otimes \llbracket L \rrbracket + \llbracket \Gamma \rrbracket \otimes \Sigma_i \llbracket A_i \rrbracket$
- If we are in the left branch, project out the result to yield $\llbracket L \rrbracket$, and return immediately
- Otherwise, compute $\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)$ in a loop, passing in a fresh copy of the context each time. This is implemented with rfix .

This simplifies significantly in the case of a where-block defining just a single label, yielding

$$\llbracket \Gamma \vdash r \text{ where } \ell(x) : \{s\} \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash r \triangleright L, \ell(A) \rrbracket; \delta^{-1}; [\pi_r, \text{rfix}(\llbracket \Gamma, x : A \vdash s \triangleright L, \ell(A) \rrbracket)]) \quad (35)$$

In particular, it is a consequence of label weakening (Lemma 5.8) that, in the case where s does not call ℓ , this simplifies further to

$$\llbracket \Gamma \vdash r \text{ where } \ell(x) : \{s\} \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash r \triangleright L, \ell(A) \rrbracket; \delta^{-1}; [\pi_r, \llbracket \Gamma, x : A \vdash s \triangleright L \rrbracket]) \quad (36)$$

5.4 Metatheory

We can now begin to state the metatheoretic properties of our denotational semantics. Before we do so, we establish the convention that whenever we have an equation involving the interpretation

$$\begin{aligned}
& \boxed{\llbracket \Gamma \vdash r \triangleright L \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket L \rrbracket} \\
& \llbracket \Gamma \vdash \text{br } \ell \ a \triangleright L \rrbracket = \llbracket \Gamma \vdash_{\perp} a : A \rrbracket; \iota_{L,\ell} \\
& \llbracket \Gamma \vdash \text{let } x = a; r \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket); \llbracket \Gamma, x : A \vdash r \triangleright L \rrbracket \\
& \llbracket \Gamma \vdash \text{let } (x, y) = e; r \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} e : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \vdash r \triangleright L \rrbracket \\
& \llbracket \Gamma \vdash \text{case } e \{ \iota_l x : r, \iota_r y : s \} \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket); \delta^{-1}; \\
& \quad \llbracket \llbracket \Gamma, x : A \vdash r \triangleright L \rrbracket, \llbracket \Gamma, y : B \vdash s \triangleright L \rrbracket \rrbracket \\
& \llbracket \Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket = \text{let}(\text{esem}_{\Gamma,L}(r)); \delta^{-1}; [\pi_r, \text{rfix}(\text{lsem}_{\Gamma,L}((\ell_i(x_i) : \{t_i\},)_i))] \\
& \text{where } \boxed{\iota_{L,\ell} : \llbracket A \rrbracket \rightarrow_{\perp} \llbracket L \rrbracket} \quad \iota_{(L,\ell(A)),\ell} = \iota_r \quad \iota_{(L,\kappa(B)),x} = \iota_l; \iota_{L,\ell} \\
& \boxed{\text{esem}_{\Gamma,L}(r) : \llbracket \Gamma \rrbracket \rightarrow \llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket} \\
& \text{esem}_{\Gamma,L}(r) = \llbracket \Gamma \vdash r \triangleright L, (\ell_i(A_i),)_i \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
& \boxed{\text{lsem}_{\Gamma,L}(\ell_i(x_i) : \{t_i\},)_i : \llbracket \Gamma \rrbracket \otimes \Sigma_i \llbracket A_i \rrbracket \rightarrow \llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket} \\
& \text{lsem}_{\Gamma,L}(\ell_i(x_i) : \{t_i\},)_i = \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_j \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+
\end{aligned}$$

Fig. 32. Denotational semantics for λ_{SSA} regions

of a derivation (e.g., $\llbracket \mathcal{D} \rrbracket = \llbracket \mathcal{D}' \rrbracket$), we assume that all the derivations (e.g., $\mathcal{D}$ and $\mathcal{D}'$) exist and are well-formed.

We begin with weakening: as shown in Figure 30, weakenings are modelled, essentially, as projections from a larger product $\llbracket \Gamma \rrbracket$ to a smaller product $\llbracket \Delta \rrbracket$, while label-weakenings are modelled as injections from a smaller coproduct $\llbracket L \rrbracket$ to a larger coproduct $\llbracket K \rrbracket$; in particular, in both cases, the morphisms are pure. A simple induction can then be used to derive the following weakening lemmas:

LEMMA 5.8 ((LABEL) WEAKENING). *Given $\Gamma \leq \Gamma'$ and $L' \leq L, K' \leq K$, we have*

- (a) *For all $\Gamma \leq \Delta$, $\llbracket \Gamma \leq \Delta \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \leq \Delta \rrbracket$*
- (b) *For all $L \leq K$, $\llbracket L' \leq K \rrbracket = \llbracket L' \leq L \rrbracket; \llbracket L \leq K \rrbracket$*
- (c) *$\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \vdash_{\epsilon} a : A \rrbracket$*
- (d) *$\llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \vdash r \triangleright L' \rrbracket; \llbracket L' \leq L \rrbracket$*
- (e) *For all $\gamma : \Gamma \mapsto \Delta$, $\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \gamma : \Gamma' \mapsto \Delta \rrbracket$*
- (f) *For all $\Gamma \vdash \sigma : L \rightsquigarrow K$, $\llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket \otimes \llbracket L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K' \rrbracket; \llbracket K' \leq K \rrbracket$*

PROOF. See Appendix C.4. □

We can now give a denotational semantics to substitutions in which a substitution $\gamma : \Gamma \mapsto \Delta$ is interpreted as a pure morphism from $\llbracket \Gamma \rrbracket$ to $\llbracket \Delta \rrbracket$, as in Figure 33. We can now state the soundness of variable substitution as follows:

THEOREM 5.9 (SOUNDNESS (SUBSTITUTION)). *Given $\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket \Delta \rrbracket$ pure, we have that*

- (a) *$\llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket$*
- (b) *$\llbracket \Gamma \vdash [\gamma]r \triangleright L \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash r \triangleright L \rrbracket$*
- (c) *$\llbracket [\gamma]\rho : \Delta \mapsto \Xi \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \rho : \Delta \mapsto \Xi \rrbracket$*
- (d) *$\llbracket \Gamma \vdash [\gamma]\sigma : L \rightsquigarrow K \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash \sigma : L \rightsquigarrow K \rrbracket$*

$$\begin{array}{c}
\boxed{\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket : \llbracket \Gamma \rrbracket \rightarrow_{\perp} \llbracket \Delta \rrbracket} \\
\llbracket \cdot : \Gamma \mapsto \cdot \rrbracket = !_{\llbracket \Gamma \rrbracket} \quad \llbracket \gamma, x \mapsto e : \Gamma \mapsto \Delta, x : A \rrbracket = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \times \llbracket \Gamma \vdash_{\perp} e : A \rrbracket \\
\boxed{\llbracket \Gamma \vdash \kappa : L \rightsquigarrow K \rrbracket : \llbracket \Gamma \rrbracket \otimes \llbracket L \rrbracket \rightarrow \llbracket K \rrbracket} \\
\llbracket \Gamma \vdash \cdot : \cdot \rightsquigarrow K \rrbracket = !_{\llbracket \Gamma \rrbracket} \otimes 0; \lambda; 0_K \\
\llbracket \kappa, \ell(x) \mapsto r \vdash \Gamma : L, \ell(A) \rightsquigarrow K \rrbracket = \delta; [\llbracket \kappa \vdash \Gamma : L \rightsquigarrow K \rrbracket, \llbracket \Gamma, x : A \vdash r \triangleright K \rrbracket]
\end{array}$$

Fig. 33. Denotational semantics for λ_{SSA} (label) substitutions

PROOF. See Appendix C.4. $\square$

In particular, this implies that the semantics of substitution composition $[\gamma]\rho$ is just composition of the denotations of γ, ρ . We can derive the following important corollary:

COROLLARY 5.10 (SOUNDNESS (SINGLE SUBSTITUTION)). *Given $\Gamma \vdash_{\perp} a : A$, we have*

(1) *Given $\Gamma, x : A \vdash_{\perp} b : B$,*

$$\llbracket \Gamma \vdash_{\epsilon} [a/x]b : B \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket); \llbracket \Gamma, x : A \vdash_{\epsilon} b : B \rrbracket = \llbracket \Gamma \vdash_{\epsilon} \text{let } x = a; b : B \rrbracket \quad (37)$$

(2) *Given $\Gamma, x : A \vdash r \triangleright L$, we have*

$$\llbracket \Gamma \vdash [a/x]r \triangleright L \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket); \llbracket \Gamma, x : A \vdash r \triangleright L \rrbracket = \llbracket \Gamma \vdash \text{let } x = a; r \triangleright L \rrbracket \quad (38)$$

PROOF. Follows immediately from the fact that

$$\llbracket x \mapsto a^1 : \Gamma \mapsto \Gamma, x : A \rrbracket = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\perp} a : A \rrbracket = \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket) \quad (39)$$

We can now move on to stating the metatheoretic properties of label-substitutions in much the same manner. In particular, in Figure 33, we interpret label substitutions $\Gamma \vdash \sigma : L \rightsquigarrow K$ as morphisms taking a copy of the context $\llbracket \Gamma \rrbracket$ and an element of the coproduct $\llbracket L \rrbracket$ to an element of the coproduct $\llbracket K \rrbracket$, with an arbitrary effect. Label substitution is then sound in general, as stated in the following theorem:

THEOREM 5.11 (SOUNDNESS (LABEL SUBSTITUTION)). *Given $\Gamma \vdash \sigma : L \rightsquigarrow K$, we have*

(a) $\llbracket \Gamma \vdash [\sigma]r \triangleright K \rrbracket = \text{let}(\llbracket \Gamma \vdash r \triangleright L \rrbracket); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket$

(b) $\llbracket \Gamma \vdash [\sigma]\sigma' : M \rightsquigarrow K \rrbracket = \Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash \sigma' : M \rightsquigarrow L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket$

PROOF. See Appendix C.5 $\square$

5.5 Equational Theory

Using the metatheory in the previous section, our goal is now to prove the equational theory given in Section 4 sound with respect to any valid λ_{SSA} model. Stated more precisely, we have the following:

THEOREM 5.12 (SOUNDNESS (EQUATIONAL THEORY)). *We have that*

(a) $\Gamma \vdash_{\epsilon} a \approx a' : A \implies \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket = \llbracket \Gamma \vdash_{\epsilon} a' : A \rrbracket$

(b) $\Gamma \vdash r \approx r' \triangleright L \implies \llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \vdash r' \triangleright L \rrbracket$

(c) $\gamma \approx \gamma' : \Gamma \mapsto \Delta \implies \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket = \llbracket \gamma' : \Gamma \mapsto \Delta \rrbracket$

(d) $\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K \implies \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \vdash \sigma' : L \rightsquigarrow K \rrbracket$

PROOF. See Appendix C.6 □

Now that we've proved the *soundness* of our equational theory, what remains is to prove that it is *complete*, i.e., that every equation which holds in all λ_{SSA} models can be derived from it, or, stated more categorically, that our syntax quotiented by the equational theory forms an initial λ_{SSA} model. Our strategy for doing this is as follows:

- (1) We begin by constructing a category of expressions $\text{Th}^\otimes(\Gamma)$ and a category of regions $\text{Th}(\Gamma, L)$ quotiented by our equational theory, and constructing a functor from the former to the latter.
- (2) We then show that the category of expressions and the category of regions have the structure of an λ_{SSA} expression model and λ_{SSA} model, respectively, and hence that expressions may be interpreted in the former and both expressions and regions in the latter.

To do so, we will first need some notation to talk about the behaviour of the equivalence classes of quotients. Suppose S and T are sets of terms. Then we will write $\Gamma \vdash_\epsilon S : A$ to mean that for every $a \in S$, we have $\Gamma \vdash_\epsilon a : A$, and similarly we will write $\Gamma \vdash_\epsilon S \approx T : A$ when for all $a \in S$ and $b \in T$, we have $\Gamma \vdash_\epsilon a \approx b : A$. We generalize in the obvious fashion to regions as well as the case when only one side of an equivalence is a set of terms.

Then, it will turn out that, for a distinguished variable $\square$ and label $\blacksquare$,

$$\begin{aligned} \Gamma, \square : \langle \Delta \rangle \vdash_\epsilon \llbracket \Delta \vdash_\epsilon a : A \rrbracket_{\text{Th}^\otimes(\Gamma)} : A \\ \Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash_\epsilon a : A \rrbracket_{\text{Th}(\Gamma, L)} \triangleright L, \blacksquare(A) \\ \Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash r \triangleright K \rrbracket_{\text{Th}(\Gamma, L)} \triangleright L, \blacksquare(\langle K \rangle) \end{aligned} \quad (40)$$

where packing of contexts into types $\langle \Delta \rangle$ is defined as in Appendix A.

- (3) Finally, we refine this result to show that

$$\begin{aligned} \Gamma, \square : [\Delta] \vdash_\epsilon \llbracket \Delta \vdash_\epsilon a : A \rrbracket_{\text{Th}^\otimes(\Gamma)} \approx \langle a \rangle : A \\ \Gamma, \square : [\Delta] \vdash \llbracket \Delta \vdash_\epsilon a : A \rrbracket_{\text{Th}(\Gamma, L)} \approx \text{ret } \langle a \rangle \triangleright L, \blacksquare(A) \\ \Gamma, \square : [\Delta] \vdash \llbracket \Delta \vdash r \triangleright K \rrbracket_{\text{Th}(\Gamma, L)} \approx \langle r \rangle \triangleright L, \blacksquare(\langle K \rangle) \end{aligned} \quad (41)$$

where $\text{ret } a := \text{br } \blacksquare a$. Since the packing operator $\langle \cdot \rangle$ on terms and regions from Appendix A is injective for pure contexts $\text{eff}(\Gamma) = \perp$, and hence in particular for $\Gamma = \cdot$, $L = \cdot$, it follows that in this case the category of expressions and the category of regions are the initial distributive λ_{SSA} expression model and λ_{SSA} model respectively.

5.6 Expressions

We'll begin by going over the entire proof of completeness for expressions, which is the simpler case. In particular, we may define the category $\text{Th}_\epsilon^\otimes(\Gamma)$ of expressions with effect ϵ as follows:

- Objects $|\text{Th}^\otimes(\Gamma)|$ are types A, B, C
- Morphisms $\text{Th}_\epsilon^\otimes(\Gamma)(A, B) = \{e \mid \Gamma, \square : A \vdash_\epsilon e : B\}$ quotiented by $\Gamma, \square : A \vdash_\epsilon e \approx e' : B$
- Identity $(\Gamma, \square : A \vdash_\perp \square : A) \in \text{Th}_\epsilon^\otimes(\Gamma)(A, A)$
- Composition $e; e' = (\text{let } \square = e; e')$, which satisfies

$$\Gamma, \square : A \vdash_\epsilon e : B, \quad \Gamma, \square : B \vdash_\epsilon e' : C \quad \implies \quad \Gamma, \square : A \vdash_\epsilon e; e' : C$$

We may verify that this satisfies the axioms of a category w.r.t. our equational theory

In general, the category of expressions $\text{Th}^\otimes(\Gamma)$ is simply then given by $\text{Th}_\top^\otimes(\Gamma)$, which can be viewed as the union of all $\text{Th}_\epsilon^\otimes(\Gamma)$.

We now need to equip $\text{Th}_\epsilon^\otimes(\Gamma)$ with the structure of a premonoidal category. Obviously, we wish to define the tensor product of types A and B to be simply $A \otimes B$; we can then begin by defining

projections

$$\Gamma, \square : A \otimes B \vdash_{\epsilon} \pi_l := \text{let } (x, y) = \square; x : A \quad \Gamma, \square : A \otimes B \vdash_{\epsilon} \text{let } (x, y) = \square; y : B \quad (42)$$

By simply using the pair constructor as a cartesian product $\langle a, b \rangle = (a, b)$, this can be shown to endow $\text{Th}_{\perp}^{\otimes}(\Gamma)$ with the structure of a cartesian category, allowing us to define the associators, symmetries, and unitors in the natural manner. If we then define tensor functors

$$- \otimes X : e \mapsto \text{let } (\square, x) = \square; (e; (\square, x)) \quad X \otimes - : e \mapsto \text{let } (x, \square) = \square; (e; (x, \square)) \quad (43)$$

we find that $\text{Th}_{\epsilon}^{\otimes}(\Gamma)$ and hence in particular $\text{Th}^{\otimes}(\Gamma)$ is endowed with the structure of a Freyd category with pure subcategory $\text{Th}_{\perp}^{\otimes}(\Gamma)$.

Similarly, we wish to show that $A + B$ is the coproduct of A and B in $\text{Th}_{\epsilon}^{\otimes}(\Gamma)$. Since we already have obvious injection morphisms

$$\Gamma, \square : A \vdash_{\epsilon} \iota_l := \iota_l \square : A + B \quad \Gamma, \square : B \vdash_{\epsilon} \iota_r := \iota_r \square : A + B \quad (44)$$

we can define the coproduct of morphisms $\Gamma, \square : A \vdash_{\epsilon} a : C$ and $\Gamma, \square : B \vdash_{\epsilon} b : C$ to be simply given by

$$\Gamma, \square : A + B \vdash_{\epsilon} [a, b] := \text{case } \square \{ \iota_l \square : a, \iota_r \square : b \} : C \quad (45)$$

It is straightforward to verify that this indeed induces a coproduct on $\text{Th}_{\epsilon}^{\otimes}(\Gamma)$ and hence on $\text{Th}^{\otimes}(\Gamma)$. All that remains is to show that $\text{Th}_{\epsilon}^{\otimes}(\Gamma)$ is in fact a *distributive* Freyd category. To do so, we may define an inverse distributor morphism

$$\Gamma, \square : A \otimes (B + C) \vdash_{\epsilon} \delta^{-1} := \text{let } (x, y) = \square; \text{case } y \{ \iota_l z : \iota_l(x, z), \iota_r z : \iota_r(x, z) \} : A \otimes B + A \otimes C \quad (46)$$

which can easily be shown to be an inverse to the obvious distributor morphism. We may now note that

$$\llbracket \cdot \rrbracket_{\text{Th}^{\otimes}(\Gamma)} = \mathbf{1}, \quad \llbracket \Delta, x : A \rrbracket_{\text{Th}^{\otimes}(\Gamma)} = \llbracket \Delta \rrbracket_{\text{Th}^{\otimes}(\Gamma)} \otimes A \quad \implies \quad \llbracket \Delta \rrbracket_{\text{Th}^{\otimes}(\Gamma)} = \langle \Gamma \rangle \quad (47)$$

Therefore, it follows that, as expected, that $\Gamma, \square : \langle \Delta \rangle \vdash_{\epsilon} \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket_{\text{Th}^{\otimes}(\Gamma)} : A$ and it remains to show that we in fact have

$$\Gamma, \square : \langle \Delta \rangle \vdash_{\epsilon} \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket_{\text{Th}^{\otimes}(\Gamma)} \approx [a] : A$$

which can be done by a relatively straightforward induction, implying, since $[\cdot]$ is injective w.r.t. our equational theory for pure contexts, the following theorem:

THEOREM 5.13 (COMPLETENESS (EXPRESSIONS)). *We have that, for all pure $\text{eff}(\Gamma) = \perp$,*

$$\Gamma \vdash_{\epsilon} e \approx e' : A \iff \llbracket \Gamma \vdash_{\epsilon} e : A \rrbracket_{\text{Th}^{\otimes}(\cdot)} = \llbracket \Gamma \vdash_{\epsilon} e' : A \rrbracket_{\text{Th}^{\otimes}(\cdot)}$$

In particular, this implies that $\text{Th}(\cdot)$ is the initial λ_{SSA} expression model

PROOF. See Appendix C.7 □

5.7 Regions

We define the category $\text{Th}(\Gamma, L)$ of regions as follows:

- Objects $|\text{Th}(\Gamma, L)|$ types A, B, C
- Morphisms $\text{Th}(\Gamma, L)(A, B) = \{r \mid \Gamma, \square : A \vdash r \triangleright L, \blacksquare(B)\}$ quotiented by $\Gamma, \square : A \vdash r \approx r' \triangleright L, \blacksquare(B)$
- Identity $\Gamma, \square : A \vdash \text{ret } \square \triangleright L, \blacksquare(A)$ where $\text{ret } a := \text{br } \blacksquare a$
- Composition $r; r' = [(\blacksquare(\square) \mapsto r')^1]r$

In particular, we may view ret as an identity-on-objects functor $\text{Th}_{\perp}^{\otimes}(\Gamma) \rightarrow \text{Th}(\Gamma, \mathbf{L})$ with action on morphisms given by given by

$$\Gamma, \square : A \vdash_{\perp} e : B \quad \mapsto \quad \Gamma, \square(A) \vdash \text{ret } e \triangleright \mathbf{L}, \blacksquare(B) \quad (48)$$

We will use this to equip $\text{Th}(\Gamma, \mathbf{L})$ with the structure of a Freyd category. In particular, taking our subcategory of pure morphisms to be the image of ret in $\text{Th}(\Gamma, \mathbf{L})$, we may define the obvious tensor functors

$$- \otimes X : r \mapsto \text{let } (\square, x) = \square; (r; \text{ret } (\square, x)) \quad X \otimes - : r \mapsto \text{let } (x, \square) = \square; (r; \text{ret } (x, \square)) \quad (49)$$

Our premonoidal structure is then completely described by requiring that ret preserves all relevant structure, i.e., that we have

$$\alpha = \text{ret } \alpha \quad \lambda = \text{ret } \lambda \quad \rho = \text{ret } \rho \quad \sigma = \text{ret } \sigma \quad \Delta = \text{ret } \Delta \quad (50)$$

Just like for expressions, we can write the coproduct of $\Gamma, \square : A \vdash s \triangleright \mathbf{L}, \blacksquare(C)$ and $\Gamma, \square : B \vdash t \triangleright \mathbf{L}, \blacksquare(C)$ in $\text{Th}(\Gamma, \mathbf{L})$ as

$$\Gamma, \square : A + B \vdash [s, t] := \text{case } \square \{ \iota_l \square : s, \iota_r \square : t \} \triangleright \mathbf{L}, \blacksquare(C) \quad (51)$$

with the obvious injections $\iota_l := \text{ret } \iota_l$ and $\iota_r = \text{ret } \iota_r$. It turns out that in this case ret preserves coproducts as well, and we can therefore easily conclude that our category is distributive by taking inverse distributor $\delta^{-1} = \text{ret } \delta^{-1}$.

All that remains now is to take $\text{Th}(\Gamma, \mathbf{L})$ from an λ_{SSA} expression model to an λ_{SSA} model by giving it an Elgot structure. We do so by defining the fixpoint of a morphism $\Gamma, \square : A \vdash r \triangleright \mathbf{L}, \blacksquare(B + A)$ as follows:

$$\Gamma, \square : A \vdash r^{\dagger} := \text{br go } \square \text{ where } \text{go}(\square : A) : \{r; \text{case } \square \{ \iota_l x : \text{ret } x, \iota_r y : \text{br go } y \} \} \triangleright \mathbf{L}, \blacksquare(B) \quad (52)$$

where go is an (arbitrary) fresh label. We can verify this indeed satisfies the axioms of an Elgot structure through a somewhat tedious calculation. We may now note that

$$\begin{aligned} \llbracket \cdot \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} &= 1, \quad \llbracket \Delta, x : A \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} = \llbracket \Delta \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} \otimes A &\implies \llbracket \Delta \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} &= \langle \Gamma \rangle \\ \llbracket \cdot^+ \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} &= 0, \quad \llbracket K, \ell(A) \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} = \llbracket K \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} + A &\implies \llbracket K \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} &= \langle K \rangle \end{aligned} \quad (53)$$

and hence that

$$\Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} \triangleright \mathbf{L}, \blacksquare(A) \quad \Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash r \triangleright K \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} \triangleright \mathbf{L}, \blacksquare(\langle K \rangle)$$

as expected. It is relatively easy to derive that

$$\Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} \approx \text{ret } \langle a \rangle \triangleright \mathbf{L}, \blacksquare(A)$$

by a relatively straightforward induction. A much more tedious induction is required to prove that

$$\Gamma, \square : \langle \Delta \rangle \vdash \llbracket \Delta \vdash r \triangleright K \rrbracket_{\text{Th}(\Gamma, \mathbf{L})} \approx [r] \triangleright \mathbf{L}, \blacksquare(\langle K \rangle)$$

since the case for where-statements is particularly complex. With a little bit more book-keeping (which can be found in the mechanization), we can state the completeness theorem as follows:

THEOREM 5.14 (COMPLETENESS (REGIONS)). *We have that, for all pure $\text{eff}(\Gamma) = \perp$,*

$$\Gamma \vdash r \approx r' \triangleright \mathbf{L} \iff \llbracket \Gamma \vdash r \triangleright \mathbf{L} \rrbracket_{\text{Th}(\cdot, \cdot)} = \llbracket \Gamma \vdash r' \triangleright \mathbf{L} \rrbracket_{\text{Th}(\cdot, \cdot)}$$

In particular, this implies that $\text{Th}(\cdot, \cdot)$ is the initial λ_{SSA} model.

PROOF. See Appendix C.7

□

6 Concrete Models

6.1 Monads and Monad Transformers

As previously stated, every strong monad over a CCC, by virtue of providing a model of higher-order effectful programming, *also* provides a model of first-order effectful programming, i.e., (acyclic) SSA. In particular, the Kleisli category C_T of every such strong monad T is a premonoidal category and induces a Freyd category with pure morphisms given by

$$(C_T)_\perp(A, B) = \{f; \eta_B : A \rightarrow TB \mid f : A \rightarrow B\} \subseteq C(A, TB) = C_T(A, B) \quad (54)$$

It follows that every strong Elgot monad over a CCC with all coproducts is an λ_{SSA} model (since, in particular, every CCC with coproducts is distributive). This allows us to very quickly amass a large collection of λ_{SSA} models from the literature on strong Elgot monads (see e.g. [28], [26]).

Probably the simplest example of an λ_{SSA} model is given by the *option monad* $\text{Option } A = A + 1$ ², with fixpoint operation

$$f^\dagger = \text{some } b \quad \text{if} \quad \exists n, f^n a = \text{some } (\iota_l b) \quad f^\dagger = \text{none} \quad \text{otherwise}$$

where

$$f^0 a = \text{some } (\iota_r a) \quad f^{n+1} a = \text{bind } (f^n a) [(\text{pure}; \iota_l), f]$$

This model allows us to interpret potentially divergent but otherwise purely functional programs. Another very important λ_{SSA} model is generated by the *powerset monad* $\mathcal{P}$ over Set , which has fixpoint operation

$$f^\dagger a = \bigcup_n \{b \mid \iota_l b \in f^n a\}$$

We can use this monad to interpret *nondeterministic* potentially divergent functional programs. We can further expand our repertoire of λ_{SSA} models by considering *monad transformers*, many of which preserve Elgot-ness. For example, if T is a strong Elgot over Set ,

- For all types R , the *reader transformer* $\text{ReaderT } R \ T \ A = R \rightarrow T \ A$ is strong Elgot with monad operations

$$\eta a = \lambda - . \eta_T a \quad \text{bind } a f = \lambda r. \text{bind}_T (a r) (\lambda a. f a r) \quad (55)$$

and fixpoint

$$f^\dagger = \lambda r. (\lambda a. f a r)^{\dagger_T} \quad (56)$$

- For all monoids W , the *writer transformer* $\text{WriterT } W \ T \ A = T (A \times W)$ is strong Elgot with monad operations

$$\eta a = \eta_T (a, 1) \quad \text{bind } a f = \text{bind}_T a (\lambda (a, w). \text{bind}_T (f a) (\lambda (b, w'). (b, w \cdot w')))) \quad (57)$$

and fixpoint

$$f^\dagger a = (\lambda (a, w). \text{bind}_T (f a) (\lambda (b, w'). \eta_T ([(\lambda c. (c, w \cdot w')), (\lambda c. (c, w \cdot w'))] b)))^{\dagger_T} (a, 1) \quad (58)$$

- For all types S , the *state transformer* $\text{StateT } S \ T \ A = S \rightarrow T (A \times S)$ is strong Elgot with monad operations

$$\eta a = \lambda s. \eta_T (a, s) \quad \text{bind } a f = \lambda s. \text{bind}_T (a s) (\lambda (a, s'). f a s') \quad (59)$$

and fixpoint

$$f^\dagger = \lambda s. (\lambda (a, s'). \text{bind}_T (f a s') (\lambda (b, s'). \eta_T ([(\lambda c. (c, s')), (\lambda c. (c, s'))] b)))^{\dagger_T} \quad (60)$$

One other important source of λ_{SSA} models is via *Elgot submonads* of monads on Set , which we define as follows:

²In a constructive setting, we may equivalently use the partial values monad $\text{Partial } A := \Sigma \phi : \text{Prop}. \phi \rightarrow A$.

Definition 6.1 (Elgot submonad). A monad (S, η', μ') is a submonad of (T, η, μ) if $\eta = \eta', \forall A, S A \subseteq T A$, and, on the set $S (S A) \subseteq T (T A)$, $\mu = \mu'$. We say S is *strong Elgot* if T is strong Elgot and, for all $f \in A \rightarrow S (A + B)$, $f^\dagger \in A \rightarrow S B$ (i.e. the fixpoint operation sends the Kleisli category of S to itself).

One useful property this definition has is that the intersection of arbitrarily many (Elgot) submonads forms an (Elgot) submonad, i.e., the (Elgot) submonads form a complete lattice.

6.2 Trace Models

A particularly important class of λ_{SSA} models are what we will call *trace models*, which can be viewed as programs which either diverge or produce a result, while also producing (potentially nondeterministic, potentially infinite) traces of events $\epsilon \in E$.

Definition 6.2 (Stream Action). A stream action $(M, I, \cdot, \Sigma)$ is a monoid actions $(M, I, \cdot)$ of finite effects M acting on divergent effects I , along with a function $\Sigma : M^\omega \rightarrow I$ mapping infinite streams of M to an element of I satisfying $\Sigma \sigma = \sigma_0 \cdot \Sigma_i \sigma_{i+1}$, where $\Sigma_i \sigma_i = \Sigma(\lambda i. \sigma_i)$.

The most important example of a stream action is the *free* stream action over a set E , called the set of events, where:

- $M = E^*$ is the free monoid over a set of “events” $\epsilon \in E$, i.e., the monoid of finite lists of events
- $I = E^{\leq \omega}$ is the set of *potentially* infinite streams of events $\epsilon \in E$
- $m \cdot i$ is defined in the obvious manner, i.e. by prepending the (finite) list m to the (potentially infinite) stream i
- $\Sigma \sigma$ is defined as the supremum $\bigsqcup_i t_i$, where $t \leq t'$ iff t is a prefix of t' , $t_0 = \perp$ (the empty stream), and $t_{i+1} = t_i \cdot \sigma_i$

Definition 6.3 (Trace Monad Transformer). Given a stream action $(M, I, \cdot, \Sigma)$, we can define the *trace monad transformer* over Set as follows: $\text{TraceT } \Sigma \ T \ A = T (A \times M + I)$ with monad operations

$$\eta = (\lambda a. \iota_l(a, 0)); \eta_T \quad \text{bind } a \ f = \text{bind}_T a \ [\lambda(a, m). m \cdot f a, \lambda i. \eta_T(\iota_r i)]$$

This yields a family of useful Elgot monads, including

- $\text{Traces? } \Sigma = \text{TraceT } \Sigma \ \mathcal{P}$, with the fixpoint of $f : A \rightarrow B + A$ given by

$$f^\dagger a = f_\infty^\dagger a \cup \bigcup_n f_n^\dagger a \quad \text{where} \quad (61)$$

$$f_n^\dagger a = \{\iota_l(b, m) \mid \iota_l(\iota_l b, m) \in [\text{id}_B, f]^n(\iota_r a)\} \cup \{\iota_r i \mid \iota_r i \in [\text{id}_B, f]^n(\iota_r a)\} \quad (62)$$

$$f^\infty a = \{\iota_r \Sigma \sigma \mid \exists \alpha : A^\omega, \alpha_0 = a \wedge \forall i, \iota_l(\iota_r \alpha_{i+1}, \sigma_i) \in f \alpha_i\} \quad (63)$$

We say a morphism $f : A \rightarrow \text{Traces? } \Sigma \ B$ is *total* if all $f a$ are nonempty, and *deterministic* if all $f a$ are subsingletons.

- $\text{Traces } \Sigma = \text{TraceT } \Sigma \ \mathcal{P}^+$, which can be viewed as an Elgot submonad of $\text{Traces? } \Sigma$ since the fixpoint of a total function is total
- $\text{Trace? } \Sigma = \text{TraceT } \Sigma \ \text{Option}$, which can be viewed as an Elgot submonad of $\text{Traces? } \Sigma$ since the fixpoint of a deterministic function is deterministic
- $\text{Trace } \Sigma = \text{TraceT } \Sigma \ \text{Id}$, which can be viewed as an Elgot submonad of $\text{Traces? } \Sigma$, and in particular the intersection $\text{Trace? } \Sigma \cap \text{Traces } \Sigma$.

As a simple, concrete example of this type of model this construction lets us define, consider the *nondeterministic printing monad*, which is simply defined as $\text{Print } A \equiv \text{Traces } \Sigma$, where $\Sigma : (\text{byte}^*)^\omega \rightarrow \text{byte}^{\leq \omega}$ denotes concatenation of bytestrings into a (potentially infinite) bytestream.

This gives us us an λ_{SSA} model supporting effectful instructions $\text{print} : \text{byte}^* \rightarrow 1$ and $\text{nondet} : 1 \rightarrow A$ (for A nonempty), with semantics $\llbracket \text{print} \rrbracket = \lambda b. \{ \iota_0((), b) \}$, $\llbracket \text{nondet} \rrbracket = \lambda(). \{ \iota_0(a, []) \mid a \in A \}$.

We can make things a little more interesting by adding a heap to the mix, defining our monad $\text{Comp} = \text{StateT Heap Print}$, where $\text{Heap} = \mathbb{N} \rightarrow \mathbb{N}$ is simply a partial function with finite support. The Kleisli category of this monad is also a Freyd category and inherits an Elgot structure from that on $\text{Set}_{\text{Print}}$; we therefore have an λ_{SSA} model supporting the instructions $\text{set} : \mathbb{N} \times \mathbb{N} \rightarrow 1$, $\text{get} : \mathbb{N} \rightarrow \mathbb{N}$, $\text{alloc} : \mathbb{N} \rightarrow \mathbb{N}$, and $\text{free} : \mathbb{N} \rightarrow 1$. We can assign semantics to set in the standard fashion, with $\llbracket \text{set} \rrbracket = \lambda(p, v) h. \{ \iota_0((), [p \mapsto v]h, []) \}$. get is a bit more tricky, since it is unclear what to do when we try to access uninitialized memory: one option is simply to return an arbitrary value, with $\llbracket \text{get} \rrbracket = \lambda p h. \{ \iota_0(v, h, []) \mid v = h \ p \vee p \notin h \}$. Finally, $\llbracket \text{alloc} \rrbracket = \lambda v h. \{ \iota_0(p, h', []) \mid h' = h \sqcup p \mapsto v \}$ simply fills a random empty heap cell with the provided value, returning a pointer to the cell, while $\llbracket \text{free} \rrbracket = \lambda p h. \{ \iota_0((), h \setminus p, []) \}$.

As we can see, the trace monad is a very powerful tool for quickly constructing families of λ_{SSA} models, at least for relatively simple side-effects. The resulting family of models is somewhat similar to *interaction trees*, which were introduced by Xia et al. [61] to give convenient denotational semantics to imperative programs: these also form an Elgot monad. Because they form an Elgot monad, they immediately form a model of λ_{SSA} . Similar techniques can be used to tackle much more complex side-effects; to demonstrate this, we will show how the model of TSO weak given in Kavanagh and Brookes [38] can be adapted to our framework.

6.3 TSO weak memory

Rather than a sequential trace consisting of a stream of events (here, *actions*) $a_1, a_2, a_3, \dots$, Kavanagh and Brookes [38] model concurrency by considering a *partially ordered multiset*, or *pomset*, of actions. The incomparable actions in our pomset are then precisely those which are executed in parallel. More formally, we define:

Definition 6.4 (Pomset). A *pomset* α over a set of actions $\mathcal{A}$ with a distinguished null action "tick" $\delta \in \mathcal{A}$ is a *nonempty* partially-ordered *carrier set* P such that every $p \in P$ has finitely many predecessors equipped with a mapping $\alpha : P \rightarrow \mathcal{A}$. A pomset is *finite* if its carrier set is. We will quotient pomsets over the removal of arbitrarily many copies of δ as long as infinite sets are only equated to other infinite pomsets. We will write unordered pomsets using multiset notation, e.g. $\{a, a, b\}$, and linearly ordered pomsets using list notation, e.g. $[a, a, b]$.

(Finite) pomsets form a monoid under sequential composition $\alpha; \beta$, which is defined by the function $[\alpha, \beta] : \text{trim}(P + Q) \rightarrow \mathcal{A}$, where $P + Q$ is given the lexicographic ordering and $\text{trim}(R)$ removes all elements of R with infinitely many predecessors. They also form a monoid under parallel composition, defining $\alpha || \beta = [\alpha, \beta] : P + Q \rightarrow \mathcal{A}$ where $P + Q$ is given the standard partial ordering (with elements of P and Q incomparable). Note in both cases the monoidal unit is $\{\delta\}$, since we only consider nonempty pomsets but allow the removal of finitely many ticks. Given any partially ordered set N and a family of pomsets α_n for $n \in N$, we can define their *sum* $\sum_n \alpha_n$ to be given by the function $(n, a) \mapsto \alpha_n a : \text{trim}(\sum_n P_n) \rightarrow \mathcal{A}$, where the dependent product $\sum_n P_n$ is given the lexicographic order. In particular, choosing $N = \mathbb{N}$ makes $\Sigma : \text{Pom}_{\text{fin}}^\omega \rightarrow \text{Pom}$ into a stream action monoid w.r.t the sequential composition monoid on finite pomsets.

For simplicity, we will consider an infinite set of named locations $x, y, z \in \text{Loc}$ which are subject to concurrent modification by all threads via TSO reads, writes, and fences. In particular, we define a *program order pomset* to be a pomset with $\mathcal{A}_{\text{PO}} = \mathcal{A}_w \cup \mathcal{A}_r \cup \{\delta\}$, where $\mathcal{A}_r$ consists of *reads* of the form $x = v$ and $\mathcal{A}_w$ consists of *writes* of the form $x := v$ for locations x and values $v \in \text{Word}$.

We may now define the *program order monad* $\text{PO } A = \text{Traces } \Sigma A$, where Σ is taken as a stream action on finite program order pomsets. This yields an λ_{SSA} model with support for

concurrent read and write operations $\text{read}_x : I_0^\otimes(1, \text{Word})$, $\text{write}_x : I_0^\otimes(\text{Word}, 1)$ with semantics $\llbracket \text{read}_x \rrbracket = R_x^{\text{PO}} = \lambda().\{(v, \{x = v\}) \mid v \in \text{Word}\}$, $\llbracket \text{write}_x \rrbracket = W_x^{\text{PO}} = \lambda v.\{(\langle(), \{x := v\})\}$. The semantics of read in particular give a hint as to how this model works: rather than tracking the state of the heap, we simply *emit* a pomset of the events that would have been generated by a given execution (in the case of a read, a read event $x = v$ whenever the read returns v , and in the case of a write, the single write event $x := v$ for a write of v), and later post-filter to obtain a set of valid executions. By doing so, our semantics remains compositional, allowing us to reason about each program fragment individually, while still allowing other program fragments to perform concurrent operations affecting our potential executions. On that note, we can define the *parallel execution* of two morphisms as follows:

$$\begin{aligned} f_0 \parallel f_1 &= \lambda(a_0, a_1). \{ \iota_0((b_0, b_1), \alpha_0 \parallel \alpha_1) \mid \iota_0(b_i, \alpha_i) \in f_i a_i \} \\ &\cup \{ \iota_1(\alpha_0 \parallel \alpha_1) \mid \iota_0(b_0, \alpha_0) \in f_0 a \vee \iota_0 \alpha_0 \in f_0 a_0 \wedge \iota_1 \alpha_1 \in f_1 a_1 \} \\ &\cup \{ \iota_1(\alpha_0 \parallel \alpha_1) \mid \iota_1 \alpha_0 \in f_0 a_0 \wedge \iota_0(a_1, \alpha_1) \in f_1 a_1 \} : \text{Set}_{\text{PO}}(A \otimes A', B \otimes B') \end{aligned} \quad (64)$$

It's tempting to try to use this to define a tensor product of morphisms to obtain a monoidal (rather than premonoidal) category, but unfortunately, sliding still fails: $(W_x^{\text{PO}} \parallel \text{id}); (\text{id} \parallel W_y^{\text{PO}})$ emits the pomset $[x := a, y := b]$, while $(\text{id} \parallel W_y^{\text{PO}}); (W_x^{\text{PO}} \parallel \text{id})$ emits the pomset $[y := b, x := a]$.

To graduate from sequential consistency to a genuine, if maximally simple, weak memory model, we introduce *load buffering* of write actions. We will implement TSO ordering by buffering all our writes, in which case they are only visible to the local thread. On the other hand, read events will first attempt to read from the buffer, and, if there is no corresponding write in the buffer, will read an arbitrary value, in both cases pushing an event to the global pomset. At any point, we may choose to *flush* some of the buffered writes. We introduce the set $\mathcal{A}_b = \{(\bar{x} := v)\}$ of *buffer write actions*, where an action $\bar{x} := v$ by a thread denotes adding a write $x := v$ to the thread's write buffer. We may then define the set of TSO actions $\mathcal{A}_{\text{TSO}} = \mathcal{A}_{\text{PO}} \cup \mathcal{A}_b$; a pomset over this set is called a *TSO pomset*. A buffer will be defined to be a list of write actions $\text{Buf} = \mathcal{A}_b^*$, which we will interpret as linear pomsets over $\mathcal{A}_{\text{TSO}}$ ordered by index (with the empty list corresponding to $\{\delta\}$). In particular, we define the monad $\text{TSO} = \text{StateT Buf (Trace } \Sigma)$, where Σ is taken as a the stream action of finite TSO pomsets on TSO pomsets. We can view PO as a submonad of this monad.

Given a buffer Buf , we can define the result of reads $[\cdot]_x : \text{Buf} \rightarrow \text{Word} \sqcup \{\perp\}$ from the buffer by induction to be: $(L; \{\bar{x} := v\})[x] = v$, $(L; \{\bar{y} := v\})[x] = L[x]$, $[\perp][x] = \perp$. Note that writes at the *end* of the buffer are prioritized, since later writes overwrite earlier ones!

The semantics of reads and writes can then be given by

$$\begin{aligned} \llbracket \text{read}_x \rrbracket &= R_x^{\text{TSO}} = \text{pflush}; (\lambda() L. \{(v, L, \{x = v\}) \mid L[x] = v \vee L[x] = \perp\}); \text{pflush} \\ \llbracket \text{write}_x \rrbracket &= W_x^{\text{TSO}} = \text{pflush}; (\lambda v L. \{(v, (L; \{\bar{x} := v\}), \{x := v\})\}); \text{pflush} \end{aligned} \quad (65)$$

where buffer flushing is implemented via the morphism

$$\text{pflush}_A = \lambda a L. \{(a, R, \alpha) \mid L = \alpha; R\} : \text{Set}_{\text{TSO}}(A, A)$$

(called *split* in [38]). To be able to perform synchronization, we will also need to introduce a fence : $I_0^\otimes(1, 1)$ instruction, which simply causes all actions before the fence to be observed before any actions after the fence. For TSO, implementing this is as simple as flushing the buffer, i.e., we define

$$\llbracket \text{fence} \rrbracket = \lambda() L. \{(\langle(), [], L; \{\delta\})\}$$

We can now define the parallel composition of morphisms in a "fork-join" style as follows: we first flush the buffer completely (i.e., taking it and sticking it at the beginning of our pomset), then execute f and g in parallel with separate buffers, *filtering out executions which completely flush the*

buffer. Since both threads end up with an empty buffer, the resulting joined buffer is also empty, giving us the following definition:

$$\begin{aligned} f_0 || f_1 = \lambda(a_0, a_1) L. \{ & \iota_0((b_0, b_1), [], L; (\alpha_0 || \alpha_1)) \mid \iota_0(b_i, [], \alpha_i) \in f_i a_i [] \} \\ & \cup \{ \iota_1(\alpha_0 || \alpha_1) \mid (\iota_0(b_0, [], \alpha_0) \in f_0 a_0 [] \vee \iota_0 \alpha_0 \in f_0 a_0 []) \wedge \iota_1 \alpha_1 \in f_1 a_1 [] \} \\ & \cup \{ \iota_1(\alpha_0 || \alpha_1) \mid \iota_1 \alpha_0 \in f_0 a_0 [] \wedge \iota_0(a_1, [], \alpha_1) \in f_1 a_1 [] \} \end{aligned} \quad (66)$$

One issue with this model is that our category currently has “too many” operations besides the ones we want: for example, it is a perfectly valid morphism to simply add an event into the buffer without any flushing. Of course, one thing we could do is consider the smallest Elgot subcategory generated by the atomic instructions we want to support, such as read_x , write_x , and fence , which can be a perfectly valid approach, but is somewhat unwieldy. Another approach, however, is to think about what “valid” morphisms might look like in general; this also has the benefit of letting us write down specifications that may not be trivially implementable in terms of actual instructions.

There are many potential properties we might want “valid” morphisms to have; in general, the less valid morphisms we admit, the more equations we have to work with when reasoning. One simple property we might consider is that a “valid” morphism must have at least one execution in which the buffer is completely flushed, regardless of initial state. This is a perfectly reasonable property to impose, since it ensures that we do not encounter degenerate cases where, for example, the parallel composition of two processes has an empty trace set even though both processes have a nonempty trace set (since neither empties the buffer). We might even want to go a little bit further and, representing the fact that a buffer flush can happen at any time between instructions, require that a nondeterministic buffer flush before or after an instruction does not change its semantics, i.e., that $\text{pflush}_A; f; \text{pflush}_B = f$. Unfortunately, this does *not* hold for the identity, since

$$\text{pflush}; \text{id}; \text{pflush} = \text{pflush}; \text{pflush} = \text{pflush} \neq \text{id} \quad (67)$$

so this would not give us a subcategory. While one possible solution is to simply add the identity back in (and this would give us a valid subcategory), we have the issue that, for example, morphisms like the associator and unitor would still be excluded. Instead, we can take into account the fact that all pflush_A are idempotent and define a new category PTSO with objects sets and morphisms of the form

$$\text{PTSO}(A, B) = \{ \text{pflush}_A; f; \text{pflush}_B \mid f \in \text{Set}_{\text{TSO}}(A, B) \} \quad (68)$$

In this category, pflush_A is the identity on A , since

$$\begin{aligned} \text{pflush}_A; \text{pflush}_A; f; \text{pflush}_B &= \text{pflush}_A; f; \text{pflush}_B \\ \text{pflush}_A; f; \text{pflush}_B; \text{pflush}_B &= \text{pflush}_A; f; \text{pflush}_B \end{aligned} \quad (69)$$

This is especially useful since it frees us from needing to remember to sprinkle pflush everywhere when specifying things (and, of course, when that level of control is desired, we can specify in Set_{TSO} and then lift to PTSO).

It turns out that this is a general construction we can do, which gives us another useful way to build up λ_{SSA} models that can often be quite difficult to express as simply monads. In particular, we have the following definition

Definition 6.5 (Idempotent envelope category). Let C be a category and d be a family of morphisms for each $A \in |C|$ such that each $d_A : C(A, A)$ is idempotent, i.e., $d_A; d_A = d_A$. We may then define the subsemicategory $\text{Ide}(C, d)$ of C to be given by

$$\text{Ide}(C, d)(A, B) = \{ f \in C(A, B) \mid f = d_A; f; d_B \} = \{ d_A; f; d_B \mid f \in C(A, B) \} \quad (70)$$

Note that $\text{Ide}(C, d)$ is *not* a subcategory unless $d_A = \text{id}_A$ for all A , in which case $\text{Ide}(C, d) = C$. However, $\text{Ide}(C, d)$ can be *independently* viewed as a category with identity d_A at A .

We have that, if C has coproducts and $d_{A+B} = d_A + d_B$, then $\text{Ide}(C, d)$ inherits coproducts from C with injections

$$\begin{aligned} d_A; \iota_l = \iota_l; d_{A+B} &= d_A; \iota_l; d_{A+B} \in \text{Ide}(C, d)(A, A+B) \\ d_B; \iota_r = \iota_r; d_{A+B} &= d_B; \iota_r; d_{A+B} \in \text{Ide}(C, d)(B, A+B) \end{aligned} \quad (71)$$

since, for $f \in \text{Ide}(C, d)(A, C)$, $g \in \text{Ide}(C, d)(A, C)$, we have that

$$d_{A+B}; [f, g]; d_C = d_A + B; [f, g]; d_C = [d_A; f; d_C, d_B; g; d_C] = [f, g] \quad (72)$$

In particular, it follows that Ide_d preserves coproducts. If C in addition has an Elgot structure, then $\text{Ide}(C, d)$ also inherits this structure (and therefore Ide_d preserves it), as for $f \in \text{Ide}(C, d)(A, B+A)$, we have

$$f^\dagger = (d_A; f; d_B + d_A)^\dagger = (d_A; f; d_B + d_A)^\dagger; d_B = d_A; (f; d_B + (d_A; d_A))^\dagger; d_B = d_A; f^\dagger; d_B \quad (73)$$

On the other hand, if C is equipped with a (symmetric) premonoidal structure and we have that

$$d_{A \otimes B} = d_A \times d_B = d_A \times d_B \quad d_{A \otimes I} = d_A \otimes I \quad d_{I \otimes A} = I \otimes d_A \quad (74)$$

(note that d does *not* have to be central!), it follows that $\text{Ide}(C, d)$ has (symmetric) premonoidal structure as well, with inherited tensor product functors and associators, unitors, and symmetries given by

$$\begin{aligned} d_{(A \otimes B) \otimes C}; \alpha_{A,B,C} &= \alpha_{A,B,C}; d_{A \otimes (B \otimes C)} = d_{(A \otimes B) \otimes C}; \alpha_{A,B,C}; d_{A \otimes (B \otimes C)} \\ d_{A \otimes I}; \lambda_A &= \lambda_A; d_A = d_{A \otimes I}; \lambda_A; d_A \quad d_{I \otimes A}; \rho_A = \rho_A; d_A = d_{I \otimes A}; \rho_A; d_A \\ d_{A \otimes B}; \sigma_{A,B} &= \sigma_{A,B}; d_{B \otimes A} = d_{A \otimes B}; \sigma_{A,B}; d_{B \otimes A} \end{aligned} \quad (75)$$

respectively. It follows that if C is distributive, so is $\text{Ide}(C, d)$, since we can verify that the distributor $[d_A \otimes (d_B; \iota_l), d_A \otimes (d_C; \iota_r)]$ has the expected inverse

$$d_{A \otimes (B+C)}; \delta^{-1}; d_{(A \otimes B) + (A \otimes C)} \quad (76)$$

If we in fact have that $d_{A \otimes B} = A \otimes d_B = d_A \otimes B$ for all A, B (which implies Equation 74) and C is equipped with the structure of a Freyd category, then so is $\text{Ide}(C, d)$, with projections and diagonal given by

$$\begin{aligned} d_{A \otimes B}; \pi_l = \pi_l; d_A &= d_{A \otimes B}; \pi_l; d_A \quad d_{A \otimes B}; \pi_r = \pi_r; d_B = d_{A \otimes B}; \pi_r; d_B \\ d_A; \Delta &= \Delta; d_{A \otimes A} = d_A; \Delta; d_{A \otimes A} \end{aligned} \quad (77)$$

Hence, it follows that if C is an λ_{SSA} (expression) model, d_A is idempotent, $d_{A+B} = d_A + d_B$, and $d_{A \otimes B} = A \otimes d_B = d_A \otimes B$, then $\text{Ide}(C, d)$ is an λ_{SSA} (expression) model as well. It turns out that all these properties hold for $d = \text{pflush}$, giving us our final categorical model for TSO weak memory, $\text{PTSO} = \text{Ide}(\text{Set}_{\text{TSO}}, \text{pflush})$.

6.4 Brookes-Style Models

In this section, we show how to construct a concurrency monad from a Brookes [12]-style model of concurrency, which we axiomatize. We will then explicitly show how to instantiate the model for sequentially consistent execution from Brookes [12]. We begin by defining a *countable closure operator*, given below:

Definition 6.6 (Countable Closure Operator). We define a (countable) closure operator $c : \mathcal{P}(T) \rightarrow \mathcal{P}(T)$ on T to be a function on sets of T which:

- is extensive: $X \subseteq c(X)$
- is idempotent: $c(c(X)) = c(X)$
- distributes over countable unions: $c(\bigcup_i X_i) = \bigcup_i c(X_i)$

We say a set X such that $c(X) = X$ is *closed* under c .

Note that this differs from the standard definition of a Kuratowski topological closure operator, which only requires us to be distributive over *finite* unions. We define the *lift* of a closure operator $c_A : \mathcal{P}(T \times A) \rightarrow \mathcal{P}(T \times A)$ to be given by

$$c_A(X) := \bigcup_{a \in A} \{(t', a) \mid t' \in c(\{t \mid (t, a) \in X\})\}$$

Identifying $\mathcal{P}(T \times A)$ with $A \rightarrow \mathcal{P}(T)$, this is equivalent to stating that $c_A(X) := c \circ X$. It is hence easy to check that this gives a closure operator on $T \times A$.

Definition 6.7 (Brookes Monad). Given a monoid of *traces* T and a closure operator $c : \mathcal{P}(T) \rightarrow \mathcal{P}(T)$, we define the *Brookes monad* B_c over c to be given by closed sets of pairs $t \cdot a$ of *traces* $t \in T$ and *results* $a \in A$. In particular, we define

$$B_c(A) := c_A(\mathcal{P}(T \times A)) \simeq A \rightarrow c(\mathcal{P}(T))$$

with

$$\eta_{B_c} a := c_A(\{1 \cdot a\}) \quad X \gg_{B_c} f := c_A(\{t \cdot t' \cdot b \mid \exists a.t \cdot a \in X, t' \cdot b \in f(a)\})$$

It is easy to see that the Brookes monad is in fact poset-enriched under the inclusion order. We say a Brookes monad is *standard* if $T = \langle S, S \rangle^*$ is the monoid of finite sequences of *rely-guarantee pairs* of *states* S .

We note that this definition gives us a Kleisli category which is not only poset-enriched, but in fact compatible with countable unions: with the usual definition $\cup_i f_i := \lambda a. \cup_i f_i(a)$, we have

$$\begin{aligned} ((\bigcup_i f_i); g)(a) &:= c_C(\{t \cdot t' \cdot c \mid \exists b.t \cdot b \in \bigcup_i f_i(a), t' \cdot c \in g(b)\}) \\ &= c(\bigcup_i \{t \cdot t' \cdot c \mid \exists b.t \cdot b \in f_i(a), t' \cdot c \in g(b)\}) \\ &= \bigcup_i c(\{t \cdot t' \cdot c \mid \exists b.t \cdot b \in f_i(a), t' \cdot c \in g(b)\}) \\ &= \bigcup_i (f_i; g)(a) \\ f; (\bigcup_i g_i)(a) &:= c(\{t \cdot t' \cdot c \mid \exists b.t \cdot b \in f(a), t' \cdot c \in \bigcup_i g_i(b)\}) \\ &= c(\bigcup_i \{t \cdot t' \cdot c \mid \exists b.t \cdot b \in f(a), t' \cdot c \in g_i(b)\}) \\ &= \bigcup_i c(\{t \cdot t' \cdot c \mid \exists b.t \cdot b \in f(a), t' \cdot c \in g_i(b)\}) \\ &= \bigcup_i (f; g_i)(a) \end{aligned}$$

and likewise for coproducts.

We can always equip B_c with an Elgot structure $(\cdot)^\dagger$ by, for $f : A \rightarrow B_c(B + A)$, defining

$$f^\dagger := \bigcup_{i \in \mathbb{N}} f_i$$

where $f_i : A \rightarrow B_c(B)$ are the *iterates* of f , defined inductively as follows

$$f_0 := 0_{A,B} = (\lambda x. \emptyset) \quad f_{i+1} := f; [\text{id}_B, f_i]$$

We note in particular that $\forall i. f_i \subseteq f_{i+1}$; similarly, if $g \subseteq f$, we have that $g_i \subseteq f_i$. We can think of analysis using this structure as corresponding to *partial correctness*, since any infinite traces are

simply discarded (a program which always diverges will have denotation $\emptyset$). It is straightforward to check that this is indeed an Elgot structure, since it satisfies:

- *Fixpoint*: we have, since $f_0 \cup g = g$,

$$f^\dagger = \bigcup_{i \in \mathbb{N}} f_i = \bigcup_{i \in \mathbb{N}} f_{i+1} = \bigcup_{i \in \mathbb{N}} (f; [\text{id}_B, f_i]) = f; [\text{id}_B, \bigcup_{i \in \mathbb{N}} f_i] = f; [\text{id}_B, f^\dagger]$$

as desired.

- *Naturality*: by induction, we show that $(f; g + A)_i = f_i; g$, since $(f; g + A)_0 = 0_{A,C} = 0_{A,B}; g$ and

$$(f; g + A)_{i+1} = f; g + A; [\text{id}_C, (f; g + A)_i] = f; [g, f_i; g] = f; [\text{id}_B, f_i]; g = f_{i+1}; g$$

and therefore

$$(f; g + A)^\dagger = \bigcup_{i \in \mathbb{N}} (f; g + A)_i = \bigcup_{i \in \mathbb{N}} (f_i; g) = (\bigcup_{i \in \mathbb{N}} f_i); g = f^\dagger; g$$

- *Codiagonal*: given $f : A \rightarrow (B + A) + A$, define $g = f; [\text{id}_{B+A}, \iota_r]$ and $h = f^\dagger$. We have

$$\begin{aligned} h_{i+1} &= f^\dagger; [\text{id}, h_i] = f; [\text{id}, f^\dagger]; [\text{id}, h_i] \\ &= f; [[\text{id}, h_i], f^\dagger; [\text{id}, h_i]] = f; [[\text{id}, h_i], h_{i+1}] \\ g_{i+1} &= f; [\text{id}_B, \iota_r]; [\text{id}, g_i] = f; [[\text{id}, g_i], g_i] \end{aligned}$$

It follows by induction that $g_i \subseteq h_i$, since $g_0 = h_0$, and

$$g_{i+1} = f; [[\text{id}, g_i], g_i] \subseteq f; [[\text{id}, h_i], h_i] \subseteq f; [[\text{id}, h_i], h_{i+1}] = h_{i+1}$$

We therefore have that $g^\dagger \subseteq h^\dagger$. On the other hand, we may show by induction on j that $f_j; [\text{id}, g^\dagger] \subseteq g^\dagger$, since $0 \subseteq g^\dagger$ and

$$f_{j+1}; [\text{id}, g^\dagger] = f; [\text{id}, f_j]; [\text{id}, g^\dagger] = f; [[\text{id}, g^\dagger], f_j; [\text{id}, g^\dagger]] \subseteq f; [[\text{id}, g^\dagger], g^\dagger] = g^\dagger$$

We now show by induction that $h_i \subseteq g^\dagger$: noting that $h_0 = 0 \subseteq g^\dagger$, it suffices to show that

$$\begin{aligned} h_{i+1} &= f; [[\text{id}, h_i], f^\dagger; [\text{id}, h_i]] \subseteq f; [[\text{id}, g^\dagger], f_j; [\text{id}, g^\dagger]] \\ &= \bigcup_{j \in \mathbb{N}} (f; [[\text{id}, g^\dagger], f_j; [\text{id}, g^\dagger]]) \subseteq \bigcup_{j \in \mathbb{N}} (f; [[\text{id}, g^\dagger], g^\dagger]) = g^\dagger \end{aligned}$$

It follows that $h^\dagger \subseteq g^\dagger$ and hence that $h^\dagger = g^\dagger$, as desired.

- *Uniformity*: given arbitrary $f : A \rightarrow B + A$, $g : X \rightarrow B + X$ and $h : X \rightarrow A$ such that $h; f = g; B + h$, for all f_i , we show by induction that $h; f_i = g_i$ since $h; f_0 = 0 = g_0$ and

$$h; f_{i+1} = h; f; [\text{id}, f_i] = g; B + h; [\text{id}, f_i] = g; [\text{id}, h; f_i] = g; [\text{id}, g_i] = g_{i+1}$$

Therefore, $h; f^\dagger = \bigcup_i h; f_i = \bigcup_i g_i = g^\dagger$, as desired.

Following Brookes [12], we can build up a model of *sequentially consistent* concurrent computation by taking a standard Brookes monad with

- States maps from locations to values $S := \text{Loc} \rightarrow \text{Val}$
- Closure operator generated by:
 - *Stuttering* $\forall \mu \in S. \cdot \twoheadrightarrow \langle \mu, \mu \rangle$
 - *Mumbling* $\forall \mu, \rho, \theta \in S. \langle \mu, \rho \rangle \langle \rho, \theta \rangle \twoheadrightarrow \langle \mu, \theta \rangle$

The semantics of a write, for example, is then given by

$$\text{write} : \text{Loc} \times \text{Val} \rightarrow \text{B}_c(1) := \lambda(\ell, v). c_1(\{\langle \mu, [\ell \mapsto v] \mu \rangle \cdot () \mid \mu \in S\})$$

More complex states (e.g. involving per-thread buffers) and closure operators can allow us to model weak memory models. In particular, Jagadeesan et al. [36] gives a Brookes model of TSO, while Dvir et al. [18] gives a Brookes model of release-acquire.

6.5 Non-examples

Weak memory is very complicated, and it has been very difficult to build models which satisfy even “obvious” properties. For example, the model of Jagadeesan et al. [35] fails to even satisfy the associativity of sequencing (i.e., $\llbracket (S_1; S_2); S_3 \rrbracket = \llbracket S_1; (S_2; S_3) \rrbracket$), and it took a nontrivial effort by Jeffrey et al. [37] to establish this property. However, properties like distributivity

$$\llbracket S; \text{if } b \{T_1\} \text{ else } \{T_2\} \rrbracket = \llbracket \text{if } b \{S; T_1\} \text{ else } \{S; T_2\} \rrbracket$$

do not hold, and there is often no semantics of loops at all. This makes them impossible to apply to SSA without extensive adaptation.

One of the easiest ways to build a model with “nice” properties is to make the model monadic; indeed, all the models presented in the above section are an example of this technique. This handles sequencing and distributivity “for free” via the monad laws and the fact that monads always preserve coproducts; however, this still requires us to verify the Conway iteration axioms to support loops, as is done in Xia et al. [62]. Our work identifies the minimal conditions required to have a compositional model of general *first-order* side effects compatible with the standard interpretation of SSA; the higher-order analogue is precisely a (complete) Elgot monad [2], as presented in Goncharov et al. [29].

7 Discussion and Related Work

7.1 Lean Formalization

We have formalized many of the lemmas and theorems presented in this paper in the Lean 4 proof assistant; as of now, our main formalization, `debruijn-ssa` (<https://github.com/imbrem/debruijn-ssa>), weighs about 29 kloc. While our paper uses named variables for simplicity, all our rules and syntax are formalized using de-Bruijn indices. Each formalized theorem is tagged as such in the respective proof. It was also necessary to fork Mathlib’s monoidal category implementation to support Freyd categories, distributivity, and Elgot structure: the resulting library, `discretion` (<https://github.com/imbrem/discretion>), weighs about 27 kloc. We are currently trying to upstream support for premonoidal categories into Mathlib.

7.2 SSA, FP and IRs

Static Single Assignment (SSA) form was first introduced as a compiler intermediate representation by Alpern et al. [4] and Rosen et al. [58], with the goal of facilitating effective reasoning about program variable equivalence. To perform optimizations like common subexpression elimination (CSE) effectively, we need to determine which expressions are equal *at a given point in time*. By transforming the program into SSA form, we introduce unique variables for each assignment and use ϕ -nodes to merge variable values at points where control paths converge. Since each variable then corresponds to a unique value at a specific point in the program’s execution, SSA unlocks the ability to perform *algebraic reasoning* about variable values over time. As a result, analyses like CSE become a matter of simple algebraic rewriting based on variable names.

Cytron et al. [16] provided the first efficient algorithm for converting 3-address code programs to SSA form using a minimal number of ϕ -nodes. They observed that a ϕ -node only needs to be

introduced at the earliest point where a variable may have different values based on control flow. This point is computable via the graph-theoretic notion of a *dominance frontier*, which identifies where control paths merge in the program's control flow graph. Since then, SSA has become the intermediate representation of choice for most production-grade compiler toolchains.

One of the most famous optimizations enabled by SSA's support for effective algebraic reasoning is *sparse conditional constant propagation (SCCP)*, introduced in Wegman and Zadeck [60]. This algorithm leverages the SSA property to reason about the possible set of values each variable may take using a lattice-based data-flow analysis, which models variable values in terms of abstract states like *unknown*, *constant*, or *overdefined*. Without SSA, one could naively consider all variable definitions, which would likely detect fewer constants due to imprecision. Alternatively, achieving more precise results would require a complex and computationally intensive reaching-definitions analysis.

However, while SSA is an excellent representation for implementing compilers, the semantics of ϕ -nodes can be quite unintuitive due to their lack of an obvious operational interpretation, making them challenging to reason about formally. Kelsey [39] establishes a correspondence between SSA and a subset of *continuation-passing style (CPS)*, a common intermediate representation for functional compilers. In particular, while not all CPS programs can be directly converted to SSA form—those using non-local returns like `longjmp` or `call/cc`—the typical outputs of CPS transformations avoid such features. Kelsey observed that many optimizations requiring flow analysis in CPS could be performed directly in SSA, often dramatically simplifying them.

Appel [5] builds on this work by informally showing that the functional subset of CPS programs “hidden inside” SSA is, in fact, simply nested, mutually tail-recursive functions, with each function corresponding to a basic block. He makes the key observation that the dominance-based scoping of SSA corresponds to the lexical scoping of functions. For a variable to be visible in a function, it must appear in the lexical scope of that variable's definition and therefore be dominated by it; otherwise, there would be no way to call the function.

In fact, the subset of functional programs identified by Appel [5] corresponds to *A-normal form (ANF)*, another functional intermediate representation advocated by Flanagan et al. [21]. Chakravarty et al. [14] formalizes this correspondence giving an algorithm to convert SSA programs to ANF, and then showing how SSA optimizations such as SCCP can be written to operate on ANF programs. The authors highlight that the semantic rigor of their notation, combined with the well-defined semantics of ANF, make their presentation of SCCP significantly more amenable to formal analysis.

Going in the other direction, Leißa et al. [43] introduce the Thorin intermediate representation, which consists of CPS extended with SSA-style dominance-based scoping. As SSA is a first-order language, it is often difficult to represent programs using closures effectively, and consequently, difficult to optimize them well. The authors give the example of LLVM, which often needs to generate a new struct and a large amount of boilerplate code for every closure, which, even after optimization, is often not significantly reduced. By contrast, Thorin retains the advantages of CPS for representing functional programs, while enabling SSA-like graph-based use-def analysis by prohibiting variable shadowing.

The primary focus of the works we have covered so far is establishing the correspondence between SSA, an imperative intermediate representation, and widely used functional intermediate representations. However, to advance beyond these correspondences and directly study the optimization and verification of SSA programs themselves, we require an equational theory underpinned by formal semantics. One approach found in the literature (and which we also use) is to relax SSA into forms that support richer substitution principles and well-defined operational

semantics, which can then be used to reason about SSA itself. Two papers which exemplify this approach are Benton and Kennedy [9] and Garbuzov et al. [23].

One of the difficulties of working with ANF as an intermediate representation is that it does not, in general, satisfy *substitution*: replacing a variable x with a compound expression like a function call can take a program out of the ANF-fragment. The *monadic intermediate language* (MIL) introduced by Benton and Kennedy [9] for use in the MLj compiler for Standard ML can be viewed as a relaxation of ANF (and hence, of SSA) to get a nicer substitution principle. Benton and Kennedy then use this flexibility to build up an equational theory justified by their operational semantics. One interesting feature is that, unlike Moggi's equational metalanguage [51] (and like our λ_{SSA} calculus), MIL enforces a stratification of *values* and *computations*, without supporting "computations of computations" (corresponding to nested monad types $T(T(A))$). This hints at Freyd categories, rather than general Kleisli categories, being a natural model of MIL-like intermediate representations.

Similarly, Garbuzov et al. [23] exhibit a correspondence between an operational semantics for SSA and an operational semantics for call-by-push-value (CBPV) [45]. They then use the normal form bisimulations of Lassen and Levy [40] to derive an equational theory for use in justifying optimizations. In particular, we can view their paper as interpreting CBPV as a relaxation of SSA more suited for developing an equational theory and for semantics work; in particular, to be able to take advantage of CBPV to give a *structural* operational semantics for (unstructured) SSA programs. The semantics of CBPV are widely studied, and hint at a large variety of potential models for SSA, but the formalization in [23] does not support any effects other than nontermination.

7.3 Formalizations of SSA

7.3.1 Other SSA type systems. Several attempts have been made to provide a type-theoretic treatment of SSA. The work most similar to ours is by Rigon et al. [56], who present a typed translation from SSA into the lambda calculus using a type-and-effect system, observing that the algorithm for converting programs to SSA form may also be viewed as a mechanism for transforming programs in a functional language with unstructured control flow into equivalent expressions.

Similarly to λ_{SSA} , they extend the lambda calculus with a mutually recursive where-binding, which allows them to directly translate unstructured SSA control flow. However, their calculus uses only a single syntactic category of expressions, whereas we attempt to model (generalized) SSA directly by distinguishing between expressions and regions. Their language also includes support for effect handlers, which are beyond the scope of our current study but represent an interesting direction for future work. While the authors do not provide an equational theory or a semantics for their language, we believe that our equational theory could be adapted to the fragment of their language without effect handlers.

An interesting alternative approach is demonstrated by Matsuno and Ohori [49], who give a type theory for what appear to be ordinary three-address code programs. However, every well-typed program can be placed into SSA-form by inserting ϕ -nodes in a fully type-directed way. This lets them model SSA without any ϕ -nodes, letting them use the standard semantics for three-address code.

Menon et al. [50] give a type-safe formalization of SSA, along with an operational semantics and formal definitions of dominance, definition/use points, and the SSA property for 3-address code. By augmenting SSA with first-class proof variables, they aim to give a representation which allows aggressive optimizations to preserve safety information. Their type system requires checking the SSA property separately from well-typedness, but is proven sound if the SSA property holds. Hua et al. [33] give another type system and direct operational semantics for standard SSA, and prove type safety for it.

Many operational semantics for SSA have arisen from compiler verification efforts. Barthe et al. [7] give an operational semantics as part of the CompCertSSA project, and give a semantics-preserving translation from three-address code into SSA. Herklotz et al. [31] formalise "gated SSA" and give semantics-preserving translations between it and ordinary SSA. Going beyond CompCertSSA, Zhao et al. [63] have studied the semantics of the LLVM IR itself as part of the Vellvm project; Li and Gunter [48] goes further and gives an operational semantics for LLVM in the (sequentially consistent) multithreaded setting.

There has been much less work on denotational semantics for SSA, or directly on its equational theory. Pop et al. [54] give an unusual denotational model of SSA in terms of the iteration structure of a program, which they use to better understand the loop-closing ϕ -nodes found both in the gated SSA representation as well as practical compilers such as GCC. Xia et al. [61] describe a toolkit for building denotational semantics for imperative languages, including the standard IMP language and a simple assembly language, using the *interaction tree monad* and *continuation tree monad*, which they show to satisfy the Elgot axioms. Recently, Chappe et al. [15] introduced *choice trees* (CTrees), which extend ITrees to support weak memory concurrency, and use this to model a subset of LLVM IR.

7.3.2 Mechanizations of SSA. CompCertSSA [6] is an attempt to extend the CompCert verified compiler [44] with an SSA-based middle-end. This is achieved by generating a pruned SSA IR from CompCert's RTL format. Instead of verifying the RTL-to-SSA translation within Coq, this translation is performed by unverified code, and a separate *translation validation* stage uses a verified checker to ensure correctness. After performing (verified) SSA optimizations, the IR is naively lowered back to RTL for the rest of CompCert's machinery to work on. Demange et al. [17] build on this work by providing realistic, verified implementations of Sparse Conditional Constant Propagation (SCCP) [60], Common Subexpression Elimination (CSE), and Global Value Numbering (GVN) [58] within a general framework of flow-insensitive static analysis for CompCertSSA.

While CompCert is a verified implementation of a new compiler in Coq, the Vellvm project [63] attempts to mechanize a subset of the LLVM intermediate representation (IR), covering the LLVM type system, operational semantics, and the well-formedness and structural properties of valid LLVM IR. The authors adopt a memory model for LLVM based on CompCert's [44], allowing them to leverage significant portions of CompCert's Coq infrastructure. Similarly, Bhat et al. [10] attempt to mechanize a well-defined subset of the Multi-Level Intermediate Representation (MLIR) in the Lean 4 theorem prover. Their formalization features a user-friendly front end to convert MLIR syntax into their calculus and scaffolding for defining and verifying peephole rewrites using tactics. Their framework has been tested on bitvector rewrites from LLVM, structured control flow, and fully homomorphic encryption; however, as of publication, only structured control flow was fully supported.

7.3.3 Completeness. Most other formalizations of SSA have given *specific* semantics for it, whether operational or denotational. One of the features which makes our work distinctive is that we give our type theory semantics relative to a categorical axiomatization. Essentially, we have parameterized our interpretation over any model satisfying the Freyd-Elgot axioms. First, this lets us show that many different concrete semantics are valid models of SSA, as can be seen in section 6. This also let us show that our equational theory is complete: we have exactly the equations valid in all Freyd-Elgot models, no more and (crucially) no less. As a result, implementors and researchers proving things about compiler optimizations do not have to deal with any of the messy details of (for example) weak memory semantics, unless they are trying to study optimizations specifically involving weak memory. All the usual equations used in control- and data-flow optimizations are independent of those details, and can be validated using the equational theory.

Conversely, researchers working on models of weak memory (and other strange models of computation such as quantum computation), can use the Freyd-Elgot axioms as a target. If they ensure their models satisfy these axioms, then using an SSA-based IR in their compiler is justified.

7.4 Compositional (Relaxed) Concurrency

The most natural way to think about concurrency is often in terms of an operational semantics on an abstract machine, in which concurrent threads interleave and interact. Such semantics, naturally, are designed to reason about an entire program at a time. Batty [8] argues that a compositional semantics of concurrency is necessary to be able to reason about properties of large software systems effectively, particularly in the presence of complicating factors such as compiler optimizations and weak memory semantics. However, it is generally very challenging to reason about a small component of a concurrent system in isolation since its behaviour may be drastically affected by other components running in parallel.

One natural approach to constructing denotational models of concurrency is to consider the extension of an abstract machine's behavior. Each thread performs a sequence of atomic actions, and so from the outside, the machine can produce any interleaving of the atomic actions of each thread. Just by itself, considering sets of possible traces is not sufficiently abstract, and so Brookes [12] takes the closure of these sets under semantics-preserving transformations, such as stuttering (introducing extra identity steps) and mumbling (which fuse sequential atomic operations, thereby hiding implementation details), to obtain a model with good equational properties such as associativity ($\llbracket \alpha; (\beta; \gamma) \rrbracket = \llbracket (\alpha; \beta); \gamma \rrbracket$) and the expected identities for branches and loops.

If the machine model is *sequentially consistent* – i.e., all allowed behaviors arise from interleavings of sequences of atomic events – then this style of trace semantics is sufficient. However, real hardware often exhibits *relaxed behaviour*, in which additional behaviours which do not correspond to the interleaving of parallel threads are allowed. Even more relaxed behaviours arise from fundamental compiler optimizations, such as re-ordering of independent reads and writes. These are perfectly valid in a single-threaded context but can introduce new behaviours in multithreaded programs – for example, another thread could distinguish the order of writes. In general, we need tools to reason about interactions with a system that is not *linearizable* (i.e., in which concurrent operations cannot be reduced to interleaved atomic operations on a single thread), of which actual hardware is only one example. There are two major approaches to this problem.

One idea we might have is to augment traces with additional structure, which we can then quotient away to maintain extensionality by using an appropriate closure operator. What's nice about this method is that we can often use structures analogous to the additional state in the machine model which leads to the relaxed behaviour in the first place. For example, in Jagadeesan et al. [36], the authors extend Brookes [12] with additional state corresponding to the contents of a thread-local buffer, and then take the closure of their trace-set with respect to buffer operations such as nondeterministic flushing. This gives them a model of TSO weak memory with good equational properties and a monadic structure, which remains intuitive, since it has a connection to the original TSO model which can also be framed in terms of the abstract machine having thread-local buffers. Dvir et al. [18] use this approach to derive a trace-based semantics for release-acquire atomics inspired by their operational semantics, showing the viability of this technique even for very complex memory models.

The idea of augmenting traces leads naturally to *games*, which we can view one variant of as sequential traces of moves between a *proponent* and an *opponent*. Game semantics were first used with great success to give a semantics to *sequential computation*; for example, Abramsky and McCusker [1] give an adequate denotational semantics for sequential Algol using Hyland-Ong

games [34]. Ghica and Murawski [25] observe that the sequentiality of Hyland-Ong games corresponds to a highly constrained, deterministic form of interleaving between concurrent processes. By generalizing away many of the rules of Hyland-Ong games, and in particular *alternation* (i.e., the proponent and the opponent must take turns), the authors obtain a form of game-semantics for concurrent programs, which they use to build a fully abstract semantics for *fine-grained concurrency* (in contrast to Brookes [12], which implements *coarse-grained* concurrency using the somewhat unrealistic await primitive).

In general, generalizing traces to represent true concurrency leads us to another approach: replacing sets of *linear* traces with (sets of) *partially ordered* structures that directly represent the concurrency of their component operations. This approach directly captures the idea that executing two operations concurrently (i.e., without specifying an order between them) is fundamentally different from executing them in some particular order. The most basic such data structure is a *partially ordered multiset*, or *pomset*, which we describe in Section 6.3 of our paper. Pomsets can naturally model sequential consistency, and, like traces, can be augmented with additional structure to model relaxed memory models such as TSO weak memory, as in Kavanagh and Brookes [38].

One issue with this approach is that it can be very challenging to determine the appropriate structures to augment pomsets with in order to model more advanced memory models. For example, Jagadeesan et al. [35] introduce *pomsets with preconditions* (PwP), which augment pomsets with logical formulae, to model weak memory behaviors. However, Jeffrey et al. [37] demonstrate that sequential composition (the semicolon operator) in PwP is not associative. This lack of associativity undermines many common program optimizations and breaks the monadic structure necessary for compositional reasoning. To address this issue, Jeffrey et al. [37] propose *pomsets with predicate transformers* (PwT), which restore the associativity of sequential composition. Despite this improvement, their semantics still do not support loops, and developing a monadic structure based on PwT that fully supports recursion and iterative constructs remains future work.

Another potential generalization of pomsets, inspired by game semantics, is the use of *event structures*, which introduce a *conflict relation* to represent many potentially conflicting executions within a single mathematical object. Castellan [13] show how to use event structures to represent concurrent executions with respect to a memory model. They study a simple language supporting parallel composition of n linear programs defined as lists of loads, stores, and arithmetic operations, for which event structures provide a denotational semantics. Paviotti et al. [53] use the event structure approach to give denotational semantics for relaxed memory concurrency in a more realistic language, including semantics for branches and loops. However, they employ step indexing in their semantics, and as a result, loop unrolling is only a refinement rather than an equation. Since event structures satisfy the axioms of axiomatic domain theory [20], it should be possible to modify this semantics into one where loop unrolling is an equation, at which point it would be a model of our calculus.

7.5 Future Work

7.5.1 Substructural Types and Effects. Currently, our treatment of effects is relatively primitive, in that, while we postulate a lattice of effects, our equational theory only distinguishes between *pure* and *impure* functions, the former having effect $\perp$. Führmann [22] studies languages with semantics given in an *abstract Kleisli category*. In particular, he introduces the notion of

- *Central* operations, which commute with all other operations; we like to call such morphisms *linear*
- *Duplicable* operations f , for which let $y = f(x)$; $(y, y) \approx (f(x), f(x))$
- *Discardable* operations f , for which let $y = f(x)$; $e \approx e$ when e does not depend on y

While, as we make use of in our calculus, pure operations are central, duplicable, and discardable, it can be useful to study each notion in isolation, as well as to consider morphisms which are central and duplicable (which we have taken to calling *relevant*) and morphisms which are both central and discardable (which we have taken to calling *affine*). Examples of morphisms which are not pure but nonetheless relevant or affine abound; for example,

- In a programming language with nondeterminism, nondeterministic functions are *impure* (since let $y = f(x)$; (y, y) is a strict refinement of $(f(x), f(x))$), but are nonetheless *affine*, because discarding the result of a nondeterministic function does not affect the program's behavior, assuming they have no other effect.
- In a programming language with nontermination, nonterminating functions with no other effect are impure (since let $y = f(x)$; e may diverge even if e does not) but are always duplicable, because duplicating a nonterminating function does not introduce new effects beyond divergence. Depending on whether the language's other effects commute with nontermination, they may in fact be relevant.

All three of these concepts still make perfect sense when interpreted as-is in a Freyd category, and are particularly useful when reasoning about models of probabilistic programming, such as Markov categories (which have many affine morphisms) [52].

Given that such relevant and affine side-effects have substitution principles which depend on how often a variable is used in a term, we call such side-effects *substructural*. We might also consider how we could support substructural, and in particular linear, *types*. Categorically, this corresponds to weakening the requirement that our base category be cartesian, instead requiring only a symmetric monoidal category. The resulting generalization of a Freyd category is called an *effectful category* by Román [57].

7.5.2 Refinement and Enrichment. Throughout this work, we have focused on the notion of program equivalence $\Gamma \vdash r \approx r' \triangleright L$. In many settings, especially when considering the nondeterminism and concurrency, we would also like to study program *refinement* $\Gamma \vdash r \sqsubseteq r' \triangleright L$. Semantically, this corresponds to an enrichment of our model in the category of partial orders; we conjecture that adding this to our semantics would require minimal changes. We may also consider how our syntax could be extended to support explicit parallelism and how we could reflect the corresponding algebraic laws (as given in, e.g., Hoare and van Staden [32]), such as

$$(P \parallel Q); (R \parallel S) \sqsubseteq (P; R) \parallel (Q; S) \quad (78)$$

in our equational theory. More generally, we can consider enrichment with further structures, such as distributive lattices and dcpos, and the language features these correspond to, such as nondeterministic choice between programs $P \vee C$. These features are particularly interesting from the perspective of programs as *specifications*, as they offer the potential for representing complex specifications within SSA.

7.5.3 Guarded Iteration. Elgot categories and monads were introduced in Elgot [19], and subsequently formalized in Adámek et al. [2]. Levy and Goncharov [46] generalize Elgot categories to *guarded Elgot categories* (and, correspondingly, *guarded Elgot monads*, whose Kleisli categories are guarded Elgot), to support partial non-unique iteration – in other words, categories where the fixed point does not always exist. Generalizing our language with support for guarded iteration could allow us to study applications of SSA to domains for which not all fixpoints exist, such as the representation of terminating languages (as, for example, one would find in a proof assistant) or productive processes.

Goncharov et al. [27] introduce a metalanguage for guarded iteration based on labelled iteration as presented by Geron and Levy [24]. Interestingly, they refer to the labels in their systems as *exceptions*, making the claim that this more closely matches their semantics. Similar to our work, they provide a denotational semantics for this language using the Kleisli category of a strong guarded *pre-iterative* monad T —that is, a monad with a guarded fixpoint operator—on a distributive category C . They then demonstrate that this semantics is sound and adequate with respect to a big-step operational semantics for a specific monad, namely $T X = (X \times \mathbb{N}^*) + \mathbb{N}^\omega$ over Set . Notably, unlike our trace monad, their monad is *guarded iterative* rather than merely iterative. The infinite branch of the coproduct $\mathbb{N}^\omega$ represents an infinite sequence of natural numbers, effectively prohibiting non-productive infinite loops — that is, recursion that does not pass through a *guard*, which in this context is emitting a natural number. Their treatment of iteration is thus somewhat more general than ours, as they support guarded iteration and require only a pre-iterative monad. However, they focus on Kleisli categories rather than providing a general treatment for Freyd categories. Additionally, their language supports functional types, whereas our language is purely first-order. We are very interested in exploring the connection between guarded labelled-iteration and SSA implied by the similarities between our respective syntaxes and semantics.

Acknowledgements

This work was supported in part by a European Research Council (ERC) Consolidator Grant for the project “TypeFoundry”, funded under the European Union’s Horizon 2020 Framework Programme (grant agreement no. 101002277).

References

- [1] Samson Abramsky and Guy McCusker. 1996. Linearity, Sharing and State: a fully abstract game semantics for Idealized Algol with active expressions. In *Linear Logic Tokyo Meeting 1996, Keio University, Mita Campus, Tokyo, Japan, March 29 - April 2, 1996 (Electronic Notes in Theoretical Computer Science, Vol. 3)*, Jean-Yves Girard, Mitsuhiro Okada, and Andre Scedrov (Eds.). Elsevier, Tokyo, Japan, 2–14. [https://doi.org/10.1016/S1571-0661\(05\)80398-6](https://doi.org/10.1016/S1571-0661(05)80398-6)
- [2] Jiri Adámek, Stefan Milius, and Jiri Velebil. 2011. Elgot theories: a new perspective on the equational properties of iteration. *Math. Struct. Comput. Sci.* 21, 2 (2011), 417–480. <https://doi.org/10.1017/S0960129510000496>
- [3] Frances E. Allen. 1970. Control flow analysis. *SIGPLAN Not.* 5, 7 (jul 1970), 1–19. <https://doi.org/10.1145/390013.808479>
- [4] B. Alpern, M. N. Wegman, and F. K. Zadeck. 1988. Detecting Equality of Variables in Programs. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (San Diego, California, USA) (POPL ’88)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/73560.73561>
- [5] Andrew W. Appel. 1998. SSA is Functional Programming. *ACM SIGPLAN Notices* 33, 4 (1998), 17–20. <https://doi.org/10.1145/278283.278285>
- [6] Gilles Barthe, Delphine Demange, and David Pichardie. 2012. A Formally Verified SSA-Based Middle-End - Static Single Assignment Meets CompCert. In *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7211)*, Helmut Seidl (Ed.). Springer, Tallinn, Estonia, 47–66. https://doi.org/10.1007/978-3-642-28869-2_3
- [7] Gilles Barthe, Delphine Demange, and David Pichardie. 2014. Formal Verification of an SSA-Based Middle-End for CompCert. *ACM Trans. Program. Lang. Syst.* 36, 1, Article 4 (mar 2014), 35 pages. <https://doi.org/10.1145/2579080>
- [8] Mark Batty. 2017. Compositional relaxed concurrency. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 375, 2104 (2017), 20150406. <https://doi.org/10.1098/rsta.2015.0406> arXiv:<https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.2015.0406>
- [9] Nick Benton and Andrew Kennedy. 1999. Monads, Effects and Transformations. *Electronic Notes in Theoretical Computer Science* 26 (1999), 3–20. [https://doi.org/10.1016/S1571-0661\(05\)80280-4](https://doi.org/10.1016/S1571-0661(05)80280-4) HOOTS ’99, Higher Order Operational Techniques in Semantics.
- [10] Siddharth Bhat, Alex Keizer, Chris Hughes, Andrés Goens, and Tobias Grosser. 2024. Verifying Peephole Rewriting in SSA Compiler IRs. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:20. <https://doi.org/10.4230/LIPIcs.ITP.2024.9>

- [11] Corrado Böhm and Giuseppe Jacopini. 1966. Flow diagrams, turing machines and languages with only two formation rules. *Commun. ACM* 9, 5 (May 1966), 366–371. <https://doi.org/10.1145/355592.365646>
- [12] Stephen D. Brookes. 1996. Full Abstraction for a Shared-Variable Parallel Language. *Inf. Comput.* 127, 2 (1996), 145–163. <https://doi.org/10.1006/INCO.1996.0056>
- [13] Simon Castellan. 2016. Weak memory models using event structures. In *Vingt-septièmes Journées Francophones des Langages Applicatifs (JFLA 2016)*, Julien Signoles (Ed.). Saint-Malo, France. <https://inria.hal.science/hal-01333582>
- [14] Manuel M. T. Chakravarty, Gabriele Keller, and Patryk Zadarnowski. 2003. A Functional Perspective on SSA Optimisation Algorithms. In *Compiler Optimization Meets Compiler Verification, COCV@ETAPS 2003, Warsaw, Poland, April 12, 2003 (Electronic Notes in Theoretical Computer Science, Vol. 82)*, Jens Knoop and Wolf Zimmermann (Eds.). Elsevier, Warsaw, Poland, 347–361. [https://doi.org/10.1016/S1571-0661\(05\)82596-4](https://doi.org/10.1016/S1571-0661(05)82596-4)
- [15] Nicolas Chappel, Ludovic Henrio, and Yannick Zakowski. 2025. Monadic Interpreters for Concurrent Memory Models: Executable Semantics of a Concurrent Subset of LLVM IR. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs (Denver, CO, USA) (CPP '25)*. Association for Computing Machinery, New York, NY, USA, 283–298. <https://doi.org/10.1145/3703595.3705890>
- [16] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1991. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.* 13, 4 (oct 1991), 451–490. <https://doi.org/10.1145/115372.115320>
- [17] Delphine Demange, David Pichardie, and Léo Stefanesco. 2015. Verifying Fast and Sparse SSA-Based Optimizations in Coq. In *Compiler Construction - 24th International Conference, CC 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9031)*, Björn Franke (Ed.). Springer, London, UK, 233–252. https://doi.org/10.1007/978-3-662-46663-6_12
- [18] Yotam Dvir, Ohad Kammar, and Ori Lahav. 2024. A Denotational Approach to Release/Acquire Concurrency. In *Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14577)*, Stephanie Weirich (Ed.). Springer, Luxembourg City, Luxembourg, 121–149. https://doi.org/10.1007/978-3-031-57267-8_5
- [19] Calvin C. Elgot. 1975. Monadic Computation And Iterative Algebraic Theories**The material reported on here evolved from research which was initiated at the University of Bristol and supported by the Science Research Council of Great Britain. In *Logic Colloquium '73*, H.E. Rose and J.C. Shepherdson (Eds.). Studies in Logic and the Foundations of Mathematics, Vol. 80. Elsevier, Amsterdam, The Netherlands, 175–230. [https://doi.org/10.1016/S0049-237X\(08\)71949-9](https://doi.org/10.1016/S0049-237X(08)71949-9)
- [20] Marcelo P. Fiore. 1994. *Axiomatic domain theory in categories of partial maps*. Ph.D. Dissertation. University of Edinburgh, UK. <https://hdl.handle.net/1842/406>
- [21] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN'93 Conference on Programming Language Design and Implementation (PLDI, Albuquerque, New Mexico, USA, June 23-25, 1993)*, Robert Cartwright (Ed.). ACM, USA, 237–247. <https://doi.org/10.1145/155090.155113>
- [22] Carsten Führmann. 1999. Direct Models for the Computational Lambda Calculus. In *Fifteenth Conference on Mathematical Foundations of Programming Semantics, MFPS 1999, Tulane University, New Orleans, LA, USA, April 28 - May 1, 1999 (Electronic Notes in Theoretical Computer Science, Vol. 20)*, Stephen D. Brookes, Achim Jung, Michael W. Mislove, and Andre Scedrov (Eds.). Elsevier, USA, 245–292. [https://doi.org/10.1016/S1571-0661\(04\)80078-1](https://doi.org/10.1016/S1571-0661(04)80078-1)
- [23] Dmitri Garbuzov, William Mansky, Christine Rizkallah, and Steve Zdancewicz. 2018. Structural Operational Semantics for Control Flow Graph Machines. *CoRR* abs/1805.05400 (2018), 27 pages. [arXiv:1805.05400](https://arxiv.org/abs/1805.05400) <https://arxiv.org/abs/1805.05400>
- [24] Bram Geron and Paul Blain Levy. 2016. Iteration and Labelled Iteration. In *The Thirty-second Conference on the Mathematical Foundations of Programming Semantics, MFPS 2016, Carnegie Mellon University, Pittsburgh, PA, USA, May 23-26, 2016 (Electronic Notes in Theoretical Computer Science, Vol. 325)*, Lars Birkedal (Ed.). Elsevier, USA, 127–146. <https://doi.org/10.1016/J.ENTCS.2016.09.035>
- [25] Dan R. Ghica and Andrzej S. Murawski. 2008. Angelic semantics of fine-grained concurrency. *Ann. Pure Appl. Log.* 151, 2-3 (2008), 89–114. <https://doi.org/10.1016/J.APAL.2007.10.005>
- [26] Sergey Goncharov. 2021. Uniform Elgot Iteration in Foundations. *CoRR* abs/2102.11828 (2021), 41 pages. [arXiv:2102.11828](https://arxiv.org/abs/2102.11828) <https://arxiv.org/abs/2102.11828>
- [27] Sergey Goncharov, Christoph Rauch, and Lutz Schröder. 2021. A metalanguage for guarded iteration. *Theor. Comput. Sci.* 880 (2021), 111–137. <https://doi.org/10.1016/J.TCS.2021.04.005>
- [28] Sergey Goncharov and Lutz Schröder. 2018. Guarded Traced Categories. In *Foundations of Software Science and Computation Structures - 21st International Conference, FOSSACS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10803)*, Christel Baier and Ugo Dal Lago (Eds.). Springer, Thessaloniki, Greece, 313–330.

- https://doi.org/10.1007/978-3-319-89366-2_17
- [29] Sergey Goncharov, Lutz Schröder, Christoph Rauch, and Julian Jakob. 2018. Unguarded Recursion on Coinductive Resumptions. *Log. Methods Comput. Sci.* 14, 3 (2018). [https://doi.org/10.23638/LMCS-14\(3:10\)2018](https://doi.org/10.23638/LMCS-14(3:10)2018)
- [30] Masahito Hasegawa. 2002. The Uniformity Principle on Traced Monoidal Categories. In *Category Theory and Computer Science, CTCS 2002, Ottawa, Canada, August 15-17, 2002 (Electronic Notes in Theoretical Computer Science, Vol. 69)*, Richard Blute and Peter Selinger (Eds.). Elsevier, Amsterdam, The Netherlands, 137–155. [https://doi.org/10.1016/S1571-0661\(04\)80563-2](https://doi.org/10.1016/S1571-0661(04)80563-2)
- [31] Yann Herklotz, Delphine Demange, and Sandrine Blazy. 2023. Mechanised Semantics for Gated Static Single Assignment. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs (Boston, MA, USA) (CPP 2023)*. Association for Computing Machinery, New York, NY, USA, 182–196. <https://doi.org/10.1145/3573105.3575681>
- [32] Tony Hoare and Stephan van Staden. 2014. The laws of programming unify process calculi. *Sci. Comput. Program.* 85 (2014), 102–114. <https://doi.org/10.1016/j.SCICO.2013.08.012>
- [33] Baojian Hua, Bo Xu, and Ying Gao. 2010. Explicitly typed static single-assignment form. In *2010 2nd International Conference on Education Technology and Computer*, Vol. 1. IEEE, Shanghai, China, V1–43–V1–47. <https://doi.org/10.1109/ICETC.2010.5529303>
- [34] J. M. E. Hyland and C.-H. Luke Ong. 2000. On Full Abstraction for PCF: I, II, and III. *Inf. Comput.* 163, 2 (2000), 285–408. <https://doi.org/10.1006/INCO.2000.2917>
- [35] Radha Jagadeesan, Alan Jeffrey, and James Riely. 2020. Pomsets with preconditions: a simple model of relaxed memory. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 194:1–194:30. <https://doi.org/10.1145/3428262>
- [36] Radha Jagadeesan, Gustavo Petri, and James Riely. 2012. Brookes Is Relaxed, Almost!. In *Foundations of Software Science and Computational Structures - 15th International Conference, FOSSACS 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7213)*, Lars Birkedal (Ed.). Springer, Tallinn, Estonia, 180–194. https://doi.org/10.1007/978-3-642-28729-9_12
- [37] Alan Jeffrey, James Riely, Mark Batty, Simon Cooksey, Ilya Kaysin, and Anton Podkopaev. 2022. The leaky semicolon: compositional semantic dependencies for relaxed-memory concurrency. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–30. <https://doi.org/10.1145/3498716>
- [38] Ryan Kavanagh and Stephen Brookes. 2018. A Denotational Semantics for SPARC TSO. *Electronic Notes in Theoretical Computer Science* 336 (2018), 223–239. <https://doi.org/10.1016/j.entcs.2018.03.025> The Thirty-third Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIII).
- [39] Richard Kelsey. 1995. A Correspondence between Continuation Passing Style and Static Single Assignment Form. In *Proceedings ACM SIGPLAN Workshop on Intermediate Representations (IR'95), San Francisco, CA, USA, January 22, 1995*, Michael D. Ernst (Ed.). ACM, USA, 13–23. <https://doi.org/10.1145/202529.202532>
- [40] Søren B. Lassen and Paul Blain Levy. 2007. Typed Normal Form Bisimulation. In *Computer Science Logic, 21st International Workshop, CSL 2007, 16th Annual Conference of the EACSL, Lausanne, Switzerland, September 11-15, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4646)*, Jacques Duparc and Thomas A. Henzinger (Eds.). Springer, Lausanne, Switzerland, 283–297. https://doi.org/10.1007/978-3-540-74915-8_23
- [41] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization (Palo Alto, California) (CGO '04)*. IEEE Computer Society, USA, 75.
- [42] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (Virtual Event, Republic of Korea) (CGO '21)*. IEEE Press, Piscataway, NJ, USA, 2–14. <https://doi.org/10.1109/CGO51591.2021.9370308>
- [43] Roland Leißa, Marcel Köster, and Sebastian Hack. 2015. A graph-based higher-order intermediate representation. In *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2015, San Francisco, CA, USA, February 07 - 11, 2015*, Kunle Olukotun, Aaron Smith, Robert Hundt, and Jason Mars (Eds.). IEEE Computer Society, San Francisco, CA, USA, 202–212. <https://doi.org/10.1109/CGO.2015.7054200>
- [44] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (2009), 107–115. <https://doi.org/10.1145/1538788.1538814>
- [45] Paul Blain Levy. 2004. *Call-By-Push-Value: A Functional/Imperative Synthesis*. Semantics Structures in Computation, Vol. 2. Springer, USA.
- [46] Paul Blain Levy and Sergey Goncharov. 2019. Coinductive Resumption Monads: Guarded Iterative and Guarded Elgot. In *8th Conference on Algebra and Coalgebra in Computer Science, CALCO 2019, June 3-6, 2019, London, United Kingdom (LIPIcs, Vol. 139)*, Markus Roggenbach and Ana Sokolova (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik,

- London, UK, 13:1–13:17. <https://doi.org/10.4230/LIPICS.CALCO.2019.13>
- [47] Paul Blain Levy, John Power, and Hayo Thielecke. 2003. Modelling environments in call-by-value programming languages. *Inf. Comput.* 185, 2 (2003), 182–210. [https://doi.org/10.1016/S0890-5401\(03\)00088-9](https://doi.org/10.1016/S0890-5401(03)00088-9)
- [48] Liyi Li and Elsa L. Gunter. 2020. K-LLVM: A Relatively Complete Semantics of LLVM IR. In *34th European Conference on Object-Oriented Programming (ECOOP 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 166)*, Robert Hirschfeld and Tobias Pape (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 7:1–7:29. <https://doi.org/10.4230/LIPIcs.ECOOP.2020.7>
- [49] Yutaka Matsuno and Atsushi Ohori. 2006. A type system equivalent to static single assignment. In *Proceedings of the 8th International ACM SIGPLAN Conference on Principles and Practice of Declarative Programming, July 10-12, 2006, Venice, Italy*, Annalisa Bossi and Michael J. Maher (Eds.). ACM, Venice, Italy, 249–260. <https://doi.org/10.1145/1140335.1140365>
- [50] Vijay Menon, Neal Glew, Brian R. Murphy, Andrew McCreight, Tatiana Shpeisman, Ali-Reza Adl-Tabatabai, and Leaf Petersen. 2006. A verifiable SSA program representation for aggressive compiler optimization. In *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2006, Charleston, South Carolina, USA, January 11-13, 2006*, J. Gregory Morrisett and Simon L. Peyton Jones (Eds.). ACM, USA, 397–408. <https://doi.org/10.1145/1111037.1111072>
- [51] Eugenio Moggi. 1991. Notions of Computation and Monads. *Inf. Comput.* 93, 1 (1991), 55–92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)
- [52] nLab authors. 2024. Markov category. <https://ncatlab.org/nlab/show/Markov+category>. Revision 62.
- [53] Marco Paviotti, Simon Cooksey, Anouk Paradis, Daniel Wright, Scott Owens, and Mark Batty. 2020. Modular Relaxed Dependencies in Weak Memory Concurrency. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, Dublin, Ireland, 599–625. https://doi.org/10.1007/978-3-030-44914-8_22
- [54] Sebastian Pop, Pierre Jouvelot, and George André Silber. 2009. In and Out of SSA : a Denotational Specification. In *Workshop Static Single-Assignment Form Seminar*. Universität des Saarlandes, Atrants, France, 15 pages. <https://minesparis-psl.hal.science/hal-00915979>
- [55] John Power and Edmund Robinson. 1997. Premonoidal Categories and Notions of Computation. *Math. Struct. Comput. Sci.* 7, 5 (1997), 453–468. <https://doi.org/10.1017/S0960129597002375>
- [56] Leonardo Filipe Rigon, Paulo Torrens, and Cristiano D. Vasconcellos. 2020. Inferring types and effects via static single assignment. In *SAC '20: The 35th ACM/SIGAPP Symposium on Applied Computing, online event, [Brno, Czech Republic], March 30 - April 3, 2020*, Chih-Cheng Hung, Tomás Cerný, Dongwan Shin, and Alessio Bechini (Eds.). ACM, Brno, Czech Republic, 1314–1321. <https://doi.org/10.1145/3341105.3373888>
- [57] Mario Román. 2023. Promonads and String Diagrams for Effectful Categories. *Electronic Proceedings in Theoretical Computer Science* 380 (aug 2023), 344–361. <https://doi.org/10.4204/eptcs.380.20>
- [58] B. K. Rosen, M. N. Wegman, and F. K. Zadeck. 1988. Global Value Numbers and Redundant Computations. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (San Diego, California, USA) (POPL '88)*. Association for Computing Machinery, New York, NY, USA, 12–27. <https://doi.org/10.1145/73560.73562>
- [59] Cranelift Development Team. 2023. Cranelift: A Simple, Fast WebAssembly Code Generator. <https://github.com/bytecodealliance/wasmtime/tree/main/cranelift>
- [60] Mark N. Wegman and F. Kenneth Zadeck. 1991. Constant Propagation with Conditional Branches. *ACM Trans. Program. Lang. Syst.* 13, 2 (1991), 181–210. <https://doi.org/10.1145/103135.103136>
- [61] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction trees: representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. <https://doi.org/10.1145/3371119>
- [62] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction trees: representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. <https://doi.org/10.1145/3371119>
- [63] Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewic. 2012. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, John Field and Michael Hicks (Eds.). ACM, Philadelphia, Pennsylvania, 427–440. <https://doi.org/10.1145/2103656.2103709>

A Records and Enums

For simplicity, we defined our language to have only binary products and sums. However, we would like to support records (named n -ary products) and enums (named n -ary sums). We will implement these on top of our language using contexts; the resulting machinery will turn out to be crucial in proving the Böhm-Jacopini theorem in Appendix A.1 and completeness in Section 5.5. We begin by defining *packing* of variable contexts as follows:

$$\langle \cdot \rangle = 1 \quad \langle \Gamma, x : A \rangle = \langle \Gamma \rangle \otimes A \quad (79)$$

While in general we destructure pairs using binary let-bindings, it will be more convenient to start defining operations on records in terms of projections. We begin by defining projections on pairs in the obvious manner:

$$\frac{\Gamma \vdash_e e : A \otimes B}{\Gamma \vdash_e \pi_l e := (\text{let } (x, y) = e; x) : A} \pi_l \quad \frac{\Gamma \vdash_e e : A \otimes B}{\Gamma \vdash_e \pi_r e := (\text{let } (x, y) = e; y) : A} \pi_r \quad (80)$$

We may then define projections on records as follows:

$$\pi_{(\Delta, x:A), y} e = \pi_{\Delta, y} (\pi_l e) \quad \pi_{(\Delta, x:A), x} e = \pi_r e \quad (81)$$

with typing rule

$$\frac{\Gamma \vdash_e e : \langle \Delta \rangle \quad \Delta(x) = A}{\Gamma \vdash_e \pi_{\Delta, x} e : A} \pi_{\text{rec}} \quad (82)$$

We will omit Δ when it is clear from context. We can define a term $\Gamma \vdash_{\perp} \text{packed}(\Gamma) : [\Gamma]$ to “pack” up the current context into a record as follows:

$$\text{packed}(\cdot) = () \quad \text{packed}(\Gamma, x : A) = (\text{packed}(\Gamma), x) \quad (83)$$

We may now define packing and unpacking substitutions $\text{pack}_y^{\otimes}(\Gamma) : \Gamma \mapsto y : \langle \Gamma \rangle$, $\text{unpack}_y^{\otimes}(\Gamma) : y : \langle \Gamma \rangle \mapsto \Gamma$ as follows:

$$\text{pack}_y^{\otimes}(\Gamma) = y \mapsto \text{packed}(\Gamma) \quad \text{unpack}_y^{\otimes}(\Gamma) = (x \mapsto \pi_{\Gamma, x} y)_{x \in \Gamma} \quad (84)$$

Our equational theory is sufficient to show that the pack and unpack substitutions are inverses of each other:

$$\begin{aligned} [\text{unpack}_y^{\otimes}(\Gamma)] \text{pack}_y^{\otimes}(\Gamma) &\approx \text{id}_{\Gamma} : \Gamma \mapsto \Gamma \\ [\text{pack}_y^{\otimes}(\Gamma)] \text{unpack}_y^{\otimes}(\Gamma) &\approx \text{id}_{y: [\Gamma]} : y : \Gamma \mapsto y : \Gamma \end{aligned} \quad (85)$$

Similarly, we may define a “packing” operation $\langle \cdot \rangle$ on label-contexts as follows:

$$\langle \cdot \rangle = \mathbf{0} \quad \langle L, \ell(A) \rangle = \langle L \rangle + A \quad (86)$$

We may then define injections on enums as follows:

$$l_{(L, \ell(A)), \kappa} e = l_{L, \kappa} (l_l e) \quad l_{(L, \ell(A)), \ell} e = l_r e \quad (87)$$

with typing rule

$$\frac{\Gamma \vdash_e e : A \quad L(\ell) = A}{\Gamma \vdash_e l_{L, \ell} e : \langle L \rangle} l_{\text{enum}} \quad (88)$$

Similarly, we can define n -ary case-statements on enums by induction as follows:

$$\begin{aligned} \text{case. } e \{ \} &= \infty \\ \text{case}_{L, \kappa(A)} e \{ (\ell_i(x_i) : t_i)_i, \kappa(y) : s \} &= \text{case } e \{ l_l x : \text{case}_L x \{ (\ell_i(A_i) : t_i)_i \}, l_r y : s \} \end{aligned} \quad (89)$$

with typing rule

$$\frac{\Gamma \vdash_e e : [L] \quad \forall i. \Gamma, x_i : A_i \vdash t_i \triangleright K}{\Gamma \vdash \text{case}_L e \{ (\ell_i(x_i) : t_i)_i \} \triangleright K} \text{case}_{\text{enum}} \quad (90)$$

where

$$\frac{}{\Gamma \vdash \infty := \text{br } \ell \text{ } () \text{ where } \ell(x) : \{\text{br } \ell \text{ } x\} \triangleright L} \infty \quad (91)$$

We may now define “packing” and “unpacking” label-substitutions $\cdot \vdash \text{pack}_\kappa^+(L) : L \rightsquigarrow \kappa([L])$, $\cdot \vdash \text{unpack}_\kappa^+(L) : \kappa([L]) \rightsquigarrow L$ as follows:

$$\text{pack}_\kappa^+(\cdot) = \cdot \quad \text{pack}_\kappa^+(L, \ell(A)) = [\kappa(x) \mapsto \text{br } \kappa \text{ } \iota_l \text{ } x] \text{pack}_\kappa^+(L), \ell(x) \mapsto \text{br } \kappa \text{ } \iota_r \text{ } x \quad (92)$$

$$\text{unpack}_\kappa^+(L) = \kappa(x) \mapsto \text{unpack}^+(L) \text{ } x \quad (93)$$

where we have

$$\frac{\Gamma \vdash_\epsilon e : L}{\Gamma \vdash \text{unpack}^+(L) \text{ } e \triangleright L} \text{unpack}^+ \quad (94)$$

defined as follows:

$$\text{unpack}^+(\cdot) \text{ } e = \infty \quad \text{unpack}^+(L, \ell(A)) \text{ } e = \text{case } e \{ \iota_l \text{ } x : \text{unpack}^+(L) \text{ } x, \iota_r \text{ } y : \text{br } \ell \text{ } y \} \quad (95)$$

We can further show that, in this case for all label contexts L , these substitutions are mutually inverse, i.e., that

$$\begin{aligned} \Gamma \vdash [\text{unpack}_\kappa^+(L)] \text{pack}_\kappa^+(L) &\approx \text{id}_L : L \rightsquigarrow L \\ \Gamma \vdash [\text{pack}_\kappa^+(L)] \text{unpack}_\kappa^+(L) &\approx \text{id}_{\kappa([L])} : \kappa([L]) \rightsquigarrow \kappa([L]) \end{aligned} \quad (96)$$

Finally, fixing a distinguished variable $\square$, it will be very useful to define a “*variable packing*” operation on expressions and regions as follows:

$$\Gamma \vdash r \triangleright L \implies \square : [\Gamma] \vdash \langle r \rangle^\otimes := [\text{unpack}_\square^\otimes(\Gamma)] r \triangleright L \quad (97)$$

In particular, for Γ pure, the packing operation $[\cdot]$ is an injection on expressions and regions w.r.t. the equational theory, since $\text{unpack}_\square^\otimes(\Gamma)$ has an inverse. Similarly, fixing a distinguished label $\blacksquare$, we may define a “*label packing*” operation on regions as follows:

$$\Gamma \vdash r \triangleright L \implies \Gamma \vdash \langle r \rangle^+ := [\text{pack}_\blacksquare^+(L)] r \triangleright \blacksquare([L]) \quad (98)$$

Since the packing substitution has an inverse, it similarly follows that label packing is an injection w.r.t. the equational theory, i.e.

$$\Gamma \vdash r \approx r' \triangleright L \iff \Gamma \vdash \langle r \rangle^+ \approx \langle r' \rangle^+ \triangleright L \quad (99)$$

Finally, we can define a “*packing*” operation on regions to be given by label-packing followed by variable-packing, or vice versa (since it turns out the operations commute), as follows:

$$\Gamma \vdash r \triangleright L \implies \square : [\Gamma] \vdash \langle r \rangle := \langle \langle r \rangle^+ \rangle^\otimes = \langle \langle r \rangle^\otimes \rangle^+ \triangleright \blacksquare(L) \quad (100)$$

We similarly have that this is an injection for Γ pure, i.e.

$$\Gamma \vdash r \approx r' \triangleright L \iff \Gamma \vdash \langle r \rangle \approx \langle r' \rangle \triangleright L \quad (101)$$

A.1 Böhm-Jacopini for SSA

Now that we have given our equational theory, we want to show it is “good enough” to reason about the properties inherent to all SSA-based languages. In particular, we wish to show that we have enough power to reason about interconversion between data-flow and control-flow. For example, a state machine can be implemented either as a switch on a state value, or as a set of mutually-tail-recursive functions (i.e., the state can be encoded in the program counter). We demonstrate this by using some of the machinery from the previous section to state and prove a form of the Böhm-Jacopini theorem [11] for SSA.

The Böhm-Jacopini theorem states that every general control-flow graph program can be rewritten to an equivalent program which uses only structured control-flow: i.e., it can be rewritten to a

program using only conditional branching, sequencing, and loops. To adapt this result to SSA, we need to express branching, sequencing and loops as SSA regions $\Gamma \vdash r \triangleright L$, so that we can build up an inductive set of structured regions. We will be maximally strict, and allow branching to an exit label in L only as the terminal statement in a structured program (and, in particular, not from within a loop!). It is obvious that we can represent conditional branching using case-statements, but we need convenient primitives for sequencing and looping. Sequencing can be expressed using a where-block as follows (with ℓ fresh):

$$\frac{\Gamma \vdash r \triangleright \blacksquare(A) \quad \Gamma, \square : A \vdash s \triangleright L}{\Gamma \vdash \text{seq}(r, s) := ([\blacksquare(x) \mapsto \text{br } \ell x]r \text{ where } \ell(\square) : \{s\}) \triangleright L}^{\text{seq}} \quad (102)$$

where “ $\square$ ” is a distinguished input variable, and “ $\blacksquare$ ” is a distinguished output label. Another, equivalent, way of writing sequencing is:

$$\Gamma \vdash \text{seq}(r, s) \approx [\blacksquare(\square) \mapsto s]r \triangleright L \quad (103)$$

That is, when we exit r , we jump to (a copy of) s .

Expressing structured looping is a bit more complicated, since we do not have mutable variables and hence cannot directly express while loops. If we recall that loops can be expressed as tail-recursive procedures which carry the loop state in the argument, then we can define a “functional do-while loop” as follows:

$$\frac{\Gamma \vdash_e e : A \quad \Gamma, \square : A \vdash r \triangleright \blacksquare(B + A)}{\Gamma \vdash \text{loop}(e, r) := (\text{br } \ell e \text{ where } \ell(\square) : \{\text{seq}(r, \text{case } \square \{ \iota_l x : \text{br } \blacksquare x, \iota_r y : \text{br } \ell y \} \}) \triangleright \blacksquare(B))}^{\text{loop}} \quad (104)$$

We can now define the inductive predicate $\Gamma \vdash^s r \triangleright L$, which says that r is a structured region with input variables Γ and output labels L , as in Figure 34. Note that this defines a subset of the well-typed regions: every region r such that $\Gamma \vdash^s r \triangleright L$ is also well-typed as $\Gamma \vdash r \triangleright L$.

It now remains to give an algorithm to convert a region r targeting label-context L into an equivalent structured region $\text{WH}_L(r)$. In particular, we define

$$\text{WH}_L(r) = [\text{unpack}_{\blacksquare}^+(L)]\text{PW}_L(r) \quad (105)$$

where we define

$$\text{PW}_L(\text{br } \ell a) = \text{br } \blacksquare \iota_{L, \ell} a$$

$$\text{PW}_L(\text{let } x = a; r) = \text{let } x = a; \text{PW}_L(r)$$

$$\text{PW}_L(\text{let } (x, y) = a; r) = \text{let } (x, y) = a; \text{PW}_L(r)$$

$$\text{PW}_L(\text{case } e \{ \iota_l x : r, \iota_r y : s \}) = \text{case } e \{ \iota_l x : \text{PW}_L(r), \iota_r y : \text{PW}_L(s) \}$$

$$\text{PW}_L(r \text{ where } (\ell_i(x_i) : \{t_i\})_i) = \text{seq}(\text{PW}_L(r), \text{case } \text{ua } \square \{ \iota_l x : \text{br } \blacksquare x, \iota_r y : \text{loop}(y, \text{case}_R \square \{ \ell_i(x_i) : \text{seq}(\text{PW}_L(t_i), \text{br } \blacksquare (\text{ua } \square)) \}) \})$$

$$\text{where } R = (\ell_i(A_i))_i$$

(106)

and

$$\text{ua}_{L, \cdot} e = e \quad \text{ua}_{L, (R, \ell(A))} e = \text{case } e \{ \iota_l x : \text{case } \text{ua}_{L, R} x \{ \iota_l z : \iota_l z, \iota_r w : \iota_r \iota_l w \}, \iota_r y : \iota_r (\iota_r y) \} \quad (107)$$

Note that the base case $\text{PW}_L(\text{br } \ell a)$ is The $\text{PW}_L(r)$ function does the actual transformation, and we can see that most of the cases are trivial except for:

- The base case, in which we simply replace a branch $\text{br } \ell a$ with a branch to the output label $\blacksquare$ with the appropriate injection $\iota_{L, \ell} a$ as parameter.

$$\begin{array}{c}
\frac{\Gamma \vdash_{\perp} a : A \quad \mathbb{L} \ell = A}{\Gamma \vdash^S \text{br } \ell \ a \triangleright \mathbb{L}} \text{ s-br} \quad \frac{\Gamma \vdash_{\epsilon} a : A \quad \Gamma, x : A \vdash^S r \triangleright \mathbb{L}}{\Gamma \vdash^S \text{let } x = a; r \triangleright \mathbb{L}} \text{ s-let}_1\text{-r} \\
\frac{\Gamma \vdash_{\epsilon} e : A \otimes B \quad \Gamma, x : A, y : B \vdash^S r \triangleright \mathbb{L}}{\Gamma \vdash^S \text{let } (x, y) = e; r \triangleright \mathbb{L}} \text{ s-let}_2\text{-r} \\
\frac{\Gamma \vdash_{\epsilon} e : A + B \quad \Gamma, x : A \vdash^S r \triangleright \mathbb{L} \quad \Gamma, y : B \vdash^S s \triangleright \mathbb{L}}{\Gamma \vdash^S \text{case } e \{ \iota_l x : r, \iota_r y : s \} \triangleright \mathbb{L}} \text{ s-case-r} \\
\frac{\Gamma \vdash^S r \triangleright \blacksquare(A) \quad \Gamma, \square : A \vdash^S s \triangleright \mathbb{L}}{\Gamma \vdash^S \text{seq}(r, s) \triangleright \mathbb{L}} \text{ s-seq} \quad \frac{\Gamma \vdash_{\epsilon} e : \blacksquare(A) \quad \Gamma, \square : A \vdash^S r \triangleright \blacksquare(B + A)}{\Gamma \vdash^S \text{loop}(e, r) \triangleright \blacksquare(B)} \text{ s-loop}
\end{array}$$

Fig. 34. Typing rules for structured regions

- The r where $(\ell_i(x_i) : \{t_i\})_i$ case, in which we replace the set of labels bound by the where clause with a tag and a while loop containing a case statement branching on the tag. (This uses the ua function, which implements associativity of n -ary coproducts. That is, given $\Gamma \vdash_{\perp} e : [L, R]$, we have that $\Gamma \vdash_{\perp} \text{ua}_L e : [L] + [R]$.)

The Böhm-Jacopini theorem for SSA can then be written:

THEOREM A.1 (BÖHM-JACOPINI FOR SSA). *For all $\Gamma \vdash r \triangleright \mathbb{L}$, we have that $\Gamma \vdash^S \text{WH}_L(r) \triangleright \mathbb{L}$ is structured, and $\Gamma \vdash r \approx \text{WH}_L(r) \triangleright \mathbb{L}$. In particular, we have that $\Gamma \vdash^S \text{PW}_L(r) \triangleright \blacksquare(\langle L \rangle)$ is structured, and $\Gamma \vdash \langle r \rangle^+ \approx \text{PW}_r(\triangleright) \blacksquare(\langle L \rangle)$.*

PROOF. See Appendix C.3 □

B The Environment Comonad

If C is a Freyd category, then given an object $R \in |C|$, the functor $R \otimes \cdot$ is a comonad with counit $\pi_r : R \otimes A \rightarrow A$ and comultiplication $\Delta_R \otimes A; \alpha : R \otimes A \rightarrow R \otimes (R \otimes A)$, often called the *environment comonad* or *coreader comonad*. In Set , the co-Kleisli category of $R \otimes \cdot$ is isomorphic to the Kleisli category of the reader monad $R \rightarrow \cdot$, and thus can be equipped with the structure of a distributive Elgot category. We might therefore intuit that, in general, if C is a distributive Elgot category, the co-Kleisli category of $R \otimes \cdot$ has this structure, even if e.g. C lacks exponentials. The rest of this section is dedicated to proving this, which, beyond providing yet another class of λ_{SSA} models, will also give us a some useful equations and notation to use in our proofs later in the appendix. We begin by giving an explicit definition:

Definition B.1 (Environment Comonad). Given a Freyd category C and an object (the *environment*) $R \in |C|$, the *environment comonad* $C_{R \otimes \cdot}$ is given by the functor $A \mapsto R \otimes A$ and has counit π_r and comultiplication $\Delta_R \otimes A$. It follows that composition of co-Kleisli morphisms $f : R \otimes A \rightarrow B$, $g : R \otimes B \rightarrow C$ is given by

$$f \gg_R g := \Delta_R; \alpha; R \otimes f; g : R \otimes A \rightarrow C \quad (108)$$

In particular, we can define an identity on objects functor $\text{env}_R : C \rightarrow C_{R \otimes \cdot}$ as follows:

$$\text{env}_R(f) := \pi_r; f \quad (109)$$

Where it is clear from context, we will leave out the environment R .

Given $f : R \otimes A \rightarrow B$, we will introduce the syntax sugar

$$\text{rlet}(f) := \Delta_R; \alpha; R \otimes f : R \otimes A \rightarrow R \otimes B \quad (110)$$

In particular, we have that $f \gg g = \text{rlet}(f); g$ and $\text{rlet}(\text{env}(f)) := R \otimes f$. It is trivial to verify (for example, by drawing string diagrams) that

$$\begin{aligned} \text{rlet}(\text{rlet}(f); g) &= \text{rlet}(f); \text{rlet}(g) \implies f \gg (g \gg h) = (f \gg g) \gg h \\ \text{rlet}(f); \pi_r = f &\implies f \gg \pi_r = f \quad \text{rlet}(\pi_r) = \text{id} \implies \pi_r \gg f = f \end{aligned} \quad (111)$$

and hence that $C_{R \otimes}$ is indeed a category. To show it is in fact a *Freyd category*, we can define tensor functors

$$\begin{aligned} f \otimes_R X &:= \alpha^{-1}; f \otimes X : R \otimes (A \otimes X) \rightarrow B \otimes X \\ X \otimes_R f &:= R \otimes \sigma; f \otimes_R X; \sigma : R \otimes (X \otimes A) \rightarrow X \otimes B \end{aligned} \quad (112)$$

We can define

- Associators $\alpha_{A,B,C}^R := \text{env}_R(\alpha_{A,B,C})$
- Unitors $\lambda_A^R := \text{env}_R(\lambda_A)$ and $\rho_A^R := \text{env}_R(\rho_A)$
- Symmetries $\sigma_{A,B}^R := \text{env}_R(\sigma_{A,B})$
- Projections $\pi_l^R := \text{env}_R(\pi_l)$ and $\pi_r^R := \text{env}_R(\pi_r)$
- Terminal morphisms $!_A^R := \text{env}_R(!_A) = !_{R \otimes A}$
- Diagonals $\Delta_A^R := \text{env}_R(\Delta_A)$
- Pure morphisms $C_{R \otimes \cdot \perp}(A, B) = C_\perp(R \otimes A, B)$

LEMMA B.2. *We can always equip $C_{R \otimes}$ with the structure of a Freyd category as described above ; furthermore, $\text{env}_R(\cdot)$ strictly preserves premonoidal structure.*

PROOF. We have that

$$\begin{aligned} \text{env}_R(f \otimes A) &= \pi_r; f \otimes A = \alpha^{-1}; (\pi_r; f) \otimes A = \text{env}_R(f) \otimes_R A \\ \text{env}_R(A \otimes f) &= \pi_r; A \otimes f = R \otimes \sigma; \alpha^{-1}; (\pi_r; f) \otimes A; \sigma = A \otimes_R \text{env}_R(f) \end{aligned} \quad (113)$$

and hence that env_R preserves products. Since env_R is a functor, to check we indeed have a Freyd category, it suffices to show that:

- $\alpha_{A,B,C}^R$ is natural: we have that, given $f : R \otimes A \rightarrow A'$, $g : R \otimes B \rightarrow B'$, $h : R \otimes C \rightarrow C'$

$$(f \otimes_R B) \otimes_R C \gg \alpha_{A',B,C}^R = \alpha_{A,B,C}^R \gg f \otimes_R (B \otimes C) \quad (114)$$

$$(A \otimes_R g) \otimes C \gg \alpha_{A,B',C}^R = \alpha_{A,B,C}^R \gg A \otimes_R (g \otimes_R C) \quad (115)$$

$$(A \otimes B) \otimes_R h \gg \alpha_{A,B,C'}^R = \alpha_{A,B,C}^R \gg A \otimes_R (B \otimes_R h) \quad (116)$$

$$(117)$$

since, for each equation, both sides are equal to the same string diagram in Figure 35a, 35b, and 35c respectively.

- λ_A^R is natural: we have that, for $f : R \otimes A \rightarrow A'$,

$$\begin{aligned} f \otimes_R 1 &\gg \lambda_{A'}^R = \Delta_R; \alpha; R \otimes (\alpha^{-1}; f \otimes 1); \pi_r; \pi_l \\ &= R \otimes \pi_l; f \\ &= \Delta_R; \alpha; R \otimes (\pi_r; \pi_l); f = \lambda_A^R \gg f \end{aligned} \quad (118)$$

In general, we note that, for all $g : R \otimes A \rightarrow A' \otimes 1$, $g \gg \lambda^R = g; \pi_l$.

- ρ_A^R is natural:

$$\begin{aligned} 1 \otimes_R f &\gg \rho_{A'}^R = \Delta_R \otimes (1 \otimes A); \alpha; R \otimes (R \otimes \sigma; \alpha^{-1}; f \otimes 1; \sigma); \pi_r; \pi_r \\ &= R \otimes \pi_r; f \\ &= \Delta_R; \alpha; R \otimes (\pi_r; \pi_r); f = \rho_A^R \gg f \end{aligned} \quad (119)$$

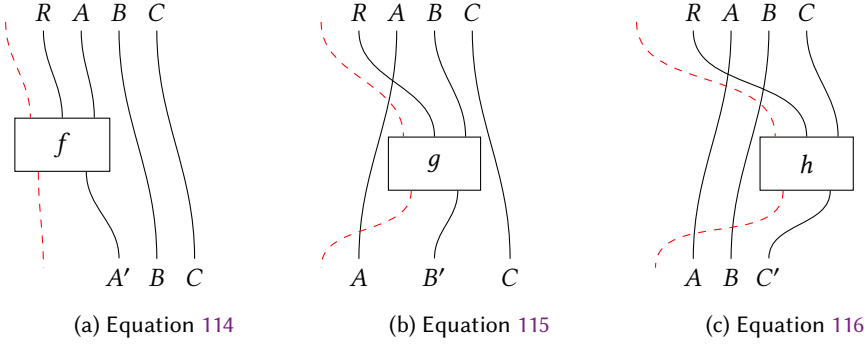


Fig. 35. Naturality of the associator in the co-Kleisli category of the environment comonad

In general, we note that, for all $g : R \otimes A \rightarrow 1 \otimes A'$, $g \gg \rho^R = g; \pi_r$.

- $\sigma_{A,B}^R$ is natural: given $f : R \otimes A \rightarrow A'$ and $g : R \otimes B \rightarrow B'$, we have that

$$\begin{aligned}
 f \otimes_R B \gg \sigma_{A',B}^R &= \Delta_R \otimes (A \otimes B); \alpha; R \otimes (\alpha^{-1}; f \otimes B); \pi_r; \sigma \\
 &= \alpha^{-1}; f \otimes B; \sigma \\
 &= R \otimes \sigma; R \otimes f \otimes B; \sigma \\
 &= \Delta_R \otimes (B \otimes A); R \otimes (\pi_r; \sigma); R \otimes \sigma; \alpha^{-1}; f \otimes B; \sigma = \sigma_{A,B}^R \gg B \otimes_R f
 \end{aligned} \tag{120}$$

and

$$\begin{aligned}
 A \otimes g \gg \sigma_{A,B'}^R &= \Delta_R \otimes (A \otimes B); \alpha; R \otimes (R \otimes \sigma; \alpha^{-1}; g \otimes A; \sigma); \pi_r; \sigma \\
 &= R \otimes \sigma; \alpha^{-1}; g \otimes A; \sigma \\
 &= \Delta_R \otimes (A \otimes B); \alpha; R \otimes (\pi_r; \sigma); \alpha^{-1}; g \otimes A = \sigma_{A,B}^R \gg g \otimes_R A
 \end{aligned} \tag{121}$$

- Pure morphisms are central: given $f : R \otimes A \rightarrow A'$ and $g : R \otimes B \rightarrow B'$ pure, we have that

$$\begin{aligned}
 f \otimes_R B \gg A' \otimes_R g &= \Delta_R; \alpha; R \otimes (\alpha^{-1}; f \otimes B; \sigma); \alpha^{-1}; g \otimes A'; \sigma \\
 &= \Delta_R; \alpha; R \otimes (R \otimes \sigma; \alpha^{-1}; g \otimes A; \sigma); \alpha^{-1}; f \otimes B' \\
 &= A \otimes_R g \gg f \otimes_R B'
 \end{aligned} \tag{122}$$

since both sides correspond to the diagram in Figure 36. We will write this as $f \otimes_R g$.

- $!_A^R$ is terminal for pure morphisms: yes, since $!_A^R$ is just $!_{R \otimes A}$ which is terminal in C
- Δ_A^R duplicates pure morphisms: we have

$$\begin{aligned}
 f \gg \Delta_B^R &= \Delta_R; \alpha; R \otimes f; \pi_r; \Delta_{A'} \\
 &= f; \Delta_{A'} = \Delta_{R \otimes A}; f \otimes f \\
 &= \Delta_R; \alpha; R \otimes (\alpha^{-1}; f \otimes A'); R \otimes \sigma; \alpha^{-1}; f \otimes A'; \sigma = \Delta_A^R \gg f \otimes_R f
 \end{aligned} \tag{123}$$

□

Now, assume C is distributive. We wish to show that $A+B$ is a coproduct in $C_{R \otimes}$, and furthermore that $C_{R \otimes}$ is distributive; note that even the former may not be the case without distributivity! We proceed as follows:

LEMMA B.3. *If C is a distributive Freyd category, then $C_{R \otimes}$ is also distributive Freyd*

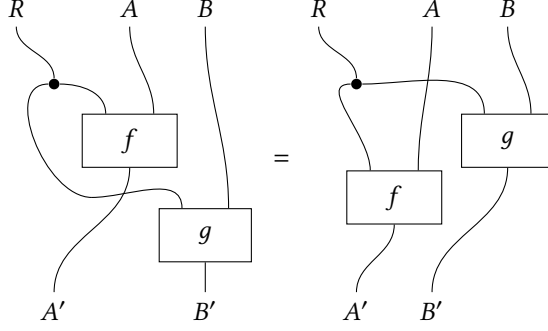


Fig. 36. Centrality of pure morphisms in the environment comonad's co-Kleisli category

PROOF. Given $f : R \otimes A \rightarrow B$ and $g : R \otimes B \rightarrow C$, we may define the coproduct and injections

$$[f, g]_R := \delta^{-1}; [f, g] : R \otimes A \rightarrow B + C \quad \iota_l^R := \pi_r; \iota_r : R \otimes A \rightarrow A + B \quad \iota_r^R := \pi_r; \iota_r : R \otimes B \rightarrow A + B \quad (124)$$

To verify this is indeed a coproduct, we can check that

$$\begin{aligned} \iota_l^R \gg [f, g]_R &= \Delta_R; \alpha; R \otimes (\pi_r; \iota_r); \delta^{-1}; [f, g] = R \otimes \iota_r; \delta^{-1}; [f, g] = f \\ \iota_r^R \gg [f, g]_R &= \Delta_R; \alpha; R \otimes (\pi_r; \iota_l); \delta^{-1}; [f, g] = R \otimes \iota_l; \delta^{-1}; [f, g] = g \end{aligned} \quad (125)$$

This morphism is obviously unique, since we have for all $h : R \otimes (A + B) \rightarrow C$

$$[\iota_l^R \gg h, \iota_r^R \gg h]_R = \delta^{-1}; [\Delta_R; \alpha; R \otimes \iota_l; \pi_r; h, \Delta_R; \alpha; R \otimes \iota_r; \pi_r; h] = \delta^{-1}; [\cancel{R \otimes \iota_r}, R \otimes \iota_l]; h \quad (126)$$

To show that $R_{C \otimes}$ is indeed distributive, we need $\delta^{R-1} := \text{env}_R(\delta^{-1}) = \pi_r; \delta^R$ to be the inverse to $\delta^R := [A \otimes \iota_l^R, A \otimes \iota_r^R]_R$, which can easily be derived from the functoriality of $\text{env}_R(\cdot)$, since

$$\text{env}_R(\delta) = \pi_r; [\iota_l; \iota_r] = \delta^{-1}; [\pi_r; \iota_l, \pi_r; \iota_r] = \delta^R \quad (127)$$

□

Note in particular that this allows us to define sums

$$\begin{aligned} f +_R g &= [f \gg \iota_l^R, g \gg \iota_r^R]_R \\ &= \delta^{-1}; [\Delta_R \otimes (A + B); \alpha; R \otimes f; \pi_r; \iota_l, \Delta_R \otimes (A + B); \alpha; R \otimes g; \pi_r; \iota_r] = \delta^{-1}; f + g \end{aligned} \quad (128)$$

Our final task is to show that, if C is a strong Elgot category, then so is $R_{C \otimes}$, and, in particular, with fixpoint operator $\text{rfix}(f)$. Note that, just like we needed distributivity to have coproducts at all, we will need strength to have an Elgot structure. We begin by stating some generally useful properties of $\text{rcase}(f)$ and $\text{rfix}(f)$. In particular, we have that: For $f : R \otimes A \rightarrow B + C$:

- Given $h : R \otimes X \rightarrow A$, we have

$$\begin{aligned} \text{rlet}(h); \text{rcase}(f) &= \Delta_R \otimes X; \alpha; R \otimes h; \Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1} \\ &= \Delta_R \otimes X; \alpha; R \otimes (\Delta_R \otimes X; \alpha; R \otimes h; f); \delta^{-1} \\ &= \text{rcase}(\text{rlet}(h); f) = \text{rcase}(h \gg f) \end{aligned} \quad (129)$$

- Given $g : R \otimes B \rightarrow X$, $h : R \otimes C \rightarrow Y$, we have

$$\begin{aligned}
 \text{rcase}(f \gg g +_R h) &= \text{rlet}(f \gg g +_R h); \delta^{-1} \\
 &= \text{rlet}(f); \text{rlet}(g +_R h); \delta^{-1} \\
 &= \text{rlet}(f); \Delta_R \otimes (B + C); \alpha; R \otimes (\delta^{-1}; g + h); \delta^{-1} \\
 &= \text{rlet}(f); \Delta_R \otimes (B + C); \alpha; R \otimes \delta^{-1}; \delta^{-1}; (R \otimes g) + (R \otimes h) \\
 &= \text{rlet}(f); \delta^{-1}; \text{rlet}(g) + \text{rlet}(h) \\
 &= \text{rcase}(f); \text{rlet}(g) + \text{rlet}(h) \\
 &= \text{rlet}(f); \text{rlet}(g) +_R \text{rlet}(h)
 \end{aligned} \tag{130}$$

Similarly, for $f : R \otimes A \rightarrow B + A$:

- Given $h : R' \rightarrow_{\perp} R$, $h \otimes A; \text{rfix}(f) = \text{rfix}(h \otimes A; f)$: we have that

$$\begin{aligned}
 h \otimes A; \text{rcase}(f) &= h \otimes A; \Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1} \\
 &= \Delta_{R'} \otimes A; \alpha; h \otimes (h \otimes A; f); \delta^{-1} \\
 &= \Delta_{R'} \otimes A; \alpha; R \otimes (h \otimes A; f); \delta^{-1}; h \otimes B + h \otimes A \\
 &= \text{rcase}(h \otimes A; f); h \otimes B + h \otimes A
 \end{aligned} \tag{131}$$

and hence by uniformity that

$$\begin{aligned}
 h \otimes A; \text{rfix}(f) &= h \otimes A; (\text{rcase}(f))^{\dagger}; \pi_r \\
 &= (\text{rcase}(h \otimes A; f); h \otimes B + R' \otimes A)^{\dagger}; \pi_r \\
 &= (\text{rcase}(h \otimes A; f))^{\dagger}; h \otimes B; \pi_r = \text{rfix}(h \otimes A; f)
 \end{aligned} \tag{132}$$

- $\text{rlet}(\text{rfix}(f)) = (\text{rcase}(f))^{\dagger}$: we have that

$$\begin{aligned}
 &(\Delta_R \otimes A; \alpha); (R \otimes \text{rcase}(f); \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A)) \\
 &= \Delta_R \otimes A; \alpha; R \otimes (\text{let}(f); \pi_l \otimes (B + A); \delta^{-1}); \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A) \\
 &= \Delta_R \otimes A; \alpha; R \otimes f; \Delta_R \otimes (B + A); \alpha; R \otimes \delta^{-1}; \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A) \\
 &= \Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; (\Delta_R \otimes A; \alpha; R \otimes \pi_r) + (\Delta_R \otimes A; \alpha) \\
 &= \Delta_{R \otimes A}; (R \otimes A) \otimes f; \pi_l \otimes (B + A); \delta^{-1}; R \otimes A + (\Delta_R \otimes A; \alpha) \\
 &= \text{rcase}(f); R \otimes A + (\Delta_R \otimes A; \alpha)
 \end{aligned} \tag{133}$$

It follows by uniformity and strength that

$$\begin{aligned}
 \text{rlet}(\text{rfix}(f)) &= \text{let}(\text{rfix}(f)); \pi_l \otimes B = \Delta_{R \otimes A}; (R \otimes A) \otimes \text{rfix}(f); \pi_l \otimes B \\
 &= \Delta_R \otimes A; \alpha; R \otimes \text{rfix}(f) \\
 &= \Delta_R \otimes A; \alpha; R \otimes ((\text{rcase}(f))^{\dagger}; \pi_r) \\
 &= \Delta_R \otimes A; \alpha; (R \otimes \text{rcase}(f); \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A))^{\dagger} \\
 &= (\text{rcase}(f))^{\dagger}
 \end{aligned} \tag{134}$$

as desired.

We also state some generally useful properties of $\text{rlet}(f)$:

- For all $f : R \otimes A \rightarrow C, g : R \otimes B \rightarrow C$,

$$\begin{aligned}
 \text{rlet}(\delta^{-1}; [f, g]) &= \Delta_R \otimes (A + B); \alpha; R \otimes (\delta^{-1}; [f, g]) \\
 &= \delta^{-1}; [\Delta_R \otimes A; \alpha; f, \Delta_R \otimes B; \alpha; g] \\
 &= \delta^{-1}; [\text{rlet}(f), \text{rlet}(g)]
 \end{aligned} \tag{135}$$

We may now state our desired result as follows:

LEMMA B.4. *If C is a premonoidal strong Elgot category, then so is $C_{R \otimes}$ with fixpoint operator $\text{rfix}(f)$.*

PROOF. We check each of the Elgot axioms as follows:

- *Fixpoint*: given $f : R \otimes A \rightarrow B + A$, we have that

$$\begin{aligned}
 \text{rfix}(f) &= (\text{rcase}(f))^{\dagger}; \pi_r = (\text{rlet}(f); \delta^{-1})^{\dagger}; \pi_r \\
 &= \text{rlet}(f); \delta^{-1}; [\text{id}_{R \otimes B}, (\text{rlet}(f); \delta^{-1})^{\dagger}]; \pi_r \\
 &= \text{rlet}(f); \delta^{-1}; [\pi_r, (\text{rcase}(f))^{\dagger}; \pi_r] = f \gg [\pi_r, \text{rfix}(f)]_R
 \end{aligned} \tag{136}$$

as desired.

- *Naturality*: given $f : R \otimes A \rightarrow B + A, g : R \otimes B \rightarrow C$, we have that

$$\begin{aligned}
 \text{rfix}(f \gg g +_R A) &= (\text{rlet}(\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; g + \pi_r); \delta^{-1})^{\dagger}; \pi_r \\
 &= (\text{rlet}(\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}); \delta^{-1}; R \otimes g + R \otimes \pi_r)^{\dagger}; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}); \delta^{-1}; (\pi_r; g) + R \otimes \pi_r)^{\dagger} \\
 &= (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; (\Delta_R \otimes B; \alpha; \pi_r; g) + (\Delta_R \otimes A; \alpha; R \otimes \pi_r))^{\dagger} \\
 &= (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; g + R \otimes A)^{\dagger} \\
 &= (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1})^{\dagger}; g = (\text{rcase}(f))^{\dagger}; g
 \end{aligned} \tag{137}$$

On the other hand, we have that

$$\begin{aligned}
 \text{rfix}(f) \gg g &= \Delta_R \otimes A; \alpha; R \otimes (\text{rcase}(f); \pi_r + R \otimes A)^{\dagger}; g \\
 &= \Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1})^{\dagger}; g \\
 &= \Delta_R \otimes A; \alpha; (R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \pi_r + R \otimes A); \delta^{-1})^{\dagger}; g \\
 &= \Delta_R \otimes A; \alpha; (R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}); \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A))^{\dagger}; g
 \end{aligned} \tag{138}$$

By uniformity, it hence suffices to show that

$$\begin{aligned}
 &(\Delta_R \otimes A; \alpha); (R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}); \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A)) \\
 &= \Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; (\Delta_R \otimes B; \alpha; R \otimes \pi_r) + (\Delta_R \otimes A; \alpha) \\
 &= \text{rcase}(f); (R \otimes B) + (\Delta_R \otimes A; \alpha)
 \end{aligned} \tag{139}$$

to yield the desired result.

- *Codiagonal*: given $f : R \otimes A \rightarrow (B + A) + A$, we have

$$\begin{aligned}
 & \text{rfix}(\text{rfix}(f)) \\
 &= (\Delta_R \otimes A; \alpha; R \otimes ((\text{rcase}(f))^\dagger; \pi_r); \delta^{-1})^\dagger; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; R \otimes (\text{rcase}(f); \pi_r + R \otimes A)^\dagger; \delta^{-1})^\dagger; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1})^\dagger; \delta^{-1})^\dagger; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1}; \delta^{-1} + R \otimes (R \otimes A))^\dagger)^\dagger; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1}; \delta^{-1} + R \otimes (R \otimes A))^\dagger; \pi_r + R \otimes A)^\dagger \\
 &= (\Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1}; (\delta^{-1}; (\pi_r + R \otimes A)) + R \otimes (R \otimes A))^\dagger)^\dagger
 \end{aligned} \tag{140}$$

On the other hand, we have

$$\begin{aligned}
 \text{rfix}(f \gg [\pi_r, i_l^R]_R) &= \text{rfix}(\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; [\pi_r, \pi_r; i_l]) \\
 &= \text{rfix}(f; [\text{id}, i_l]) \\
 &= (\Delta_R \otimes A; \alpha; R \otimes (f; [\text{id}, i_l]); \delta^{-1})^\dagger; \pi_r \\
 &= (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \delta^{-1} + R \otimes A; [\text{id}, i_l])^\dagger; \pi_r \\
 &= ((\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \delta^{-1} + R \otimes A)^\dagger)^\dagger; \pi_r \\
 &= ((\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \delta^{-1} + R \otimes A)^\dagger; \pi_r + R \otimes A)^\dagger \\
 &= ((\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \delta^{-1} + R \otimes A; (\pi_r + R \otimes A) + R \otimes A)^\dagger)^\dagger
 \end{aligned} \tag{141}$$

By uniformity, it therefore suffices to show that

$$\begin{aligned}
 & (\Delta_R \otimes A; \alpha; (R \otimes (\text{rcase}(f); \pi_r + R \otimes A); \delta^{-1}; (\delta^{-1}; (\pi_r + R \otimes A)) + R \otimes (R \otimes A)) \\
 &= \Delta_R \otimes A; \alpha; R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \pi_r + R \otimes A); \delta^{-1}; (\delta^{-1}; (\pi_r + R \otimes A)) + R \otimes (R \otimes A) \\
 &= \Delta_R \otimes A; \alpha; R \otimes (\Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}); \\
 &\quad \delta^{-1}; R \otimes \pi_r + R \otimes (R \otimes A); (\delta^{-1}; (\pi_r + R \otimes A)) + R \otimes (R \otimes A) \\
 &= \Delta_R \otimes A; \alpha; R \otimes f; \delta^{-1}; \delta^{-1} + (R \otimes A); (\pi_r + R \otimes A) + (\Delta_R \otimes A; \alpha)
 \end{aligned} \tag{142}$$

to obtain the desired result.

- *Uniformity*: Given $f : R \otimes A \rightarrow B + A$, $g : R \otimes X \rightarrow B + X$ and $h : R \otimes X \rightarrow_\perp A$ such that $h \gg f = g \gg B +_R h$, we have that

$$\begin{aligned}
 \text{rlet}(h); \text{rcase}(f) &= \text{rcase}(h \gg f) \\
 &= \text{rcase}(g \gg B +_R h) \\
 &= \text{rcase}(g); (R \otimes B) + \text{rlet}(h)
 \end{aligned} \tag{143}$$

and hence by uniformity that

$$h \gg \text{rfix}(f) = \text{rlet}(h); (\text{rcase}(f))^\dagger; \pi_r = (\text{rcase}(g))^\dagger; \pi_r = \text{rfix}(g) \tag{144}$$

□

C Definitions and Proofs

C.1 Syntactic Metatheory

$$(\gamma, x \mapsto e)(x) = e \quad (\gamma, y \mapsto e)(x) = \gamma(x) \quad (\cdot)(x) = x$$

$$\begin{aligned} [\gamma]x &= \gamma(x) & [\gamma](\text{let } x = a; e) &= \text{let } x = [\gamma]a; [\gamma]e & [\gamma](a, b) &= ([\gamma]a, [\gamma]b) & [\gamma]() &= () \\ [\gamma](\text{let } (x, y) = a; e) &= \text{let } (x, y) = [\gamma]a; [\gamma]e & [\gamma](\iota_l a) &= \iota_l [\gamma]a & [\gamma](\iota_r b) &= \iota_r [\gamma]b \\ [\gamma](\text{case } e \{ \iota_l x : a, \iota_r y : b \}) &= \text{case } [\gamma]e \{ \iota_l x : [\gamma]a, \iota_r y : [\gamma]b \} \\ [\gamma](\text{abort } a) &= \text{abort } [\gamma]a \end{aligned}$$

$$\begin{aligned} [\gamma](\text{br } \ell a) &= \text{br } \ell [\gamma]a & [\gamma](\text{let } x = a; r) &= \text{let } x = [\gamma]a; [\gamma]r \\ [\gamma](\text{let } (x, y) = e; r) &= \text{let } (x, y) = [\gamma]e; [\gamma]r \\ [\gamma](\text{case } e \{ \iota_l x : r, \iota_r y : s \}) &= \text{case } [\gamma]e \{ \iota_l x : [\gamma]r, \iota_r y : [\gamma]s \} \\ [\gamma](r \text{ where } (\ell_i(x_i) : \{t_i\},)_i) &= [\gamma]r \text{ where } (\ell_i(x_i) : \{[\gamma]t_i\},)_i \end{aligned}$$

$$[\gamma](\cdot) = \cdot \quad [\gamma](\gamma', x \mapsto e) = ([\gamma]\gamma', x \mapsto [\gamma]e)$$

$$[\gamma](\cdot) = \cdot \quad [\gamma](\sigma, \ell(x) \mapsto r) = ([\gamma]\sigma, \ell(x) \mapsto [\gamma]r)$$

Fig. 37. Capture-avoiding substitution for λ_{SSA} terms, regions, and (label) substitutions; in particular, we assume bound variables and labels are α -converted so as not to appear in γ/σ .

C.2 Lexical SSA and ANF

LEMMA 4.4 (A-NORMALIZATION). *Given an arbitrary region r , we have that*

- $\text{IsANF}(\text{ANF}(r))$
- *If $\Gamma \vdash r \triangleright L$, then $\Gamma \vdash r \approx \text{ANF}(r) \triangleright L$*

Similarly, given an arbitrary variable x , expression a and region r If we are also given an arbitrary expression a , then

- $\text{IsANF}(r) \implies \text{IsANF}(\text{ANF}_{\text{let}}(x, a, r))$
- *If $\Gamma \vdash_{\epsilon} a : A$ and $\Gamma, x : A \vdash r \triangleright L$, then $\Gamma \vdash \text{let } x = a; r \approx \text{ANF}_{\text{let}}(x, a, r) \triangleright L$*

PROOF. We may show that $\text{ANF}(r)$ is always in ANF and that $\text{ANF}_{\text{let}}(x, a, r)$ is in ANF if r is by a straightforward induction. To prove the rest of the lemma, we begin by proving the correctness of $\text{ANF}_{\text{let}}(x, a, r)$ by induction on expressions a :

- If $\Gamma \vdash_{\epsilon}^{\text{anf}} a : A$ is atomic, we are done; otherwise
- If $a = f e$, then, since e is a subterm of a , by induction, we have

$$\text{let } x = a; r \approx \text{let } y = e; \text{let } x = f y; r \approx \text{ANF}_{\text{let}}(y, e, \text{let } x = f y; r) = \text{ANF}_{\text{let}}(x, a, r) \quad (145)$$

as desired.

- The cases for pairs, injections, and aborts containing expressions are analogous

- If $a = \text{case } e \{ \iota_l y : b, \iota_r z : c \}$, then we may rewrite

$$\text{let } x = a; r \approx \text{let } w = e; \text{case } w \{ \iota_l y : \text{let } b = x; r, \iota_r z : \text{let } c = x; r \} \quad (146)$$

Since r is in ANF, by induction (since b, c are subterms of a), this is equivalent to

$$\text{let } w = e; \text{case } w \{ \iota_l y : \text{ANF}_{\text{let}}(b, x, r), \iota_r z : \text{ANF}_{\text{let}}(c, x, r) \} \quad (147)$$

which is equal to $\text{ANF}_{\text{let}}(x, a, r)$, as desired.

- The cases for unary and binary let are analogous to the above

We may now prove the correctness of $\text{ANF}(r)$ by a straightforward induction on r :

- If $r = \text{br } \ell \ a$, by β -reduction, we have that

$$r \approx \text{let } x = a; \text{br } \ell \ x \approx \text{ANF}_{\text{let}}(x, a, \text{br } \ell \ x) = \text{ANF}(r) \quad (148)$$

- If $r = \text{let } x = a; r'$, then by induction we have that

$$r \approx \text{let } x = a; \text{ANF}(r') \approx \text{ANF}_{\text{let}}(x, a, \text{ANF}(r')) = \text{ANF}(r) \quad (149)$$

as desired.

- If $r = \text{let } (x, y) = a; r'$, then by induction we have that

$$\begin{aligned} r &\approx \text{let } z = a; \text{let } (x, y) = z; r' \\ &\approx \text{let } z = a; \text{let } (x, y) = z; \text{ANF}(r') \\ &\approx \text{ANF}_{\text{let}}(z, a, \text{let } (x, y) = z; \text{ANF}(r')) \approx \text{ANF}(r) \end{aligned} \quad (150)$$

The proof for case-statements is analogous

- The case for control-flow graphs follows trivially by induction

□

LEMMA 4.5 (SSA CONVERSION). *Given an arbitrary region r ,*

- $\text{IsSSA}(\text{SSA}(r))$
- *If $\Gamma \vdash r \triangleright L$, then $\Gamma \vdash r \approx \text{SSA}(r) \triangleright L$*

In particular,

- *If $\text{IsANF}(r)$ and $\forall i, \text{IsSSA}(t_i)$, then $\text{IsSSA}(\text{SSA}_a(r, (\ell_i(x_i) : \{t_i\},)_i))$*
- *If $\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_i$ and $\forall i, \Gamma \vdash t_i \triangleright L, (\ell_i(A_i),)_i$, then $\Gamma \vdash (r \text{ where } (\ell_i(x_i) : \{t_i\},)_i) \approx \text{SSA}_a(r, (\ell_i(x_i) : \{t_i\},)_i) \triangleright L$*

PROOF. Given that $\text{SSA}_a(r, G) \approx r$ where G for r in ANF, it is trivial to see that

$$\text{SSA}(r) := \text{SSA}_a(\text{ANF}(r), \cdot) \approx (\text{ANF}(r) \text{ where } \cdot) \approx \text{ANF}(r) \approx r \quad (151)$$

We hence only need to prove the second part of the lemma. We proceed by induction on ANF regions r as follows:

- If r is a terminator, this holds trivially by reflexivity
- If $r = \text{let } x = a; r'$, then by induction, we have that

$$\text{SSA}_a(r, G) := \text{let } x = a; \text{SSA}_a(r', G) \approx \text{let } x = a; r' \text{ where } G \approx \text{let } x = a; r' \text{ where } G \approx r \text{ where } G \quad (152)$$

as desired. The case for binary let-statements is analogous.

- If $r = \text{case } a \{ \iota_l x : s, \iota_r y : t \}$, then by induction, we have that

$$\begin{aligned} \text{SSA}_a(r, G) &:= (\text{case } a \{ \iota_l x : \text{br } \ell_l x, \iota_r y : \text{br } \ell_r y \}) \text{ where } G, \ell_l(x) : \{ \text{SSA}(s) \}, \ell_r(y) : \{ \text{SSA}(t) \} \\ &:= (\text{case } a \{ \iota_l x : \text{br } \ell_l x, \iota_r y : \text{br } \ell_r y \}) \text{ where } G, \ell_l(x) : \{ s \}, \ell_r(y) : \{ t \} \\ &\approx ((\text{case } a \{ \iota_l x : \text{br } \ell_l x, \iota_r y : \text{br } \ell_r y \}) \text{ where } \ell_l(x) : \{ s \}, \ell_r(y) : \{ t \}) \text{ where } G \\ &\approx \text{case } a \{ \iota_l x : s, \iota_r y : t \} \text{ where } G \approx r \text{ where } G \end{aligned} \tag{153}$$

- If $r = r'$ where $\ell_i(x_i) : \{ t_i \},)_i$, then by induction, we have that

$$\begin{aligned} \text{SSA}_a(r, G) &:= \text{SSA}_a(r', G, (\ell_i(x_i) : \{ \text{SSA}(t_i) \},)_i) \\ &\approx r' \text{ where } G, (\ell_i(x_i) : \{ t_i \},)_i \\ &\approx (r' \text{ where } (\ell_i(x_i) : \{ t_i \},)_i \text{ where } G \approx r \text{ where } G \end{aligned} \tag{154}$$

□

LEMMA 4.6 (PERMUTATION INVARIANCE). *If $G \simeq G'$ and $\Gamma \vdash \text{reg}(G) \triangleright L$, then $\Gamma \vdash \text{reg}(G') \triangleright L$ and $\Gamma \vdash \text{reg}(G) \approx \text{reg}(G') \triangleright L$*

PROOF. We proceed by induction on the size of G . If G consists only of an entry block, there is only one permutation, so we are done. Otherwise, assume $G = \beta, (\ell_i(x_i) : \{ t_i \},)_{i \in I}$. Furthermore, let:

- $\ell_i(x_i) : \{ \beta_i \}$ be the children of β in G in order
- $G_i = (\kappa_{i,j}(y_{i,j}) : \{ t_{i,j} \},)_{j \in I}$ be the CFG composed of the descendants of β_i in G , with β_i as entry block, in order
- $\ell'_i(x'_i) : \{ \beta'_i \}$ be the children of β in G' in order
- G'_i be the CFG composed of the descendants of β'_i in G' , with β'_i as entry block, in order

By assumption, there exists some permutation $\sigma : I \rightarrow I$ such that $G' = \beta, (\ell_{\sigma_i}(x_{\sigma_i}) : \{ t_{\sigma_i} \},)_{i \in I}$. It follows that, since the dominance relation on labels is permutation-invariant, there exists some permutation ρ such that $\forall i \in I, \ell'_i(x'_i) : \{ \beta'_i \} = \ell_{\rho_i}(x_{\rho_i}) : \{ \beta_{\rho_i} \}$, as well as permutations τ_i such that $G'_i = (\ell_{\rho_i, \tau_{ij}}(x_{\rho_i, \tau_{ij}}) : \{ t_{\rho_i, \tau_{ij}} \},)_{j \in I}$, implying in particular that $G'_i \simeq G_{\rho_i}$. By induction, we hence have that, for all i , $\text{reg}(G'_i) \approx \text{reg}(G_{\rho_i})$, and hence that

$$\begin{aligned} \text{reg}(G') &:= \text{bb}(\beta, (\ell_{\rho_i}(x_{\rho_i}) : \{ \text{reg}(G'_i) \},)_{i \in I}) \\ &\approx \text{bb}(\beta, (\ell_{\rho_i}(x_{\rho_i}) : \{ \text{reg}(G_{\rho_i}) \},)_{i \in I}) \\ &\approx \text{bb}(\beta, (\ell_i(x_i) : \{ \text{reg}(G_i) \},)_{i \in I}) \approx \text{reg}(G) \end{aligned} \tag{155}$$

□

LEMMA 4.7 (CFG CONVERSION). *Given a lexical SSA region r (i.e., assuming $\text{lsSSA}(r)$) s.t. $\Gamma \vdash r \triangleright L$, we have that $\Gamma \vdash r \approx \text{reg}(\text{cfg}(r)) \triangleright L$.*

PROOF. We will proceed by induction on the length of $G = \text{cfg}(r)$. If G consists of only an entry block β , then we trivially have that $r = \text{reg}(\text{cfg}(r))$, and so we are done. Otherwise, assume $r = \text{bb}(\beta, (\ell_i(x_i) : \{ t_i \},)_{i \in I})$. Clearly, $\text{cfg}(r)$ will have entry block β ; moreover, since every block in $\text{cfg}(r)$ other than those of the form $\text{entry}(t_i)$ can only be reached from within the region t_i (due to lexical scoping of labels), we have $\beta_i = \text{entry}(t_{\rho_i})$ for some injection ρ , where β_i are the children of β in the dominance tree of G . In particular, we can write $\{ \ell_i(x_i) : \{ \text{entry}(t_i) \}, \}_i$ as the disjoint union of:

- The immediate children of β , $\kappa_i(y_i) : \{ \beta_i \}$, where $\kappa_i = \ell_{\rho_i}$, $y_i = x_{\rho_i}$, and $\beta_i = \text{entry}(t_{\rho_i})$

- The nodes dominated by each β_i but not immediately dominated by β ; we will write the collection of such nodes for each i as $\kappa_{i,j}(y_{i,j}) : \{\beta_{i,j}\}$, where $\kappa_{i,j} = \ell_{\rho_{i,j}}$, $y_{i,j} = x_{\rho_{i,j}}$, and $\beta_{i,j} = \text{entry}(t_{\rho_{i,j}})$.

As in the algorithm to compute $\text{reg}(\cdot)$, let G_i denote the control-flow graph with entry block β_i consisting of all blocks dominated by β_i . We may write

$$G \simeq \beta, (\kappa_i(y_i) : \{G_i\})_{i \in I} \quad \forall i \in I, G_i = \beta_i, (\kappa_{i,j}(y_{i,j}) : \{\beta_{i,j}\})_{j \in I}, R_i \quad (156)$$

where R_i is the “remainder” of the control-flow graph G_i . Note the equations for G_i are actually equalities, rather than being equivalence up to permutation $\simeq$, since the $\beta_{i,j}$, appear in G before any other elements, being immediate children of β . It follows that

$$\text{reg}(\text{cfg}(r)) = \text{reg}(\text{cfg}(\text{bb}(\beta, (\ell_i(x_i) : \{t_i\})_i))) = \text{bb}(\beta, (\kappa_i(y_i) : \{\text{reg}(G_i)\})_i) \quad (157)$$

Now, define the lexical SSA regions

$$r_i = \text{bb}(\beta_i, ((\kappa_{i,j}(y_{i,j}) : \{t_{\rho_{i,j}}\})_j, \text{children}(t_i))) \quad (158)$$

It is easy to show that $\text{cfg}(r_i) \simeq G_i$, since every basic block dominated by β_i must be dominated via either some $t_{\rho_{i,j}}$ or some child of t_i . Since by induction $\text{reg}(\text{cfg}(r_i)) \approx r_i$, and by the previous lemma $\text{reg}(\text{cfg}(r_i)) \approx \text{reg}(G_i)$, it follows that

$$\text{reg}(\text{cfg}(r)) \approx \text{bb}(\beta, (\kappa_i(y_i) : \{r_i\})_i) \quad (159)$$

Define

$$T_i = (\kappa_j(y_j) : \{r_j\})_{j < i}, (\kappa_j(y_j) : \{t_j\}), (\kappa_{j,k}(y_{j,k}) : \{t_{\rho_{j,k}}\})_{k, j \geq i} \quad (160)$$

Using Equation 27, we may show that, for all i , $\text{bb}(\beta, T_{i+1}) \approx \text{bb}(\beta, T_i)$, since

- For all j , $t_{\rho_{i,j}}$ cannot use variables defined in β_i , or r would not typecheck
- For all j , if r_k or $t_{\rho_{k,j'}}$ calls $\kappa_{i,j}$, then $k = i$, or $\beta_{i,j}$ would not be dominated by β_i
- Similarly, β cannot call $\kappa_{i,j}$ or $\beta_{i,j}$ would be a direct descendant of β

We hence have by induction that

$$\text{reg}(\text{cfg}(r)) \approx \text{bb}(\beta, T_N) \approx \text{bb}(\beta, T_0) \approx r \quad (161)$$

as desired, since T_0 is a permutation of $(\ell_i(x_i) : \{t_i\})_i$. $\square$

C.3 Böhm-Jacopini

LEMMA C.1. *The following facts hold:*

- Given $\Gamma \vdash r \triangleright \blacksquare(A)$ and $\Gamma, \square : A \vdash s \triangleright L$, we have that

$$\begin{aligned} \llbracket \Gamma \vdash \text{seq}(r, s) \rrbracket &:= ([\blacksquare(x) \mapsto \text{br } \ell \ x] r \text{ where } \ell(\square) : \{s\} \triangleright L) \\ &= \text{let}(\llbracket \Gamma \vdash [\blacksquare(x) \mapsto \text{br } \ell \ x] r \triangleright \ell(A) \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_A^+; \llbracket \Gamma, \square : A \vdash s \triangleright L \rrbracket) \\ &= \text{let}(\llbracket \Gamma \vdash r \triangleright \ell(A) \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_A^+; \llbracket \Gamma, \square : A \vdash s \triangleright L \rrbracket) \end{aligned} \quad (162)$$

- Given $\Gamma \vdash_e e : A$, $\Gamma, \square : A \vdash r \triangleright \blacksquare(B + A)$, we have that

$$\begin{aligned} \llbracket \Gamma \vdash \text{loop}(e, r) \rrbracket &:= (\text{br } \ell \ e \text{ where } \ell(\square) : \{\text{seq}(r, \text{case } \square \{ \iota_l \ x : \text{br } \blacksquare \ x, \iota_r \ y : \text{br } \ell \ y \} \}) \triangleright \blacksquare(B) \rrbracket) \\ &= \text{let}(\llbracket \Gamma \vdash_e e : A \rrbracket; \text{rfix}(\llbracket \Gamma, \square : A \vdash r \triangleright \blacksquare(B + A) \rrbracket; \alpha_{B+A}^+)) \end{aligned} \quad (163)$$

- Given $R = (\ell_i(A_i))_i$ and $\Gamma, x_i : A_i \vdash t_i \triangleright L$, we have

$$\begin{aligned} \llbracket \Gamma, c : \langle R \rangle \vdash \text{case}_R \ c \ \{ \ell_i(x_i) : t_i \} \triangleright L \rrbracket \\ = \llbracket \Gamma \rrbracket \otimes \alpha_{\sum_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L \rrbracket,]_i \end{aligned} \quad (164)$$

We have that

$$\llbracket \Gamma, \square : \perp \vdash_{[L]} \text{ua } \square : \langle L \rangle + \langle R \rangle \rrbracket = \pi_r; \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+ \quad (165)$$

We have that

$$\llbracket \Gamma \vdash \text{pack}_\kappa^+(L) : L \rightsquigarrow \kappa(\langle L \rangle) \rrbracket = \pi_r; \alpha_{0+\llbracket L \rrbracket}^+ \quad (166)$$

and

$$\llbracket \Gamma \vdash \text{unpack}_\kappa^+(L) : \kappa(\langle L \rangle) \rightsquigarrow L \rrbracket = \pi_r; \alpha_{\llbracket L \rrbracket}^+ \quad (167)$$

In particular, given $\Gamma \vdash r \triangleright L$, we have that

$$\llbracket \Gamma \vdash \langle r \rangle^+ \triangleright \blacksquare(\langle L \rangle) \rrbracket = \llbracket \Gamma \vdash r \triangleright L \rrbracket; \alpha_{0+\llbracket L \rrbracket}^+ \quad (168)$$

THEOREM A.1 (BÖHM-JACOPINI FOR SSA). *For all $\Gamma \vdash r \triangleright L$, we have that $\Gamma \vdash^s \text{WH}_L(r) \triangleright L$ is structured, and $\Gamma \vdash r \approx \text{WH}_L(r) \triangleright L$. In particular, we have that $\Gamma \vdash^s \text{PW}_L(r) \triangleright \blacksquare(\langle L \rangle)$ is structured, and $\Gamma \vdash \langle r \rangle^+ \approx \text{PW}_L(r) \triangleright \blacksquare(\langle L \rangle)$.*

PROOF. We begin by showing the correctness of $\text{PW}_L(r)$ by induction on r :

- If $r = \text{br } \ell \ a$, we have that $\text{PW}_L(a) = \text{pack}^+(L)_\ell(a) = [\text{br } \ell \ a]^+$ by definition
- If $r = \text{let } x = a; s$, we have by induction that

$$\text{PW}_L(\text{let } x = a; s) = \text{let } x = a; \text{PW}_L(s) \approx \text{let } x = a; [s]^+ = [\text{let } x = a; s]^+$$

- If $r = \text{let } (x, y) = a; s$, we have by induction that

$$\text{PW}_L(\text{let } (x, y) = a; s) = \text{let } (x, y) = a; \text{PW}_L(s) \approx \text{let } (x, y) = a; \langle s \rangle^+ = \langle \text{let } (x, y) = a; s \rangle^+$$

- If $r = \text{case } a \{ \iota_l \ x : s, \iota_r \ y : t \}$, we have by induction that

$$\begin{aligned} \text{PW}_L(\text{case } a \{ \iota_l \ x : s, \iota_r \ y : t \}) &= \text{case } a \{ \iota_l \ x : \text{PW}_L(s), \iota_r \ y : \text{PW}_L(t) \} \\ &\approx \text{case } a \{ \iota_l \ x : \langle s \rangle^+, \iota_r \ y : \langle t \rangle^+ \} \\ &= \langle \text{case } a \{ \iota_l \ x : s, \iota_r \ y : t \} \rangle^+ \end{aligned}$$

- Assume $r = s$ where $(\ell_i(t_i) : \{x_i\},)_i$. Define $R = (\ell_i(A_i),)_i$. By induction, we have that $\forall i, \text{PW}_{L,R}(t_i) \approx \langle t_i \rangle^+$, and hence by soundness

$$\llbracket \Gamma, x_i : A_i \vdash \text{PW}_L(t_i) \triangleright \blacksquare(\langle L, R \rangle) \rrbracket = \llbracket \Gamma, x_i : A_i \vdash \langle t_i \rangle^+ \triangleright \blacksquare(\langle L, R \rangle) \rrbracket \quad (169)$$

Now, define

$$D = \text{case}_L \square \{ \ell_i(x_i) : \text{seq}(\text{PW}_L(t_i), \text{br } \blacksquare(\text{ua } \square)) \}$$

and $L = \text{loop}(y, D)$. It follows that

$$\begin{aligned} &\llbracket \Gamma, \square : [R] \vdash D \triangleright \blacksquare(\langle L \rangle + \langle R \rangle) \rrbracket \\ &= \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \text{seq}(\text{PW}_L(t_i), \text{br } \blacksquare(\text{ua } \square)) \triangleright \blacksquare(\langle L \rangle + \langle R \rangle) \rrbracket,]_i \\ &= \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \text{PW}_L(t_i) \triangleright \blacksquare(\langle L, R \rangle) \rrbracket; \alpha_{0+(\llbracket L \rrbracket + \llbracket R \rrbracket)}^+,]_i \\ &= \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \langle t_i \rangle^+ \triangleright \blacksquare(\langle L, R \rangle) \rrbracket; \alpha_{0+(\llbracket L \rrbracket + \llbracket R \rrbracket)}^+,]_i \\ &= \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \alpha_{0+(\llbracket L \rrbracket + \llbracket R \rrbracket)}^+,]_i \end{aligned} \quad (170)$$

and therefore that

$$\begin{aligned} &\llbracket \Gamma, y : \langle R \rangle \vdash L \triangleright \blacksquare(\langle L \rangle) \rrbracket \\ &= \text{let}(\llbracket \Gamma, y : \langle R \rangle \vdash \perp \ y : A \rrbracket; \text{rfix}(\llbracket \Gamma, \square : \langle R \rangle \vdash D \triangleright \blacksquare(\langle L \rangle + \langle R \rangle) \rrbracket; \alpha_{(0+\llbracket L \rrbracket + \llbracket R \rrbracket)}^+)) \\ &= \text{rfix}(\llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright \langle L, R \rangle \rrbracket; \alpha_{(0+\llbracket L \rrbracket + \llbracket R \rrbracket)}^+,]_i) \\ &= \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \text{rfix}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i); \alpha_{0+\llbracket L \rrbracket}^+) \end{aligned} \quad (171)$$

Hence, we have that

$$\begin{aligned}
 & \llbracket \Gamma, \square : \langle L, R \rangle \vdash \text{case ua } \square \{ \iota_l x : \text{br } \blacksquare x, \iota_r y : L \} \triangleright \blacksquare(\langle L \rangle) \rrbracket \\
 &= \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; [\llbracket \Gamma, x : \langle L \rangle \vdash \text{br } \blacksquare x \triangleright \blacksquare(\langle L \rangle) \rrbracket, \llbracket \Gamma, y : \langle R \rangle \vdash L \triangleright \blacksquare(\langle L \rangle) \rrbracket] \\
 &= \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; [\pi_r; \alpha_{0+\llbracket L \rrbracket}^+, \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}^+; \text{rfix}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)); \alpha_{0+\llbracket L \rrbracket}^+] \\
 &= \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+; \delta^{-1}; [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)); \alpha_{0+\llbracket L \rrbracket}^+]
 \end{aligned} \tag{172}$$

It hence suffices by completeness (Theorem 5.14) to show that

$$\begin{aligned}
 & \llbracket \Gamma \vdash \text{seq}(\text{PW}_L(r), \text{case ua } \square \{ \iota_l x : \text{br } \blacksquare x, \iota_r y : L \} \triangleright \blacksquare(\langle L \rangle)) \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash \text{PW}_L(r) \triangleright \blacksquare(\langle L, R \rangle) \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L, R \rrbracket}^+; \\
 & \quad \llbracket \Gamma, \square : \langle L, R \rangle \vdash \text{case ua } \square \{ \iota_l x : \text{br } \blacksquare x, \iota_r y : L \} \triangleright \blacksquare(\langle L \rangle) \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash \text{PW}_L(r) \triangleright \blacksquare(\langle L, R \rangle) \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+; \delta^{-1}; \\
 & \quad [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)); \alpha_{0+\llbracket L \rrbracket}^+] \\
 &= \llbracket \Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket; \alpha_{0+\llbracket L \rrbracket}^+ \\
 &= \llbracket \Gamma \vdash [r \text{ where } (\ell_i(x_i) : \{t_i\},)_i]^+ \triangleright L \rrbracket
 \end{aligned} \tag{173}$$

□

C.4 Substitution

LEMMA C.2. *The following facts hold:*

(a) *For all $f : A \rightarrow B \otimes C, g : (A \otimes B) \otimes C \rightarrow D$, we have*

$$\text{let}(f); \alpha; \text{let}(g); (\pi_l; \pi_l) \otimes D = \text{let}(\text{let}(f); \alpha; g) \tag{174}$$

(b) *For all $f : A \rightarrow B + C, g : A \otimes B \rightarrow D, h : A \otimes C \rightarrow D$, we have*

$$\text{let}(f); \delta^{-1}; [\text{let}(g); \pi_l \otimes D, \text{let}(h); \pi_l \otimes D] = \text{let}(\text{let}(f); \delta^{-1}; [g, h]) \tag{175}$$

(c) *For all $f_i : R \otimes A_i \rightarrow B$, we have*

$$\delta_{\Sigma}^{-1}; [\text{let}(f_i); \pi_l \otimes B,]_i = \text{let}(\delta_{\Sigma}^{-1}; [f_i]_i); \pi_l \otimes B \tag{176}$$

i.e.,

$$\delta_{\Sigma}^{-1}; [\text{rlet}(f_i),]_i = \text{rlet}(\delta_{\Sigma}^{-1}; [f_i]_i) \tag{177}$$

(d) *For all $f : A \rightarrow B + C, g : A \otimes B \rightarrow B', h : A \otimes C \rightarrow C'$, we have*

$$\begin{aligned}
 \text{let}(\text{let}(f); \delta^{-1}; g + h) &= \text{let}(f); \delta^{-1}; [\text{let}(g); \pi_l \otimes \iota_l, \text{let}(h); \pi_l \otimes \iota_r] \\
 &= \text{let}(f); \delta^{-1}; (\text{let}(g); \pi_l \otimes B') + (\text{let}(h); \pi_l \otimes C'); \delta \\
 &= \text{let}(f); \delta^{-1}; \text{rlet}(g) + \text{rlet}(h); \delta
 \end{aligned} \tag{178}$$

In particular, we have

$$\begin{aligned}
 \text{let}(\text{let}(f); \delta^{-1}; \pi_r + h) &= \text{let}(f); \delta^{-1}; [A \otimes \iota_l, \text{let}(h); \pi_l \otimes \iota_r] \\
 &= \text{let}(f); \delta^{-1}; (A \otimes B) + (\text{let}(h); \pi_l \otimes C'); \delta \\
 &= \text{let}(f); \delta^{-1}; (A \otimes B) + \text{rlet}(h); \delta
 \end{aligned} \tag{179}$$

and

$$\begin{aligned}
 \text{let}(\text{let}(f); \delta^{-1}; g + \pi_r) &= \text{let}(f); \delta^{-1}; [\text{let}(g); \pi_l \otimes \iota_l, A \otimes \iota_r] \\
 &= \text{let}(f); \delta^{-1}; (\text{let}(g); \pi_l \otimes B') + (A \otimes C); \delta \\
 &= \text{let}(f); \delta^{-1}; \text{rlet}(g) + (A \otimes C); \delta
 \end{aligned} \tag{180}$$

LEMMA 5.8 ((LABEL) WEAKENING). *Given $\Gamma \leq \Gamma'$ and $L' \leq L$, $K' \leq K$, we have*

- (a) *For all $\Gamma \leq \Delta$, $\llbracket \Gamma \leq \Delta \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \leq \Delta \rrbracket$*
- (b) *For all $L \leq K$, $\llbracket L' \leq K \rrbracket = \llbracket L' \leq L \rrbracket; \llbracket L \leq K \rrbracket$*
- (c) *$\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \vdash_{\epsilon} a : A \rrbracket$*
- (d) *$\llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \Gamma' \vdash r \triangleright L' \rrbracket; \llbracket L' \leq L \rrbracket$*
- (e) *For all $\gamma : \Gamma \mapsto \Delta$, $\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket; \llbracket \gamma : \Gamma' \mapsto \Delta \rrbracket$*
- (f) *For all $\Gamma \vdash \sigma : L \rightsquigarrow K$, $\llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \leq \Gamma' \rrbracket \otimes \llbracket L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K' \rrbracket; \llbracket K' \leq K \rrbracket$*

PROOF. To show that variable weakenings compose (a), we proceed by induction on the derivation of $\Gamma \leq \Gamma'$ as follows:

- wk-nil: if $\Gamma, \Gamma' = \cdot$, then $\Delta = \cdot$, and so we trivially have $\llbracket \cdot \leq \cdot \rrbracket; \llbracket \cdot \leq \cdot \rrbracket = \llbracket \cdot \leq \cdot \rrbracket = \text{id}$
- wk-skip: we have $\Gamma = \Xi, x : A$, and so by induction

$$\llbracket \Xi, x : A \leq \Gamma' \rrbracket; \llbracket \Gamma' \leq \Delta \rrbracket = \pi_l; \llbracket \Xi \leq \Gamma' \rrbracket; \llbracket \Gamma' \leq \Delta \rrbracket = \pi_l; \llbracket \Xi \leq \Delta \rrbracket = \llbracket \Xi, x : A \leq \Delta \rrbracket \quad (181)$$

as desired

- wk-cons: we have $\Gamma = \Xi, x : A$ and $\Gamma' = \Xi', x : A$. We proceed by case analysis on $\Gamma' \leq \Delta$:
 - wk-skip: we have

$$\begin{aligned} \llbracket \Xi, x : A \leq \Xi', x : A \rrbracket; \llbracket \Xi' \leq \Delta \rrbracket &= \llbracket \Xi \leq \Xi' \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Xi', x : A \leq \Delta \rrbracket \\ &= \llbracket \Xi \leq \Xi' \rrbracket \otimes \llbracket A \rrbracket; \pi_l; \llbracket \Xi' \leq \Delta \rrbracket \\ &= \pi_l; \llbracket \Xi \leq \Xi' \rrbracket; \llbracket \Xi' \leq \Delta \rrbracket \\ &= \pi_l; \llbracket \Xi \leq \Delta \rrbracket \\ &= \llbracket \Xi, x : A \leq \Delta \rrbracket \end{aligned} \quad (182)$$

as desired

- wk-cons: we have $\Delta = \Delta', x : A$, and so by induction

$$\begin{aligned} \llbracket \Xi, x : A \leq \Xi', x : A \rrbracket; \llbracket \Xi', x : A \leq \Delta', x : A \rrbracket &= \llbracket \Xi \leq \Xi' \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Xi' \leq \Delta' \rrbracket \otimes \llbracket A \rrbracket \\ &= (\llbracket \Xi \leq \Xi' \rrbracket; \llbracket \Xi' \leq \Delta' \rrbracket) \otimes \llbracket A \rrbracket \\ &= \llbracket \Xi \leq \Delta' \rrbracket \otimes \llbracket A \rrbracket \\ &= \llbracket \Xi, x : A \leq \Delta \rrbracket \end{aligned} \quad (183)$$

as desired

We can analogously show (b) (i.e., that label weakenings compose) by induction on the derivation of $L \leq K$ as follows:

- lwk-nil: if $L = K = \cdot$, then $L' = \cdot$, so the result follows trivially from the fact that $\llbracket \cdot \rrbracket = L'$ is the initial object
- lwk-skip: we have $K = K', \ell(A)$, and therefore

$$\llbracket L' \leq L \rrbracket; \llbracket L \leq K', \ell(A) \rrbracket = \llbracket L' \leq L \rrbracket; \llbracket L \leq K' \rrbracket; \iota_r = \llbracket L' \leq K' \rrbracket; \iota_r = \llbracket L' \leq K \rrbracket \quad (184)$$

- lwk-cons: we have $L = R, \ell(A)$ and $K = K', \ell(A)$. We proceed by case splitting on $L \leq K'$:
 - lwk-skip: we have

$$\begin{aligned} \llbracket L' \leq R, \ell(A) \rrbracket; \llbracket R, \ell(A) \leq K', \ell(A) \rrbracket &= \llbracket L' \leq R \rrbracket; \iota_r; \llbracket R \leq K' \rrbracket + \llbracket A \rrbracket \\ &= \llbracket L' \leq R \rrbracket; \llbracket R \leq K' \rrbracket; \iota_r \\ &= \llbracket L' \leq K' \rrbracket; \iota_r \\ &= \llbracket L' \leq K', \ell(A) \rrbracket \end{aligned} \quad (185)$$

– lwk-cons: we have that $L' = R', \ell(A)$, and therefore

$$\begin{aligned}
 \llbracket R', \ell(A) \leq R, \ell(A) \rrbracket; \llbracket R, \ell(A) \leq K', \ell(A) \rrbracket &= \llbracket R' \leq R \rrbracket + \llbracket A \rrbracket; \llbracket R \leq K' \rrbracket + \llbracket A \rrbracket \\
 &= (\llbracket R' \leq R \rrbracket; \llbracket R \leq K' \rrbracket) + \llbracket A \rrbracket \\
 &= \llbracket R' \leq K' \rrbracket + \llbracket A \rrbracket \\
 &= \llbracket R', \ell(A) \leq K', \ell(A) \rrbracket
 \end{aligned} \tag{186}$$

We can now show weakening for expressions $\Delta \vdash_\epsilon a : A$ (c) by induction on the typing derivation as follows:

- var: we need to show that

$$\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon x : A \rrbracket = \llbracket \Gamma \leq \Delta \rrbracket; \pi_{\Delta, x} = \llbracket \Gamma \vdash_\epsilon x : A \rrbracket = \pi_{\Gamma, x} \tag{187}$$

we proceed by induction on $\Gamma \leq \Delta$:

- wk-nil: this case yields a contradiction, since if $\Delta = \cdot$ it cannot define x .
- wk-cons: given $\Gamma = \Gamma', y : B, \Delta = \Delta', y : B$,
 - * If $x = y$, then $B = A$, and

$$\llbracket \Gamma', x : A \leq \Delta', x : A \rrbracket; \pi_{(\Delta, x:A), x} = \llbracket \Gamma' \leq \Delta' \rrbracket \otimes \llbracket A \rrbracket; \pi_r = \pi_r = \pi_{(\Gamma, x:A), x} \tag{188}$$

as desired.

- * Otherwise, we have by induction that

$$\begin{aligned}
 \llbracket \Gamma', y : B \leq \Delta', y : B \rrbracket; \pi_{(\Delta', y:B), x} &= \llbracket \Gamma' \leq \Delta' \rrbracket \otimes \llbracket B \rrbracket; \pi_l; \pi_{\Delta', x} \\
 &= \pi_l; \llbracket \Gamma' \leq \Delta' \rrbracket; \pi_{\Delta', x} = \pi_l; \pi_{\Gamma', x} = \pi_{\Gamma, x}
 \end{aligned} \tag{189}$$

- wk-skip: we have $\Gamma = \Gamma', y : B$, and hence

$$\llbracket \Gamma', y : B \leq \Delta \rrbracket; \pi_{\Delta, x} = \pi_l; \llbracket \Gamma' \leq \Delta \rrbracket; \pi_{\Delta, x} = \pi_l; \pi_{\Gamma', x} = \pi_{\Gamma, x} \tag{190}$$

- let₁: we have

$$\begin{aligned}
 &\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon \text{let } x = a; b : B \rrbracket \\
 &= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_\epsilon a : A \rrbracket); \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma, x : A \leq \Delta, x : A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \llbracket \Gamma \vdash_\epsilon \text{let } x = a; b : B \rrbracket
 \end{aligned} \tag{191}$$

- unit: follows immediately since weakenings are pure and **1** is the terminal object.

- pair: we have

$$\begin{aligned}
 &\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon (a, b) : A \otimes B \rrbracket \\
 &= \llbracket \Gamma \leq \Delta \rrbracket; \Delta_\Delta; \llbracket \Delta \vdash_\epsilon a : A \rrbracket \otimes \llbracket \Delta \vdash_\epsilon b : B \rrbracket \\
 &= \Delta_\Gamma; (\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \rrbracket) \otimes (\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon b : B \rrbracket) \\
 &= \Delta_\Gamma; \llbracket \Gamma \vdash_\epsilon a : A \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon b : B \rrbracket \\
 &= \llbracket \Gamma \vdash_\epsilon (a, b) : A \otimes B \rrbracket
 \end{aligned} \tag{192}$$

- let_2 : we have

$$\begin{aligned}
 & \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_{\epsilon} \text{let } (x, y) = a; b : B \rrbracket \\
 &= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_{\epsilon} a : A \otimes B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash_{\epsilon} b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_{\epsilon} a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, x : A, y : B \vdash_{\epsilon} b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \leq \Delta, x : A, y : B \rrbracket; \llbracket \Delta, x : A, y : B \vdash_{\epsilon} b : B \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \vdash_{\epsilon} b : B \rrbracket \\
 &= \llbracket \Gamma \vdash_{\epsilon} \text{let } (x, y) = a; b : B \rrbracket
 \end{aligned} \tag{193}$$

- case: we have

$$\begin{aligned}
 & \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_{\epsilon} \text{case } a \{ \iota_l x : s, \iota_r y : t \} : B \rrbracket \\
 &= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_{\epsilon} a : A \rrbracket); \delta^{-1}; [\llbracket \Delta, x : A \vdash_{\epsilon} s : B \rrbracket, \llbracket \Delta, y : A \vdash_{\epsilon} t : B \rrbracket] \\
 &= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket); \delta^{-1}; \\
 & \quad [\llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_{\epsilon} s : C \rrbracket, \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash_{\epsilon} t : C \rrbracket] \\
 &= \text{let}(\llbracket \Delta \vdash_{\epsilon} a : A \rrbracket); \\
 & \quad [\llbracket \Gamma, x : A \leq \Delta, x : A \rrbracket; \llbracket \Delta, x : A \vdash_{\epsilon} s : C \rrbracket, \llbracket \Gamma, y : B \leq \Delta, y : B \rrbracket; \llbracket \Delta, y : B \vdash_{\epsilon} t : C \rrbracket] \\
 &= \text{let}(\llbracket \Delta \vdash_{\epsilon} a : A \rrbracket); [\llbracket \Gamma, x : A \vdash_{\epsilon} s : C \rrbracket, \llbracket \Gamma, y : B \vdash_{\epsilon} t : C \rrbracket]
 \end{aligned} \tag{194}$$

- op: we have

$$\begin{aligned}
 & \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Gamma \vdash_{\epsilon} f a : B \rrbracket = \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket; \llbracket f \in \mathcal{I}_{\epsilon}(A, B) \rrbracket \\
 &= \llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket f \in \mathcal{I}_{\epsilon}(A, B) \rrbracket \\
 &= \llbracket \Gamma \vdash_{\epsilon} f a : B \rrbracket
 \end{aligned} \tag{195}$$

- inl , inr : analogous to the op case

Similarly, we can show weakening for regions $\Delta \vdash r \triangleright L$ (d) by induction on the typing derivation as follows:

- br: we have that

$$\begin{aligned}
 & \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash \text{br } \ell a \triangleright L \rrbracket; \llbracket L \leq K \rrbracket = \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_{\perp} a : A \rrbracket; \iota_{L, \ell}; \llbracket L \leq K \rrbracket \\
 &= \llbracket \Gamma \vdash_{\perp} a : A \rrbracket; \iota_{L, \ell}; \llbracket L \leq K \rrbracket
 \end{aligned} \tag{196}$$

It hence suffices to show that $\iota_{L, \ell}; \llbracket L \leq K \rrbracket = \iota_{K, \ell}$, which we can do by induction on $L \leq K$:

- lw-k-nil : this case yields a contradiction, since if $K = \cdot$ it cannot define the label ℓ .
- lw-k-skip : we have $K = K', \kappa(B)$, and hence

$$\iota_{L', \ell}; \llbracket L \leq K', \kappa(B) \rrbracket = \iota_{L', \ell}; \llbracket L \leq K', \kappa(B) \rrbracket; \iota_l = \iota_{K', \ell}; \iota_l = \iota_{K, \ell} \tag{197}$$

- lw-k-cons : we have $L = L', \kappa(B)$ and $K = K', \kappa(B)$.

- * If $\kappa = \ell$, then $B = A$ and

$$\iota_{(L', \ell(A)), \ell}; \llbracket L, \ell(A) \leq K', \ell(A) \rrbracket = \iota_r; \llbracket L' \leq K' \rrbracket + \llbracket A \rrbracket = \iota_r = \iota_{K, \ell} \tag{198}$$

- * Otherwise, we have by induction that

$$\begin{aligned}
 & \iota_{(L', \kappa(B)), \ell}; \llbracket L, \kappa(B) \leq K', \kappa(B) \rrbracket = \iota_{L', \ell}; \iota_l; \llbracket L' \leq K' \rrbracket + \llbracket B \rrbracket \\
 &= \iota_{L', \ell}; \llbracket L' \leq K' \rrbracket; \iota_l \\
 &= \iota_{K', \ell}; \iota_l = \iota_{K, \ell}
 \end{aligned} \tag{199}$$

- let₁-r: we have by induction that

$$\begin{aligned}
& \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash \text{let } x = a; r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_\epsilon a : A \rrbracket); \llbracket \Delta, x : A \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma, x : A \leq \Delta, x : A \rrbracket; \llbracket \Delta, x : A \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \rrbracket); \llbracket \Gamma, x : A \vdash r \triangleright K \rrbracket \\
&= \llbracket \Gamma \vdash \text{let } x = a; r \triangleright K \rrbracket
\end{aligned} \tag{200}$$

- let₂-r: we have by induction that

$$\begin{aligned}
& \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash \text{let } (x, y) = a; r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_\epsilon a : A \otimes B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, x : A, y : B \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \leq \Delta, x : A, y : B \rrbracket; \llbracket \Delta, x : A, y : B \vdash r \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \vdash r \triangleright K \rrbracket
\end{aligned} \tag{201}$$

- case-r: we have by induction that

$$\begin{aligned}
& \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash \text{case } a \{ \iota_l x : r, \iota_r y : s \} \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash_\epsilon a : A \rrbracket); \delta^{-1}; \\
&\quad \llbracket \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket, \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket \rrbracket; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \rrbracket); \delta^{-1}; [\\
&\quad \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket; \llbracket L \leq K \rrbracket, \\
&\quad \llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket; \llbracket L \leq K \rrbracket] \\
&= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon a : A \rrbracket); \delta^{-1}; [\\
&\quad \llbracket \Gamma, x : A \leq \Delta, x : A \rrbracket; \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket; \llbracket L \leq K \rrbracket, \\
&\quad \llbracket \Gamma, y : B \leq \Delta, y : B \rrbracket; \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket; \llbracket L \leq K \rrbracket] \\
&= \text{let}(\llbracket \Gamma \vdash_\epsilon a : A \rrbracket); \delta^{-1}; [\llbracket \Gamma, x : A \vdash s \triangleright K \rrbracket, \llbracket \Gamma, y : B \vdash t \triangleright K \rrbracket] \\
&= \llbracket \Gamma \vdash \text{case } a \{ \iota_l x : s, \iota_r y : t \} \triangleright K \rrbracket
\end{aligned} \tag{202}$$

- cfg: Let $L = \text{lsem}_{\Delta, L}((\ell_i(x_i) : \{t_i\},)_i)$ and $R = (\ell_i(A_i),)_i$. We have by induction that

$$\begin{aligned}
& \llbracket \Gamma \leq \Delta \rrbracket; L; \llbracket L \leq K \rrbracket + \Sigma_i \llbracket A_i \rrbracket \\
&= \llbracket \Gamma \leq \Delta \rrbracket; \delta_\Sigma^{-1}; [(\llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket,)_i]; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+; \llbracket L \leq K \rrbracket + \Sigma_i \llbracket A_i \rrbracket \\
&= \delta_\Sigma^{-1}; [\llbracket \Gamma \leq \Delta \rrbracket \otimes \llbracket A_i \rrbracket; (\llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \llbracket L \leq K \rrbracket + \llbracket R \rrbracket,)_i]; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \delta_\Sigma^{-1}; [(\llbracket \Gamma, x_i : A_i \leq \Delta, x_i : A_i \rrbracket; \llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \llbracket L, R \leq K, R \rrbracket,)_i]; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \delta_\Sigma^{-1}; [(\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright K, R \rrbracket,)_i]; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i)
\end{aligned} \tag{203}$$

It follows that

$$\begin{aligned}
& \llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket; \llbracket L \leq K \rrbracket \\
&= \llbracket \Gamma \leq \Delta \rrbracket; \text{let}(\llbracket \Delta \vdash r \triangleright L, R \rrbracket; \llbracket \Delta \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+}; \delta^{-1}; [\pi_r, \text{rfix}(L)]; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \leq \Delta \rrbracket; \llbracket \Delta \vdash r \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+}; \delta^{-1}; [\pi_r, \llbracket \Gamma \leq \Delta \rrbracket; \text{rfix}(L)]; \llbracket L \leq K \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+}; [\pi_r; \llbracket L \leq K \rrbracket, \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i))] \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}; \llbracket L \leq K \rrbracket + \sum_i \llbracket A_i \rrbracket); [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i))] \quad (204) \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}; \llbracket L \leq K \rrbracket + \sum_i \llbracket A_i \rrbracket); [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i))] \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \llbracket L, R \leq K, R \rrbracket; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}); [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i))] \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright K, R \rrbracket; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}); [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{t_i\},)_i))] \\
&= \llbracket \Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright K \rrbracket
\end{aligned}$$

Weakening for substitutions (e) and label substitutions (f) then follow by a trivial induction. $\square$

LEMMA C.3 (SUBSTITUTION PROJECTION). *For $\gamma : \Gamma \mapsto \Delta$, $\text{eff}(\Delta) = \perp$ and $\Delta(x) = A$, we have*

$$\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \pi_{\Delta, x} = \llbracket \Gamma \vdash_e [\gamma]x : A \rrbracket \quad (205)$$

PROOF. We proceed by induction on Δ :

- If $\Delta = \cdot$, then $\Delta(x) = A$ is a contradiction.
- If $\Delta = \Delta', x : A$, then $\gamma = \gamma', x \mapsto [\gamma]x$, so we have

$$\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \pi_{\Delta, x} = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \gamma' : \Gamma \mapsto \Delta' \rrbracket \otimes \llbracket \Gamma \vdash_{\perp} [\gamma]x : A \rrbracket; \pi_r = \llbracket \Gamma \vdash_e [\gamma]x : A \rrbracket \quad (206)$$

as desired.

- If $\Delta = \Delta', y : B$ (with $y \neq x$), then $\gamma = \gamma', y \mapsto [\gamma]y$, so by induction we have

$$\begin{aligned}
& \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \pi_{\Delta, x} = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \gamma' : \Gamma \mapsto \Delta' \rrbracket \otimes \llbracket \Gamma \vdash_{\perp} [\gamma]y : B \rrbracket; \pi_l; \pi_{\Delta', x} \\
&= \llbracket \gamma' : \Gamma \mapsto \Delta' \rrbracket; \pi_{\Delta', x} = \llbracket \Gamma \vdash_e [\gamma']x : A \rrbracket = \llbracket \Gamma \vdash_e [\gamma]x : A \rrbracket \quad (207)
\end{aligned}$$

$\square$

THEOREM 5.9 (SOUNDNESS (SUBSTITUTION)). *Given $\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket \Delta \rrbracket$ pure, we have that*

- $\llbracket \Gamma \vdash_e [\gamma]a : A \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_e a : A \rrbracket$
- $\llbracket \Gamma \vdash [\gamma]r \triangleright L \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash r \triangleright L \rrbracket$
- $\llbracket [\gamma]\rho : \Delta \mapsto \Xi \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \rho : \Delta \mapsto \Xi \rrbracket$
- $\llbracket \Gamma \vdash [\gamma]\sigma : L \rightsquigarrow K \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash \sigma : L \rightsquigarrow K \rrbracket$

PROOF. Fix $\gamma : \Gamma \mapsto \Delta$ with $\text{eff}(\Delta) = \perp$. We will begin by showing the soundness of substitution for expressions (a): we proceed by induction on the derivation $\Delta \vdash_e e : E$:

- If $e = x$ is a variable, then by Lemma C.3, we have

$$\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_e x : A \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \pi_{\Delta, x} = \llbracket \Gamma \vdash_e [\gamma]x : A \rrbracket \quad (208)$$

as desired.

- If $e = f a$ is an operation, then by induction we have that

$$\begin{aligned}
& \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Gamma \vdash_e f a : B \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_e a : A \rrbracket; \llbracket f \in \mathcal{I}_B(\epsilon, A) \rrbracket \\
&= \llbracket \Gamma \vdash_e [\gamma]a : A \rrbracket; \llbracket f \in \mathcal{I}_B(\epsilon, A) \rrbracket = \llbracket \Gamma \vdash_e [\gamma](f a) : B \rrbracket \quad (209)
\end{aligned}$$

as desired. The cases for left injections, right injections, and abort are analogous

- If $e = (\text{let } x = a; b)$ is a unary let-binding, then by induction we have that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \llbracket (\gamma, x \mapsto x) : (\Gamma, x : A) \mapsto (\Delta, x : A) \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \llbracket \Gamma, x : A \vdash_\epsilon [\gamma, x \mapsto x] b : B \rrbracket = \llbracket \Gamma, x : A \vdash_\epsilon [\gamma] b : B \rrbracket
 \end{aligned} \tag{210}$$

as we can assume that x is a fresh variable. Hence, it follows that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon \text{let } x = a; b : B \rrbracket \\
 &= \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_\epsilon a : A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon [\gamma] a : A \rrbracket; \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : B \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon [\gamma] a : A \rrbracket; \llbracket \Gamma, x : A \vdash_\epsilon [\gamma] b : B \rrbracket \\
 &= \llbracket \Gamma \vdash_\epsilon [\gamma] (\text{let } x = a; b) : B \rrbracket
 \end{aligned} \tag{211}$$

as desired.

- If $e = (a, b)$ is a pair, then by induction we have that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon (a, b) : A \times B \rrbracket \\
 &= \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \vdash_\epsilon a : A \rrbracket \ltimes \llbracket \Delta \vdash_\epsilon b : B \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; (\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket \Delta \vdash_\epsilon a : A \rrbracket) \ltimes (\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket \Delta \vdash_\epsilon b : B \rrbracket) \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \vdash_\epsilon [\gamma] a : A \rrbracket \ltimes \llbracket \Gamma \vdash_\epsilon [\gamma] b : B \rrbracket = \llbracket \Gamma \vdash_\epsilon [\gamma] (a, b) : A \times B \rrbracket
 \end{aligned} \tag{212}$$

- If $e = (\text{let } (x, y) = a; b)$ is a binary let-binding, then by induction we have that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash_\epsilon b : C \rrbracket \\
 &= \alpha; \llbracket (\gamma, x \mapsto x, y \mapsto y) : (\Gamma, x : A, y : B) \mapsto (\Delta, x : A, y : B) \rrbracket; \llbracket \Delta, x : A, y : B \vdash_\epsilon b : C \rrbracket \\
 &= \llbracket \Gamma, x : A, y : B \vdash_\epsilon [\gamma, x \mapsto x, y \mapsto y] b : C \rrbracket = \llbracket \Gamma, x : A, y : B \vdash_\epsilon [\gamma] b : C \rrbracket
 \end{aligned} \tag{213}$$

as we can assume that x, y are fresh variables. Hence, it follows that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_\epsilon \text{let } (x, y) = a; b : C \rrbracket \\
 &= \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_\epsilon a : A \otimes B \rrbracket; \alpha; \llbracket \Delta, x : A, y : B \vdash_\epsilon b : C \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon [\gamma] a : A \otimes B \rrbracket; \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash_\epsilon b : C \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon [\gamma] a : A \otimes B \rrbracket; \llbracket \Gamma, x : A, y : B \vdash_\epsilon [\gamma] b : C \rrbracket \\
 &= \llbracket \Gamma \vdash_\epsilon [\gamma] (\text{let } (x, y) = a; b) : C \rrbracket
 \end{aligned} \tag{214}$$

as desired.

- If $e = \text{case } a \{ \iota_l x : b, \iota_r y : c \}$ is a case-expression, then by induction we have that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : C \rrbracket \\
 &= \llbracket (\gamma, x \mapsto x) : (\Gamma, x : A) \mapsto (\Delta, x : A) \rrbracket; \llbracket \Delta, x : A \vdash_\epsilon b : C \rrbracket \\
 &= \llbracket \Gamma, x : A \vdash_\epsilon [\gamma, x \mapsto x] b : C \rrbracket = \llbracket \Gamma, x : A \vdash_\epsilon [\gamma] b : C \rrbracket
 \end{aligned} \tag{215}$$

and

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash_\epsilon c : C \rrbracket \\
 &= \llbracket (\gamma, y \mapsto y) : (\Gamma, y : B) \mapsto (\Delta, y : B) \rrbracket; \llbracket \Delta, y : B \vdash_\epsilon c : C \rrbracket \\
 &= \llbracket \Gamma, y : B \vdash_\epsilon [\gamma, y \mapsto y] c : C \rrbracket = \llbracket \Gamma, y : B \vdash_\epsilon [\gamma] c : C \rrbracket
 \end{aligned} \tag{216}$$

as we can assume that x, y are fresh variables. Hence, it follows that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_{\epsilon} a : A + B \rrbracket; \delta_{\llbracket \Delta \rrbracket}^{-1}; \llbracket \llbracket \Delta, x : A \vdash_{\epsilon} b : C \rrbracket, \llbracket \Delta, y : B \vdash_{\epsilon} c : C \rrbracket \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A + B \rrbracket; \delta_{\llbracket \Gamma \rrbracket}^{-1}; \\
 & \quad \llbracket \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash_{\epsilon} b : C \rrbracket, \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash_{\epsilon} c : C \rrbracket \rrbracket \quad (217) \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A + B \rrbracket; \delta_{\llbracket \Gamma \rrbracket}^{-1}; \llbracket \llbracket \Gamma, x : A \vdash_{\epsilon} [\gamma]b : C \rrbracket, \llbracket \Gamma, y : B \vdash_{\epsilon} [\gamma]c : C \rrbracket \rrbracket \\
 &= \llbracket \Gamma \vdash_{\epsilon} [\gamma](\text{case } a \{ \iota_l x : b, \iota_r y : c \}) : C \rrbracket
 \end{aligned}$$

as desired.

- If $e = ()$ is the null expression, since $\llbracket \gamma : \Gamma \mapsto \Delta \rrbracket$ is pure, the desired result holds trivially since 1 is the terminal object in the category of pure morphisms.

We may now prove the soundness of substitution for regions as follows: assuming $\Gamma \vdash r \triangleright L$, we proceed by induction on r as follows:

- If $r = \text{br } \ell \ a$, then we have that

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash \text{br } \ell \ A \triangleright L \rrbracket = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash_{\perp} a : A \rrbracket; \iota_{L,\ell} \\
 &= \llbracket \Gamma \vdash_{\perp} [\gamma]a : A \rrbracket; \iota_{L,\ell} = \llbracket \Gamma \vdash [\gamma](\text{br } \ell \ a) \triangleright L \rrbracket \quad (218)
 \end{aligned}$$

- If $r = (\text{let } x = a; t)$, then we have that, by induction,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash t \triangleright L \rrbracket \\
 &= \llbracket (\gamma, x \mapsto x) : (\Gamma, x : A) \mapsto (\Delta, x : A) \rrbracket; \llbracket \Delta, x : A \vdash t \triangleright L \rrbracket \quad (219) \\
 &= \llbracket \Gamma, x : A \vdash [\gamma, x \mapsto x]t \triangleright L \rrbracket = \llbracket \Gamma, x : A \vdash [\gamma]t \triangleright L \rrbracket
 \end{aligned}$$

since x can be taken to be a free variable. Hence,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash \text{let } x = a; t \triangleright L \rrbracket \\
 &= \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_{\epsilon} a : A \rrbracket; \llbracket \Delta, x : A \vdash t \triangleright L \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A \rrbracket; \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash t \triangleright L \rrbracket \quad (220) \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A \rrbracket; \llbracket \Gamma, x : A \vdash [\gamma]t \triangleright L \rrbracket \\
 &= \llbracket \Gamma \vdash [\gamma](\text{let } x = a; t) \triangleright L \rrbracket
 \end{aligned}$$

- If $r = (\text{let } (x, y) = a; t)$, then we have that, by induction,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash t \triangleright L \rrbracket \\
 &= \alpha; \llbracket (\gamma, x \mapsto x, y \mapsto y) : (\Gamma, x : A, y : B) \mapsto (\Delta, x : A, y : B) \rrbracket; \llbracket \Delta, x : A, y : B \vdash t \triangleright L \rrbracket \quad (221) \\
 &= \llbracket \Gamma, x : A, y : B \vdash [\gamma, x \mapsto x, y \mapsto y]t \triangleright L \rrbracket = \llbracket \Gamma, x : A, y : B \vdash [\gamma]t \triangleright L \rrbracket
 \end{aligned}$$

since x, y can be taken to be free variables. Hence,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \llbracket \Delta \vdash \text{let } (x, y) = a; t \triangleright L \rrbracket \\
 &= \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_{\epsilon} a : A \otimes B \rrbracket; \alpha; \llbracket \Delta, x : A, y : B \vdash t \triangleright L \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A \otimes B \rrbracket; \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket); \alpha; \llbracket \Delta, x : A, y : B \vdash t \triangleright L \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} [\gamma]a : A \otimes B \rrbracket; \llbracket \Gamma, x : A, y : B \vdash [\gamma]t \triangleright L \rrbracket \\
 &= \llbracket \Gamma \vdash [\gamma](\text{let } (x, y) = a; t) \triangleright L \rrbracket \quad (222)
 \end{aligned}$$

- If $r = \text{case } a \{ \iota_l x : s, \iota_r y : t \}$, then we have that, by induction

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket \\
 &= \llbracket (\gamma, x \mapsto x) : (\Gamma, x : A) \mapsto (\Delta, x : A) \rrbracket; \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket \\
 &= \llbracket \Gamma, x : A \vdash [\gamma, x \mapsto x]s \triangleright L \rrbracket = \llbracket \Gamma, x : A \vdash [\gamma]s \triangleright L \rrbracket
 \end{aligned} \tag{223}$$

and

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket \\
 &= \llbracket (\gamma, y \mapsto y) : (\Gamma, y : B) \mapsto (\Delta, y : B) \rrbracket; \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket \\
 &= \llbracket \Gamma, y : B \vdash [\gamma, y \mapsto y]t \triangleright L \rrbracket = \llbracket \Gamma, y : B \vdash [\gamma]t \triangleright L \rrbracket
 \end{aligned} \tag{224}$$

since x, y can be taken to be free variables. Hence,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket; \Delta_{\llbracket \Delta \rrbracket}; \llbracket \Delta \rrbracket \otimes \llbracket \Delta \vdash_e a : A + B \rrbracket; \delta_{\llbracket \Delta \rrbracket}^{-1}; [\llbracket \Delta, x : A \vdash s \triangleright L \rrbracket, \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket] \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e [\gamma]a : A + B \rrbracket; \delta_{\llbracket \Gamma \rrbracket}^{-1}; \\
 & \quad \llbracket \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A \rrbracket; \llbracket \Delta, x : A \vdash s \triangleright L \rrbracket, \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket B \rrbracket; \llbracket \Delta, y : B \vdash t \triangleright L \rrbracket \rrbracket \\
 &= \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e [\gamma]a : A + B \rrbracket; \delta_{\llbracket \Gamma \rrbracket}^{-1}; [\llbracket \Gamma, x : A \vdash [\gamma]s \triangleright L \rrbracket, \llbracket \Gamma, y : B \vdash [\gamma]t \triangleright L \rrbracket] \\
 &= \llbracket \Gamma \vdash [\gamma](\text{case } a \{ \iota_l x : s, \iota_r y : t \}) \triangleright L \rrbracket
 \end{aligned} \tag{225}$$

as desired.

- Assume $r = s$ where $(\ell_i(x_i) : \{t_i\},)_i$. Define $R = (\ell_i(A_i),)_i$ and $S = \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket$. We have by induction that, for all i ,

$$\begin{aligned}
 & \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket \otimes \llbracket A_i \rrbracket; \llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket \\
 &= \llbracket (\gamma, x_i \mapsto x_i) : (\Gamma, x_i : A_i) \mapsto (\Delta, x_i : A_i) \rrbracket; \llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket \\
 &= \llbracket \Gamma, x_i : A_i \vdash [\gamma, x_i \mapsto x_i]t_i \triangleright L, R \rrbracket = \llbracket \Gamma, x_i : A_i \vdash [\gamma]t_i \triangleright L, R \rrbracket
 \end{aligned} \tag{226}$$

and therefore that

$$\begin{aligned}
 & S \otimes \Sigma_i \llbracket A_i \rrbracket; \text{lsem}_{\Delta, L}((\ell_i(x_i) : \{t_i\},)_i) \\
 &= S \otimes \Sigma_i \llbracket A_i \rrbracket; \delta_{\Sigma}^{-1}; [\llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
 &= \delta_{\Sigma}^{-1}; [S \otimes \llbracket A_i \rrbracket; \llbracket \Delta, x_i : A_i \vdash t_i \triangleright L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
 &= \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash [\gamma]t_i \triangleright L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
 &= \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{[\gamma]t_i\},)_i)
 \end{aligned} \tag{227}$$

It follows that

$$\begin{aligned}
 & S; \llbracket \Delta \vdash s \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket \\
 &= S; \text{let}(\llbracket \Delta \vdash s \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{lsem}_{\Delta, L}((\ell_i(x_i) : \{t_i\},)_i)] \\
 &= \text{let}(S; \llbracket \Delta \vdash s \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); S \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket); \delta^{-1}; [\pi_r, \text{lsem}_{\Delta, L}((\ell_i(x_i) : \{t_i\},)_i)] \\
 &= \text{let}(\llbracket \Gamma \vdash [\gamma]s \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, S \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket)]; \text{lsem}_{\Delta, L}((\ell_i(x_i) : \{t_i\},)_i)] \\
 &= \text{let}(\llbracket \Gamma \vdash [\gamma]s \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{[\gamma]t_i\},)_i)] \\
 &= \llbracket \Gamma \vdash [\gamma]s \text{ where } (\ell_i(x_i) : \{[\gamma]t_i\},)_i \triangleright L \rrbracket
 \end{aligned} \tag{228}$$

as desired.

Composition of substitutions then follows by a trivial induction, as does substitution for label substitutions. $\square$

C.5 Label Substitution

LEMMA C.4 (LABEL SUBSTITUTION INJECTION). *For $\Gamma \vdash \sigma : L \rightsquigarrow K$ and $L(\ell) = A$, we have*

$$\llbracket \Gamma \rrbracket \otimes \iota_{L,\ell}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma, x : A \vdash [\gamma](\text{br } \ell \ x) \triangleright K \rrbracket \quad (229)$$

PROOF. We proceed by induction on L :

- If $L = \cdot$, then $L(\ell) = A$ is a contradiction
- If $L = L', \ell(A)$, then $\sigma = \sigma', \ell(x) \mapsto [\sigma](\text{br } \ell \ x)$, so we have

$$\begin{aligned} & \llbracket \Gamma \rrbracket \otimes \iota_{L,\ell}; \llbracket \Gamma \vdash \sigma : L, \ell(A) \rightsquigarrow K \rrbracket \\ &= \llbracket \Gamma \rrbracket \otimes \iota_r; \delta^{-1}; [\llbracket \Gamma \vdash \sigma' : L \rightsquigarrow K \rrbracket, \llbracket \Gamma, x : A \vdash [\sigma](\text{br } \ell \ x) \triangleright K \rrbracket] \\ &= \llbracket \Gamma, x : A \vdash [\sigma](\text{br } \ell \ x) \triangleright K \rrbracket \end{aligned} \quad (230)$$

- If $L = L', \kappa(A)$, then $\sigma = \sigma', \kappa(x) \mapsto [\sigma](\text{br } \kappa \ x)$, so by induction we have

$$\begin{aligned} & \llbracket \Gamma \rrbracket \otimes \iota_{L,\ell}; \llbracket \Gamma \vdash \sigma : L, \ell(A) \rightsquigarrow K \rrbracket \\ &= \llbracket \Gamma \rrbracket \otimes (\iota_{L',\ell}; \iota_l); \delta^{-1}; [\llbracket \Gamma \vdash \sigma' : L \rightsquigarrow K \rrbracket, \llbracket \Gamma, x : B \vdash [\sigma](\text{br } \kappa \ x) \triangleright K \rrbracket] \\ &= \llbracket \Gamma \rrbracket \otimes \iota_{L',\ell}; \llbracket \Gamma \vdash \sigma' : L \rightsquigarrow K \rrbracket \\ &= \llbracket \Gamma, x : A \vdash [\sigma'](\text{br } \ell \ x) \triangleright K \rrbracket \\ &= \llbracket \Gamma, x : A \vdash [\sigma](\text{br } \ell \ x) \triangleright K \rrbracket \end{aligned} \quad (231)$$

□

LEMMA C.5 (LABEL SUBSTITUTION SPLITTING). *For $\Gamma \vdash \sigma : L \rightsquigarrow K$, where $L = (\ell_i(A_i),)_i$, we have*

$$\llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright K \rrbracket,]_i \quad (232)$$

and therefore

$$\delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright K \rrbracket,]_i = \alpha_{\llbracket L \rrbracket}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket \quad (233)$$

In particular, we have that, given $\forall i, \Gamma, x_i : A_i \vdash t_i \triangleright K$,

$$\llbracket \Gamma \vdash (\ell_i(x_i) : \{A_i\},)_i : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \rrbracket \otimes \alpha_{\Sigma_i \llbracket A_i \rrbracket}; \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright K \rrbracket,]_i \quad (234)$$

LEMMA C.6 (LABEL SUBSTITUTION EXTENSION). *Given $\sigma = \sigma_l, \sigma_r, \Gamma \vdash \sigma : L, R \rightsquigarrow L', R', \Gamma \vdash \sigma_l : L \rightsquigarrow L', \Gamma \vdash \sigma_r : R \rightsquigarrow R'$, we have*

$$\llbracket \Gamma \vdash \sigma : L, R \rightsquigarrow L', R' \rrbracket = \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}; \llbracket \Gamma \vdash \sigma_l : L \rightsquigarrow L' \rrbracket + \llbracket \Gamma \vdash \sigma_r : R \rightsquigarrow R' \rrbracket; \alpha_{\llbracket L', R' \rrbracket} \quad (235)$$

In particular, for $\Gamma \vdash \sigma : L \rightsquigarrow K$, we have

$$\llbracket \Gamma \vdash \sigma^{\uparrow} : R, L \rightsquigarrow R, K \rrbracket = \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket R \rrbracket + \llbracket L \rrbracket}^+; \delta^{-1}; \pi_r + \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket; \alpha_{\llbracket R, K \rrbracket}^+ \quad (236)$$

$$\llbracket \Gamma \vdash \sigma^{\downarrow} : L, R \rightsquigarrow K, R \rrbracket = \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r; \alpha_{\llbracket K, R \rrbracket}^+ \quad (237)$$

since

$$\Gamma \vdash \text{id} : R \rightsquigarrow R = \pi_r \quad (238)$$

THEOREM 5.11 (SOUNDNESS (LABEL SUBSTITUTION)). *Given $\Gamma \vdash \sigma : L \rightsquigarrow K$, we have*

- $\llbracket \Gamma \vdash [\sigma]r \triangleright K \rrbracket = \text{let}(\llbracket \Gamma \vdash r \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)$
- $\llbracket \Gamma \vdash [\sigma]\sigma' : M \rightsquigarrow K \rrbracket = \Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash \sigma' : M \rightsquigarrow L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket$

PROOF. Fix $\Gamma \vdash \sigma : L \rightsquigarrow K$. We begin by proving the soundness of label substitution for regions as follows: assuming $\Gamma \vdash r \triangleright L$, we proceed by induction on r as follows:

- If $r = \text{br } \ell \ a$, then by Lemma C.4 we have that

$$\begin{aligned}
 \llbracket \Gamma \vdash [\sigma](\text{br } \ell \ a) \triangleright K \rrbracket &= \llbracket \Gamma \vdash [a/x][\sigma](\text{br } \ell \ x) \triangleright K \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket; \llbracket \Gamma, x : A \vdash [\sigma](\text{br } \ell \ x) \triangleright K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket; \llbracket \Gamma \rrbracket \otimes \iota_{L,\ell}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\perp} a : A \rrbracket; \iota_{L,\ell}); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash \text{br } \ell \ a \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)
 \end{aligned} \tag{239}$$

as desired.

- If $r = (\text{let } x = a; t)$, then we have by induction that

$$\begin{aligned}
 \llbracket \Gamma \vdash [\sigma](\text{let } x = a; t) \triangleright K \rrbracket &= \llbracket \Gamma \vdash \text{let } x = a; [\sigma]t \triangleright K \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket \Gamma, x : A \vdash [\sigma]t \triangleright K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\llbracket \Gamma, x : A \vdash t \triangleright L \rrbracket; \llbracket \Gamma, x : A \vdash \sigma : L \rightsquigarrow K \rrbracket)) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\llbracket \Gamma, x : A \vdash t \triangleright L \rrbracket; \pi_l \otimes \llbracket L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)) \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket \Gamma, x : A \vdash t \triangleright L \rrbracket); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash \text{let } x = a; t \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)
 \end{aligned} \tag{240}$$

as desired.

- If $r = (\text{let } (x, y) = a; t)$, then we have by induction that

$$\begin{aligned}
 \llbracket \Gamma \vdash [\sigma](\text{let } (x, y) = a; t) \triangleright K \rrbracket &= \llbracket \Gamma \vdash \text{let } (x, y) = a; [\sigma]t \triangleright K \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket; \alpha; \llbracket \Gamma, x : A, y : B \vdash [\sigma]t \triangleright K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket; \alpha; \text{let}(\llbracket \Gamma, x : A, y : B \vdash t \triangleright L \rrbracket; \llbracket \Gamma, x : A, y : B \vdash \sigma : L \rightsquigarrow K \rrbracket)) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket; \alpha; \text{let}(\llbracket \Gamma, x : A, y : B \vdash t \triangleright L \rrbracket; (\pi_l; \pi_l) \otimes \llbracket L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)) \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \otimes B \rrbracket; \alpha; \llbracket \Gamma, x : A, y : B \vdash t \triangleright L \rrbracket); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash \text{let } (x, y) = a; t \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)
 \end{aligned} \tag{241}$$

as desired, since

- If $r = \text{case } a \{ \iota_l x : s, \iota_r y : t \}$, then we have by induction that

$$\begin{aligned}
 \llbracket \Gamma \vdash [\sigma](\text{case } a \{ \iota_l x : s, \iota_r y : t \}) \triangleright K \rrbracket &= \llbracket \Gamma \vdash \text{case } a \{ \iota_l x : [\sigma]s, \iota_r y : [\sigma]t \} \triangleright K \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A + B \rrbracket; \delta^{-1}; [\llbracket \Gamma, x : A \vdash [\sigma]s \triangleright K \rrbracket, \llbracket \Gamma, y : B \vdash [\sigma]t \triangleright K \rrbracket]) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A + B \rrbracket; \delta^{-1}; [\\
 &\quad \text{let}(\llbracket \Gamma, x : A \vdash s \triangleright L \rrbracket; \llbracket \Gamma, x : A \vdash \sigma : L \rightsquigarrow K \rrbracket, \\
 &\quad \text{let}(\llbracket \Gamma, y : B \vdash t \triangleright L \rrbracket; \llbracket \Gamma, y : B \vdash \sigma : L \rightsquigarrow K \rrbracket)) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A + B \rrbracket; \delta^{-1}; [\\
 &\quad [\text{let}(\llbracket \Gamma, x : A \vdash s \triangleright L \rrbracket; \pi_l \otimes \llbracket L \rrbracket, \text{let}(\llbracket \Gamma, y : B \vdash t \triangleright L \rrbracket; \pi_l \otimes \llbracket L \rrbracket); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket) \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A + B \rrbracket; \delta^{-1}; [\llbracket \Gamma, x : A \vdash s \triangleright L \rrbracket, \llbracket \Gamma, y : B \vdash t \triangleright L \rrbracket]); \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash \text{case } a \{ \iota_l x : s, \iota_r y : t \} \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)
 \end{aligned} \tag{242}$$

as desired.

- If $r = s$ where $(\ell_i(x_i) : \{t_i\},)_i$, then we have by induction, taking $R = (\ell_i(A_i),)_i$,

$$\begin{aligned}
 \text{esem}_{\Gamma, K}([\sigma^\dagger]s) &= \llbracket \Gamma \vdash [\sigma^\dagger]s \triangleright K, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \\
 &= \text{let}(\llbracket \Gamma \vdash s \triangleright L, R \rrbracket; \llbracket \Gamma \vdash \sigma^\dagger : L, R \rightsquigarrow K, R \rrbracket; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \\
 &= \text{let}(\llbracket \Gamma \vdash s \triangleright L, R \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \quad (243) \\
 &= \text{let}(\llbracket \Gamma \vdash s \triangleright L, R \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r \\
 &= \text{let}(\text{esem}_{\Gamma, L}(s)); \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r
 \end{aligned}$$

and

$$\begin{aligned}
 \text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{[\sigma^\dagger]t_i\},)_i) &= \delta_{\Sigma}^{-1}; \llbracket \llbracket \Gamma, x_i : A_i \vdash [\sigma^\dagger]t_i \triangleright K, R \rrbracket \rrbracket_i; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \\
 &= \delta_{\Sigma}^{-1}; [\text{let}(\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \llbracket \Gamma, x_i : A_i \vdash \sigma^\dagger : L, R \rightsquigarrow K, R \rrbracket \rrbracket_i; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \\
 &= \delta_{\Sigma}^{-1}; [\text{let}(\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \pi_l \otimes \llbracket L, R \rrbracket \rrbracket_i; \llbracket \Gamma \vdash \sigma^\dagger : L, R \rightsquigarrow K, R \rrbracket; \alpha_{\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \\
 &= \delta_{\Sigma}^{-1}; [\text{let}(\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \pi_l \otimes \alpha_{\llbracket L \rrbracket + \sum_j \llbracket A_j \rrbracket}^+ \rrbracket_i; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r \quad (244) \\
 &= \text{let}(\delta_{\Sigma}^{-1}; \llbracket \llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket A_j \rrbracket}^+ \rrbracket_i; \pi_l \otimes -; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r \\
 &= \text{let}(\delta_{\Sigma}^{-1}; \llbracket \llbracket \Gamma, x_i : A_i \vdash t_i \triangleright L, R \rrbracket, \rrbracket_i; \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}^+ \rrbracket_i; \pi_l \otimes -; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r \\
 &= \text{let}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)); \pi_l \otimes -; \delta^{-1}; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket + \pi_r
 \end{aligned}$$

Letting $L = \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)$ and $S = \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket$, we have that

$$\begin{aligned}
 \text{rcase}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{[\sigma^\dagger]t_i\},)_i)) \\
 &= \text{let}(\text{let}(L); \pi_l \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1}; S + \pi_r); \pi_l \otimes (\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1} \\
 &= \text{let}(\text{let}(L)); (\llbracket \Gamma \rrbracket \otimes \sum_i \llbracket A_i \rrbracket) \otimes (\pi_l \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1}; S + \pi_r); \pi_l \otimes (\llbracket K \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1} \\
 &= \text{let}(L); \Delta_{\llbracket \Gamma \rrbracket \otimes \sum_i \llbracket A_i \rrbracket} \otimes -; \alpha; (\llbracket \Gamma \rrbracket \otimes \sum_i \llbracket A_i \rrbracket) \otimes (\pi_l \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1}; S + \pi_r); \pi_l \otimes -; \delta^{-1} \\
 &= \text{let}(L); (\pi_l; \Delta_{\llbracket \Gamma \rrbracket}) \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \alpha; \llbracket \Gamma \rrbracket \otimes (\delta^{-1}; S + \pi_r); \delta^{-1} \\
 &= \text{let}(L); (\pi_l; \Delta_{\llbracket \Gamma \rrbracket}) \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \alpha; \llbracket \Gamma \rrbracket \otimes \delta^{-1}; \delta^{-1}; (\llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \rrbracket) \otimes S + (\llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \rrbracket) \otimes \pi_r \\
 &= \text{let}(L); \pi_l \otimes (\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket); \delta^{-1}; (\Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes S) + (\Delta_{\llbracket \Gamma \rrbracket} \otimes \sum_i \llbracket A_i \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \pi_r) \\
 &= \text{rcase}(L); \Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes S + \llbracket \Gamma \rrbracket \otimes \sum_i \llbracket A_i \rrbracket \quad (245)
 \end{aligned}$$

implying by naturality that

$$\begin{aligned}
 \text{rfix}(\text{lsem}_{\Gamma, K}((\ell_i(x_i) : \{[\sigma^\dagger]t_i\},)_i)) &= (\text{rcase}(L); \Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes S + \llbracket \Gamma \rrbracket \otimes \sum_i \llbracket A_i \rrbracket)^\dagger; \pi_r \\
 &= (\text{rcase}(L))^\dagger; \Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes S; \pi_r \\
 &= (\text{rcase}(L))^\dagger; S \quad (246)
 \end{aligned}$$

Hence, we have that

$$\begin{aligned}
& \llbracket \Gamma \vdash [\sigma](s \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright K) \rrbracket = \llbracket \Gamma \vdash [\sigma^\dagger]s \text{ where } (\ell_i(x_i) : \{[\sigma^\dagger]t_i\},)_i \triangleright K \rrbracket \\
& = \text{let}(\text{esem}_{\Gamma,K}([\sigma^\dagger]s); \delta^{-1}; [\pi_r, \text{rfix}(\text{lsem}_{\Gamma,K}((\ell_i(x_i) : \{[\sigma^\dagger]t_i\},)_i))] \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \delta^{-1}; S + \pi_r); \delta^{-1}; [\pi_r, (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \llbracket \Gamma \rrbracket \otimes (\delta^{-1}; S + \pi_r); \delta^{-1}; [\pi_r, (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \llbracket \Gamma \rrbracket \otimes \delta^{-1}; \delta^{-1}; [\llbracket \Gamma \rrbracket \otimes S; \pi_r, \llbracket \Gamma \rrbracket \otimes \pi_r; (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \llbracket \Gamma \rrbracket \otimes \delta^{-1}; \delta^{-1}; [\pi_r, \llbracket \Gamma \rrbracket \otimes \pi_r; (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \Delta_{\llbracket \Gamma \rrbracket} \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket); \alpha; \llbracket \Gamma \rrbracket \otimes \delta^{-1}; \delta^{-1}; [\pi_r, \llbracket \Gamma \rrbracket \otimes \pi_r; (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \delta^{-1}; [\Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \pi_r, \Delta_{\llbracket \Gamma \rrbracket} \otimes \Sigma_i \llbracket A_i \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \pi_r; (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \delta^{-1}; [\Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \pi_r, (\text{rcase}(L))^\dagger]; S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \delta^{-1}; [\Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \pi_r, \text{let}(\text{rfix}(L)); \pi_l \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket)]); S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \delta^{-1}; [\Delta_{\llbracket \Gamma \rrbracket} \otimes \llbracket L \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \pi_r, \Delta_{\llbracket \Gamma \rrbracket} \otimes \Sigma_i \llbracket A_i \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \text{rfix}(L)]); S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \Delta_{\llbracket \Gamma \rrbracket} \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket); \alpha; \llbracket \Gamma \rrbracket \otimes \delta^{-1}; \delta^{-1}; [\llbracket \Gamma \rrbracket \otimes \pi_r, \llbracket \Gamma \rrbracket \otimes \text{rfix}(L)]); S) \\
& = \text{let}(\text{esem}_{\Gamma,L}(s); \Delta_{\llbracket \Gamma \rrbracket} \otimes (\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket); \alpha; \llbracket \Gamma \rrbracket \otimes (\delta^{-1}; [\pi_r, \text{rfix}(L)])); S) \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \llbracket \Gamma \rrbracket \otimes (\delta^{-1}; [\pi_r, \text{rfix}(L)])); S) \\
& = \text{let}(\text{let}(\text{esem}_{\Gamma,L}(s)); \delta^{-1}; [\pi_r, \text{rfix}(L)]); S) \\
& = \text{let}(\llbracket \Gamma \vdash s \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket; \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket)
\end{aligned}$$

(247)

as desired.

Composition of label substitutions then follows by a trivial induction. $\square$

C.6 Equational Theory

THEOREM 5.12 (SOUNDNESS (EQUATIONAL THEORY)). *We have that*

- (a) $\Gamma \vdash_\epsilon a \approx a' : A \implies \llbracket \Gamma \vdash_\epsilon a : A \rrbracket = \llbracket \Gamma \vdash_\epsilon a' : A \rrbracket$
- (b) $\Gamma \vdash r \approx r' \triangleright L \implies \llbracket \Gamma \vdash r \triangleright L \rrbracket = \llbracket \Gamma \vdash r' \triangleright L \rrbracket$
- (c) $\gamma \approx \gamma' : \Gamma \mapsto \Delta \implies \llbracket \gamma : \Gamma \mapsto \Delta \rrbracket = \llbracket \gamma' : \Gamma \mapsto \Delta \rrbracket$
- (d) $\sigma \approx \sigma' \vdash \Gamma : L \rightsquigarrow K \implies \llbracket \Gamma \vdash \sigma : L \rightsquigarrow K \rrbracket = \llbracket \Gamma \vdash \sigma' : L \rightsquigarrow K \rrbracket$

PROOF. We begin our proof by showing soundness of the equational theory for expressions, i.e. (a), by rule induction.

- *Congruence*: these follow trivially by induction
- *initial, terminal*: both of these follow trivially from the universal property of the initial/terminal object, respectively.
- *let₁-β*: this follows directly from Corollary 5.10
- *let₁-η*: we have

$$\begin{aligned}
& \llbracket \Gamma \vdash_\epsilon \text{let } x = a; x : A \rrbracket \\
& = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon a : A \rrbracket; \llbracket \Gamma, x : A \vdash_\epsilon x : A \rrbracket \\
& = \Delta_{\llbracket \Gamma \rrbracket}; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_\epsilon a : A \rrbracket; \pi_r \\
& = \llbracket \Gamma \vdash_\epsilon a : A \rrbracket
\end{aligned}$$

as desired.

- let₁-op: we have

$$\begin{aligned}
 & \llbracket \Gamma \vdash_{\epsilon} \text{let } x = a; \text{let } y = f \ x; \ c : C \rrbracket \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\llbracket \Gamma, x : A \vdash_{\epsilon} f \ x : B \rrbracket; \llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\pi_r; \llbracket f \rrbracket); \pi_l \otimes \llbracket B \rrbracket; \llbracket \Gamma, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \pi_r; \llbracket f \rrbracket); \llbracket \Gamma, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket f \rrbracket); \llbracket \Gamma, y : B \vdash_{\epsilon} c : C \rrbracket \\
 &= \llbracket \Gamma \vdash_{\epsilon} \text{let } y = f \ a; \ c : C \rrbracket
 \end{aligned}$$

as desired. The let₁-abort case is analogous.

- let₁-let₁: we have that

$$\begin{aligned}
 & \llbracket \Gamma \vdash_{\epsilon} \text{let } y = (\text{let } x = a; \ b; \ c) : C \rrbracket \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \llbracket \Gamma, x : A \vdash_{\epsilon} b : B \rrbracket); \llbracket \Gamma, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\llbracket \Gamma, x : A \vdash_{\epsilon} b : B \rrbracket; \pi_l \otimes \llbracket B \rrbracket; \llbracket \Gamma, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \text{let}(\llbracket \Gamma \vdash_{\epsilon} a : A \rrbracket; \text{let}(\llbracket \Gamma, x : A \vdash_{\epsilon} b : B \rrbracket; \llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket) \\
 &= \llbracket \Gamma \vdash_{\epsilon} \text{let } x = a; \text{let } y = b; \ c : C \rrbracket
 \end{aligned}$$

as desired.

- let₁-let₂: we have that

$$\begin{aligned}
 & \llbracket \Gamma \vdash_{\epsilon} \text{let } z = (\text{let } (x, y) = e; \ c); \ d : D \rrbracket \\
 &= \text{let}(\text{let}(\llbracket \Gamma \vdash_{\epsilon} e : A \otimes B \rrbracket; \alpha; \llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket); \llbracket \Gamma, z : C \vdash_{\epsilon} d : D \rrbracket) \\
 &= \text{let}(\Gamma \vdash_{\epsilon} e : A \otimes B; \alpha; \text{let}(\llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket; (\pi_l; \pi_l) \otimes \llbracket C \rrbracket; \llbracket \Gamma, z : C \vdash_{\epsilon} d : D \rrbracket) \\
 &= \text{let}(\Gamma \vdash_{\epsilon} e : A \otimes B; \alpha; \text{let}(\llbracket \Gamma, x : A, y : B \vdash_{\epsilon} c : C \rrbracket; \llbracket \Gamma, x : A, y : B, z : C \vdash_{\epsilon} d : D \rrbracket) \\
 &= \llbracket \Gamma \vdash_{\epsilon} \text{let } (x, y) = e; \text{let } z = c; \ d : D \rrbracket
 \end{aligned}$$

as desired.

- let₁-case: follows from the properties of the coproduct; in particular, we have that

$$\begin{aligned}
& \llbracket \Gamma \vdash_e \text{case } e \{ \iota_l x : \text{let } z = a; d, \iota_r y : \text{let } z = b; d \} : D \rrbracket \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \delta^{-1}; [\\
&\quad \Delta; \llbracket \Gamma, x : A \rrbracket \otimes \llbracket \Gamma, x : A \vdash_e a : C \rrbracket; \llbracket \Gamma, x : A, z : C \vdash_e d : D \rrbracket, \\
&\quad \Delta; \llbracket \Gamma, y : B \rrbracket \otimes \llbracket \Gamma, y : B \vdash_e b : C \rrbracket; \llbracket \Gamma, y : B, z : C \vdash_e d : D \rrbracket] \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \delta^{-1}; [\\
&\quad \Delta \otimes \llbracket A \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, x : A \vdash_e a : C \rrbracket; \llbracket \Gamma, z : C \vdash_e d : D \rrbracket, \\
&\quad \Delta \otimes \llbracket B \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, y : B \vdash_e b : C \rrbracket; \llbracket \Gamma, z : C \vdash_e d : D \rrbracket] \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \delta^{-1}; [\\
&\quad \Delta \otimes \llbracket A \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, x : A \vdash_e a : C \rrbracket, \\
&\quad \Delta \otimes \llbracket B \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, y : B \vdash_e b : C \rrbracket]; \llbracket \Gamma, z : C \vdash_e d : D \rrbracket \\
&= \Delta; \Delta \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \delta^{-1}; \\
&\quad [\alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, x : A \vdash_e a : C \rrbracket, \alpha; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma, y : B \vdash_e b : C \rrbracket]; \llbracket \Gamma, z : C \vdash_e d : D \rrbracket \\
&= \Delta; \Delta \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \alpha; \llbracket \Gamma \rrbracket \otimes (\delta^{-1}; [\llbracket \Gamma, x : A \vdash_e a : C \rrbracket, \llbracket \Gamma, y : B \vdash_e b : C \rrbracket]); \\
&\quad \llbracket \Gamma, z : C \vdash_e d : D \rrbracket \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes (\Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A + B \rrbracket; \delta^{-1}; [\llbracket \Gamma, x : A \vdash_e a : C \rrbracket, \llbracket \Gamma, y : B \vdash_e b : C \rrbracket]); \\
&\quad \llbracket \Gamma, z : C \vdash_e d : D \rrbracket \\
&= \llbracket \Gamma \vdash_e \text{let } z = (\text{case } e \{ \iota_l x : a, \iota_r y : b \}); d : D \rrbracket
\end{aligned}$$

- let₂-bind: we have

$$\begin{aligned}
& \llbracket \Gamma \vdash_e \text{let } z = e; \text{let } (x, y) = z; c : C \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_e e : A \otimes B \rrbracket); \text{let}(\llbracket \Gamma, z : A \otimes B \vdash_e z : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, z : A \otimes B, x : A, y : B \vdash_e c : C \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_e e : A \otimes B \rrbracket); \text{let}(\pi_r); \pi_l \otimes (\llbracket A \rrbracket \otimes \llbracket B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \vdash_e c : C \rrbracket \\
&= \text{let}(\text{let}(\llbracket \Gamma \vdash_e e : A \otimes B \rrbracket); \pi_r); \alpha; \llbracket \Gamma, x : A, y : B \vdash_e c : C \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash_e e : A \otimes B \rrbracket); \alpha; \llbracket \Gamma, x : A, y : B \vdash_e c : C \rrbracket \\
&= \llbracket \Gamma \vdash_e \text{let } (x, y) = e; c : C \rrbracket
\end{aligned}$$

- let₂-η: follows from the properties of the product; in particular, we have that

$$\begin{aligned}
& \llbracket \Gamma \vdash_e \text{let } (x, y) = e; (x, y) : A \otimes B \rrbracket \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A \otimes B \rrbracket; \Delta; \llbracket \Gamma, x : A, y : B \vdash_{\perp} x : A \rrbracket \otimes \llbracket \Gamma, x : A, y : B \vdash_{\perp} y : B \rrbracket \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e e : A \otimes B \rrbracket; \Delta; (\pi_l; \pi_r) \otimes \pi_r = \llbracket \Gamma \vdash_e e : A \otimes B \rrbracket
\end{aligned}$$

- case-inl: follows from the properties of the coproduct and inverse distributor; in particular, we have that

$$\begin{aligned}
& \llbracket \Gamma \vdash_e \text{case } \iota_l a \{ \iota_l x : c, \iota_r y : d \} : C \rrbracket \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes (\llbracket \Gamma \vdash_e a : A \rrbracket; \iota_l); \delta^{-1}; [\llbracket \Gamma, x : A \vdash_e c : C \rrbracket, \llbracket \Gamma, y : B \vdash_e d : C \rrbracket] \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e a : A \rrbracket; \iota_l; [\llbracket \Gamma, x : A \vdash_e c : C \rrbracket, \llbracket \Gamma, y : B \vdash_e d : C \rrbracket] \\
&= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_e a : A \rrbracket; \iota_l; \llbracket \Gamma, x : A \vdash_e c : C \rrbracket \\
&= \llbracket \Gamma \vdash_e \text{let } x = a; c : C \rrbracket
\end{aligned}$$

We can validate case-inr analogously

- case- η : follows from the properties of the coproduct and distributor; in particular, we have

$$\begin{aligned} & \llbracket \Gamma \vdash_{\epsilon} \text{case } e \{ \iota_l x : \iota_l x, \iota_r y : \iota_r y \} : A + B \rrbracket \\ &= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket; \delta^{-1}; [\llbracket \Gamma, x : A \vdash_{\epsilon} x : A \rrbracket; \iota_l, \llbracket \Gamma, y : B \vdash_{\epsilon} y : B; \iota_r \rrbracket] \\ &= \Delta; \llbracket \Gamma \rrbracket \otimes \llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket; \delta^{-1}; (\pi_r + \pi_r) = \llbracket \Gamma \vdash_{\epsilon} e : A + B \rrbracket \end{aligned}$$

We now proceed to tackle the equational theory for regions r in the same manner. In particular, we proceed by rule induction as follows:

- cfg- β_1 : Define $P = \llbracket \Gamma \vdash_{\perp} a : A_k \rrbracket$, $L = \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)$ and $R = (\ell_i(A_i),)_i$. We have that

$$\begin{aligned} & \llbracket \Gamma \vdash \text{br } \ell_k a \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket \\ &= \text{let}(P; \iota_{(L, R), \ell_k}; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P; \iota_k); \llbracket \Gamma \rrbracket \otimes \iota_r; \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P; \iota_k); \text{rfix}(L) \\ &= \text{let}(P; \iota_k); \text{rcase}(L); [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P); \text{rcase}(\llbracket \Gamma \rrbracket \otimes \iota_k; L); [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P); \text{rcase}(\llbracket \Gamma, x_k : A_k \vdash t_k \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P); \text{rlet}(\llbracket \Gamma, x_k : A_k \vdash t_k \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(\text{let}(P); \llbracket \Gamma, x_k : A_k \vdash t_k \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(\llbracket \Gamma \vdash \text{let } x_k = a; t_k \triangleright L, R \rrbracket; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \llbracket \Gamma \vdash \text{let } x_k = a; t_k \text{ where } (\ell_i(x_i) : \{A_i\},)_i \triangleright L \rrbracket \end{aligned} \tag{248}$$

as desired.

- cfg- β_2 : Define $P = \llbracket \Gamma \vdash_{\perp} b : B \rrbracket$, $L = \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)$ and $R = (\ell_i(A_i),)_i$. We have that

$$\begin{aligned} & \llbracket \Gamma \vdash \text{br } \kappa b \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket \\ &= \text{let}(P; \iota_{(L, R), \kappa}; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P; \iota_{L, \kappa}); \llbracket \Gamma \rrbracket \otimes \iota_l; \delta^{-1}; [\pi_r, \text{rfix}(L)] \\ &= \text{let}(P; \iota_{L, \kappa}); \pi_r = P; \iota_{L, \kappa} = \llbracket \Gamma \vdash \text{br } \kappa b \triangleright L \rrbracket \end{aligned} \tag{249}$$

as desired.

- **cfg- η** : Let $L = (\kappa_j(B_j),)_j$, and define $R = (\ell_i(A_i),)_i$ and $L = \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\},)_i)$. Using the **cfg- β_1** and **cfg- β_2** cases proved above, we have that

$$\begin{aligned}
& \llbracket \Gamma \vdash \text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\} : L, R \rightsquigarrow L \rrbracket \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\\
& \quad \llbracket \Gamma \vdash \text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\} : L \rightsquigarrow L \rrbracket, \\
& \quad \llbracket \Gamma \vdash \text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\} : R \rightsquigarrow L \rrbracket] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{\sum_i \llbracket B_i \rrbracket + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\\
& \quad \delta_{\Sigma}^{-1}; [\llbracket \Gamma, y_j : B_j \vdash \text{br } \kappa_j y_j \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket,]_j, \\
& \quad \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_j : A_j \vdash \text{br } \ell_j x_j \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket,]_j] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{\sum_i \llbracket B_i \rrbracket + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\\
& \quad \delta_{\Sigma}^{-1}; [\llbracket \Gamma, y_j : B_j \vdash_{\perp} y_j : B_j \rrbracket; \iota_{L, \kappa_j},]_j, \\
& \quad \delta_{\Sigma}^{-1}; [\text{let}(\llbracket \Gamma, x_j : A_j \vdash_{\perp} x_j : A_j \rrbracket; \iota_j); \text{rfix}(\pi_l \otimes \Sigma_k \llbracket A_k \rrbracket; L)]_j] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{\sum_i \llbracket B_i \rrbracket + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\delta_{\Sigma}^{-1}; [\pi_r; \iota_{L, \kappa_j},]_j, \delta_{\Sigma}^{-1}; [\text{let}(\pi_r; \iota_j); \pi_l \otimes \Sigma_k \llbracket A_k \rrbracket; \text{rfix}(L)]] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{\sum_i \llbracket B_i \rrbracket + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\pi_r; \alpha_L, \text{rlet}(\delta_{\Sigma}^{-1}; [\pi_r; \iota_j]_j); \text{rfix}(L)] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{L + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\pi_r, \text{rlet}(\pi_r); \text{rfix}(L)] \\
&= \llbracket \Gamma \rrbracket \otimes \alpha_{L + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\pi_r, \text{rfix}(L)]
\end{aligned} \tag{250}$$

It follows by label-substitution that that

$$\begin{aligned}
& \llbracket \Gamma \vdash [\text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\}] r \triangleright L \rrbracket \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \llbracket \Gamma \vdash \text{cfgs } \{(\ell_i(x_i) : \{t_i\},)_i\} : L, R \rightsquigarrow L \rrbracket) \\
&= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket; \llbracket \Gamma \rrbracket \otimes \alpha_{L + \sum_i \llbracket A_i \rrbracket}; \delta^{-1}; [\pi_r, \text{rfix}(L)]) \\
&= \text{esem}_{\Gamma, L}(r); \delta^{-1}; [\pi_r, \text{rfix}(L)] \\
& \llbracket \Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_i \triangleright L \rrbracket
\end{aligned} \tag{251}$$

as desired.

- **codiag**: Define $R = \llbracket \Gamma \vdash r \triangleright L, \ell(A) \rrbracket$, and $S = \llbracket \Gamma, y : A \vdash s \triangleright L, \ell(A), \kappa(A) \rrbracket$. We have that

$$\begin{aligned}
& \llbracket \Gamma \vdash r \text{ where } \ell(x) : \{\text{br } \kappa x \text{ where } \kappa(y) : \{s\}\} \triangleright L \rrbracket \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\llbracket \Gamma, x : A \vdash \text{br } \kappa x \text{ where } \kappa(y) : \{s\} \triangleright L, \ell(A) \rrbracket)] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\text{let}(\pi_r; \iota_r); \delta^{-1}; [\pi_r, \text{rfix}(\llbracket \Gamma, x : A, y : A \vdash s \triangleright L, \ell(A), \kappa(A) \rrbracket)])] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\text{let}(\pi_r; \iota_r); \delta^{-1}; [\pi_l \otimes A; \pi_r, \text{rfix}(\pi_l \otimes \llbracket A \rrbracket; S)])] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\text{let}(\pi_r; \iota_r); \pi_l \otimes \llbracket A \rrbracket; \delta^{-1}; [\pi_r, \text{rfix}(S)])] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\iota_r^{\llbracket \Gamma \rrbracket} \gg [\pi_r, \text{rfix}(S)]_{\llbracket \Gamma \rrbracket})] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\text{rfix}(S))] = \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(S \gg [\pi_r, \iota_r^{\llbracket \Gamma \rrbracket}]_{\llbracket \Gamma \rrbracket})] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\text{let}(S); \delta^{-1}; [\pi_r, \pi_r; \iota_r])] = \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(S; [\text{id}, \iota_r])] \\
&= \text{let}(R); \delta^{-1}; [\text{id}, \text{rfix}(\llbracket \Gamma, y : A \vdash [\ell/\kappa]s \triangleright L, \ell(A) \rrbracket)] = \llbracket \Gamma \vdash r \text{ where } \ell(y) : \{[\ell/\kappa]s\} \triangleright L \rrbracket
\end{aligned} \tag{252}$$

as desired.

- uni: Define $R = \llbracket \Gamma \vdash r \triangleright L, \ell(A) \rrbracket$, $E = \llbracket \Gamma, x : A \vdash_\perp e : B \rrbracket$, $S = \llbracket \Gamma, y : B \vdash s \triangleright L, \kappa(B) \rrbracket$, and $T = \llbracket \Gamma, x : A \vdash s \triangleright L, \ell(A) \rrbracket$. We have by induction that

$$\begin{aligned} \llbracket \Gamma, x : A \vdash \text{let } y = e; s \triangleright L, \kappa(B) \rrbracket &= \text{rlet}(E); S = \\ \llbracket \Gamma, x : A \vdash t \text{ where } \ell(x) : \{\text{br } \kappa \ e\} \triangleright L, \kappa(B) \rrbracket &= \text{rcase}(T); \llbracket L \rrbracket + E \end{aligned} \quad (253)$$

It follows in particular that

$$\text{rlet}(E); \text{rfix}(S) = \text{rfix}(T) \quad (254)$$

and hence that

$$\begin{aligned} \llbracket \Gamma \vdash (r \text{ where } \ell(x) : \{\text{br } \kappa \ e\}) \text{ where } \kappa(y) : \{s\} \triangleright L \rrbracket &= \\ = \text{let}(\text{let}(R); \delta^{-1}; \pi_r + E); \delta^{-1}; [\pi_r, \text{rfix}(S)] &= \\ = \text{let}(R); \delta^{-1}; (\llbracket \Gamma \rrbracket \otimes \llbracket L \rrbracket) + \text{rlet}(E); [\pi_r, \text{rfix}(S)] &= \text{let}(R); \delta^{-1}; [\pi_r, \text{rlet}(E); \text{rfix}(S)] \\ = \text{let}(R); \delta^{-1}; [\pi_r, \text{rfix}(T)] = \llbracket \Gamma \vdash r \text{ where } \ell(x) : \{t\} \triangleright L \rrbracket \end{aligned} \quad (255)$$

as desired.

- dinat: We define $R = (\ell_i(A_i))_i$, $R' = (\kappa_j(B_j))_j$, $S = \llbracket \Gamma \vdash \sigma : R \rightsquigarrow R' \rrbracket$, and $S' = \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket R \rrbracket}^+; S; \alpha_{\sum_j \llbracket B_j \rrbracket}^+$. We have that

$$\begin{aligned} \text{esem}_{\Gamma, L}([\sigma^1]r) &= \llbracket \Gamma \vdash [\sigma^1]r \triangleright L, R' \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket B_j \rrbracket}^+ \\ &= \text{let}(\llbracket \Gamma \vdash r \triangleright L, R \rrbracket); \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; \pi_r + (S; \alpha_{\sum_j \llbracket B_j \rrbracket}^+) \\ &= \text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; \pi_r + S' \end{aligned} \quad (256)$$

and, writing $L = \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{t_i\})_i)$,

$$\begin{aligned} \text{let}((\kappa_i(x_i) : \{[\sigma^1]t_i\})_i) &= \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : B_i \vdash [\sigma^1]t_i \triangleright L, R' \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket B_j \rrbracket}^+ \\ &= \delta_{\Sigma}^{-1}; [\text{let}(\llbracket \Gamma, x_i : B_i \vdash t_i \triangleright L, R \rrbracket); \llbracket \Gamma, x_i : B_i \vdash \sigma^1 : L, R \rightsquigarrow L, R' \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket B_j \rrbracket}^+ \\ &= \delta_{\Sigma}^{-1}; [\text{rlet}(\llbracket \Gamma, x_i : B_i \vdash t_i \triangleright L, R \rrbracket),]_i; \llbracket \Gamma \vdash \sigma^1 : L, R \rightsquigarrow L, R' \rrbracket; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket B_j \rrbracket}^+ \\ &= \text{rlet}(\delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : B_i \vdash t_i \triangleright L, R \rrbracket,]_i; \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R \rrbracket}^+; \delta^{-1}; \pi_r + (S; \alpha_{\sum_j \llbracket B_j \rrbracket}^+)) \\ &= \text{rlet}(\delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : B_i \vdash t_i \triangleright L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \sum_j \llbracket B_j \rrbracket}^+); \delta^{-1}; \pi_r + S' \\ &= \text{rcase}(L); \pi_r + S' \end{aligned} \quad (257)$$

Furthermore, we have that, letting $G = \llbracket \Gamma \vdash (\kappa_j(x_j) \mapsto t_j)_j : R' \rightsquigarrow L, R \rrbracket$,

$$\begin{aligned} \llbracket \Gamma, x_i : A_i \vdash [(\kappa_j(x_j) \mapsto t_j)_j]^1 (\sigma \ell_i x_i) \triangleright L, R \rrbracket &= \\ = \text{let}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright L, R' \rrbracket); \llbracket \Gamma, x_i : A_i \vdash (\kappa_j(x_j) \mapsto t_j)_j^1 : L, R' \rightsquigarrow L, R \rrbracket &= \\ = \text{let}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright L, R' \rrbracket); \pi_l \otimes \llbracket L, R' \rrbracket; \llbracket \Gamma \vdash (\kappa_j(x_j) \mapsto t_j)_j^1 : L, R' \rightsquigarrow L, R \rrbracket &= \\ = \text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright L, R' \rrbracket); \llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket L \rrbracket + \llbracket R' \rrbracket}^+; \delta^{-1}; [\pi_r; \iota_l; \alpha_{\llbracket L, R \rrbracket}^+, G] &= \\ = \text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright L, R' \rrbracket; \alpha_{\llbracket L \rrbracket + \llbracket R' \rrbracket}^+); \delta^{-1}; [\pi_r; \iota_l; \alpha_{\llbracket L, R \rrbracket}^+, G] &= \\ = \text{rcase}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright L, R' \rrbracket; \alpha_{\llbracket L \rrbracket + \llbracket R' \rrbracket}^+); [\pi_r; \iota_l; \alpha_{\llbracket L, R \rrbracket}^+, G] &= \\ = \text{rcase}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket; \iota_r); [\pi_r; \iota_l; \alpha_{\llbracket L, R \rrbracket}^+, G] &= \\ = \text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket); G \end{aligned} \quad (258)$$

It follows that

$$\begin{aligned}
& \text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{[(\kappa_j(x_j) \mapsto t_j),]^1](\sigma \ell_i x_i)\},)_i) \\
&= \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash [(\kappa_j(x_j) \mapsto t_j),]^1](\sigma \ell_i x_i) \triangleright L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \delta_{\Sigma}^{-1}; [\text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket); G]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \delta_{\Sigma}^{-1}; [\text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket); G]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \delta_{\Sigma}^{-1}; [\text{rlet}(\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket); G]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \text{rlet}(\delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket]_i; G; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+) \\
&= \text{rlet}(\delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash \sigma \ell_i x_i \triangleright R' \rrbracket]_i; G; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+) \\
&= \text{rlet}(\llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket R \rrbracket}; S); G; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \text{rlet}(S'); [\llbracket \Gamma \rrbracket \otimes \alpha_{\llbracket R \rrbracket}; [\Gamma \vdash (\kappa_j(x_j) \mapsto t_j),]_j : R' \rightsquigarrow L, R]; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \text{rlet}(S'); \delta_{\Sigma}^{-1}; [\llbracket \Gamma, x_i : A_i \vdash t_i \triangleright R' L, R \rrbracket,]_i; \alpha_{\llbracket L \rrbracket + \Sigma_i \llbracket A_i \rrbracket}^+ \\
&= \text{rlet}(S'); L
\end{aligned} \tag{259}$$

We therefore have

$$\begin{aligned}
& \llbracket \Gamma \vdash [\sigma^1]r \text{ where } (\kappa_i(x_i) : \{[\sigma^1]t_i\},)_i \triangleright L \rrbracket \\
&= \text{let}(\text{esem}_{\Gamma, L}([\sigma^1]r)); \delta^{-1}; [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, L}((\kappa_i(x_i) : \{[\sigma^1]t_i\},)_i))] \\
&= \text{let}(\text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; \pi_r + S'); \delta^{-1}; [\pi_r, \text{rfix}(\text{rcase}(L); \pi_r + S')] \\
&= \text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; [\llbracket \Gamma \rrbracket \otimes \llbracket L \rrbracket + \text{rlet}(S); [\pi_r, \text{rfix}(\text{rcase}(L); \pi_r + S')]] \\
&= \text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; [\pi_r, \text{rlet}(S'); \text{rfix}(\text{rcase}(L); \pi_r + S')] \\
&= \text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; [\pi_r, \text{rfix}(\text{rlet}(S'); L)] \\
&= \text{let}(\text{esem}_{\Gamma, L}(r)); \delta^{-1}; [\pi_r, \text{rfix}(\text{lsem}_{\Gamma, L}((\ell_i(x_i) : \{[(\kappa_j(x_j) \mapsto t_j),]^1](\sigma \ell_i x_i)\},)_i))] \\
&= \llbracket \Gamma \vdash r \text{ where } [(\kappa_j(x_j) \mapsto t_j),]^1](\sigma \ell_i x_i),)_i \triangleright L \rrbracket
\end{aligned} \tag{260}$$

as desired.

All other cases are analogous to those for expressions, and so omitted. $\square$

LEMMA C.7 (where-FUSION). *The rewrite rules cfg-fuse_1 (Eqn. 16) and cfg-fuse_2 are sound, where we define*

$$\begin{array}{c}
\forall i \in I. \Gamma, x_i : A_i \vdash t_i \triangleright L, (\ell_j(A_j),)_{j \in I}, \kappa(B) \\
\Gamma, y : B \vdash s \triangleright L, (\ell_j(A_j),)_{j \in I}, \kappa(B), (\ell'_{j'}(A'_{j'}),)_{j' \in I'}, \\
\hline
\Gamma \vdash r \triangleright L, (\ell_i(A_i),)_i, \kappa(B) \quad \forall i' \in I'. \Gamma, x'_{i'} : A'_{i'} \vdash t'_{i'} \triangleright L, (\ell_j(A_j),)_{j \in I}, \kappa(B), (\ell'_{j'}(A'_{j'}),)_{j' \in I'} \\
\Gamma \vdash r \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I}, \kappa(y) : \{s \text{ where } (\ell'_{i'}(x'_{i'}) : \{t'_{i'}\},)_{i' \in I'}\} \\
\approx r \text{ where } (\ell_i(x_i) : \{t_i\},)_{i \in I}, \kappa(y) : \{s\}, (\ell'_{i'}(x'_{i'}) : \{t'_{i'}\},)_{i' \in I'} \triangleright L
\end{array} \text{cfg-fuse}_2 \tag{261}$$

4754

4755

4757
4758
4759
4760
4761
4762
4763
4764
4765
4766
4767
4768
4769
4770
4771
4772
4773
4774
4775
4776
4777
4778

4779

4781

4782

4784
4785
4786
4787
4788
4789

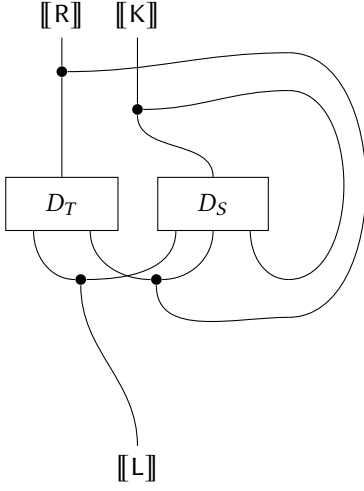
4790
4791
4792
4793

4796

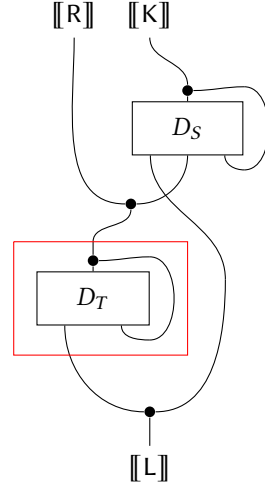
4797

4799

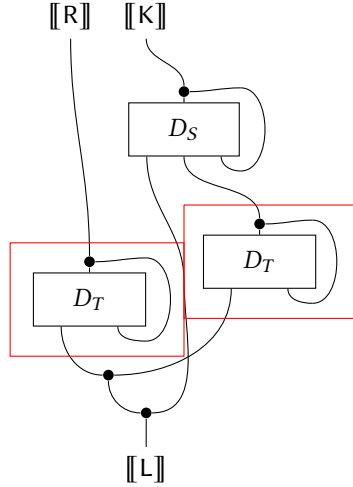
4800



(a) $\text{rfix}(D_G)$, with D_G drawn as per the right of Eqn. 264



(b) Equivalent to 38a by isotopy and associativity of the codiagonal. We highlight $\text{rfix}(D_T)$, which is used in the next step.



(c) $[\text{rfix}(D_T), \text{rfix}(D_S); \alpha_{\llbracket L \rrbracket + \sum_j \llbracket A_j \rrbracket}^+] \gg [\pi_r, \text{rfix}(D_T)]_{\llbracket R \rrbracket} \llbracket \Gamma \rrbracket$.
Equivalent to 38b by duplication of $\text{rfix}(D_T)$.

Fig. 38. String diagrams validating the soundness of cfg-fuse_1 (Eqn. 16), drawn in the co-Kleisli category induced by $\llbracket \Gamma \rrbracket$

PROOF. We begin by showing $\text{Th}^\otimes(\Gamma)$ is an λ_{SSA} expression model by validating all the equations for a distributive Freyd category. These are formalized as follows:

- $\text{Th}^\otimes(\Gamma)$ is a category: formalized in Rewrite/Term/Compose/Seq.lean
- $\text{Th}^\otimes(\Gamma)$ is a Freyd category: formalized in Rewrite/Term/Compose/Product.lean
- $\text{Th}^\otimes(\Gamma)$ has coproducts: formalized in Rewrite/Term/Compose/Sum.lean
- $\text{Th}^\otimes(\Gamma)$ is distributive: formalized in Rewrite/Term/Compose/Distrib.lean

We then prove that the packing of each type constructor is equivalent to its denotational semantics in $\text{Th}^\otimes(\Gamma)$ in `Rewrite/Term/Compose/Completeness.lean`. Finally, we show that packing and unpacking are mutual inverses for Γ pure in `Term.Eqv.packed_unpacked` and `Term.Eqv.unpacked_packed` in `Rewrite/Term/Structural/Product.lean`, completing the proof of initiality. $\square$

THEOREM 5.14 (COMPLETENESS (REGIONS)). *We have that, for all pure $\text{eff}(\Gamma) = \perp$,*

$$\Gamma \vdash r \approx r' \triangleright L \iff \llbracket \Gamma \vdash r \triangleright L \rrbracket_{\text{Th}(\cdot, \cdot)} = \llbracket \Gamma \vdash r' \triangleright L \rrbracket_{\text{Th}(\cdot, \cdot)}$$

In particular, this implies that $\text{Th}(\cdot, \cdot)$ is the initial λ_{SSA} model.

PROOF. We begin by showing $\text{Th}(\Gamma, L)$ is an λ_{SSA} region model by validating all the equations for a distributive Elgot category. These are formalized as follows:

- $\text{Th}(\Gamma, L)$ is a category: formalized in `Rewrite/Region/Compose/Seq.lean`
- $\text{Th}(\Gamma, L)$ is a Freyd category: formalized in `Rewrite/Region/Compose/Product.lean`
- $\text{Th}(\Gamma, L)$ has coproducts: formalized in `Rewrite/Region/Compose/Sum.lean`
- $\text{Th}(\Gamma, L)$ is distributive: formalized in `Rewrite/Region/Compose/Distrib.lean`
- $\text{Th}(\Gamma, L)$ is a strong Elgot category: formalized in `Rewrite/Region/Compose/Elgot.lean`

We then prove that the packing of each type constructor is equivalent to its denotational semantics in $\text{Th}(\Gamma, L)$ in `Rewrite/Region/Compose/Completeness.lean`. Finally, we show that packing and unpacking are mutual inverses for Γ pure in `Region.Eqv.packed_unpacked` and `Region.Eqv.unpacked_packed` in `Rewrite/Region/Structural.lean`, completing the proof of initiality. $\square$